

Nuclei Studio Supply Documents

Table of contents

Home

●	Nuclei Studio Supply Documents	14
●	Documents	14

Failure to Start QEMU in Nuclei Studio Due to Insufficient Memory

●	Failure to Start QEMU in Nuclei Studio Due to Insufficient Memory	16
●	Problem Description	16
●	Solution	16

Error When Debugging Programs with QEMU in Nuclei Studio on Windows 11

●	Error When Debugging Programs with QEMU in Nuclei Studio on Windows 11	17
●	Problem Description	17
●	Solution	17

How to print memory usage in Nuclei Studio

●	How to print memory usage in Nuclei Studio	18
●	Problem Description	18
●	Solution	19

Errors Occur When Using Pre-build Command/Post-build Command During Project Build

●	Errors Occur When Using Pre-build Command/Post-build Command During Project Build	20
●	Problem Description	20
●	Solution	22

Upgrading npk.yml to Support Nuclei Studio 2023.10

●	Upgrading npk.yml to Support Nuclei Studio 2023.10	23
●	Toolchain Upgrade in npk.yml	23
●	Upgrade of Extensions (ARCHEXT) Beyond the Standard IMAFDC	25
●	libncrt Upgrade	27
●	Eliminating Link Warnings	30
●	Detailed Changes of npk.yml in Nuclei SDK 0.5.0	31

GCC13 auto generated RVV instructions when RVV enabled

●	GCC13 auto generated RVV instructions when RVV enabled	32
●	Problem Description	32
●	Solution	32

Updating Nuclei Studio 2023.10 to the Latest Fixed Version

●	Updating Nuclei Studio 2023.10 to the Latest Fixed Version	34
●	Problem Description	34

●	How to Upgrade Nuclei Studio 2023.10 to the Latest Version	34
●	Upgrading Nuclei Studio 2023.10 Downloaded Before November 18, 2023	34
●	Downloading the Latest Version from the Official Website	37
●	References	38

Issues with OpenOCD When Operating on NOR Flash Larger Than 16 MB

●	Issues with OpenOCD When Operating on NOR Flash Larger Than 16 MB	39
●	Problem Description	39
●	Solution	39

Upgrading the Project Toolchain to GCC 13 by Modifying the .cproject File

●	Upgrading the Project Toolchain to GCC 13 by Modifying the .cproject File	40
●	Problem Description	40
●	Modifying Toolchain-Related Configuration	40
●	Modifying RISC-V Extension-Related Configuration	43
●	Modifying libnrt C Library-Related Configuration	45
●	Adding Configuration to Eliminate Link Warnings	47

Compiling Projects from the Command Line in Nuclei Studio

●	Compiling Projects from the Command Line in Nuclei Studio	49
●	Problem Description	49
●	Solution	49

Error Reported by OpenOCD When Flashing a Program: Error: Device ID 0x1e200a6d is not known as FESPI capable

● Error Reported by OpenOCD When Flashing a Program: Error: Device ID 0x1e200a6d is not known as FESPI capable	72
● Problem Description	72
● Solution	72

About the Inconsistency Between Dhrystone Benchmark Scores in the IDE and NSDK 0.5.0 Command-Line Scores

● About the Inconsistency Between Dhrystone Benchmark Scores in the IDE and NSDK 0.5.0 Command-Line Scores	74
● Problem Description	74
● Solution	74

Error: Couldn't find an available hardware trigger / Error: can't add breakpoint: resource not available

● Error: Couldn't find an available hardware trigger / Error: can't add breakpoint: resource not available	78
● Problem Description	78
● Solution	79

cannot find -ln crt_balanced: No such file or directory

● cannot find -ln crt_balanced: No such file or directory	80
● Problem Description	80
● Solution	82

UnsatisfiedLinkError of swt-win32-4965r8.dll on Windows 7

● UnsatisfiedLinkError of swt-win32-4965r8.dll on Windows 7	83
● Problem Description	83
● Solution	86

Issues You May Encounter When Using the Profiling Feature

● Issues You May Encounter When Using the Profiling Feature	87
● Issue 1: The log output reports insufficient on-chip memory, meaning there is not enough memory to store gprof/gcov data	87
● Solution	88
● Issue 2: Data parsing fails because the data collected in the Console or Terminal is incomplete	88
● Solution	90
● Issue 3: After deleting the gmon.out file, parsing again pops up a No files have been generated error dialog	92
● Solution	93
● Issue 4: Compilation error when importing amrwb_profiling_demo.zip in Nuclei Studio on Linux	94

An Example of Using the Profiling Feature in Nuclei Studio for Performance Tuning

●	An Example of Using the Profiling Feature in Nuclei Studio for Performance Tuning	96
●	Problem Description	96
●	Solution	96
●	1 Environment Preparation	96
●	2 Profiling Feature	97
●	3 Call Graph Feature	108
●	4 Code coverage Feature	110
●	5 Additional Notes	113

Demonstrate NICE/VNICE Instruction Acceleration of the Nuclei Model through Profiling

●	Demonstrate NICE/VNICE Instruction Acceleration of the Nuclei Model through Profiling	115
●	Background	115
●	Nuclei Model Profiling	115
●	NICE/VNICE Custom Instruction Acceleration	115
●	Solution	116
●	Environment Preparation	116
●	Model Profiling	116
●	step1: Create a new demo_vnice project	116
●	step2: Port the aes_demo bare-metal test case based on the demo_vnice project	
●	step3: Simulate the program with the model	117 118
●	step4: Parse the gprof data	121
●	step5: Replace hotspot functions with NICE/VNICE instructions	121

●	step6: Implement NICE/VNICE instructions in the Nuclei Model	126
●	step7: Re-analyze the hotspot functions	129

Rapid Verification of NICE/VNICE Instruction Acceleration with Nuclei Model and NICE Wizard

●	Rapid Verification of NICE/VNICE Instruction Acceleration with Nuclei Model and NICE Wizard	133
●	Background	133
●	xlmodel_nice	133
●	Nuclei NICE Wizard	133
●	test code	133
●	Solution	134
●	Environment Preparation	134
●	Running the Original Program on Nuclei Model	134
●	NICE Instruction Replacement	137
●	VNICE Instruction Replacement	143
●	Summary	147

Flash Programming Use Case

●	Flash Programming Use Case	148
●	Solution	148
●	Environment Preparation	148
●	Flash Programming Demo	148
●	Summary	157

Using the Live Watch Feature

● Using the Live Watch Feature	158
● Background	158
● Solution	158
● Environment Preparation	158
● Live Watch Usage Demo	158
● Summary	165

Tutorial for Adding Custom Assembly Instructions in LLVM

● Tutorial for Adding Custom Assembly Instructions in LLVM	166
● Recognizing the Custom Extension Name	166
● Recognizing Custom Assembly Instructions	166
● Usage Instructions	168
● References	168

How to Use the DebugMap Registers Provided by Nuclei to Analyze Error Scenarios

● How to Use the DebugMap Registers Provided by Nuclei to Analyze Error Scenarios	169
● First, Confirm That the Hardware Supports the DebugMap Feature	169
● What Are DebugMap Registers	169
● DebugMap Output Information in OpenOCD	169
● Possible Error Scenarios	170
● How to Correctly Use DebugMap to Analyze Error Scenarios	171

● Reading Other DebugMap Registers via OpenOCD	171
--	-----

Tutorial for Adding Custom Assembly Instructions in binutils

● Tutorial for Adding Custom Assembly Instructions in binutils	172
● Recognizing the Custom Extension Name	172
● Recognizing Custom Assembly Instructions	173
● Usage Instructions	174
● References:	175

Guide to Using the Nuclei Cross-Trigger Feature in OpenOCD

● Guide to Using the Nuclei Cross-Trigger Feature in OpenOCD	176
● Feature Overview	176
● Configuration File Examples	176
● 1. Synchronized Halt Group Configuration	176
● 2. Synchronized Resume Group Configuration	177
● Command-Line Verification Steps	178
● 1. Synchronized Halt Group Verification	178
● 2. Synchronized Resume Group Verification	179
● IDE Verification Steps	180
● 1. Synchronized Halt Group Verification	180
● 2. Synchronized Resume Group Verification	181

User Guide for FreeRTOS Debugging Support in OpenOCD

●	User Guide for FreeRTOS Debugging Support in OpenOCD	183
●	Environment Preparation	183
●	Usage Steps	183
●	Usage Notes	186

How to Debug with Multiple Hummingbird Debuggers Simultaneously

●	How to Debug with Multiple Hummingbird Debuggers Simultaneously	187
●	Problem Description	187
●	Solution	187
●	Download FT_PROG	187
●	View the Serial Number	188
●	Modify the Serial Number	189
●	Update the OpenOCD Configuration	190
●	Linux	190
●	Windows	190
●	References	190

Quick Porting from evalsoc to customsoc Based on Nuclei SDK

●	Quick Porting from evalsoc to customsoc Based on Nuclei SDK	192
●	Overview	192
●	Solution	192
●	Environment Preparation	192

●	Adaptation Modifications	192
●	1 Modify the CPU feature description macro file	192
●	2 Modify the CPU feature ISA configuration	193
●	3 Modify the memory map of link addresses	193
●	4 Modify the OpenOCD configuration file	193
●	5 Modify the SysTimer frequency	193
●	6 Modify the CPU core frequency	193
●	7 Modify the UART driver	194
●	8 Modify the UART baud rate	194
●	9 Modify _premain_init	194
●	10 Remove the code used internally by Nuclei	194
●	11 Check peripheral addresses	194
●	Test Run	195
●	Adjust Names	196
●	Streamline the Code	196
●	IAR Project	196
●	IDE Project Support	197
●	References	197

Using the OVERLAY Command in Linker Scripts

●	Using the OVERLAY Command in Linker Scripts	198
●	Problem Description	198
●	Solution	198
●	Sample Program	198
●	Overlay Layout	199
●	Writing the Linker Script	199
●	Test Results	200

● Notes	202
● References	202

Nuclei Studio Supply Documents

Deploy MkDocs passing pages-build-deployment passing

This repository is utilized for providing supply documents, user guides, wikis, and facilitating discussions related to Nuclei Studio.

Note

- The latest version of Nuclei Studio IDE is 2025.10, which can be found in <https://github.com/Nuclei-Software/nuclei-studio/releases/tag/2025.10>
- In Ubuntu 20.04, you must install `libncursesw5 libtinfo5 libfdt1 libpixmap-1-0 libpng16-16 libasound2 libglib2.0-0` to make `riscv64-unknown-elf-gdb` and `qemu` able to run.

PDF Version can be found here: https://doc.nucleisys.com/nuclei_studio_supply/pdf/nuclei_studio_supply.pdf

- Nuclei Studio IDE Documentation: https://doc.nucleisys.com/nuclei_tools/ide/index.html
- Nuclei Tools(Toolchain/OpenOCD/Qemu/Model) Documentation: https://doc.nucleisys.com/nuclei_tools/
- Nuclei Studio NPK Introduction:
 - <https://github.com/Nuclei-Software/nuclei-sdk/wiki/Nuclei-Studio-NPK-Introduction>
 - https://doc.nucleisys.com/nuclei_tools/ide/npkoverview.html

Please create new doc based on [Doc Template](#)

Click [this link](#) to see online version.

If you come across any spelling errors or areas that need improvement in the document, feel free to submit a Pull Request or Issue to help us enhance it!

Documents

Generated by `python3 update.py` @ 2026-07-28 05:04:12

- [1. Failure to Start QEMU in Nuclei Studio Due to Insufficient Memory](#)
- [2. Error When Debugging Programs with QEMU in Nuclei Studio on Windows 11](#)
- [3. How to print memory usage in Nuclei Studio](#)
- [4. Errors Occur When Using Pre-build Command/Post-build Command During Project Build](#)
- [5. Upgrading npk.yml to Support Nuclei Studio 2023.10](#)
- [6. GCC13 auto generated RVV instructions when RVV enabled](#)

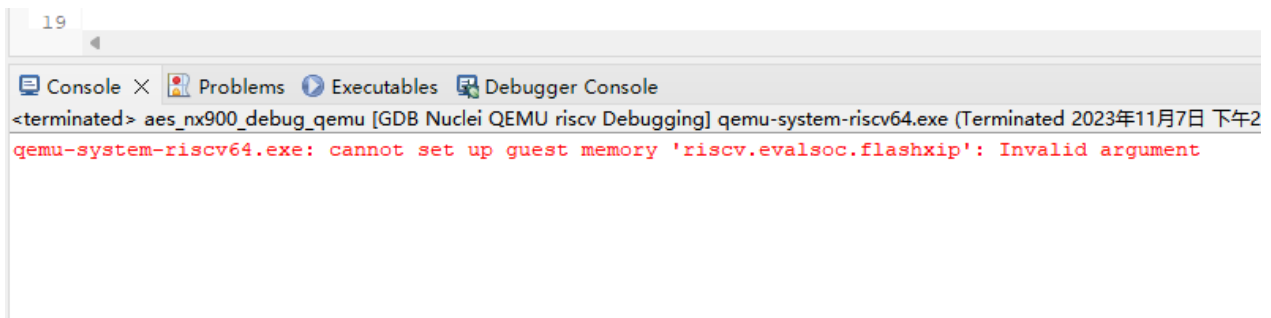
- [7. Updating Nuclei Studio 2023.10 to the Latest Fixed Version](#)
- [8. Issues with OpenOCD When Operating on NOR Flash Larger Than 16 MB](#)
- [9. Upgrading the Project Toolchain to GCC 13 by Modifying the .cproject File](#)
- [10. Compiling Projects from the Command Line in Nuclei Studio](#)
- [11. Error Reported by OpenOCD When Flashing a Program: Error: Device ID 0x1e200a6d is not known as FESPI capable](#)
- [12. About the Inconsistency Between Dhrystone Benchmark Scores in the IDE and NSDK 0.5.0 Command-Line Scores](#)
- [13. Error: Couldn't find an available hardware trigger / Error: can't add breakpoint: resource not available](#)
- [14. cannot find -Incrtr_balanced: No such file or directory](#)
- [15. UnsatisfiedLinkError of swt-win32-4965r8.dll on Windows 7](#)
- [16. Issues You May Encounter When Using the Profiling Feature](#)
- [17. An Example of Using the Profiling Feature in Nuclei Studio for Performance Tuning](#)
- [18. Demonstrate NICE/VNICE Instruction Acceleration of the Nuclei Model through Profiling](#)
- [19. Rapid Verification of NICE/VNICE Instruction Acceleration with Nuclei Model and NICE Wizard](#)
- [20. Flash Programming Use Case](#)
- [21. Using the Live Watch Feature](#)
- [22. Tutorial for Adding Custom Assembly Instructions in LLVM](#)
- [23. How to Use the DebugMap Registers Provided by Nuclei to Analyze Error Scenarios](#)
- [24. Tutorial for Adding Custom Assembly Instructions in binutils](#)
- [25. Guide to Using the Nuclei Cross-Trigger Feature in OpenOCD](#)
- [26. User Guide for FreeRTOS Debugging Support in OpenOCD](#)
- [27. How to Debug with Multiple Hummingbird Debuggers Simultaneously](#)
- [28. Quick Porting from evalsoc to customsoc Based on Nuclei SDK](#)
- [29. Using the OVERLAY Command in Linker Scripts](#)

Failure to Start QEMU in Nuclei Studio Due to Insufficient Memory

Problem Description

During actual development, it has been observed that when many processes are running on the computer at the same time, or when the computer's system memory is insufficient, the following error may occur when debugging a program with QEMU in Nuclei Studio:

```
qemu-system-riscv64.exe: cannot set up guest memory  
'riscv.evalsoc.flashxip' Invalid argument
```



Solution

In general, this problem can be resolved by closing some applications to free up memory for QEMU to use.

Error When Debugging Programs with QEMU in Nuclei Studio on Windows 11

Problem Description

When developing with Nuclei Studio on Windows 11, the following errors may occur when debugging a program with QEMU. This is caused by missing related dependencies on Windows 11, but it generally does not affect the correct use of QEMU, so this error can be ignored.

```
qemu-system-riscv32.exe: warning: GLib-GIO: Unexpectedly, UWP app
'Microsoft.ScreenSketch_11.2309.16.0_x64__8wekyb3d8bbwe' (AUMId
'Microsoft.ScreenSketch_8wekyb3d8bbwe!App') supports 29 extensions
but has no verbs
qemu-system-riscv32.exe: warning: GLib-GIO: Unexpectedly, UWP app
'Clipchamp.Clipchamp_2.8.1.0_neutral_yxz26nhyzhst' (AUMId
'Clipchamp.Clipchamp_yxz26nhyzhst!App') supports 41 extensions but
has no verbs
```

```
<terminated> 050hello_debug_qemu [GDB Nuclei QEMU riscv Debugging] qemu-system-riscv32.exe (Terminated 2023年11月7日 下午2:52:59)
GDB Server listening on: 'tcp::1234'...
Nuclei SDK Build Time: Nov 7 2023, 14:46:26

Download Mode: ILM

CPU Frequency 2295653007 Hz

CPU HartID: 0

qemu-system-riscv32.exe: warning: GLib-GIO: Unexpectedly, UWP app 'Microsoft.ScreenSketch_11.2309.16.0_x64__8wekyb3d8bbwe' (AUMId 'Microsoft.ScreenSketch_8wekyb3d8bbwe!App') supports 29 ext
qemu-system-riscv32.exe: warning: GLib-GIO: Unexpectedly, UWP app 'Clipchamp.Clipchamp_2.8.1.0_neutral_yxz26nhyzhst' (AUMId 'Clipchamp.Clipchamp_yxz26nhyzhst!App') supports 41 extensions
qemu-system-riscv32.exe: QEMU: Terminated via GDBStub
```

Solution

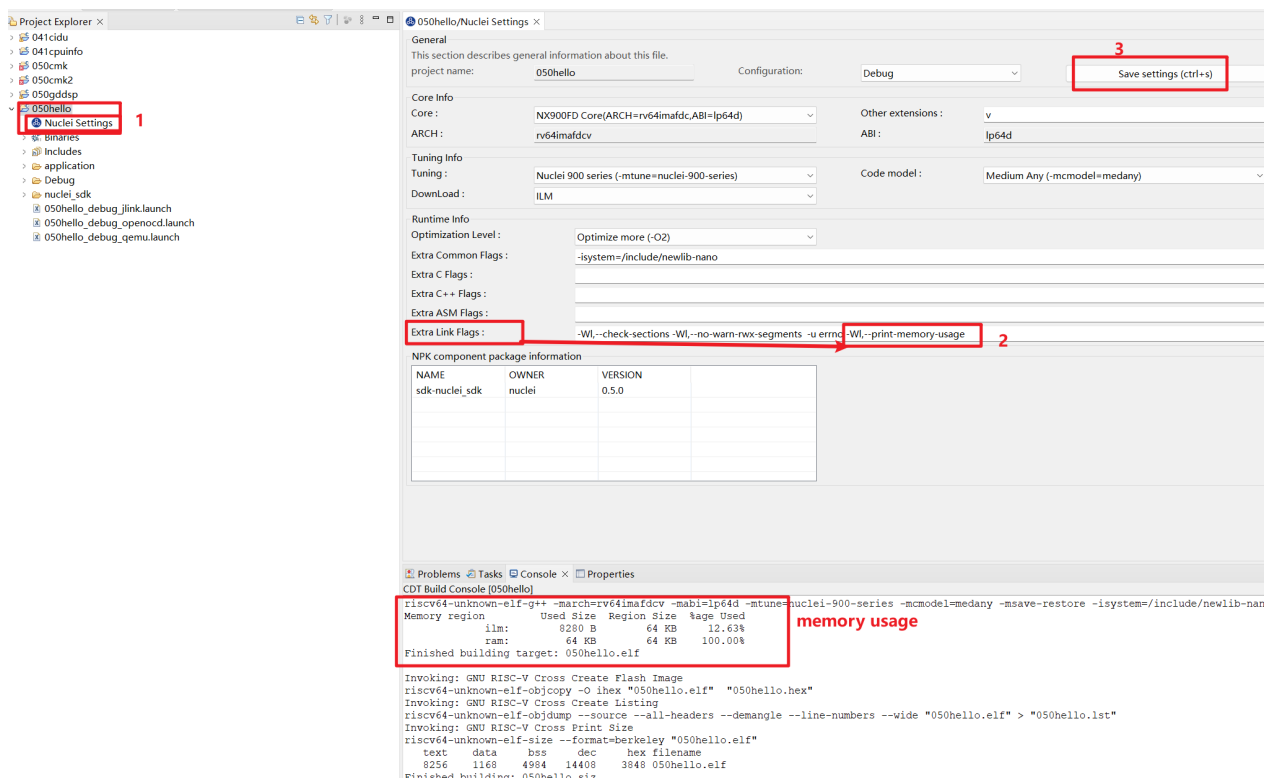
This is a system library matching issue that exists on Windows 10/11. It does not affect the normal use of QEMU and can be ignored.

How to print memory usage in Nuclei Studio

Problem Description

In order to print memory usage when compile an application, you can do it like this:

Click Nuclei Settings in selected project, and pass extra `-Wl,--print-memory-usage` in Extra Link Flags, and save settings, and then build this project, you will be able to see memory usage.



Building target: 050hello.elf

Invoking: GNU RISC-V Cross C++ Linker

... ..

Memory region	Used Size	Region Size	%age Used
ilm:	8280 B	64 KB	12.63%
ram:	64 KB	64 KB	100.00%

Finished building target: 050hello.elf

Why the ram usage here is 100% used?

For Nuclei SDK or NMSIS template linker script, the stack is placed at the bottom of ram memory, so the ram usage is 100%.

Solution

Add extra link option `-Wl,--print-memory-usage` will solve this issue.

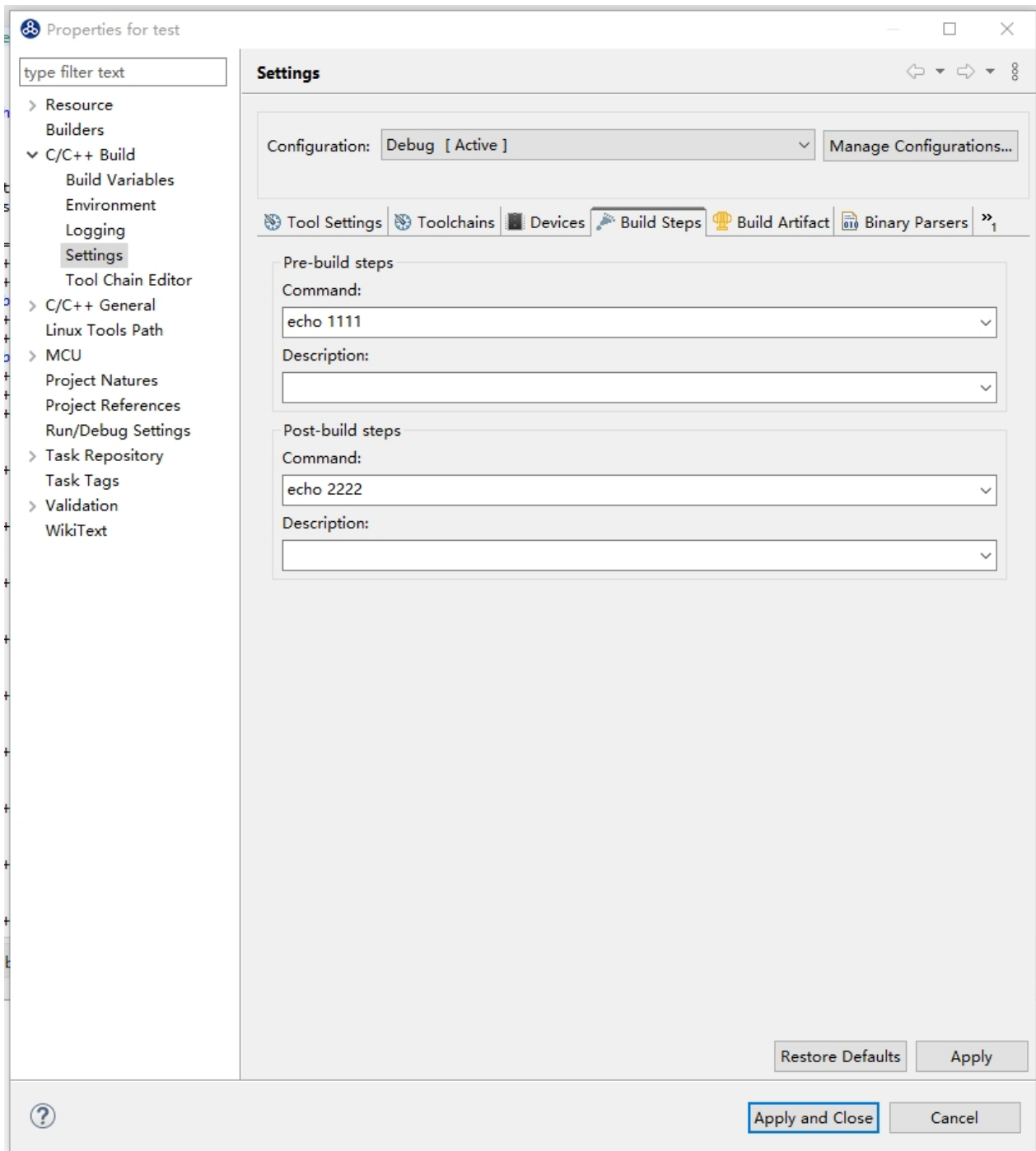
Errors Occur When Using Pre-build Command/Post-build Command During Project Build¶

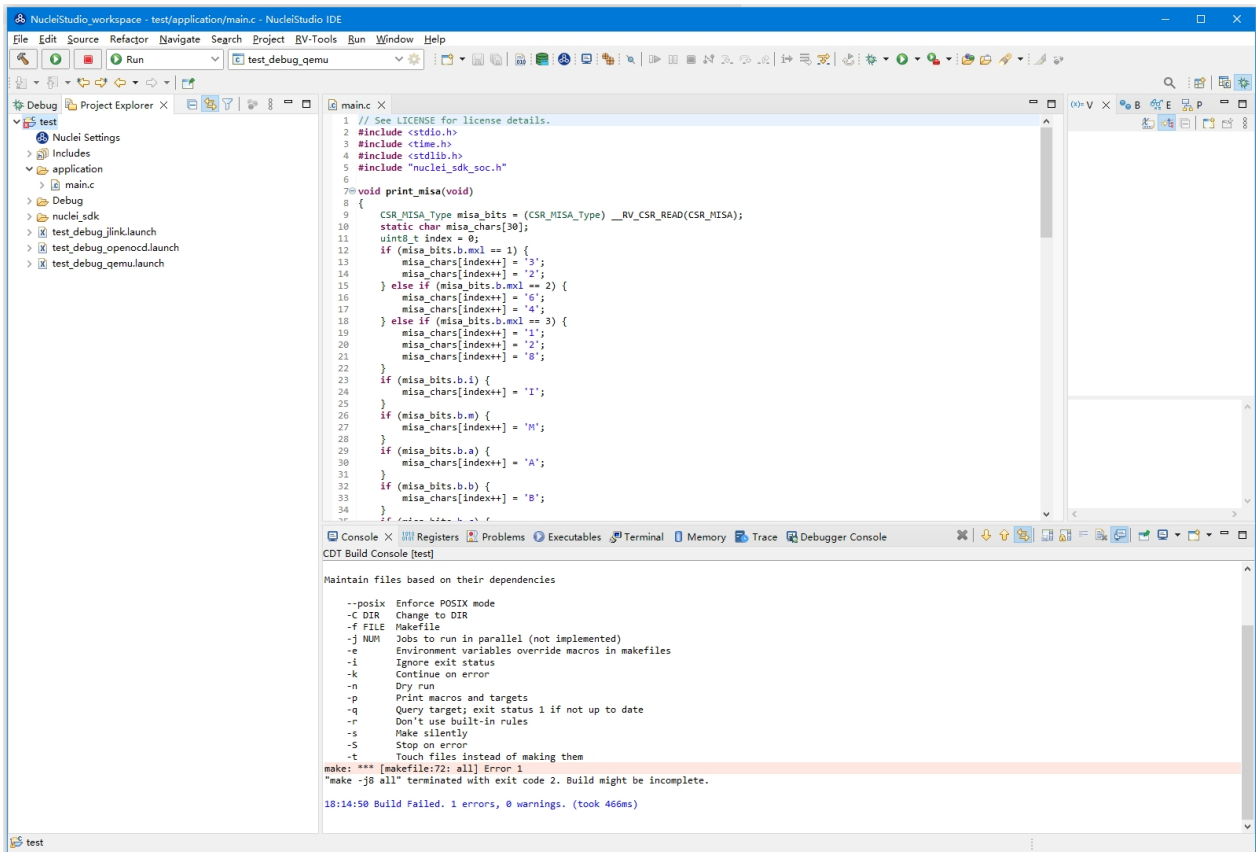
Problem Description¶

This issue is fixed in the corrected 2023.10 version uploaded on Nuclei Studio 2023.10.17. See [this article](#)

See [eclipse-embed-cdt/eclipse-plugins#597](#)

In Nuclei Studio 2023.10, if the Pre-build Command/Post-build Command is needed during project compilation, an error will occur because the build-tools integrated in Nuclei Studio is version v4.4.0, while the method used by the upstream CDT to handle Pre-build Command/Post-build Command does not work properly with build-tools v4.4.0.





Solution

When encountering this situation, you can download https://www.nucleisys.com/upload/files/toochain/build-tools/build-tools_202002.zip and replace the build-tools in the toolchain to resolve the issue.

Upgrading to the latest 2024.06 or 2025.02 versions also resolves this problem.

NucleiStudio\toolchain\build-tools

Upgrading npk.yml to Support Nuclei Studio 2023.10¶

In Nuclei Studio 2023.10, an important change is the support for GCC 13. Therefore, previously released NPK Packages also need corresponding changes to better work with Nuclei Studio 2023.10. The changes include the following points.

Note that the new version of npk.yml no longer supports the previous 2022.12 version of the IDE

Toolchain Upgrade in npk.yml¶

In NPK, we define buildconfig to customize various parameters used when building a project. Nuclei Studio uses the type field to identify which toolchain is used, such as gcc, clang, etc., and uses type->toolchain_name & cross_prefix to identify the specific distribution within that toolchain. To upgrade an SDK to support GCC 13, comparing the following two examples makes it easy to see that you only need to modify toolchain_name: RISC-V GCC/Newlib and cross_prefix: riscv64-unknown-elf-, so that the SDK supports selecting the GCC 13 toolchain when creating a project.

The following is the buildconfig configuration that supports gcc 10 (some parameters are hidden for the sake of example; define the actual parameters according to your situation).

```
## Build Configuration
buildconfig:
  - type: gcc
    description: Nuclei GNU Toolchain
    cross_prefix: riscv-nuclei-elf- # optional
    common_flags: # flags need to be combined together across all
packages
    ldflags:
    cflags:
    asmflags:
    cxxflags:
    common_defines:
    prebuild_steps: # could be override by app/bsp type
      command:
      description:
    postbuild_steps: # could be override by app/bsp type
      command:
      description:
```

The following is the buildconfig configuration that supports GCC 13 and Clang (some parameters are hidden for the sake of example; define the actual parameters according to your situation).

```
## Build Configuration
buildconfig:
  - type: gcc
    description: Nuclei GNU Toolchain
    # When upgrading to GCC13, make the following two-line changes
    here
    # And for all npk.yml files, any file containing buildconfig
    needs to be modified, not limited to ssp/bsp types, but also
    including bsp/app/mwp/osp/sdk types
    toolchain_name: RISC-V GCC/Newlib
    cross_prefix: riscv64-unknown-elf- # optional
    common_flags: # flags need to be combined together across all
    packages
    ldflags:
    cflags:
    asmflags:
    cxxflags:
    common_defines:
    prebuild_steps: # could be override by app/bsp type
      command:
      description:
    postbuild_steps: # could be override by app/bsp type
      command:
      description:
  - type: clang
    description: Nuclei LLVM Toolchain
    toolchain_name: RISC-V Clang/Newlib
    cross_prefix: riscv64-unknown-elf- # optional
    common_flags: # flags need to be combined together across all
    packages
    ldflags:
    cflags:
    asmflags:
    cxxflags:
    common_defines:
    prebuild_steps: # could be override by app/bsp type
      command:
      description:
    postbuild_steps: # could be override by app/bsp type
      command:
      description:
```

Upgrade of Extensions (ARCHEXT) Beyond the Standard IMAFDC¶

The following example uses the npk.yml upgrade of evalsoc in Nuclei SDK 0.5.0, considering only GCC support. If you need to consider CLANG support, please refer to the detailed changes of evalsoc's npk.yml in the SDK

In GCC 13, there are significant changes to the use of RISC-V instruction extensions. For details, see Section 2.1.4 of the Nuclei Studio User Guide and the ARCH_EXT description in Nuclei SDK.

- [Nuclei Studio User Guide](#)
- [ARCH_EXT Description](#)

When upgrading npk.yml, if the SDK uses RISC-V instruction extensions beyond the standard IMAFDC, such as B/P/K/V, the corresponding configuration also needs to be upgraded.

In NPK, RISC-V instruction extensions are passed to Nuclei Studio in the form of `-march=xxx`. When Nuclei Studio receives the relevant configuration, it stores it and applies it during compilation. Taking the npk.yml in Nuclei SDK as an example, the value of `-march=` can be obtained through the configuration below. It is easy to see that what relates to RISC-V instruction extensions is the NPK variable `nuclei_archext`.

```
## (Some parameters are hidden for the sake of example; define the
## actual parameters according to your situation)
## Build Configuration
buildconfig:
  - type: gcc
    description: Nuclei RISC-V GNU Toolchain #must
    cross_prefix: riscv-nuclei-elf- # optional
    common_flags: # flags need to be combined together across all
packages
    # The value passed to -march here is concatenated from the
two variables nuclei_core.arch and nuclei_archext
    # For example, if nuclei_core.arch is set to rv32imafdc and
nuclei_archext is set to _zba_zbb_zbc_zbs_xldspn1x,
    # then what is passed is -
march=rv32imafdc_zba_zbb_zbc_zbs_xldspn1x
    # If your march is known and fixed, you can directly specify
the -march/-mabi options here, without passing them through the
configuration field
    - flags: -march=${nuclei_core.arch}${join(
{nuclei_archext},'')} -mabi=${nuclei_core.abi}
    ldflags:
```

```

cflags:
asmflags:
cxxflags:
common_defines:
prebuild_steps: # could be override by app/bsp type
  command:
  description:
postbuild_steps: # could be override by app/bsp type
  command:
  description:

```

In the old version of the SDK, `nuclei_archext` was defined as a `multicheckbox` that users could select by themselves, while in the new version of the SDK, `nuclei_archext` is defined as a text input box, so that users can use RISC-V instruction extensions more flexibly. If in certain projects or scenarios you want to preset some RISC-V instruction extensions, it is recommended to simply give a default value. You can refer to the sample code below.

- Syntax for supporting Nuclei RISC-V Toolchain 2022.12

```

## In the old version of the SDK, nuclei_archext was defined as a
multicheckbox
## (Some parameters are hidden for the sake of example; define
the actual parameters according to your situation)
nuclei_archext:
  default_value: []
  type: multicheckbox
  global: true
  description: Nuclei ARCH Extensions
  choices:
    - name: b
      description: Bitmanip Extension
    - name: p
      description: Packed SIMD Extension
    - name: v
      description: Vector Extension

```

- Syntax for supporting Nuclei RISC-V Toolchain 2023.10

```

## In the new version of the SDK, nuclei_archext is defined as
a text input box
## Package Configurations
configuration:
nuclei_archext:
  default_value: "_zba_zbb_zbc_zbs"

```



```

type: text
global: true
# hints and tips are introduced in Nuclei Studio 2023.10
# used to show tool tips and input hints
tips: "Possible other ISA extensions, seperated by
underscores, like '_zba_zbb_zbc_zbs_xlDSPn1x'"
hints: "_zba_zbb_zbc_zbs_xlDSPn1x"
description: Nuclei ARCH Extensions

```

The final display effect when creating a project is as follows

Project name: cs1

Project Filter by: no filter Filters:

Project Example: Coremark Benchmark @app-nsdk_coremark

Toolchain: RISC-V GCC/Newlib (riscv64-unknown-elf-gcc)

Download/Run Mode: ILM download mode, program will be download into ilm/ram and run c

Nuclei RISC-V Core: N307FD Core(ARCH=rv32imafdc, ABI=ilp32d)

Nuclei ARCH Extensions: _zba_zbb_zbc_zbs

Nuclei Cache Extensions: ☒ ICache ☒ DCache ☐ CCM

Nuclei SMP Count: 0

Boot HartID: 0

Heap Size: 4K

Stack Size Per CPU: 4K

Enable Semihosting: ☐

Standard C Library: newlib nano with printf float

Select NMSIS Library: No NMSIS Library used

Select Benchmark Options: Options for Nuclei 300 Series

? < Back Next > Finish Cancel

libnrt Upgrade

libnrt has also changed somewhat compared to before. Before using libnrt in NPK, both the old and new versions of the SDK define a variable `stdclib` in configuration, whose value is a

dropdown box that allows selecting different values. The difference lies in how `stdlib` is used in `common_flags` or elsewhere after it is obtained.

For some notes about `stdlib`, see [here](#)

```
## Define the stdlib variable
## (Some parameters are hidden for the sake of example; define the
## actual parameters according to your situation)
## Package Configurations
configuration:
  stdlib:
    default_value: newlib_nano
    type: choice
    global: true
    description: Standard C Library
    choices:
      - name: newlib_full
        description: newlib with full feature
      - name: newlib_fast
        description: newlib nano with printf/scanf float
      - name: newlib_small
        description: newlib nano with printf float
      - name: newlib_nano
        description: newlib nano without printf/scanf float
      - name: libncrt_fast
        description: nuclei c runtime library, optimized for speed
      - name: libncrt_balanced
        description: nuclei c runtime library, balanced, full
feature
  - name: libncrt_small
    description: nuclei c runtime library, optimized for size,
full feature
  - name: libncrt_nano
    description: nuclei c runtime library, optimized for size,
no float support
  - name: libncrt_pico
    description: nuclei c runtime library, optimized for size,
no long/long long support
  - name: nostd
    description: no std c library will be used, and don't
search the standard system directories for header files
  - name: nospec
    description: no std c library will be used, not pass any --
specs options
```

In the new version of the SDK, if `--specs=libncrt_XXX.specs` is used or the linked libraries contain `-lncrt_XXX` (indicating the use of the libncrt C library), it needs to be changed to `-lncrt_XXX -lfileops_uart -lheapops_basic`. This is also the principle for changing an old SDK into a new SDK that supports GCC 13.

The configuration below shows the npk variable `stdclib` in the old SDK. When the variable `stdclib` starts with `libncrt`, a `--specs=${stdclib}.specs` is defined directly. According to the principle we mentioned above, this should be changed to setting `-l$(subst(${stdclib},lib,)) -lfileops_uart -lheapops_basic`, so the syntax in the new SDK becomes the configuration below.

```
## Using the stdclib variable in the old SDK
## (Some parameters are hidden for the sake of example; define the
## actual parameters according to your situation)
## Build Configuration
buildconfig:
  - type: gcc
    description: Nuclei GNU Toolchain
    cross_prefix: riscv-nuclei-elf- # optional
    common_flags: # flags need to be combined together across all
packages
    - flags: --specs=${stdclib}.specs
      condition: $( startswith(${stdclib}, "libncrt") )
  ldflags:
  cflags:
  asmflags:
  cxxflags:
  common_defines:
  prebuild_steps: # could be override by app/bsp type
    command:
    description:
  postbuild_steps: # could be override by app/bsp type
    command:
    description:
```

Changed to

```
## Using the stdclib variable in the new SDK
## (Some parameters are hidden for the sake of example; define the
## actual parameters according to your situation)
## Build Configuration
buildconfig:
  - type: gcc
    description: Nuclei GNU Toolchain
    toolchain_name: RISC-V GCC/Newlib
```

```

cross_prefix: riscv64-unknown-elf- # optional
common_flags: # flags need to be combined together across all
packages
  - flags: --specs=${stdclib}.specs
    condition: $( startswith(${stdclib}, "libncrt") )
ldflags:
  - flags: -l$(subst(${stdclib},lib,)) -lheapops_basic -
lfileops_uart
    condition: $( startswith(${stdclib}, "libncrt") )
cflags:
asmflags:
cxxflags:
common_defines:
prebuild_steps: # could be override by app/bsp type
  command:
  description:
postbuild_steps: # could be override by app/bsp type
  command:
  description:

```

Eliminating Link Warnings

In Nuclei Studio 2023.10, the integrated GCC 13 produces warnings during use. Adding the link option `-Wl, --no-warn-rwx-segments` can hide the warnings.

For details, refer to the following configuration (some parameters are hidden for the sake of example; define the actual parameters according to your situation)

```

## Build Configuration
buildconfig:
  - type: gcc
    description: Nuclei GNU Toolchain
    toolchain_name: RISC-V GCC/Newlib
    cross_prefix: riscv64-unknown-elf- # optional
    common_flags: # flags need to be combined together across all
packages
    ldflags:
      # Used to eliminate warnings in the gcc13 linking stage
      - flags: -Wl, --no-warn-rwx-segments
    cflags:
    asmflags:
    cxxflags:
    common_defines:
    prebuild_steps: # could be override by app/bsp type
      command:

```

```
description:  
postbuild_steps: # could be override by app/bsp type  
command:  
description:
```

Detailed Changes of npk.yml in Nuclei SDK 0.5.0

For the npk.yml changes supporting Nuclei Studio + Nuclei RISC-V Toolchain 2023.10, you can refer to the changes in nuclei-sdk 0.5.0.

- Changes for gd32vf103: `git diff 0.4.1..0.5.0 SoC/gd32vf103/***npk.yml`
- Changes for evalsoc: `git diff 0.4.1..0.5.0 SoC/evalsoc/***npk.yml`
- Changes for NMSIS: `git diff 0.4.1..0.5.0 NMSIS/***npk.yml`
- Changes for application: `git diff 0.4.1..0.5.0 application/***npk.yml`
- Changes for RTOS: `git diff 0.4.1..0.5.0 OS/***npk.yml`

The commands to view the code changes are as follows

```
git clone https://github.com/Nuclei-Software/nuclei-sdk/  
cd nuclei-sdk  
git fetch --all  
git diff 0.4.1..0.5.0 SoC/gd32vf103/***npk.yml  
git diff 0.4.1..0.5.0 SoC/evalsoc/***npk.yml  
git diff 0.4.1..0.5.0 NMSIS/***npk.yml  
git diff 0.4.1..0.5.0 application/***npk.yml  
git diff 0.4.1..0.5.0 OS/***npk.yml
```

GCC13 auto generated RVV instructions when RVV enabled¶

Problem Description¶

If you are using Nuclei SDK 0.5.0 with Nuclei RISC-V Toolchain 2023.10, and when compiling some examples with RVV enabled, it may generate RVV instructions, which is called auto-vectorization.

Take application/baremetal/benchmark/dhrystone for example:

```
cd application/baremetal/benchmark/dhrystone
# enable extra vector extension, which means the -march=rv64imafdcv
make CORE=nx900fd ARCH_EXT=v clean
make CORE=nx900fd ARCH_EXT=v dasm
```

Then if you check the dhrystone.dasm, you will be able to see RVV instructions:

Solution¶

These auto-generated instructions may affect your hardware performance, so if you want to disable it, you don't need to pass the RVV extension when compiling the application.

```
$ cat dhrystone.dasm |grep vs
      800003e2:  cc3ff057          vsetivli      zero,
31,e8,m8,ta,ma
      800003f8:  02038427          vse8.v  v8,(t2)
      8000040c:  020b8027          vse8.v  v0,(s7)
      800004a2:  cc3ff057          vsetivli      zero,
31,e8,m8,ta,ma
      800004b2:  02098827          vse8.v  v16,(s3)
      80000524:  cc3ff057          vsetivli      zero,
31,e8,m8,ta,ma
      80000530:  02098c27          vse8.v  v24,(s3)
      80000df2:  cdb3f057          vsetivli      zero,
7,e64,m8,ta,ma
      80000dfa:  0204f427          vse64.v v8,(s1)
      80000e20:  cdb3f057          vsetivli      zero,
7,e64,m8,ta,ma
      80000e28:  02047027          vse64.v v0,(s0)
```

You can check https://gcc.gnu.org/bugzilla/show_bug.cgi?id=112537 for more details.

In GCC 14.x, if you want to disable the RISC-V RVV automatic vectorization, you can use the options `-fno-tree-loop-vectorize` `-fno-tree-slp-vectorize`.

In GCC 13.x, you need to pass `--param=riscv-autovec-preference=none`.

Updating Nuclei Studio 2023.10 to the Latest Fixed Version¶

The Nuclei Studio 2023.10 version uploaded on 2023.11.06 had some issues. We have fixed them and replaced the online 2023.10 version at 2023.11.17 13:30.

Problem Description¶

The Nuclei Studio 2023.10 version released on November 6, 2023 contained some issues that affected the user experience:

- The busybox in build tools had a problem, causing compilation failures when make is used with pre- and post-build steps
- Corner cases in Nuclei Settings may fail in certain scenarios
- The way Nuclei Settings is opened affects how other files in the project are opened
- When using the V extension in QEMU, the RVV length was not passed in
- Fixed an issue where a project with the same name could be created when opening a brand-new workspace and creating a new project; reopening the workspace resolves this problem

We made the following changes to fix the issues above:

- Modified and released Nuclei Studio Plugins 2.1.0, uploaded to the plugin update site
- Modified and released Windows build-tools 1.2, replacing the online Windows Build Tools 2023.10
- Released a new Nuclei Studio 2023.10, replacing the online Nuclei Studio 2023.10

How to Upgrade Nuclei Studio 2023.10 to the Latest Version¶

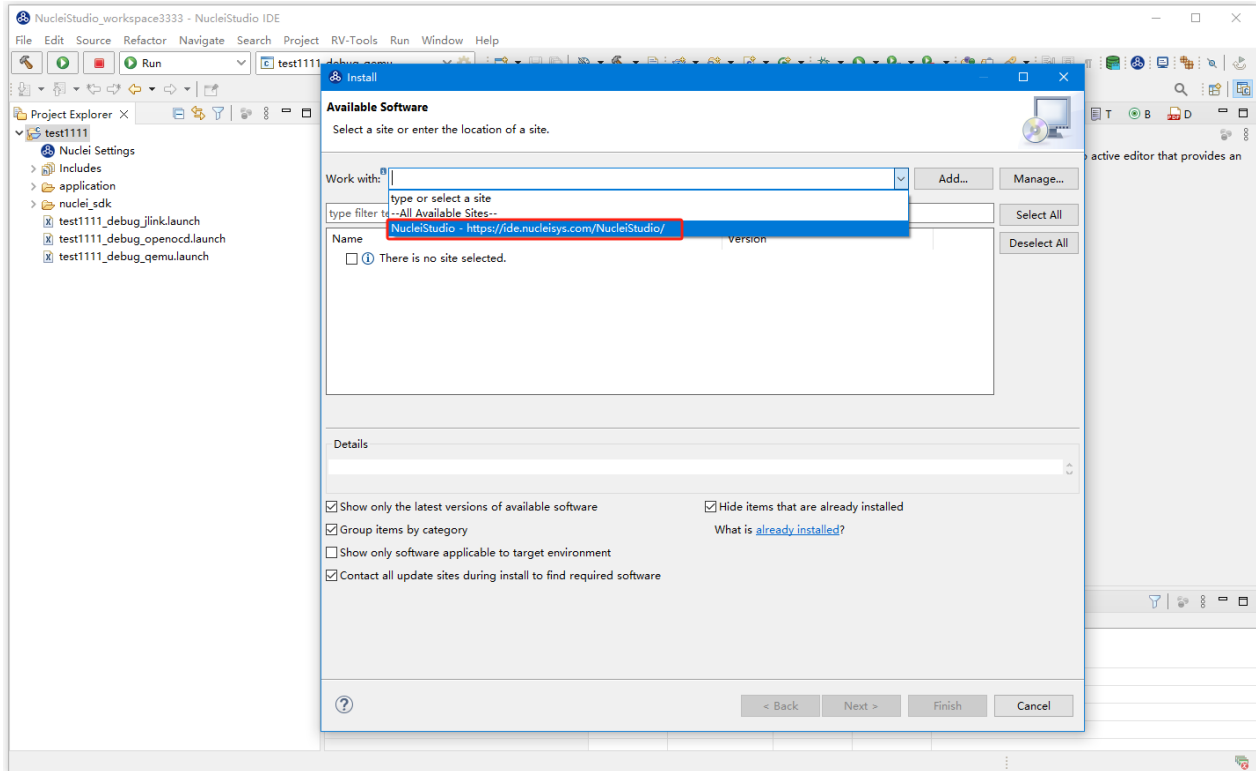
If your Nuclei Studio 2023.10 was downloaded before November 18, 2023, the issues described above may affect your user experience. You can either upgrade manually or download our latest release from the official website.

Upgrading Nuclei Studio 2023.10 Downloaded Before November 18, 2023¶

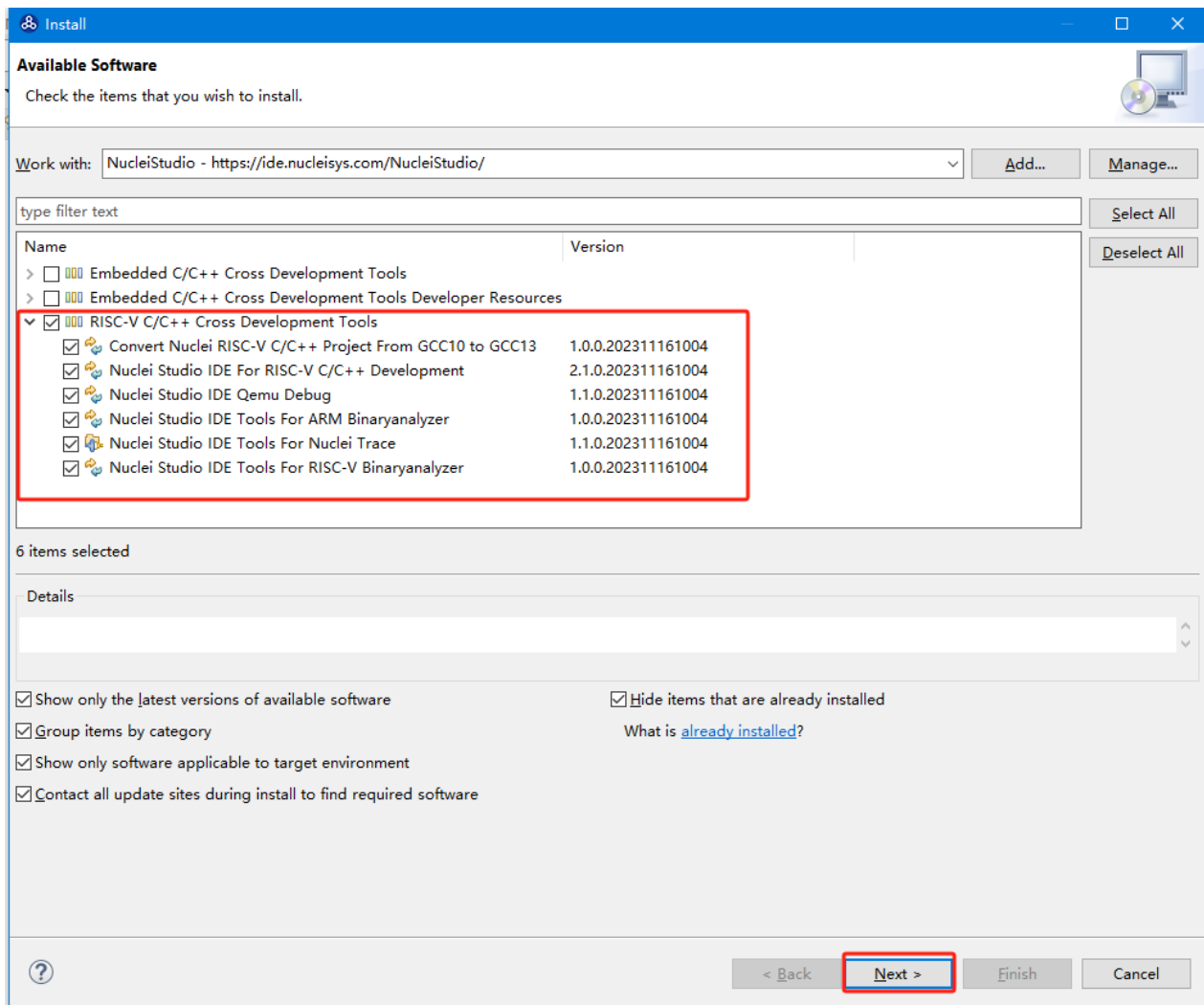
If you downloaded Nuclei Studio 2023.10 before November 18, 2023, you can update your Nuclei Studio 2023.10 to the latest version in the following ways.

1. Upgrade Nuclei Studio Plugins

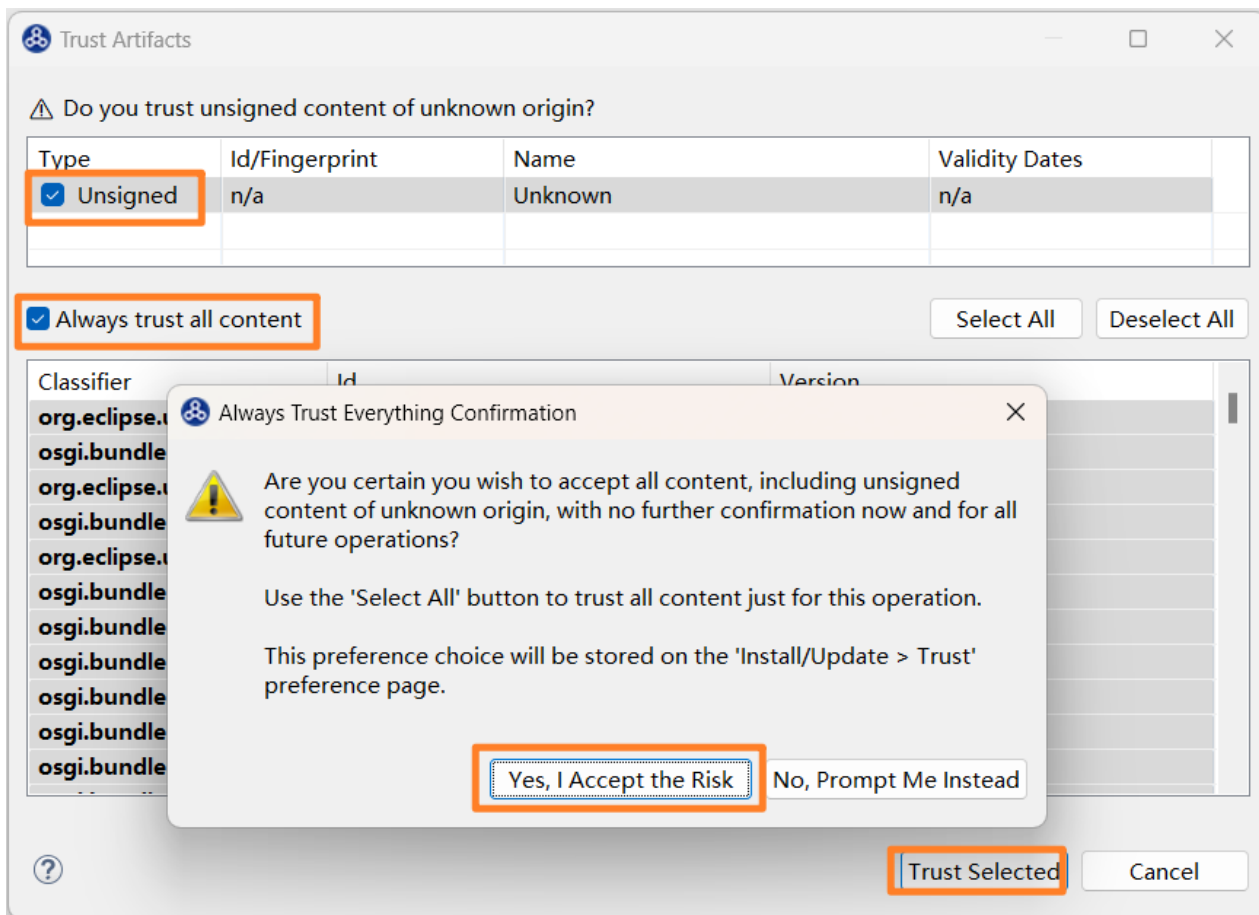
In the Nuclei Studio menu, go to Help->Install New Software, then in the Install dialog's Work with field, select NucleiStudio - <https://ide.nucleisys.com/NucleiStudio/>. All plugins pending update will be listed below.



In the plugin list that pops up, select the plugins to upgrade. We select RISC-V C/C++ Cross Development Tools, then click Next.



During the upgrade, when Nuclei Studio asks about Trust Artifacts, proceed as shown in the figure below: select Trust Selected. Once the upgrade is complete, Nuclei Studio will restart. At this point, the Nuclei Studio Plugins upgrade is finished.



2. Upgrade build-tools

The Linux version does not require this step; you only need to make sure the make tool is installed on your system.

Download `build-tools-1.2` and replace the contents of `NucleiStudio\toolchain\build-tools` in Nuclei Studio 2023.10.

For details on this part, refer to [Errors when using Pre-build Command/Post-build Command during project compilation](#).

- [Download build-tools-1.2](#)

With these two steps, the upgrade of Nuclei Studio 2023.10 is complete.

Downloading the Latest Version from the Official Website¶

If you do not want to perform a manual upgrade, you can download the latest Nuclei Studio 2023.10 directly from our website.

- [Download Windows version](#)
- [Download Linux version](#)

References¶

- [Nuclei Studio FAQs](#)
- [Continuously updated supplementary documentation for Nuclei Studio/Tools](#)
- [Nuclei Studio Issues](#)

Issues with OpenOCD When Operating on NOR Flash Larger Than 16 MB

Problem Description

To access the 0 ~ 16M address range, the SPI controller needs to send three bytes of address information; this is called 3-byte address mode. To access the 16M ~ 2G address range, the SPI controller needs to send four bytes of address information; this is called 4-byte address mode.

Both the normal SPI mode and XIP mode of the nuspi controller default to 3-byte address mode.

Solution

We have developed two sets of SPI drivers in OpenOCD, namely nuspi and custom, both of which support 3-byte mode and 4-byte mode. The nuspi driver can automatically switch between modes by checking the address being accessed.

There are many ways to read/verify data in flash with OpenOCD, which can be grouped into two categories: one reads flash data directly through XIP, and the other reads flash data by calling the driver through normal SPI.

Therefore, when reading flash data directly through XIP, there is a limitation that only the first 16M of the address range can be read. Commands affected by this include:

- `flash verify_image filename [offset] [type]`
- `dump_image filename address size`
- the `gdb x` command
- and other commands that read memory directly

Of course, OpenOCD also provides flash-reading commands that directly call the SPI driver registered in the `cfg` file, such as:

- `flash read_bank num filename [offset [length]]`
- `flash verify_bank num filename [offset]`

Upgrading the Project Toolchain to GCC 13 by Modifying the .cproject File

Problem Description

Nuclei Studio 2023.10 is a major version upgrade of the IDE, in which the bundled toolchain was upgraded from gcc10 to gcc13, and the toolchain prefix also changed.

See <https://github.com/Nuclei-Software/nuclei-studio/releases/tag/2023.10>

Although the 2023.10 IDE provides a [one-click upgrade tool by right-clicking a project](#) (see [Chapter 8 of the IDE manual](#)), it can only convert one project at a time, which is not friendly for projects that need to convert a large number of projects in batch. Therefore, we list here the approach you can refer to if you want to write a script to convert and upgrade projects.

The following changes only apply to upgrading gcc10 projects created with versions of Nuclei Studio earlier than 2023.10. If batch conversion is required, the script should first check whether the project is a riscv gcc10 project.

Modifying Toolchain-Related Configuration

The gcc used in versions of Nuclei Studio earlier than 2023.10 incorporated many customizations, whereas the gcc used in Nuclei Studio 2023.10 inherits the features and some naming conventions of the official version. In the project's .cproject file, the following values mainly need to be modified. If the value of `ilg.gnuclib.managedbuild.cross.riscv.option.toolchain.name` is RISC-V Nuclei GCC, the value of `ilg.gnuclib.managedbuild.cross.riscv.option.toolchain.id` is 3901352267, and the value of `ilg.gnuclib.managedbuild.cross.riscv.option.command.prefix` is `riscv-nuclei-elf-`, it means the project was created using GCC 10. To make the project support GCC 13, the following changes are required:

- Change the value of `toolchain.name` from RISC-V Nuclei GCC to RISC-V GCC/Newlib
- Change the value of `toolchain.id` from 3901352267 to 2262347901
- Change the value of `command.prefix` from `riscv-nuclei-elf-` to `riscv64-unknown-elf-`

Contents of the .cproject file before the change

```
<option
id="ilg.gnuclib.managedbuild.cross.riscv.option.toolchain.name.
129748485"
```

```
superClass="ilg.gnuclipse.managedbuild.cross.riscv.option.toolchain.name"
value="RISC-V Nuclei GCC" valueType="string"/>
<option
id="ilg.gnuclipse.managedbuild.cross.riscv.option.toolchain.id.
1143901706"
superClass="ilg.gnuclipse.managedbuild.cross.riscv.option.toolchain.id"
value="3901352267" valueType="string"/>
<option
id="ilg.gnuclipse.managedbuild.cross.riscv.option.command.prefix.
1270840820"
superClass="ilg.gnuclipse.managedbuild.cross.riscv.option.command.prefix"
value="riscv-nuclei-elf-" valueType="string"/>
<option
id="ilg.gnuclipse.managedbuild.cross.riscv.option.command.c.
718590769"
superClass="ilg.gnuclipse.managedbuild.cross.riscv.option.command.c"
value="gcc" valueType="string"/>
<option
id="ilg.gnuclipse.managedbuild.cross.riscv.option.command.cpp.
243660928"
superClass="ilg.gnuclipse.managedbuild.cross.riscv.option.command.cpp"
value="g++" valueType="string"/>
<option
id="ilg.gnuclipse.managedbuild.cross.riscv.option.command.ar.
416250093"
superClass="ilg.gnuclipse.managedbuild.cross.riscv.option.command.ar"
value="ar" valueType="string"/>
<option
id="ilg.gnuclipse.managedbuild.cross.riscv.option.command.objcopy.
741068581"
superClass="ilg.gnuclipse.managedbuild.cross.riscv.option.command.objcopy"
value="objcopy" valueType="string"/>
<option
id="ilg.gnuclipse.managedbuild.cross.riscv.option.command.objdump.
1474975752"
superClass="ilg.gnuclipse.managedbuild.cross.riscv.option.command.objdump"
value="objdump" valueType="string"/>
<option
id="ilg.gnuclipse.managedbuild.cross.riscv.option.command.size.
2085350427"
superClass="ilg.gnuclipse.managedbuild.cross.riscv.option.command.size"
value="size" valueType="string"/>
<option
id="ilg.gnuclipse.managedbuild.cross.riscv.option.command.make.
1355881376"
superClass="ilg.gnuclipse.managedbuild.cross.riscv.option.command.make"
value="make" valueType="string"/>
```

```
<option
id="ilg.gnuclueclipse.managedbuild.cross.riscv.option.command.rm.
1330665916"
superClass="ilg.gnuclueclipse.managedbuild.cross.riscv.option.command.rm"
value="rm" valueType="string"/>
```

Contents of the .cproject file after the change

```
<option
id="ilg.gnuclueclipse.managedbuild.cross.riscv.option.toolchain.name.
129748485"
superClass="ilg.gnuclueclipse.managedbuild.cross.riscv.option.toolchain.name"
value="RISC-V GCC/Newlib" valueType="string"/>
<option
id="ilg.gnuclueclipse.managedbuild.cross.riscv.option.toolchain.id.
1143901706"
superClass="ilg.gnuclueclipse.managedbuild.cross.riscv.option.toolchain.id"
value="2262347901" valueType="string"/>
<option
id="ilg.gnuclueclipse.managedbuild.cross.riscv.option.command.prefix.
1270840820"
superClass="ilg.gnuclueclipse.managedbuild.cross.riscv.option.command.prefix"
value="riscv64-unknown-elf-" valueType="string"/>
<option
id="ilg.gnuclueclipse.managedbuild.cross.riscv.option.command.c.
718590769"
superClass="ilg.gnuclueclipse.managedbuild.cross.riscv.option.command.c"
value="gcc" valueType="string"/>
<option
id="ilg.gnuclueclipse.managedbuild.cross.riscv.option.command.cpp.
243660928"
superClass="ilg.gnuclueclipse.managedbuild.cross.riscv.option.command.cpp"
value="g++" valueType="string"/>
<option
id="ilg.gnuclueclipse.managedbuild.cross.riscv.option.command.ar.
416250093"
superClass="ilg.gnuclueclipse.managedbuild.cross.riscv.option.command.ar"
value="ar" valueType="string"/>
<option
id="ilg.gnuclueclipse.managedbuild.cross.riscv.option.command.objcopy.
741068581"
superClass="ilg.gnuclueclipse.managedbuild.cross.riscv.option.command.objcopy"
value="objcopy" valueType="string"/>
<option
id="ilg.gnuclueclipse.managedbuild.cross.riscv.option.command.objdump.
1474975752"
```



```

superClass="ilg.gnuclipse.managedbuild.cross.riscv.option.command.objdump"
value="objdump" valueType="string"/>
<option
id="ilg.gnuclipse.managedbuild.cross.riscv.option.command.size.
2085350427"
superClass="ilg.gnuclipse.managedbuild.cross.riscv.option.command.size"
value="size" valueType="string"/>
<option
id="ilg.gnuclipse.managedbuild.cross.riscv.option.command.make.
1355881376"
superClass="ilg.gnuclipse.managedbuild.cross.riscv.option.command.make"
value="make" valueType="string"/>
<option
id="ilg.gnuclipse.managedbuild.cross.riscv.option.command.rm.
1330665916"
superClass="ilg.gnuclipse.managedbuild.cross.riscv.option.command.rm"
value="rm" valueType="string"/>

```

Modifying RISC-V Extension-Related Configuration ¶

In projects created with versions of Nuclei Studio earlier than 2023.10, RISC-V extensions were stored in four separate boolean values, whereas in projects created with Nuclei Studio 2023.10, they are stored in a single string value. Therefore, we need to find the four old values in the project's .cproject file, convert them into the new RISC-V extension string according to the rules,

	store	it	in
--	-------	----	----

ilg.gnuclipse.managedbuild.cross.riscv.option.target.isa.extensions, and clear or delete the four old separate boolean values.

```

# The four separate boolean values
ilg.gnuclipse.managedbuild.cross.riscv.option.target.isa.extensions.rvb
ilg.gnuclipse.managedbuild.cross.riscv.option.target.isa.extensions.rvk
ilg.gnuclipse.managedbuild.cross.riscv.option.target.isa.extensions.dsp
ilg.gnuclipse.managedbuild.cross.riscv.option.target.isa.extensions.vector

# The single string value
ilg.gnuclipse.managedbuild.cross.riscv.option.target.isa.extensions

```

1. First, determine whether the arch corresponding to the project is rv32/rv64 based on
ilg.gnuclipse.managedbuild.cross.riscv.option.target.isa.base

2. Second, determine whether f/d is included based on
`ilg.gnuclueclipse.managedbuild.cross.riscv.option.target.isa.fp`
3. Finally, convert to the correct RISC-V extension string according to the corresponding conversion rules

Conversion rules (note in particular that the value of p needs to be appended at the end of the RISC-V extension string):

- b -> `_zba_zbb_zbc_zbs`
- k -> `_zk_zks`
- v -> `rv32f/d : _zve32f, rv64f: _zve64f, rv64fd: v`
- p -> `rv64: _xxldsp, rv32: _xxldspn1x`

For example, suppose there is an N307FD project whose `arch=rv32imafdcbpv` (gcc10). We can tell that it is rv32, includes fd, and uses the bpv extensions. According to the conversion rules, the resulting RISC-V extension string is `_zba_zbb_zbc_zbs_zve32f_xxldspn1x`.

Contents of the .cproject file before the change

```
<option
id="ilg.gnuclueclipse.managedbuild.cross.riscv.option.target.isa.base.
489743203"
superClass="ilg.gnuclueclipse.managedbuild.cross.riscv.option.target.isa.base"
value="ilg.gnuclueclipse.managedbuild.cross.riscv.option.target.arch.rv32i"
valueType="enumerated"/>
<option
id="ilg.gnuclueclipse.managedbuild.cross.riscv.option.target.isa.fp.
1936924005"
superClass="ilg.gnuclueclipse.managedbuild.cross.riscv.option.target.isa.fp"
value="ilg.gnuclueclipse.managedbuild.cross.riscv.option.isa.fp.double"
valueType="enumerated"/>
<option
id="ilg.gnuclueclipse.managedbuild.cross.riscv.option.target.isa.extension.rvb.
168405526"
superClass="ilg.gnuclueclipse.managedbuild.cross.riscv.option.target.isa.extens
value="true" valueType="boolean"/>
<option
id="ilg.gnuclueclipse.managedbuild.cross.riscv.option.target.isa.extension.dsp.
565204765"
superClass="ilg.gnuclueclipse.managedbuild.cross.riscv.option.target.isa.extens
value="true" valueType="boolean"/>
<option
id="ilg.gnuclueclipse.managedbuild.cross.riscv.option.target.isa.extension.vect
1142078455"
superClass="ilg.gnuclueclipse.managedbuild.cross.riscv.option.target.isa.extens
value="true" valueType="boolean"/>
```

Contents of the .cproject file after the change

```
<option
id="ilg.gnuarm.eclipse.managedbuild.cross.riscv.option.target.isa.extensions.
1832321358"
superClass="ilg.gnuarm.eclipse.managedbuild.cross.riscv.option.target.isa.extensions
value="_zba_zbb_zbc_zbs_zve32f_xxldspnx" valueType="string"/>
```

Modifying libncrt C Library-Related Configuration

In projects created with versions of Nuclei Studio earlier than 2023.10, when the libncrt C library is used, the project includes a `--specs=libncrt_xxx.specs` option, or the linked libraries include `-lncrt_xxx`. In projects created with Nuclei Studio 2023.10, if the libncrt C library is used, the `--specs=libncrt_xxx.specs` approach needs to be changed to `-lncrt_xxx`, and additionally `-lncrt_small` `-lheapops_basic` `-lfileops_uart` need to be added at link time, while the general target compile options need to include `-isystem=/include/libncrt`

Examples: `* -lncrt_small -> -lncrt_small -lheapops_basic -lfileops_uart * --specs=libncrt_small.specs -> -lncrt_small -lheapops_basic -lfileops_uart`

1. Check whether `--specs=libncrt_xxx.specs` exists in the .cproject file. If it exists, it means this is a project that uses libncrt, and you can proceed with the subsequent steps
2. If `--specs=libncrt_xxx.specs` exists, delete it first
3. If `-lm` exists, delete it first
4. Check whether `m` exists in
`ilg.gnuarm.eclipse.managedbuild.cross.riscv.option.c.linker.libs` or
`ilg.gnuarm.eclipse.managedbuild.cross.riscv.option.cpp.linker.libs`;
 if it exists, delete it first
5. Check whether `ncrt_xxx` exists in
`ilg.gnuarm.eclipse.managedbuild.cross.riscv.option.c.linker.libs` or
`ilg.gnuarm.eclipse.managedbuild.cross.riscv.option.cpp.linker.libs`
6. Based on the above results, add the corresponding values to
`ilg.gnuarm.eclipse.managedbuild.cross.riscv.option.c.linker.libs` or
`ilg.gnuarm.eclipse.managedbuild.cross.riscv.option.cpp.linker.libs`
 ◦ If `--specs=libncrt_xxx.specs` exists, add `ncrt_xxx`; or if `ncrt_xxx` exists,
 you also need to add `heapops_basic` and `fileops_uart`
7. Add `-isystem=/include/libncrt` to
`ilg.gnuarm.eclipse.managedbuild.cross.riscv.option.target.other`

```
# --specs=libncrt_xxx1.specs may exist in the following string
values
ilg.gnuarm.eclipse.managedbuild.cross.riscv.option.target.other
```

```

ilg.gnuclipse.managedbuild.cross.riscv.option.optimization.other
r
ilg.gnuclipse.managedbuild.cross.riscv.option.warnings.other
ilg.gnuclipse.managedbuild.cross.riscv.option.debugging.other
ilg.gnuclipse.managedbuild.cross.riscv.option.assembler.otherwa
rnings
ilg.gnuclipse.managedbuild.cross.riscv.option.assembler.other
ilg.gnuclipse.managedbuild.cross.riscv.option.c.compiler.othero
ptimizations
ilg.gnuclipse.managedbuild.cross.riscv.option.c.compiler.otherw
arnings
ilg.gnuclipse.managedbuild.cross.riscv.option.c.compiler.other
ilg.gnuclipse.managedbuild.cross.riscv.option.c.linker.other
ilg.gnuclipse.managedbuild.cross.riscv.option.cpp.compiler.othe
roptimizations
ilg.gnuclipse.managedbuild.cross.riscv.option.cpp.compiler.othe
rwarnings
ilg.gnuclipse.managedbuild.cross.riscv.option.cpp.compiler.othe
r
ilg.gnuclipse.managedbuild.cross.riscv.option.cpp.linker.other
ilg.gnuclipse.managedbuild.cross.riscv.option.createflash.other
ilg.gnuclipse.managedbuild.cross.riscv.option.createlisting.oth
er
ilg.gnuclipse.managedbuild.cross.riscv.option.printsize.other

```

For example, the project uses `--specs=libncrt_balanced.specs`

Contents of the .cproject file before the change

```

<option
id="ilg.gnuclipse.managedbuild.cross.riscv.option.target.other.
1735566114"
superClass="ilg.gnuclipse.managedbuild.cross.riscv.option.target.other"
value=" " valueType="string"/>
<option
id="ilg.gnuclipse.managedbuild.cross.riscv.option.optimization.other.
443378574"
superClass="ilg.gnuclipse.managedbuild.cross.riscv.option.optimization.other"
value="--specs=libncrt_balanced.specs" valueType="string"/>

```

Contents of the .cproject file after the change

```

<option
id="ilg.gnuclipse.managedbuild.cross.riscv.option.target.other.
1735566114"

```

```

superClass="ilg.gnuclipse.managedbuild.cross.riscv.option.target.other"
value="-isystem=/include/libncrt " valueType="string"/>
<option IS_BUILTIN_EMPTY="false" IS_VALUE_EMPTY="false"
id="ilg.gnuclipse.managedbuild.cross.riscv.option.cpp.linker.libs.
146128417"
superClass="ilg.gnuclipse.managedbuild.cross.riscv.option.cpp.linker.libs"
valueType="libs">
  <listOptionValue builtIn="false" value="ncrt_balanced"/>
  <listOptionValue builtIn="false" value="fileops_uart"/>
  <listOptionValue builtIn="false" value="heapops_basic"/>
</option>

```

Adding Configuration to Eliminate Link Warnings¶

Many warning messages are generated when using GCC 13. You can add the `-Wl,--no-warn-rwx-segments` parameter to the link options to disable these warning messages.

For details, see https://sourceware.org/binutils/docs/ld/Options.html#index-_002d_002dwarn_002drwx_002dsegments

Contents of the .cproject file before the change

```

<option
id="ilg.gnuclipse.managedbuild.cross.riscv.option.cpp.linker.other.
1000044097"
superClass="ilg.gnuclipse.managedbuild.cross.riscv.option.cpp.linker.other"
value="" valueType="string"/>

```

Contents of the .cproject file after the change

```

<option
id="ilg.gnuclipse.managedbuild.cross.riscv.option.cpp.linker.other.
1000044097"
superClass="ilg.gnuclipse.managedbuild.cross.riscv.option.cpp.linker.other"
value="-Wl,--no-warn-rwx-segments" valueType="string"/>

```

After completing the above changes, reload the project, and the project can be compiled, debugged, and run normally under Nuclei Studio 2023.10.

Note:

In this document, in all referenced examples of the .cproject file, in entries like `id="ilg.gnuclipse.managedbuild.cross.riscv.option.target.other.1735566114"`, 1735566114 is a hash value generated by Nuclei Studio. It

varies across different times and different projects, and it does not affect the configuration. If it can be kept the same as the original value, try to keep it the same.

Compiling Projects from the Command Line in Nuclei Studio

Problem Description

Many customers ask how to use the IDE's headless mode in Nuclei Studio to build and compile projects.

Solution

When running any command line that starts with `NucleiStudio.exe`, a pop-up window appears showing the execution log. To suppress this pop-up, change the command to `eclipse.exe`

The following documentation was tested with the 2024.06 version of the IDE and is provided as supplementary information.

Because NucleiStudio 2024.06 runs on a Java 21 environment, and many users do not have a Java 21 environment installed locally, running the command may fail with an error because the corresponding JRE cannot be found. To resolve this issue, you can either install a Java 21 environment on your local machine (users can search for relevant installation tutorials themselves), or use the `-vm` parameter on the command line to specify the path of the JRE bundled with NucleiStudio 2024.06.

```
NucleiStudio.exe -vm "<user_nucleistudio_path>/plugins/
org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.3.v202
40426-1530/jre/bin" --launcher.suppressErrors -nosplash -
application org.eclipse.cdt.managedbuilder.core.headlessbuild -
data C:\NucleiStudio_workspace -cleanBuild test/Debug -Debug
```

The following is a set of commands for batch importing and batch building projects

Create a workspace and batch import projects

```
NucleiStudio.exe -vm "<user_nucleistudio_path>/plugins/
org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.3.v202
40426-1530/jre/bin" --launcher.suppressErrors -noSplash -
application org.eclipse.cdt.managedbuilder.core.headlessbuild -
data $CI_PROJECT_DIR -importAll $CI_PROJECT_DIR
```

Build the imported projects

```
NucleiStudio.exe -vm "<user_nucleistudio_path>/plugins/
org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.3.v202
40426-1530/jre/bin" --launcher.suppressErrors -noSplash -
application org.eclipse.cdt.managedbuilder.core.headlessbuild -
data $CI_PROJECT_DIR -build ${TARGET_PHASE}_Project/Debug
```

The following documentation was tested with the 2023.10 version of the IDE; other versions may require some adjustments to work properly.

Nuclei Studio is a graphical (GUI) code editing tool, but in certain scenarios users need to build projects quickly from the command line. In Nuclei Studio, this can be done with a single command. After downloading Nuclei Studio, make sure the project test to be built has already been created in the Nuclei Studio workspace, and that Nuclei Studio is not running, then execute the following command to build the project.

Note: Make sure the NucleiStudio PATH has been added to the system, so that NucleiStudio.exe/NucleiStudio can be executed.

The following example uses Windows

```
NucleiStudio.exe --launcher.suppressErrors -nosplash -application
org.eclipse.cdt.managedbuilder.core.headlessbuild -data C:
\NucleiStudio_workspace -cleanBuild test/Debug -Debug
```

--launcher.suppressErrors is used to suppress the Eclipse error pop-up window when a build error occurs.

If you need to use the 2022.12 version of the IDE, you must first add the paths of gcc/bin and build-tools/bin under the toolchain directory to the system PATH, and then replace NucleiStudio.exe with eclipsesec.exe

For the 2022.12 version, the example commands are as follows:

```
# Change this to your own IDE path
set NSIDE=D:\NucleiStudio_IDE_202212-win64\NucleiStudio
# The system PATH must be set correctly
set PATH=%NSIDE%\toolchain\gcc\bin;%NSIDE%\toolchain\build-
tools\bin;%PATH%
# Note that NucleiStudio.exe is replaced with eclipsesec.exe
%NSIDE%\eclipsesec.exe --launcher.suppressErrors -nosplash -
application org.eclipse.cdt.managedbuilder.core.headlessbuild -
data C:\NucleiStudio_workspace -cleanBuild test/Debug
```


The example command for the 2023.10 version opens an additional command-line window for output.

```

E:\NucleiStudio_IDE_202310-win64\NucleiStudio>NucleiStudio.exe --launcher.suppressErrors -n
osplash --application org.eclipse.cdt.managedbuilder.core.headlessbuild --data C:\Users\11653\NucleiStudio_workspace2023
-cleanBuild test2222/Debug -Debug

NucleiStudio.exe --launcher.suppressErrors -nosplash --application org.eclipse.cdt.managedbuilder.core.headlessbu...
sdk/SoC/hbirdv2/Common/Source/Stubs/lseek.o ./hbird_sdk/SoC/hbirdv2/Common/Source/Stubs/open.o ./hbird_sdk/SoC/hbirdv2/C
ommon/Source/Stubs/read.o ./hbird_sdk/SoC/hbirdv2/Common/Source/Stubs/sbrk.o ./hbird_sdk/SoC/hbirdv2/Common/Source/Stubs
/stat.o ./hbird_sdk/SoC/hbirdv2/Common/Source/Stubs/times.o ./hbird_sdk/SoC/hbirdv2/Common/Source/Stubs/unlink.o ./hbird
_sdk/SoC/hbirdv2/Common/Source/Stubs/wait.o ./hbird_sdk/SoC/hbirdv2/Common/Source/Stubs/write.o ./hbird_sdk/SoC/hbirdv2
/Common/Source/GCC/intexc_hbirdv2.o ./hbird_sdk/SoC/hbirdv2/Common/Source/GCC/startup_hbirdv2.o ./hbird_sdk/SoC/hbirdv2
/Common/Source/Drivers/hbirdv2_gpio.o ./hbird_sdk/SoC/hbirdv2/Common/Source/Drivers/hbirdv2_i2c.o ./hbird_sdk/SoC/hbirdv
2/Common/Source/Drivers/hbirdv2_pwm.o ./hbird_sdk/SoC/hbirdv2/Common/Source/Drivers/hbirdv2_spi.o ./hbird_sdk/SoC/hbirdv
2/Common/Source/Drivers/hbirdv2_uart.o ./hbird_sdk/SoC/hbirdv2/Common/Source/Drivers/htif.o ./hbird_sdk/SoC/hbirdv2/Com
mon/Source/hbirdv2_common.o ./hbird_sdk/SoC/hbirdv2/Common/Source/system_hbirdv2.o ./application/main.o test2222.hex te
st2222.lst test2222.siz ./hbird_sdk/SoC/hbirdv2/Common/Source/GCC/intexc_hbirdv2.d ./hbird_sdk/SoC/hbirdv2/Common/Source
/GCC/startup_hbirdv2.d ./hbird_sdk/SoC/hbirdv2/Common/Source/Stubs/clock_getres.d ./hbird_sdk/SoC/hbirdv2/Common/Source
/Stubs/clock_gettime.d ./hbird_sdk/SoC/hbirdv2/Common/Source/Stubs/clock_settime.d ./hbird_sdk/SoC/hbirdv2/Common/Source
/Stubs/close.d ./hbird_sdk/SoC/hbirdv2/Common/Source/Stubs/execve.d ./hbird_sdk/SoC/hbirdv2/Common/Source/Stubs/exit.d ./
hbird_sdk/SoC/hbirdv2/Common/Source/Stubs/fork.d ./hbird_sdk/SoC/hbirdv2/Common/Source/Stubs/fstat.d ./hbird_sdk/SoC/hbi
rdv2/Common/Source/Stubs/getpid.d ./hbird_sdk/SoC/hbirdv2/Common/Source/Stubs/gettimeofday.d ./hbird_sdk/SoC/hbirdv2/Com
mon/Source/Stubs/isatty.d ./hbird_sdk/SoC/hbirdv2/Common/Source/Stubs/kill.d ./hbird_sdk/SoC/hbirdv2/Common/Source/Stubs
/link.d ./hbird_sdk/SoC/hbirdv2/Common/Source/Stubs/lseek.d ./hbird_sdk/SoC/hbirdv2/Common/Source/Stubs/open.d ./hbird_s
dk/SoC/hbirdv2/Common/Source/Stubs/read.d ./hbird_sdk/SoC/hbirdv2/Common/Source/Stubs/sbrk.d ./hbird_sdk/SoC/hbirdv2/Com
mon/Source/Stubs/stat.d ./hbird_sdk/SoC/hbirdv2/Common/Source/Stubs/times.d ./hbird_sdk/SoC/hbirdv2/Common/Source/Stubs
/unlink.d ./hbird_sdk/SoC/hbirdv2/Common/Source/Stubs/wait.d ./hbird_sdk/SoC/hbirdv2/Common/Source/Stubs/write.d ./hbird
_sdk/SoC/hbirdv2/Common/Source/Drivers/hbirdv2_gpio.d ./hbird_sdk/SoC/hbirdv2/Common/Source/Drivers/hbirdv2_i2c.d ./hbir
d_sdk/SoC/hbirdv2/Common/Source/Drivers/hbirdv2_pwm.d ./hbird_sdk/SoC/hbirdv2/Common/Source/Drivers/hbirdv2_spi.d ./hbir
d_sdk/SoC/hbirdv2/Common/Source/Drivers/hbirdv2_uart.d ./hbird_sdk/SoC/hbirdv2/Common/Source/Drivers/htif.d ./hbird_sdk
/SoC/hbirdv2/Common/Source/hbirdv2_common.d ./hbird_sdk/SoC/hbirdv2/Common/Source/system_hbirdv2.d ./application/main.d
test2222.elf
15:34:36 Build Finished. 0 errors, 0 warnings. (took 326ms)

```

- **NucleiStudio.exe**: This is the Nuclei Studio launcher application, located in the Nuclei Studio installation directory.
- **--launcher.suppressErrors**: This parameter is used to suppress error messages when Nuclei Studio starts.
- **-nosplash**: This parameter disables the splash screen at startup. This means no brief loading screen is displayed when Eclipse starts.
- **-application**: This parameter specifies the application to run. Here, `org.eclipse.cdt.managedbuilder.core.headlessbuild` refers to the headless build application. This application performs build operations without a graphical user interface (GUI).
- **-data**: This parameter specifies the workspace path. It tells Nuclei Studio where to store data, such as workspaces, projects, and files.
- **-build**: This parameter specifies the project to build; `test/Debug` means building the Debug configuration of the test project; projects created by Nuclei Studio generally have two configurations, Debug and Release. If no configuration is specified, both Debug and Release are built by default, and you can see the Debug and Release directories under the project directory after the build.

```

├─.settings
├─application
├─Debug
│   └─application
│       └─nuclei_sdk
└─nuclei_sdk

```

```
└─Release
    └─application
        └─nuclei_sdk
```

- -cleanBuild: This parameter is similar to -build, except that it cleans the workspace before building. Using -cleanBuild is recommended.
- -Debug: This parameter specifies that the build process runs in Debug mode, outputting detailed build logs during compilation. Without this parameter, the command runs silently with no output.

The following is the output of the example command above, for reference

```
17:00:17 **** Clean-only build of configuration Debug for project
test ****
make -j8 clean
rm -rf ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
chown.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
clock_getres.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
clock_gettime.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/clock_settime.o ./nuclei_sdk/SoC/evalsoc/Common/Source/
Stubs/newlib/close.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/environ.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/errno.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
execve.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
exit.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
fork.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
fstat.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
getpid.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
gettimeofday.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
isatty.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
kill.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
link.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
lseek.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
open.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
read.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
readlink.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
sbrk.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
stat.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
symlink.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
times.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
unlink.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
wait.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
write.o ./nuclei_sdk/SoC/evalsoc/Common/Source/GCC/
intexc_evalsoc.o ./nuclei_sdk/SoC/evalsoc/Common/Source/GCC/
intexc_evalsoc_s.o ./nuclei_sdk/SoC/evalsoc/Common/Source/GCC/
startup_evalsoc.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Drivers/
```

```

evalsoc_uart.o ./nuclei_sdk/SoC/evalsoc/Common/Source/
evalsoc_common.o ./nuclei_sdk/SoC/evalsoc/Common/Source/
system_evalsoc.o ./application/main.o test.hex test.lst
test.siz ./nuclei_sdk/SoC/evalsoc/Common/Source/GCC/
intexc_evalsoc.d ./nuclei_sdk/SoC/evalsoc/Common/Source/GCC/
intexc_evalsoc_s.d ./nuclei_sdk/SoC/evalsoc/Common/Source/GCC/
startup_evalsoc.d ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/chown.d ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
clock_getres.d ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
clock_gettime.d ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/clock_settime.d ./nuclei_sdk/SoC/evalsoc/Common/Source/
Stubs/newlib/close.d ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/envIRON.d ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/errno.d ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
execve.d ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
exit.d ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
fork.d ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
fstat.d ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
getpid.d ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
gettimeofday.d ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
isatty.d ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
kill.d ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
link.d ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
lseek.d ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
open.d ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
read.d ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
readlink.d ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
sbrk.d ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
stat.d ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
symlink.d ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
times.d ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
unlink.d ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
wait.d ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
write.d ./nuclei_sdk/SoC/evalsoc/Common/Source/Drivers/
evalsoc_uart.d ./nuclei_sdk/SoC/evalsoc/Common/Source/
evalsoc_common.d ./nuclei_sdk/SoC/evalsoc/Common/Source/
system_evalsoc.d ./application/main.d test.elf

```

17:00:17 Build Finished. 0 errors, 0 warnings. (took 371ms)

```

17:00:18 **** Build of configuration Debug for project test ****
make -j8 all
Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/chown.c
Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/clock_getres.c

```

Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/clock_gettime.c

Invoking: GNU RISC-V Cross C Compiler

Invoking: GNU RISC-V Cross C Compiler

```
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mcmmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING="ILM" -
I"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -
I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Include" -I"C:\NucleiStudio_workspace\test\application" -
std=gnu11 -MMD -MP -MF"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/chown.d" -MT"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/chown.o" -c -o "nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/chown.o" "../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/chown.c"
```

Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/clock_settime.c

```
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mcmmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING="ILM" -
I"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -
I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Include" -I"C:\NucleiStudio_workspace\test\application" -std=gnu11 -MMD -MP
-MF"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
clock_getres.d" -MT"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/clock_getres.o" -c -o "nuclei_sdk/SoC/evalsoc/Common/Source/
Stubs/newlib/clock_getres.o" "../nuclei_sdk/SoC/evalsoc/Common/
Source/Stubs/newlib/clock_getres.c"
```

Invoking: GNU RISC-V Cross C Compiler

```
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mcmmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING="ILM" -
```

```

I"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -
I"C:
\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Incl
-I"C:\NucleiStudio_workspace\test\application" -std=gnu11 -MMD -MP
-MF"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
clock_gettime.d" -MT"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/clock_gettime.o" -c -o "nuclei_sdk/SoC/evalsoc/Common/
Source/Stubs/newlib/clock_gettime.o" "../nuclei_sdk/SoC/evalsoc/
Common/Source/Stubs/newlib/clock_gettime.c"
Invoking: GNU RISC-V Cross C Compiler
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING="ILM" -
I"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -
I"C:
\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Incl
-I"C:\NucleiStudio_workspace\test\application" -std=gnu11 -MMD -MP
-MF"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
clock_gettime.d" -MT"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/clock_gettime.o" -c -o "nuclei_sdk/SoC/evalsoc/Common/
Source/Stubs/newlib/clock_gettime.o" "../nuclei_sdk/SoC/evalsoc/
Common/Source/Stubs/newlib/clock_gettime.c"
Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/close.c
Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/envron.c
Invoking: GNU RISC-V Cross C Compiler
Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/errno.c
Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/execve.c
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING="ILM" -
I"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -
I"C:
\nuclei_sdk\SoC\evalsoc\Common\Include"

```



```

-I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Incl
-I"C:\NucleiStudio_workspace\test\application" -std=gnu11 -MMD -MP
-MF"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/close.d" -
MT"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/close.o" -c -o
"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/close.o" "../
nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/close.c"
Invoking: GNU RISC-V Cross C Compiler
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING="ILM" -
I"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -
I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Incl
-I"C:\NucleiStudio_workspace\test\application" -std=gnu11 -MMD -MP
-MF"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/environ.d" -
MT"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/environ.o" -c
-o "nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/environ.o"
"../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/environ.c"
Invoking: GNU RISC-V Cross C Compiler
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING="ILM" -
I"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -
I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Incl
-I"C:\NucleiStudio_workspace\test\application" -std=gnu11 -MMD -MP
-MF"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/errno.d" -
MT"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/errno.o" -c -o
"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/errno.o" "../
nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/errno.c"
Invoking: GNU RISC-V Cross C Compiler
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -

```

```

DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING=\"ILM\" -
I"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -
I"C:
\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Include"
-I"C:\NucleiStudio_workspace\test\application" -std=gnu11 -MMD -MP
-MF"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/execve.d" -
MT"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/execve.o" -c -
o "nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/execve.o" "../
nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/execve.c"
Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/envIRON.c
Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/chown.c
Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/clock_getres.c
Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/clock_gettime.c
Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/close.c
Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/clock_gettime.c
Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/exit.c
Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/fork.c
Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/fstat.c
Invoking: GNU RISC-V Cross C Compiler
Invoking: GNU RISC-V Cross C Compiler
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mcmode=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING=\"ILM\" -
I"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -
I"C:
\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Include"
-I"C:\NucleiStudio_workspace\test\application" -std=gnu11 -MMD -MP
-MF"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/exit.d" -
MT"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/exit.o" -c -o
"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/exit.o" "../
nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/exit.c"

```

Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/getpid.c

Invoking: GNU RISC-V Cross C Compiler

```
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mcmmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING="ILM" -
I"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -
I"C:
```

```
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
```

```
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Incl
```

```
-I"C:\NucleiStudio_workspace\test\application" -std=gnu11 -MMD -MP
-MF"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/fork.d" -
MT"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/fork.o" -c -o
"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/fork.o" "../
nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/fork.c"
```

Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/execve.c

```
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mcmmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING="ILM" -
I"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -
I"C:
```

```
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
```

```
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Incl
```

```
-I"C:\NucleiStudio_workspace\test\application" -std=gnu11 -MMD -MP
-MF"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/fstat.d" -
MT"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/fstat.o" -c -o
"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/fstat.o" "../
nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/fstat.c"
```

Invoking: GNU RISC-V Cross C Compiler

```
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mcmmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING="ILM" -
I"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -
I"C:
```



```

\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Incl
-I"C:\NucleiStudio_workspace\test\application" -std=gnu11 -MMD -MP
-MF"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/getpid.d" -
MT"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/getpid.o" -c -
o "nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/getpid.o" "../
nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/getpid.c"
Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/gettimeofday.c
Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/isatty.c
Invoking: GNU RISC-V Cross C Compiler
Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/kill.c
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING="ILM" -
I"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -
I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Incl
-I"C:\NucleiStudio_workspace\test\application" -std=gnu11 -MMD -MP
-MF"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
gettimeofday.d" -MT"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/gettimeofday.o" -c -o "nuclei_sdk/SoC/evalsoc/Common/Source/
Stubs/newlib/gettimeofday.o" "../nuclei_sdk/SoC/evalsoc/Common/
Source/Stubs/newlib/gettimeofday.c"
Invoking: GNU RISC-V Cross C Compiler
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING="ILM" -
I"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -
I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Incl
-I"C:\NucleiStudio_workspace\test\application" -std=gnu11 -MMD -MP
-MF"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/isatty.d" -
MT"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/isatty.o" -c -

```

```

o "nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/isatty.o" "../
nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/isatty.c"
Invoking: GNU RISC-V Cross C Compiler
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mcmmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING="ILM" -
I"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -
I"C:
\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Incl
-I"C:\NucleiStudio_workspace\test\application" -std=gnu11 -MMD -MP
-MF"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/kill.d" -
MT"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/kill.o" -c -o
"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/kill.o" "../
nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/kill.c"
Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/exit.c

Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/errno.c
Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/fork.c
Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/link.c

Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/lseek.c
Invoking: GNU RISC-V Cross C Compiler
Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/gettimeofday.c
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mcmmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING="ILM" -
I"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -
I"C:
\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Incl
-I"C:\NucleiStudio_workspace\test\application" -std=gnu11 -MMD -MP

```

```
-MF"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/link.d" -
MT"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/link.o" -c -o
"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/link.o" "../
nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/link.c"
```

Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/open.c

Invoking: GNU RISC-V Cross C Compiler

```
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mcmmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
```

```
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING="ILM" -
I"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -
I"C:
```

```
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
```

```
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Inc
```

```
-I"C:\NucleiStudio_workspace\test\application" -std=gnu11 -MMD -MP
-MF"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/lseek.d" -
MT"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/lseek.o" -c -o
"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/lseek.o" "../
nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/lseek.c"
```

Invoking: GNU RISC-V Cross C Compiler

```
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mcmmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
```

```
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING="ILM" -
I"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -
I"C:
```

```
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
```

```
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Inc
```

```
-I"C:\NucleiStudio_workspace\test\application" -std=gnu11 -MMD -MP
-MF"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/open.d" -
MT"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/open.o" -c -o
"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/open.o" "../
nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/open.c"
```

Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/read.c

Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/kill.c

Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/getpid.c

Invoking: GNU RISC-V Cross C Compiler

```
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mcmmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
```

```
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING=\"ILM\" -
I\"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include\" -
I\"C:
```

```
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Common\Include\"
-I\"C:
```

```
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Incl
```

```
-I\"C:\NucleiStudio_workspace\test\application\" -std=gnu11 -MMD -MP
-MF\"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/read.d\" -
MT\"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/read.o\" -c -o
\"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/read.o\" \"../
nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/read.c\"
```

Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/readlink.c

Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/sbrk.c

Invoking: GNU RISC-V Cross C Compiler

```
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mcmmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
```

```
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING=\"ILM\" -
I\"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include\" -
I\"C:
```

```
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Common\Include\"
-I\"C:
```

```
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Incl
```

```
-I\"C:\NucleiStudio_workspace\test\application\" -std=gnu11 -MMD -MP
-MF\"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/readlink.d\" -
MT\"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/readlink.o\" -c
-o \"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/readlink.o\"
\"../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/readlink.c\"
```

Invoking: GNU RISC-V Cross C Compiler

```
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mcmmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
```

```
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING=\"ILM\" -
```

```

I"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -
I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Incl
-I"C:\NucleiStudio_workspace\test\application" -std=gnu11 -MMD -MP
-MF"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/sbrk.d" -
MT"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/sbrk.o" -c -o
"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/sbrk.o" "../
nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/sbrk.c"
Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/isatty.c

Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/fstat.c
Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/link.c
Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/lseek.c
Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/read.c

Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/stat.c

Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/symlink.c
Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/times.c
Invoking: GNU RISC-V Cross C Compiler
Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/unlink.c
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mcmmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING="ILM" -
I"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -
I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Incl
-I"C:\NucleiStudio_workspace\test\application" -std=gnu11 -MMD -MP
-MF"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/stat.d" -

```

```

MT"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/stat.o" -c -o
"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/stat.o" "../
nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/stat.c"
Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/readlink.c
Invoking: GNU RISC-V Cross C Compiler
Invoking: GNU RISC-V Cross C Compiler
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mcmmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING="ILM" -
I"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -
I"C:
\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Include"
-I"C:\NucleiStudio_workspace\test\application" -std=gnu11 -MMD -MP
-MF"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/symlink.d" -
MT"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/symlink.o" -c
-o "nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/symlink.o"
"../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/symlink.c"
Invoking: GNU RISC-V Cross C Compiler
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mcmmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING="ILM" -
I"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -
I"C:
\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Include"
-I"C:\NucleiStudio_workspace\test\application" -std=gnu11 -MMD -MP
-MF"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/times.d" -
MT"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/times.o" -c -o
"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/times.o" "../
nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/times.c"
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mcmmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING="ILM" -
I"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -

```



```

I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Incl
-I"C:\NucleiStudio_workspace\test\application" -std=gnu11 -MMD -MP
-MF"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/unlink.d" -
MT"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/unlink.o" -c -
o "nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/unlink.o" "../
nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/unlink.c"

Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/wait.c
Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/write.c
Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/sbrk.c
Invoking: GNU RISC-V Cross C Compiler
Invoking: GNU RISC-V Cross C Compiler

riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mcmmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING="ILM" -
I"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -
I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Incl
-I"C:\NucleiStudio_workspace\test\application" -std=gnu11 -MMD -MP
-MF"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/wait.d" -
MT"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/wait.o" -c -o
"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/wait.o" "../
nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/wait.c"
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mcmmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING="ILM" -
I"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -
I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Incl
-I"C:\NucleiStudio_workspace\test\application" -std=gnu11 -MMD -MP

```

```

-MF"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/write.d" -
MT"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/write.o" -c -o
"nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/write.o" "../
nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/write.c"
Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/GCC/
intexc_evalsoc.S
Invoking: GNU RISC-V Cross Assembler
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mcmmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -x assembler-with-cpp -D__IDE_RV_CORE=n307fd -
DBOOT_HARTID=0 -DRUNMODE_IC_EN=0 -DRUNMODE_DC_EN=0 -
DRUNMODE_CCM_EN=0 -DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -
DDOWNLOAD_MODE_STRING="ILM" -I"C:
\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Inc
-I"C:\NucleiStudio_workspace\test\application" -MMD -MP -
MF"nuclei_sdk/SoC/evalsoc/Common/Source/GCC/intexc_evalsoc.d" -
MT"nuclei_sdk/SoC/evalsoc/Common/Source/GCC/intexc_evalsoc.o" -c -o
"nuclei_sdk/SoC/evalsoc/Common/Source/GCC/intexc_evalsoc.o" "../
nuclei_sdk/SoC/evalsoc/Common/Source/GCC/intexc_evalsoc.S"
Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/unlink.c

Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/symlink.c
Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/GCC/
intexc_evalsoc_s.S

Invoking: GNU RISC-V Cross Assembler
Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/GCC/
startup_evalsoc.S
Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/GCC/
intexc_evalsoc.S
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mcmmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -x assembler-with-cpp -D__IDE_RV_CORE=n307fd -
DBOOT_HARTID=0 -DRUNMODE_IC_EN=0 -DRUNMODE_DC_EN=0 -
DRUNMODE_CCM_EN=0 -DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -
DDOWNLOAD_MODE_STRING="ILM" -I"C:
\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Inc

```



```
-I"C:\NucleiStudio_workspace\test\application" -MMD -MP -
MF"nuclei_sdk/SoC/evalsoc/Common/Source/GCC/intexc_evalsoc_s.d" -
MT"nuclei_sdk/SoC/evalsoc/Common/Source/GCC/intexc_evalsoc_s.o" -c
-o "nuclei_sdk/SoC/evalsoc/Common/Source/GCC/intexc_evalsoc_s.o"
"../nuclei_sdk/SoC/evalsoc/Common/Source/GCC/intexc_evalsoc_s.S"
Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/open.c
Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/wait.c
Invoking: GNU RISC-V Cross Assembler
```

```
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mcmmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -x assembler-with-cpp -D__IDE_RV_CORE=n307fd -
DBOOT_HARTID=0 -DRUNMODE_IC_EN=0 -DRUNMODE_DC_EN=0 -
DRUNMODE_CCM_EN=0 -DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -
DDOWNLOAD_MODE_STRING="ILM" -I"C:
\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Inc
-I"C:\NucleiStudio_workspace\test\application" -MMD -MP -
MF"nuclei_sdk/SoC/evalsoc/Common/Source/GCC/startup_evalsoc.d" -
MT"nuclei_sdk/SoC/evalsoc/Common/Source/GCC/startup_evalsoc.o" -c -
o "nuclei_sdk/SoC/evalsoc/Common/Source/GCC/startup_evalsoc.o" "../
nuclei_sdk/SoC/evalsoc/Common/Source/GCC/startup_evalsoc.S"
Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/Drivers/
evalsoc_uart.c
Invoking: GNU RISC-V Cross C Compiler
Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/
evalsoc_common.c
Building file: ../nuclei_sdk/SoC/evalsoc/Common/Source/
system_evalsoc.c
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mcmmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING="ILM" -
I"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -
I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
\NucleiStudio_workspace\test\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Inc
```

```

-I"C:\NucleiStudio_workspace\test\application" -std=gnu11 -MMD -MP
-MF"nuclei_sdk/SoC/evalsoc/Common/Source/Drivers/evalsoc_uart.d" -
MT"nuclei_sdk/SoC/evalsoc/Common/Source/Drivers/evalsoc_uart.o" -c
-o "nuclei_sdk/SoC/evalsoc/Common/Source/Drivers/evalsoc_uart.o"
"./nuclei_sdk/SoC/evalsoc/Common/Source/Drivers/evalsoc_uart.c"
Invoking: GNU RISC-V Cross C Compiler
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mcmmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING="ILM" -
I"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -
I"C:
\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Include"
-I"C:\NucleiStudio_workspace\test\application" -std=gnu11 -MMD -MP
-MF"nuclei_sdk/SoC/evalsoc/Common/Source/evalsoc_common.d" -
MT"nuclei_sdk/SoC/evalsoc/Common/Source/evalsoc_common.o" -c -o
"nuclei_sdk/SoC/evalsoc/Common/Source/evalsoc_common.o" "../
nuclei_sdk/SoC/evalsoc/Common/Source/evalsoc_common.c"
Invoking: GNU RISC-V Cross C Compiler
riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mcmmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING="ILM" -
I"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -
I"C:
\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Include"
-I"C:\NucleiStudio_workspace\test\application" -std=gnu11 -MMD -MP
-MF"nuclei_sdk/SoC/evalsoc/Common/Source/system_evalsoc.d" -
MT"nuclei_sdk/SoC/evalsoc/Common/Source/system_evalsoc.o" -c -o
"nuclei_sdk/SoC/evalsoc/Common/Source/system_evalsoc.o" "../
nuclei_sdk/SoC/evalsoc/Common/Source/system_evalsoc.c"
Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/stat.c
Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/write.c

Building file: ../application/main.c
Invoking: GNU RISC-V Cross C Compiler

```

```

riscv64-unknown-elf-gcc -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mcmmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -D__IDE_RV_CORE=n307fd -DBOOT_HARTID=0 -DRUNMODE_IC_EN=0
-DRUNMODE_DC_EN=0 -DRUNMODE_CCM_EN=0 -
DDOWNLOAD_MODE=DOWNLOAD_MODE_ILM -DDOWNLOAD_MODE_STRING="ILM" -
I"C:\NucleiStudio_workspace\test\nuclei_sdk\NMSIS\Core\Include" -
I"C:
\nuclei_sdk\SoC\evalsoc\Common\Include"
-I"C:
\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Inc
-I"C:\NucleiStudio_workspace\test\application" -std=gnu11 -MMD -MP
-MF"application/main.d" -MT"application/main.o" -c -o "application/
main.o" "../application/main.c"
Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/GCC/
startup_evalsoc.S

Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/times.c

Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/GCC/
intexc_evalsoc_s.S

Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/Drivers/
evalsoc_uart.c

Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/
evalsoc_common.c

Finished building: ../nuclei_sdk/SoC/evalsoc/Common/Source/
system_evalsoc.c

Finished building: ../application/main.c

Building target: test.elf
Invoking: GNU RISC-V Cross C++ Linker
riscv64-unknown-elf-g++ -march=rv32imafdc -mabi=ilp32d -
mtune=nuclei-300-series -mcmmodel=medlow -msave-restore -isystem=/
include/newlib-nano -O2 -ffunction-sections -fdata-sections -fno-
common -g -T "C:
\nuclei_sdk\SoC\evalsoc\Board\nuclei_fpga_eval\Sour
-nostartfiles -nodefaultlibs -Xlinker --gc-sections -Wl,-
Map,"test.map" -Wl,--check-sections -Wl,--no-warn-rwx-segments -u
_isatty -u _write -u _sbrk -u _read -u _close -u _fstat -u _lseek -
u errno -o "test.elf" ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/chown.o ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
clock_getres.o ../nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/

```

```

clock_gettime.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/clock_settime.o ./nuclei_sdk/SoC/evalsoc/Common/Source/
Stubs/newlib/close.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/
newlib/errno.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
execve.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
exit.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
fork.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
fstat.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
getpid.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
gettimeofday.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
isatty.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
kill.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
link.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
lseek.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
open.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
read.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
readlink.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
sbrk.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
stat.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
symlink.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
times.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
unlink.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
wait.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Stubs/newlib/
write.o ./nuclei_sdk/SoC/evalsoc/Common/Source/GCC/
intexc_evalsoc.o ./nuclei_sdk/SoC/evalsoc/Common/Source/GCC/
intexc_evalsoc_s.o ./nuclei_sdk/SoC/evalsoc/Common/Source/GCC/
startup_evalsoc.o ./nuclei_sdk/SoC/evalsoc/Common/Source/Drivers/
evalsoc_uart.o ./nuclei_sdk/SoC/evalsoc/Common/Source/
evalsoc_common.o ./nuclei_sdk/SoC/evalsoc/Common/Source/
system_evalsoc.o ./application/main.o -lstdc++ -lc_nano -lgcc
Finished building target: test.elf

```

```

Invoking: GNU RISC-V Cross Create Flash Image
riscv64-unknown-elf-objcopy -O ihex "test.elf" "test.hex"
Invoking: GNU RISC-V Cross Create Listing
riscv64-unknown-elf-objdump --source --all-headers --demangle --
line-numbers --wide "test.elf" > "test.lst"
Invoking: GNU RISC-V Cross Print Size
riscv64-unknown-elf-size --format=berkeley "test.elf"

```

text	data	bss	dec	hex	filename
8824	1272	4592	14688	3960	test.elf

```

Finished building: test.siz
Finished building: test.hex

```

```

Finished building: test.lst

```

```
17:00:23 Build Finished. 0 errors, 0 warnings. (took 5s.75ms)
```

The following are the parameters provided by `org.eclipse.cdt.managedbuilder.core.headlessbuild`, for reference.

```
-data          {/path/to/workspace}
-remove        {[uri:]/path/to/project}
-removeAll     {[uri:]/path/to/projectTreeURI} Remove all projects
under URI
-import        {[uri:]/path/to/project}
-importAll     {[uri:]/path/to/projectTreeURI} Import all projects
under URI
-build         {project_name_reg_ex{/config_reg_ex} | all}
-cleanBuild    {project_name_reg_ex{/config_reg_ex} | all}
-markerType    Marker types to fail build on {all | cdt |
marker_id}
-no-indexer    Disable indexer
-verbose       Verbose progress monitor updates
-printErrorMarkers Print all error markers
-I             {include_path} additional include_path to add to
tools
-include       {include_file} additional include_file to pass to
tools
-D             {preproc_define} addition preprocessor defines to
pass to the tools
-E            {var=value} replace/add value to environment
variable when running all tools
-Ea           {var=value} append value to environment variable
when running all tools
-Ep           {var=value} prepend value to environment variable
when running all tools
-Er           {var} remove/unset the given environment variable
-T            {toolid} {optionid=value} replace a tool option
value in each configuration built
-Ta           {toolid} {optionid=value} append to a tool option
value in each configuration built
-Tp           {toolid} {optionid=value} prepend to a tool option
value in each configuration built
-Tr           {toolid} {optionid=value} remove a tool option value
in each configuration built
              Tool option values are parsed as a string, comma
              separated list of strings or a boolean based on the options type
```

Error Reported by OpenOCD When Flashing a Program: Error: Device ID 0x1e200a6d is not known as FESPI capable¶

Problem Description¶

In Nuclei Studio 2023.10, the following error may be reported when flashing a program:

See <https://github.com/riscv-mcu/hbird-sdk/issues/8>

```
Info : Using libusb driver
Info : clock speed 1000 kHz
Info : JTAG tap: riscv.cpu tap/device found: 0x1e200a6d (mfg:
0x536 (Nuclei System Technology Co Ltd), part: 0xe200, ver: 0x1)
Info : [riscv.cpu] Found 0 triggers
halted at 0x200000b2 due to debug interrupt
Info : Examined RISCv core; XLEN=32, misa=0x40001105
[riscv.cpu] Target successfully examined.
Info : starting gdb server for riscv.cpu on 3333
Info : Listening on port 3333 for gdb connections
Error: Device ID 0x1e200a6d is not known as FESPI capable
Error: auto_probe failed
```

Solution¶

In OpenOCD 2023.10, flash bank `$_FLASHNAME` was changed from `fespi` to `nuspi`, so the `fespi` setting in the OpenOCD configuration file of the project needs to be changed to `nuspi`. Taking the Hummingbird (hbird) project as an example, modify `hbird_sdk/SoC/hbirdv2/Board/mcu200t/openocd_hbirdv2.cfg` to the following configuration, and the project will then work properly.

```
adapter_khz      1000

interface ftdi
ftdi_vid_pid 0x0403 0x6010
ftdi_oscan1_mode off

transport select jtag
```

```

ftdi_layout_init 0x0008 0x001b
ftdi_layout_signal nSRST -oe 0x0020 -data 0x0020
ftdi_layout_signal TCK -data 0x0001
ftdi_layout_signal TDI -data 0x0002
ftdi_layout_signal TDO -input 0x0004
ftdi_layout_signal TMS -data 0x0008
ftdi_layout_signal JTAG_SEL -data 0x0100 -oe 0x0100

set _CHIPNAME riscv
jtag newtap $_CHIPNAME cpu -irlen 5

set _TARGETNAME $_CHIPNAME.cpu
target create $_TARGETNAME riscv -chain-position $_TARGETNAME
$_TARGETNAME configure -work-area-phys 0x80000000 -work-area-size
10000 -work-area-backup 1

set _FLASHNAME $_CHIPNAME.flash
flash bank $_FLASHNAME nuspi 0x20000000 0 0 0 $_TARGETNAME
# Set the ILM space also as flash, to make sure it can be add
breakpoint with hardware trigger
#flash bank onboard_ilm nuspi 0x80000000 0 0 0 $_TARGETNAME

# Expose Nuclei self-defined CSRS range
770-800,835-850,1984-2032,2064-2070
# See https://github.com/riscv/riscv-gnu-toolchain/issues/
319#issuecomment-358397306
# Then user can view the csr register value in gdb using: info reg
csr775 for CSR MTVT(0x307)
riscv expose_csrs 770-800,835-850,1984-2032,2064-2070

init
#reset
if {[ info exists pulse_srst]} {
    ftdi_set_signal nSRST 0
    ftdi_set_signal nSRST z
}
halt
# We must turn on this because otherwise the IDE version debug
cannot download the program into flash
flash protect 0 0 last off

```


About the Inconsistency Between Dhrystone Benchmark Scores in the IDE and NSDK 0.5.0 Command-Line Scores¶

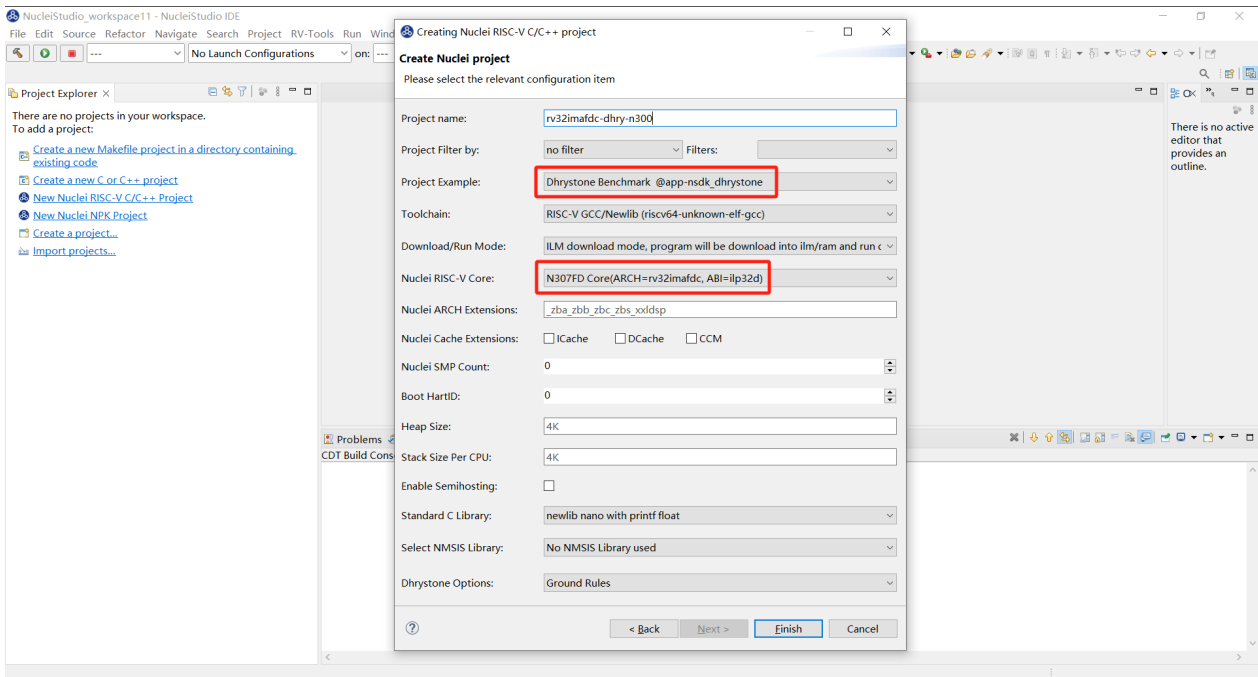
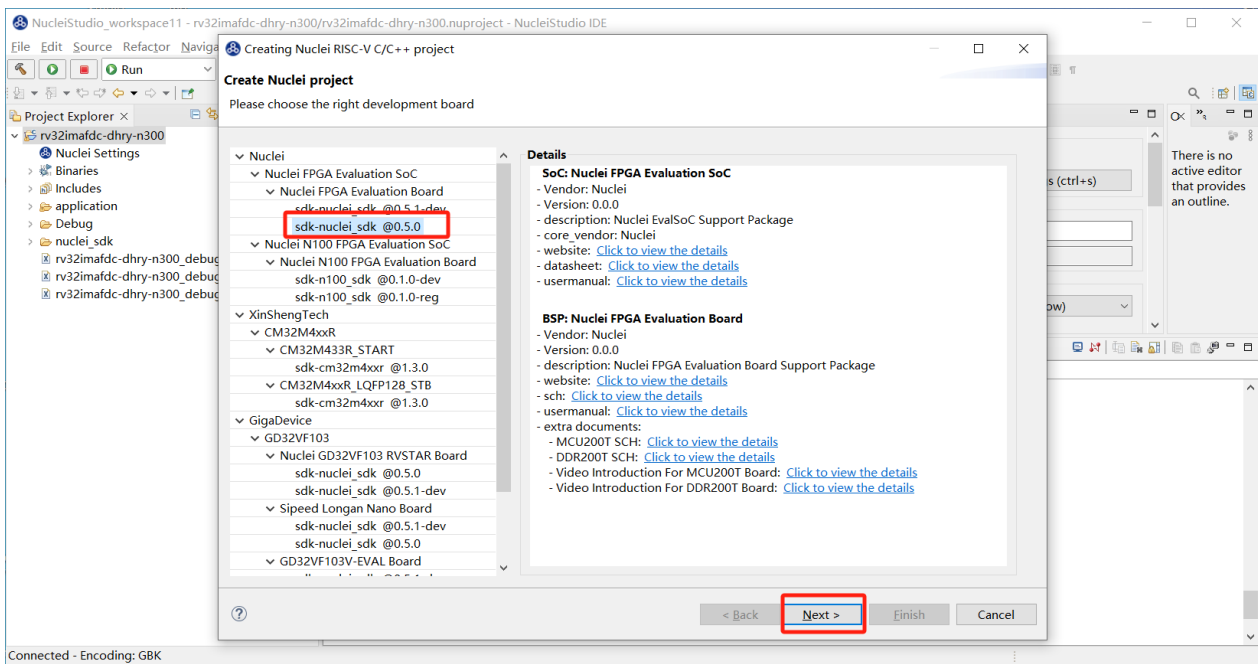
Problem Description¶

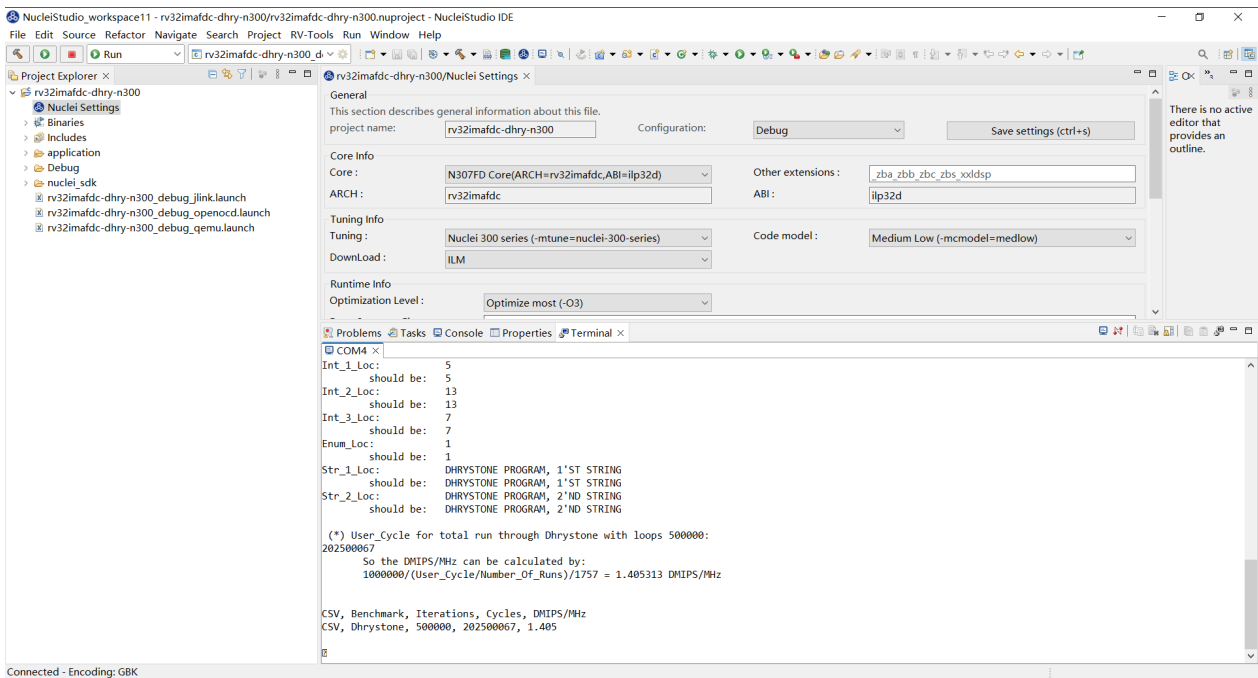
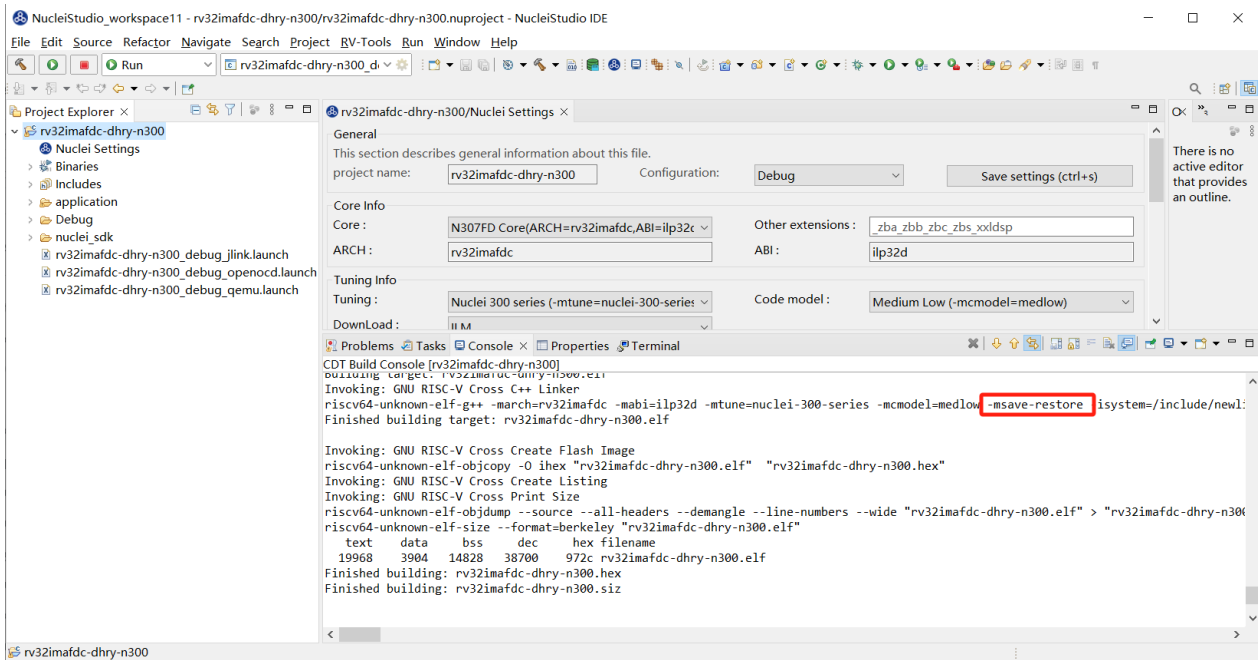
In version 0.5.0 of `sdk-nuclei_sdk`, the `-msave-restore` option is enabled by default so that some programs can compile without errors when using the `libncrt` library in the IDE. However, when creating a Dhrystone example project with the `newlib` library selected, this option causes the benchmark score to drop, and the result does not reflect the CPU's true performance.

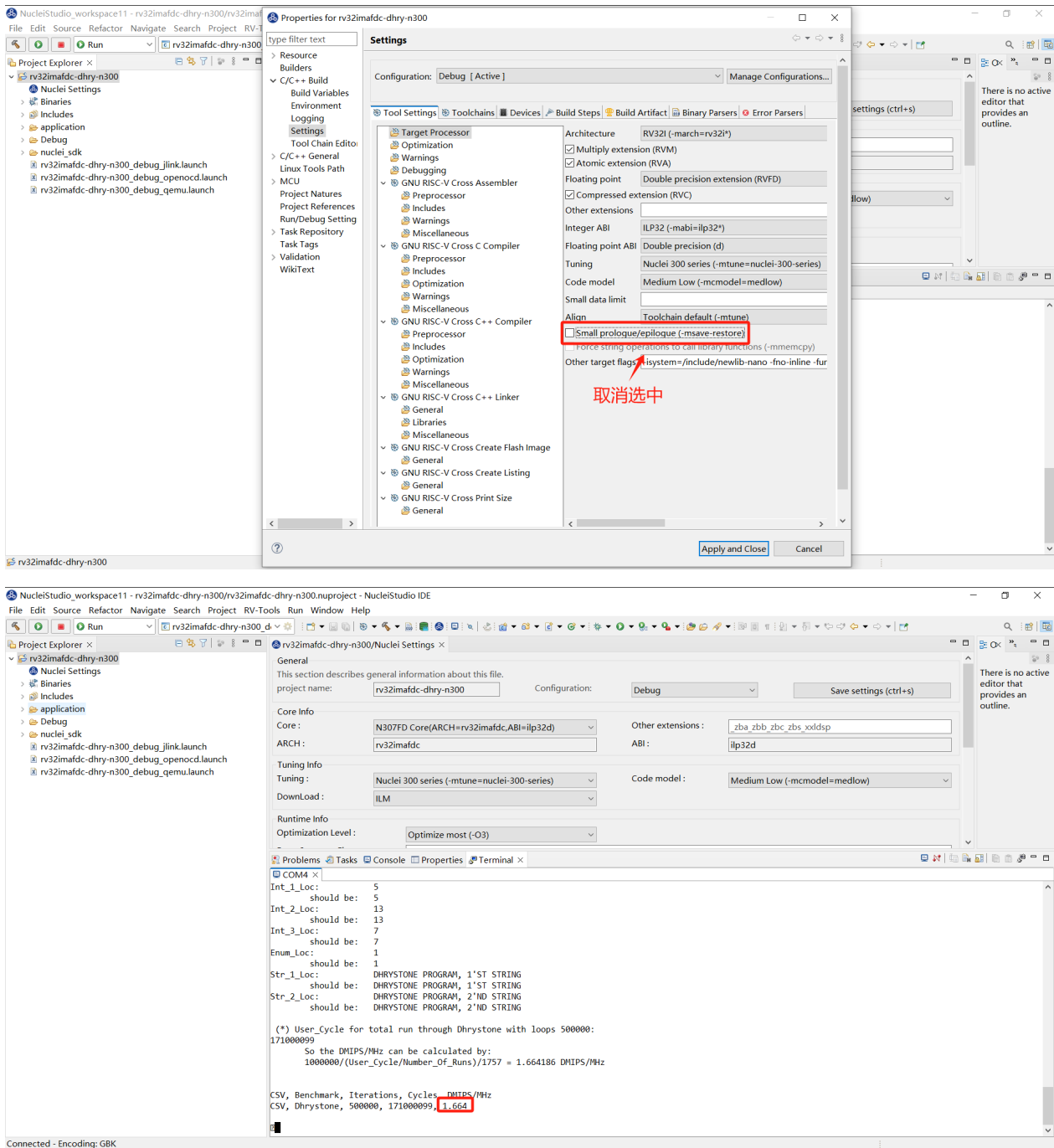
Solution¶

When running the benchmark, you need to go to the corresponding project's Properties -> C/C++ Build -> Settings and deselect Small prologue/epilogue(`-msave-restore`). The detailed steps and screenshots are as follows:

1. Download the `sdk-nuclei_sdk` 0.5.0 NPK component package.
2. Create a new Nuclei RISC-V C/C++ project.
3. During project creation, select Dhrystone Benchmark and N307FD Core, and keep the default settings for the other options. If you build and run it directly at this point, the benchmark score is 1.405.
4. However, for actual benchmarking, you must first deselect the `-msave-restore` option; the resulting benchmark score is 1.664.







Error: Couldn't find an available hardware trigger / Error: can't add breakpoint: resource not available¶

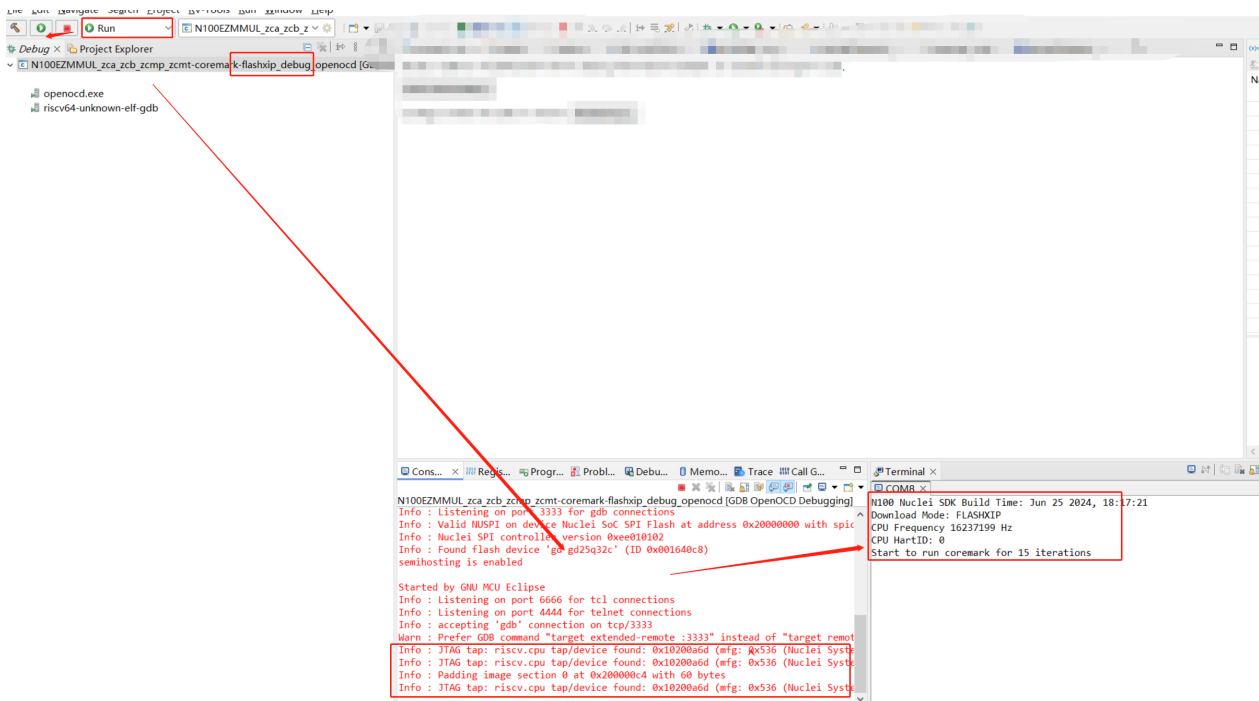
Problem Description¶

When using OpenOCD in NucleiStudio to debug hbird/hbirdv2 processors (which do not support hardware breakpoints) or the Nuclei 100 series processors, an Error is reported when the program runs from Flash/FlashXip.

```
Error: Couldn't find an available hardware trigger.
Error: can't add breakpoint: resource not available
```

```
Info : coreid=0, 32 : 0x0
Info : Examined RISC-V core; found 1 harts
Info : hart 0: XLEN=32, misa=0x40001104
[riscv.cpu] Target successfully examined.
Info : starting gdb server for riscv.cpu on 3333
Info : Listening on port 3333 for gdb connections
Info : Valid NUSPI on device Nuclei SoC SPI Flash at address 0x20000000 with spictrl regbase at 0x10014000
Info : Nuclei SPI controller version 0xee010102
Info : Found flash device 'gd25q32c' (ID 0x001640c8)
semihosting is enabled

Started by GNU MCU Eclipse
Info : Listening on port 6666 for tcl connections
Info : Listening on port 4444 for telnet connections
Info : accepting 'gdb' connection on tcp/3333
Warn : Prefer GDB command "target extended-remote :3333" instead of "target remote :3333"
Info : JTAG tap: riscv.cpu tap/device found: 0x10200a6d (mfg: 0x536 (Nuclei System Technology Co Ltd), part: 0x0200, ver: 0x1)
Info : JTAG tap: riscv.cpu tap/device found: 0x10200a6d (mfg: 0x536 (Nuclei System Technology Co Ltd), part: 0x0200, ver: 0x1)
Info : Padding image section 0 at 0x200000c4 with 60 bytes
Info : JTAG tap: riscv.cpu tap/device found: 0x10200a6d (mfg: 0x536 (Nuclei System Technology Co Ltd), part: 0x0200, ver: 0x1)
Info : [riscv.cpu] Found 0 triggers
Error: Couldn't find an available hardware trigger.
Error: can't add breakpoint: resource not available
```



This happens because the CPU being run does not support hardware breakpoints, which prevents the IDE's debug functionality from working properly when the program runs from Flash — the IDE needs to set a temporary breakpoint. If you only need to download and run the program, switch to Run mode and the program will run normally.

Solution

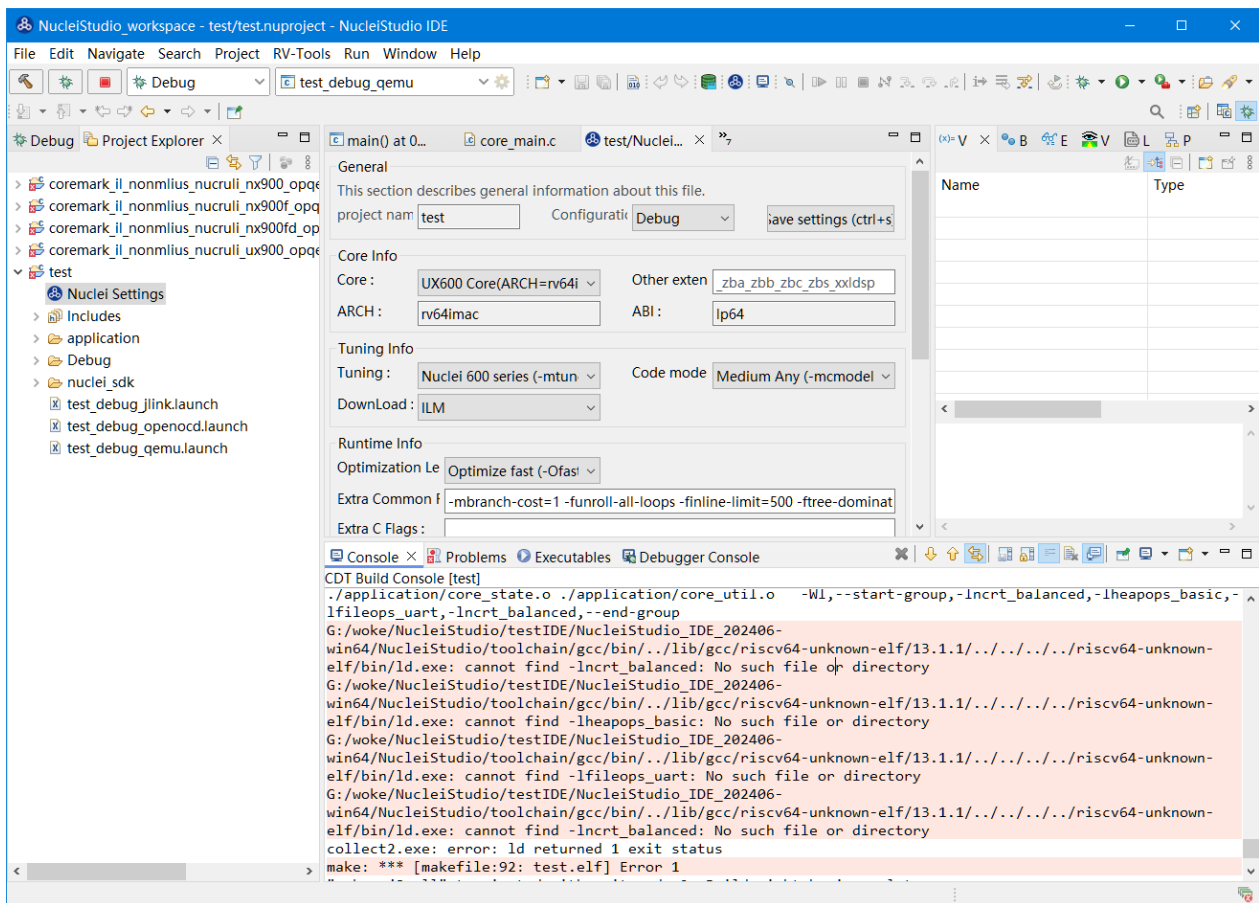
When debugging this type of processor, if debugging is required, the program must be compiled to run from RAM.

cannot find -ln crt_balanced: No such file or directory¶

Problem Description¶

When building a project in Nuclei Studio, the following error messages are reported:

```
G:/NucleiStudio/toolchain/gcc/bin/./lib/gcc/riscv64-unknown-elf/13.1.1/././././././riscv64-unknown-elf/bin/ld.exe: cannot find -ln crt_balanced: No such file or directory
G:/NucleiStudio/toolchain/gcc/bin/./lib/gcc/riscv64-unknown-elf/13.1.1/././././././riscv64-unknown-elf/bin/ld.exe: cannot find -lheapops_basic: No such file or directory
G:/NucleiStudio/toolchain/gcc/bin/./lib/gcc/riscv64-unknown-elf/13.1.1/././././././riscv64-unknown-elf/bin/ld.exe: cannot find -lfileops_uart: No such file or directory
G:/NucleiStudio/toolchain/gcc/bin/./lib/gcc/riscv64-unknown-elf/13.1.1/././././././riscv64-unknown-elf/bin/ld.exe: cannot find -ln crt_balanced: No such file or directory
```



This happens because when creating the project, we created a 64-bit project, and in the Standard C Library settings, extensions with `-ln crt_balanced` and `-lfileops_uart` were selected. However, these extensions do not support 64-bit, which causes the build to fail.

Create Nuclei RISC-V C/C++ project using npk sdk-nuclei_sdk @0.6.0-dev

Create project for SoC:Nuclei FPGA Evaluation SoC,Board:Nuclei FPGA Evaluation B

Project name:

Project Filter by: Filters:

Project Example:

Toolchain:

Download/Run Mode:

Nuclei RISC-V Core:

ARCH Extensions(ARCH_EXT=):

Nuclei Cache Extensions: ☐ ICache ☐ DCache ☐ CCM

Nuclei SMP Count:

Boot HartID:

Heap Size:

Stack Size Per CPU:

Enable Semihosting: ☐

Standard C Library(STDCLIB=):

Select NMSIS Library:

Solution

-ln crt_balanced and -lfileops_uart do not support 64-bit processors. When creating a project for such processors, avoid using the libn crt library.

UnsatisfiedLinkError of swt-win32-4965r8.dll on Windows 7

Problem Description

When using NucleiStudio 2024.06 on Windows 7 or Windows 8, users find that it fails to start. The following error can be seen in the log files under the NucleiStudio\configuration directory:

```
!ENTRY org.eclipse.osgi 4 0 2024-07-16 10:41:57.010
!MESSAGE Application error
!STACK 1
java.lang.UnsatisfiedLinkError: Could not load SWT library.
Reasons:
    C:\NucleiStudio\configuration\org.eclipse.osgi\492\0\cp\swt-
win32-4965r11.dll: 找不到指定的程序。
    no swt-win32 in java.library.path: C:\NucleiStudio;C:
\Windows\Sun\Java\bin;C:\Windows\system32;C:\Windows;C:/
NucleiStudio//plugins/
org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.
0.3.v20240426-1530/jre/bin/server;C:/NucleiStudio//plugins/
org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.
0.3.v20240426-1530/jre/bin;C:\Java\JCDK3.0.4_ClassicEdition\bin;C:
\Java\jdk1.6.0_26\bin;C:\Java\jdk1.6.0_26\lib;C:
\Windows\system32;C:\Windows;C:\Windows\System32\Wbem;C:
\Windows\System32\WindowsPowerShell\v1.0\;C:\Program
Files\TortoiseSVN\bin;C:\Program Files (x86)\Microsoft SQL
Server\90\Tools\bin\;D:\Python25;C:\NucleiStudio;;.
    no swt in java.library.path: C:\NucleiStudio;C:
\Windows\Sun\Java\bin;C:\Windows\system32;C:\Windows;C:/
NucleiStudio//plugins/
org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.
0.3.v20240426-1530/jre/bin/server;C:/NucleiStudio//plugins/
org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.
0.3.v20240426-1530/jre/bin;C:\Java\JCDK3.0.4_ClassicEdition\bin;C:
\Java\jdk1.6.0_26\bin;C:\Java\jdk1.6.0_26\lib;C:
\Windows\system32;C:\Windows;C:\Windows\System32\Wbem;C:
\Windows\System32\WindowsPowerShell\v1.0\;C:\Program
Files\TortoiseSVN\bin;C:\Program Files (x86)\Microsoft SQL
Server\90\Tools\bin\;D:\Python25;C:\NucleiStudio;;.
    C:\Users\username\.swt\lib\win32\x86_64\swt-win32-4965r11.dll:
找不到指定的程序。
```

```
Can't load library: C:
\Users\username\.swt\lib\win32\x86_64\swt-win32.dll
Can't load library: C:
\Users\username\.swt\lib\win32\x86_64\swt.dll
C:\Users\username\.swt\lib\win32\x86_64\swt-win32-4965r11.dll:
找不到指定的程序。
```

```
at org.eclipse.swt.internal.Library.loadLibrary(Library.java:
345)
at org.eclipse.swt.internal.Library.loadLibrary(Library.java:
254)
at org.eclipse.swt.internal.C.<clinit>(C.java:19)
at
org.eclipse.swt.internal.win32.STARTUPINFO.<clinit>(STARTUPINFO.java:
42)
at org.eclipse.swt.widgets.Display.<clinit>(Display.java:149)
at
org.eclipse.ui.internal.Workbench.createDisplay(Workbench.java:721)
at org.eclipse.ui.PlatformUI.createDisplay(PlatformUI.java:185)
at
org.eclipse.ui.internal.ide.application.IDEApplication.createDisplay(IDEApplica
182)
at
org.eclipse.ui.internal.ide.application.IDEApplication.start(IDEApplication.jav
125)
at
org.eclipse.equinox.internal.app.EclipseAppHandle.run(EclipseAppHandle.java:
208)
at
org.eclipse.core.runtime.internal.adaptor.EclipseAppLauncher.runApplication(Ecl
143)
at
org.eclipse.core.runtime.internal.adaptor.EclipseAppLauncher.start(EclipseAppLa
109)
at
org.eclipse.core.runtime.adaptor.EclipseStarter.run(EclipseStarter.java:
439)
at
org.eclipse.core.runtime.adaptor.EclipseStarter.run(EclipseStarter.java:
271)
at java.base/
jdk.internal.reflect.DirectMethodHandleAccessor.invoke(DirectMethodHandleAccess
103)
at java.base/java.lang.reflect.Method.invoke(Method.java:580)
at org.eclipse.equinox.launcher.Main.invokeFramework(Main.java:
668)
```

```
at org.eclipse.equinox.launcher.Main.basicRun(Main.java:605)
at org.eclipse.equinox.launcher.Main.run(Main.java:1481)
```

Note

The version number in the SWT library file name (e.g., swt-win32-4965r8.dll, swt-win32-4965r11.dll) depends on the Eclipse/SWT plugin version bundled with each Nuclei Studio release. Different IDE versions ship with different SWT plugin versions, so the exact DLL name in your log may differ from the one in the title of this document.

This is because Eclipse 2024.06 uses certain features that have specific operating system requirements. For details, refer to <https://github.com/eclipse-platform/eclipse.platform.swt/issues/1252>

That commit references `SystemParametersInfoForDpi`. See <https://learn.microsoft.com/en-us/windows/win32/api/winuser/nf-winuser-systemparametersinfofordpi>

And [f809372](#) references `GetSystemMetricsForDpi`. See <https://learn.microsoft.com/en-us/windows/win32/api/winuser/nf-winuser-getsystemmetricsfordpi>

Both of these functions require a minimum version of Windows 10:

Minimum supported client	Windows 10, version 1607 [desktop apps only]
Minimum supported server	Windows Server 2016 [desktop apps only]



But this does still not explain problems with MacOS.

In addition, the official Eclipse documentation states that the operating systems against which Eclipse is tested no longer include compatibility for certain OS versions. For details, refer to https://eclipse.dev/eclipse/development/plans/eclipse_project_plan_4_32.xml#target_environments

Operating System	Version	Hardware	JRE	Windowing System
Windows	10 11	x86 64-bit	OpenJDK 17.0.8 (LTS) OpenJDK 21 (LTS) Oracle Java 17.0.8 (LTS) Oracle Java 21 (LTS)	Win32
Red Hat Enterprise Linux	9.0	x86 64-bit aarch64	OpenJDK 17.0.8 (LTS) OpenJDK 21 (LTS) Oracle Java 17.0.8 (LTS) Oracle Java 21 (LTS)	GTK 3
		Power 64-bit LE	OpenJDK 17.0.8 (LTS)	
SUSE Linux Enterprise Server	15 SP4	x86 64-bit	OpenJDK 17.0.8 (LTS) OpenJDK 21 (LTS)	GTK 3
		Power 64-bit LE	OpenJDK 17.0.8 (LTS)	
Ubuntu Long Term Support	22.04	x86 64-bit aarch64	OpenJDK 17.0.8 (LTS) OpenJDK 21 (LTS)	GTK 3
Apple macOS	12 13	x86 64-bit	OpenJDK 17.0.8 (LTS) OpenJDK 21 (LTS) Oracle Java 17.0.8 (LTS) Oracle Java 21 (LTS)	Cocoa
	12 13	M1 (arm64)	OpenJDK 17.0.8 (LTS) OpenJDK 21 (LTS)	

Since NucleiStudio 2024.06 is based on Eclipse 2024.06, it is affected by the same type of issue.

Solution

Please use NucleiStudio 2024.06 on Windows 10 or later.

If you want to use NucleiStudio on older operating systems such as Windows 7 or Windows 8, consider using NucleiStudio 2024.02 or an earlier version.

Issues You May Encounter When Using the Profiling Feature¶

You may encounter some issues when using the Profiling feature. They are documented below:

- Issue 1: The log output reports insufficient on-chip memory, meaning there is not enough memory to store gprof/gcov data
- Issue 2: When collecting data via serial port output, the printed output is overwritten/truncated, so the data collected in the Console or Terminal is incomplete, causing data parsing to fail and a No files have been generated error dialog to pop up
- Issue 3: After deleting the gmon.out file, parsing again triggers a No files have been generated error dialog

Issue 1: The log output reports insufficient on-chip memory, meaning there is not enough memory to store gprof/gcov data¶

gprof/gcov data needs to be stored in on-chip memory. The amount of memory required depends on the size of the use case (ranging from tens to hundreds of KB), so make sure the on-chip memory is large enough.



The screenshot shows a debugger interface with tabs for Problems, Tasks, Console, Properties, Call Graph, and gcov. The Console tab is active, displaying the output of a GDB session for 'amrwb_profiling_demo_debug_qemu [GDB Nuclei QEMU riscv Debugging]'. The output includes several 'CSV, encode' lines followed by 'finish'. A red box highlights the error message '_mcleanup: tos overflow', which is followed by a red text annotation in Chinese: '说明片上RAM不足，需要大一点的存储空间' (Indicates on-chip RAM is insufficient, need a larger storage space). Below the error, the console shows 'Dump coverage data start' followed by a large hex dump. The dump consists of multiple lines of hexadecimal values, including '616463672a3133420cdce3f90000000000000001', '0c0000007b208975a32139fe888879e90000a101', '28000000330000000000000033000000000000', '33000000000000003300000000000033000000', '000000000000000010c0000006b130344d23a9d77', '3eb2bbc00000a1011000000010000000000000', '01000000000000000000000010c00000091dee512', '618e0f5fb943f3810000a101200000001000000', '00000000100000000000000010000000000000', and '0100000000000000'. The dump ends with 'CREATE: C:\Users\shuzhuo\demo\amrwb_profiling_demo\Debug\application\wrapper.gcda'.

```

amrwb_profiling_demo_debug_qemu [GDB Nuclei QEMU riscv Debugging]
CSV, encode, 3054837
CSV, encode, 2992645
CSV, encode, 2842156
finish
_mcleanup: tos overflow
说明片上RAM不足，需要大一点的存储空间
Dump coverage data start
616463672a3133420cdce3f90000000000000001
0c0000007b208975a32139fe888879e90000a101
28000000330000000000000033000000000000
33000000000000003300000000000033000000
000000000000000010c0000006b130344d23a9d77
3eb2bbc00000a1011000000010000000000000
01000000000000000000000010c00000091dee512
618e0f5fb943f3810000a101200000001000000
00000000100000000000000010000000000000
0100000000000000
CREATE: C:\Users\shuzhuo\demo\amrwb_profiling_demo\Debug\application\wrapper.gcda

```

Solution¶

First, make sure the memory size configured in the software matches the actual hardware size (ilm/sram/flash/ddr/). Otherwise, you need to adapt the memory layout in the software linker script:

For example, if downloading in `DOWNLOAD=ilm` mode, you can adapt it according to the actual ilm and dlm sizes of the hardware. For `nuclei_sdk 0.6.0`, the file to modify is `nuclei-sdk/SoC/evalsoc/Board/nuclei_fpga_eval/Source/GCC/gcc_evalsoc_ilm.ld`

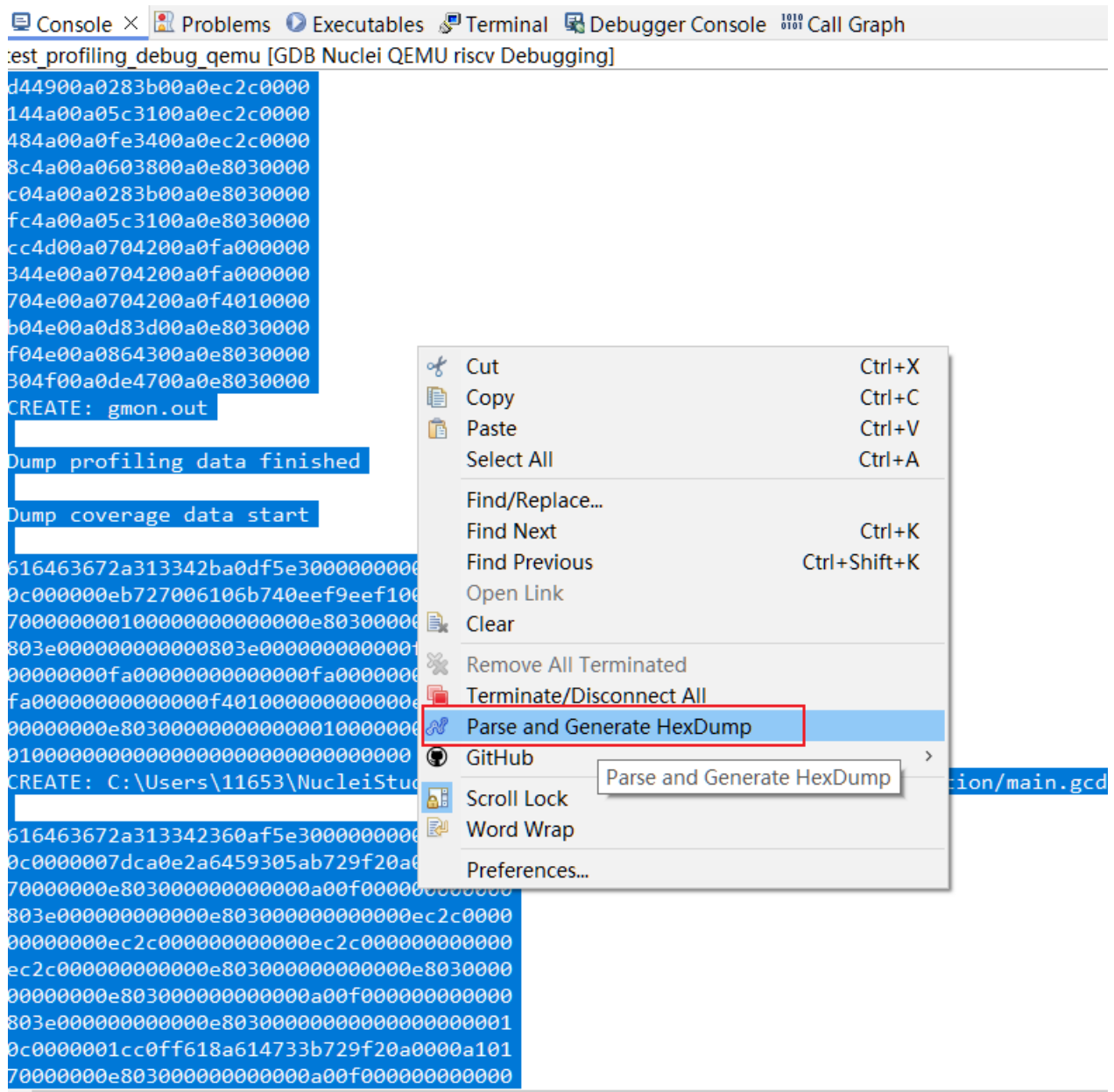
```
INCLUDE evalsoc.memory

MEMORY
{
    ilm (rxa!w) : ORIGIN = ILM_MEMORY_BASE,    LENGTH =
    ILM_MEMORY_SIZE
    ram (wxa!r) : ORIGIN = DLM_MEMORY_BASE,    LENGTH =
    DLM_MEMORY_SIZE
}
```

If `DOWNLOAD=ilm` mode does not provide enough memory, you can use a download mode with more memory (such as `DOWNLOAD=ddr`).

Issue 2: Data parsing fails because the data collected in the Console or Terminal is incomplete¶

In NucleiStudio 2024.06, when using the Profiling feature with serial port output, parsing the data with `Parse` and `Generate Hexdump` may pop up a `No files have been generated` error dialog, and the corresponding `gmon.out` file or `*.gcno` file is not generated in the end. This may be because the serial port data was overwritten/truncated, resulting in incomplete data and parsing failure.



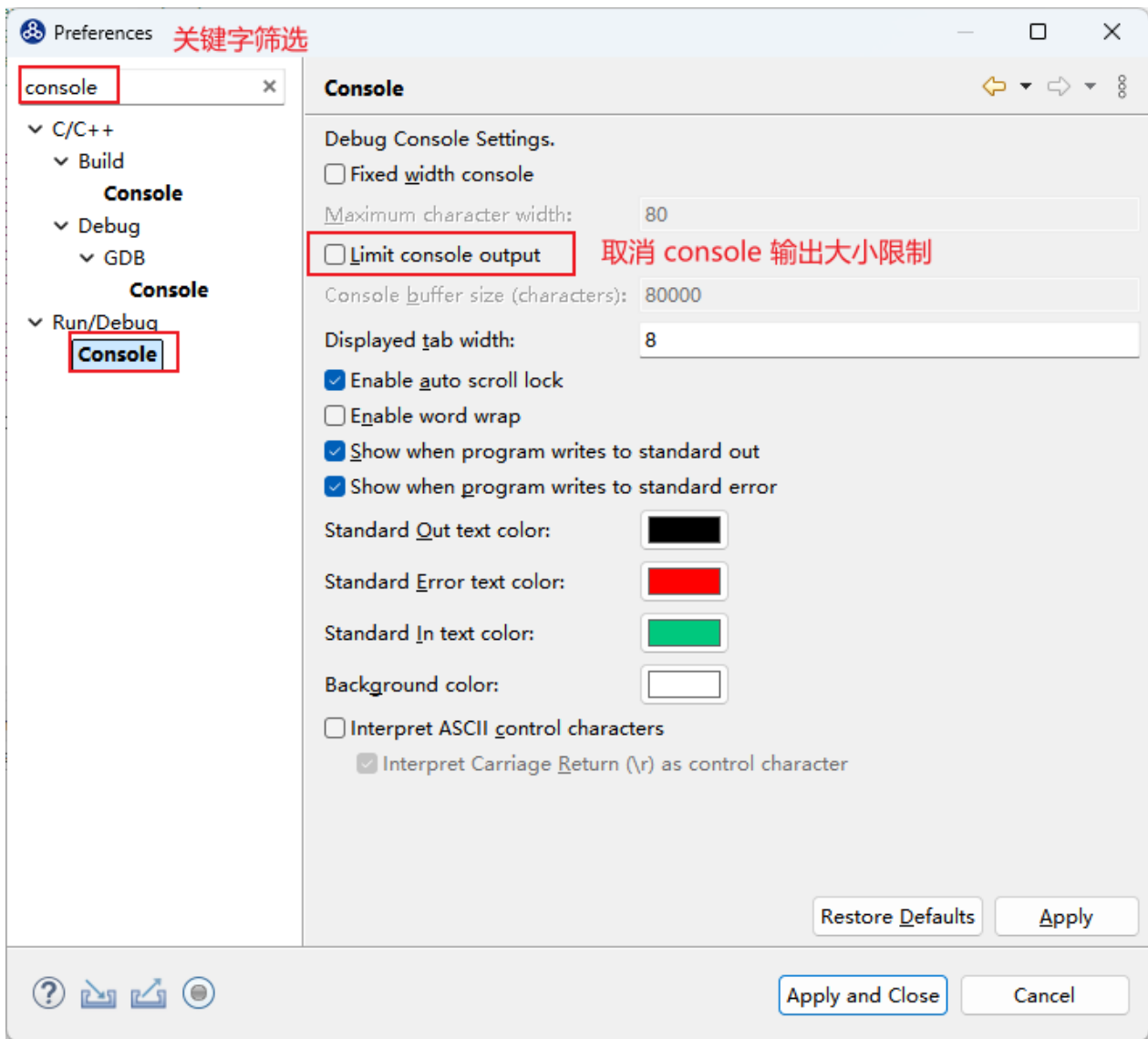
Solution

The Console and Terminal limit the number of output lines. When the output length exceeds the limit, the earlier content is overwritten/truncated, making the content incomplete and causing parsing to fail.

You need to increase the output size limit of the Console or Terminal to ensure that data is not overwritten/truncated.

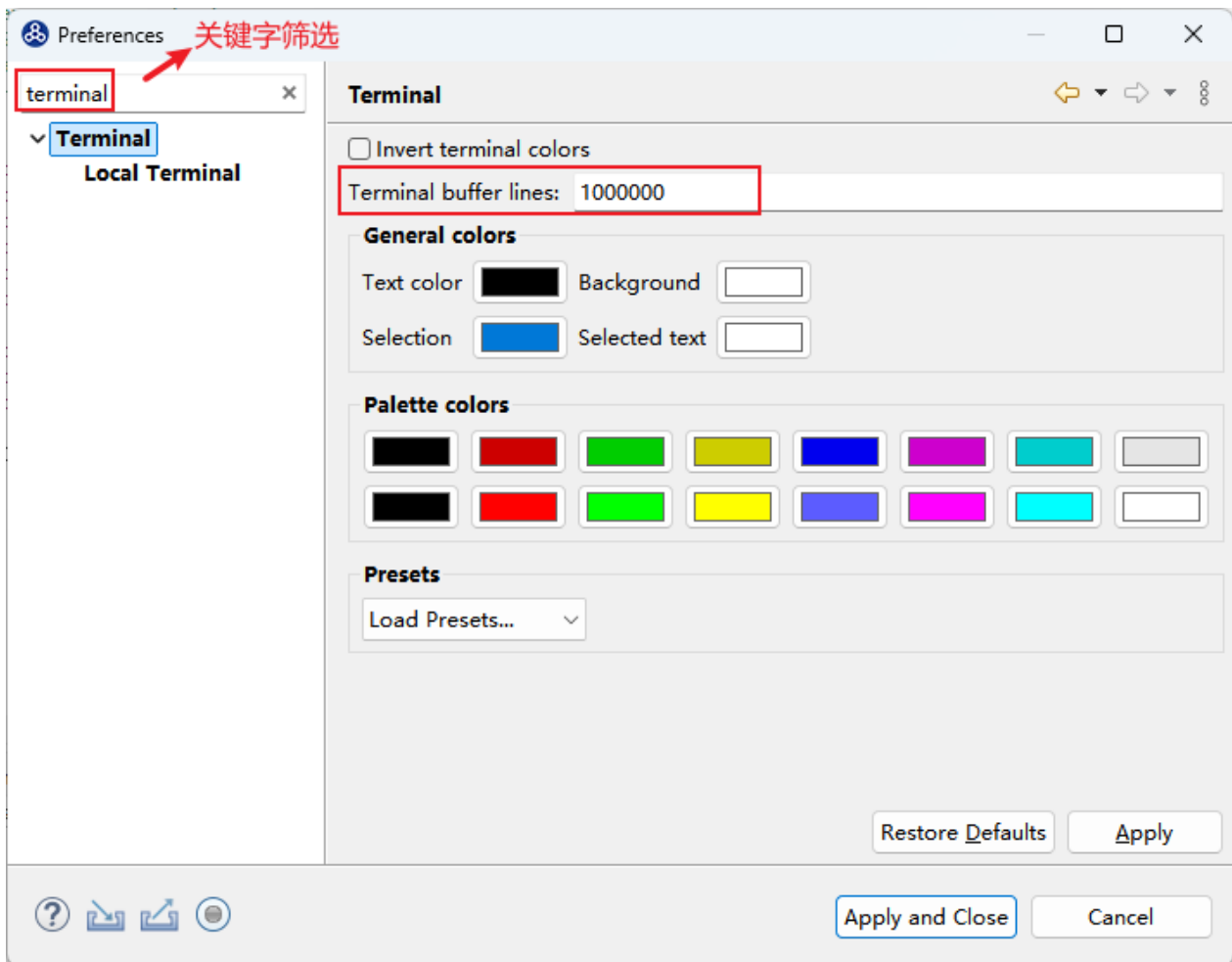
- It is recommended to set the Console output limit to unlimited.

Go to Window->Preference to open the following interface:



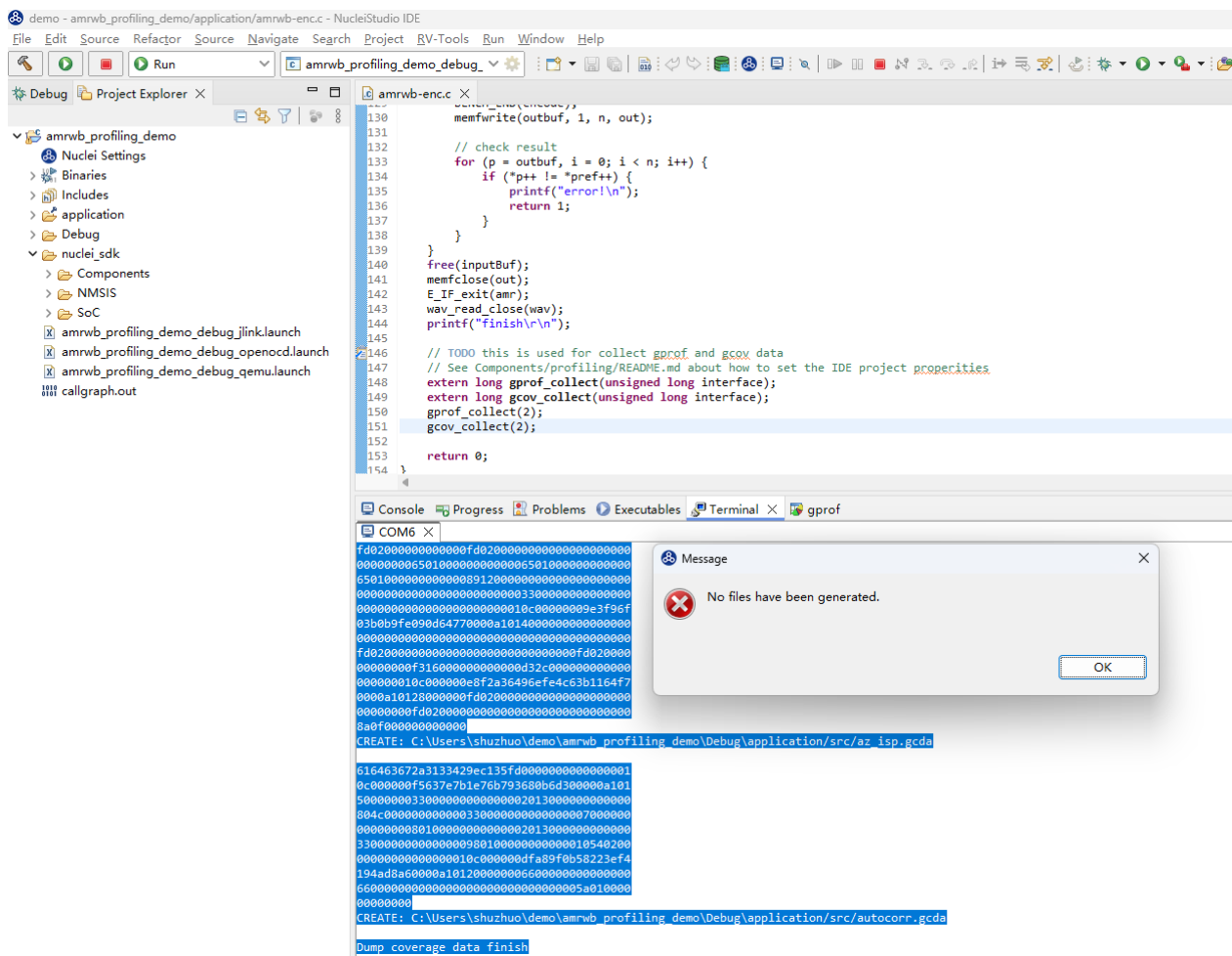
- It is recommended to set the Terminal output limit to a larger value.

Go to Window->Preference to open the following interface:



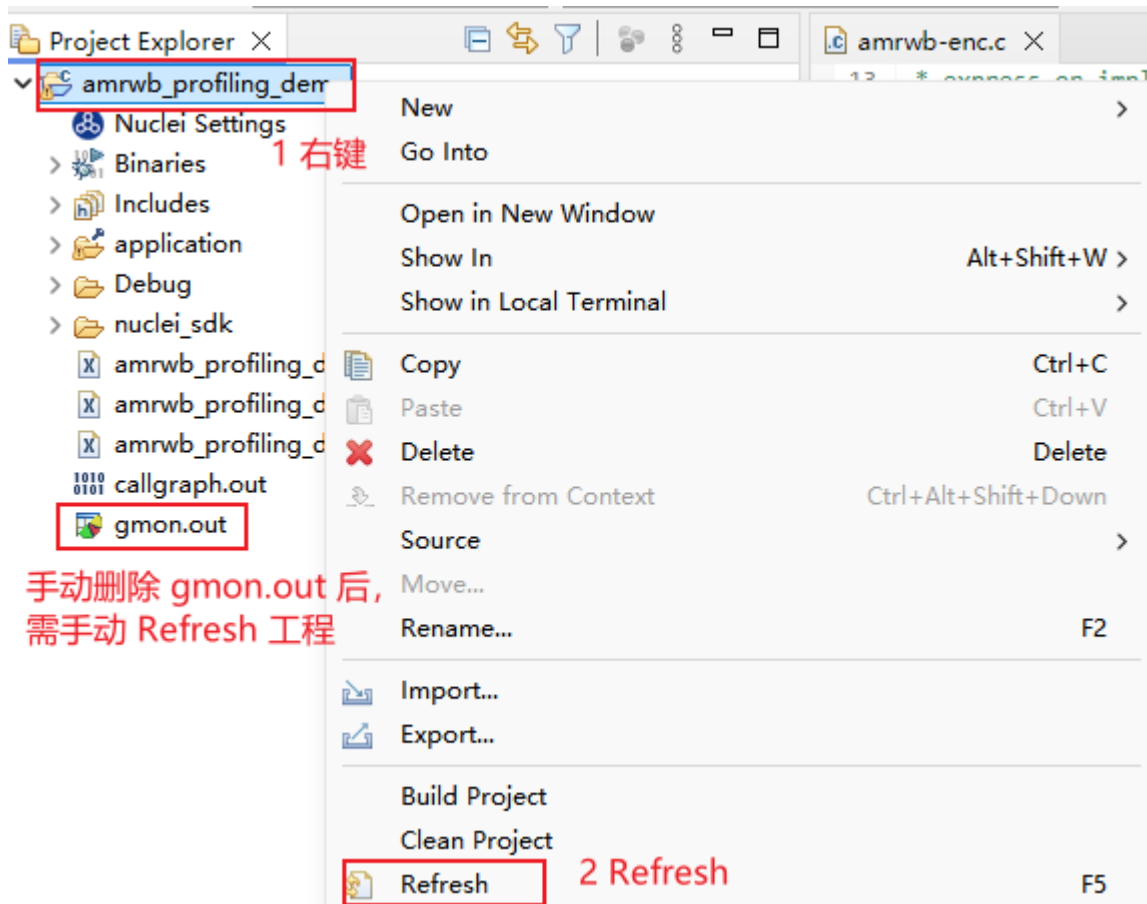
Issue 3: After deleting the gmon.out file, parsing again pops up a No files have been generated error dialog¶

After manually deleting the gmon.out file in the project folder, parsing again shows a No files have been generated error dialog.



Solution

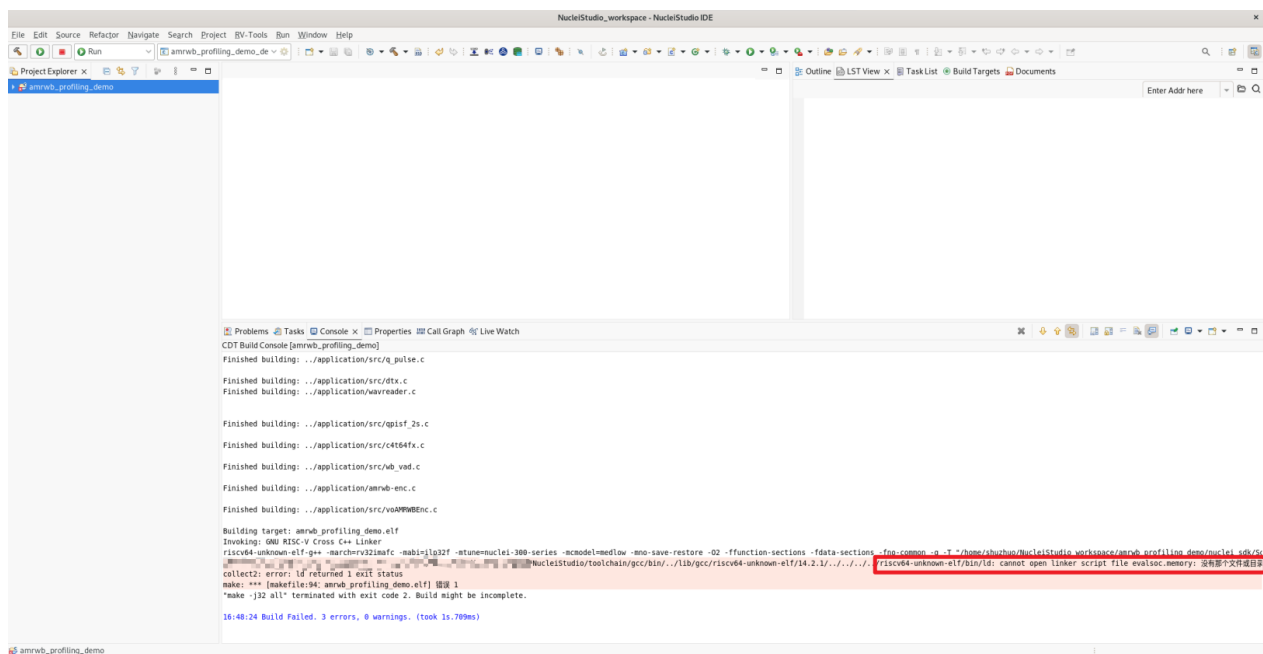
After manually deleting the `gmon.out` file, you need to manually refresh the project.



Issue 4: Compilation error when importing amrwb_profiling_demo.zip in Nuclei Studio on Linux

The use case created in the document "[Issues You May Encounter When Using the Profiling Feature](#)" has a problem, causing a compilation error when importing amrwb_profiling_demo.zip into Nuclei Studio on Linux.

The specific error is: evalsoc.memory: No such file or directory



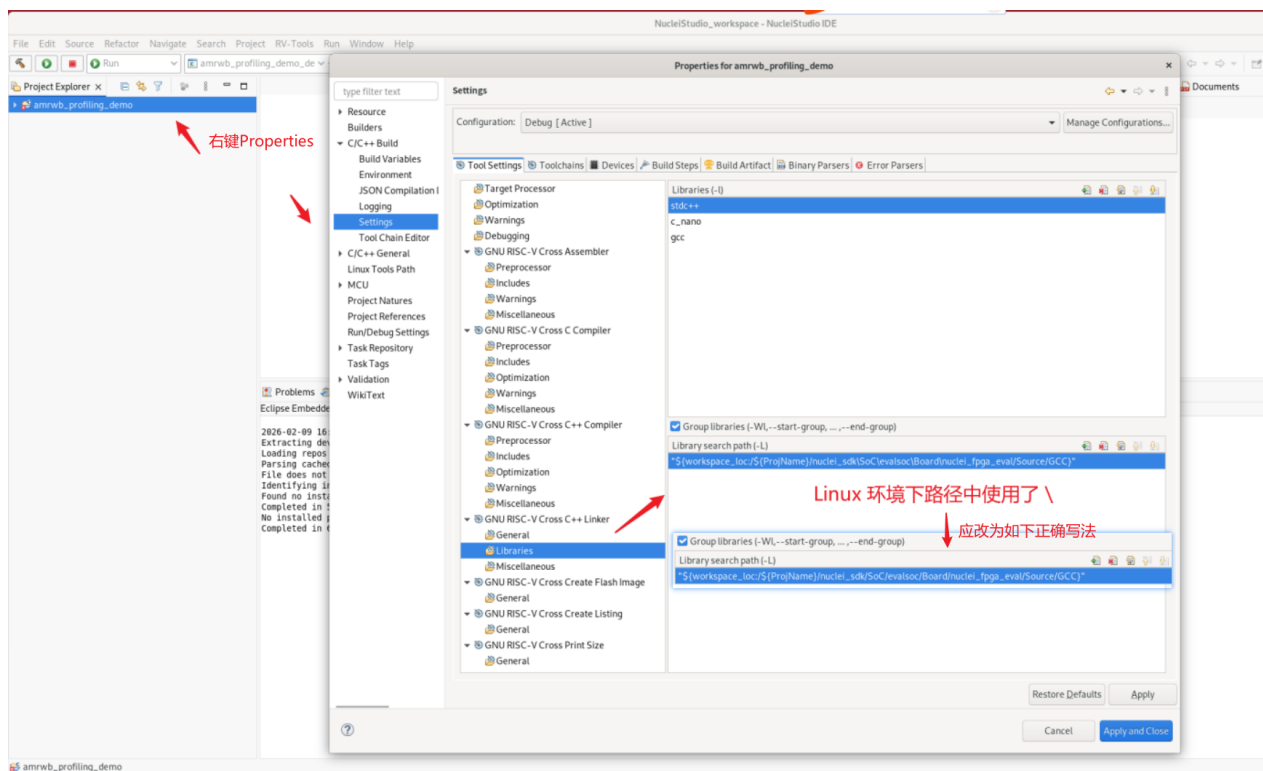
Cause: Windows path separators were mixed into the Linux environment

Solution:

You can use either of the following two methods:

Method 1. The zip package in the document has already been fixed; simply download the new use case package

Method 2. As shown in the figure below, change the \ in the path to /



An Example of Using the Profiling Feature in Nuclei Studio for Performance Tuning

This document is based on hands-on testing with the 2024.06 Windows/Linux version of Nuclei Studio.

Problem Description

Nuclei Studio 2024.06 provides the Profiling feature, the Call Graph feature, and the Code coverage feature for user convenience. A brief description is as follows:

- Profiling feature: Based on the binutils gprof tool, it can be used to analyze function call relationships, call counts, and execution time. Hotspot functions captured through Profiling can be used to analyze program bottlenecks for performance optimization.
- Call Graph feature: Based on the Profiling feature, it displays function call relationships, call counts, and execution time in graph form, making analysis easier for developers.
- Code coverage feature: Based on the gcov tool provided by the gcc compiler, it can be used to view the code coverage of source files, helping developers determine whether test cases are sufficient and whether they cover all branches and paths of the code under test.

The relevant chapters of [NucleiStudio_User_Guide.pdf](#) already describe these features in detail. This document demonstrates their practical application through an example.

Solution

1 Environment Preparation

Required materials:

- Nuclei Studio: [NucleiStudio 2024.06](#); the Windows version is used as an example
- Use case: [AMR-WB-enc](#), i.e., the Adaptive Multi-Rate Wideband encoding audio algorithm, is used as an example; users may port their own use cases

Porting the amrwbenc bare-metal use case based on nuclei-sdk v0.6.0:

Open Nuclei Studio and create an amrwbenc project, then port the amrwbenc source code so that the final use case runs correctly. Users may port their own use cases; the porting details differ from case to case, and this step is not the focus of this document, so it is skipped.

2 Profiling Feature

The Profiling feature in Nuclei Studio is based on the binutils gprof tool. During compilation, the specified source files must be compiled with the specific compiler option `-pg`. After a successful build, an ELF file is obtained, which is then run on the actual development board to collect the required `gmon.out` file, and finally the results are displayed graphically in the IDE. Therefore, gprof data collection code also needs to be added at the end of the use case. There are two ways to do this:

- Method 1: Port the gprof data collection code into your own project. The code can refer to [Profiling README](#)
- Method 2: Modify the Profiling demo in Nuclei Studio, i.e., replace the use case portion of the Profiling demo project with your own use case

The following example demonstrates the latter method:

Step 1: Create a new Profiling demo project

File->New->New Nuclei RISC-V C/C++ Project, select Nuclei FPGA Evaluation Board->`sdk-nuclei_sdk @0.6.0`

Note: Nuclei SDK version 0.6.0 or later must be selected to support the Profiling and Code coverage features

You can choose different toolchains according to your needs:

1. RISC-V GCC/Newlib(`riscv-unknown-elf-gcc`)
2. RISC-V Clang/Newlib(`riscv-unknown-elf-clang`)
3. Terapines ZCC(`zcc`)

Note: Nuclei SDK version 0.7.1 or later must be selected to support the Terapines ZCC(`zcc`) toolchain. If you select the Terapines ZCC(`zcc`) toolchain to use the profiling feature, please also change the Standard C Library(`STDCLIB=`) option to `newlib_small` Note: For the RISC-V Clang and Terapines ZCC toolchains, the latest SDK (0.8.1) and earlier versions currently only support the profiling feature; the Code coverage feature is not yet supported

The screenshot shows the 'Create project for SoC:Nuclei FPGA Evaluation SoC,Board:Nuclei FPGA Evaluation Board' window. Red boxes and arrows highlight specific configuration steps:

- 1 工程名**: Points to the 'Project name' field containing 'amrwb_profiling_demo'.
- 2 使用关键字, 快速筛选 Profiling demo**: Points to the 'Project Filter by' dropdown set to 'keywords' and the 'Filters' dropdown set to 'baremetal'.
- 3 选择 Profiling demo**: Points to the 'Project Example' dropdown set to 'Profiling demo to show how to use gprof and gcov @app-nsdk_demo_profiling'.
- 4 选择 ilm 模式**: Points to the 'Download/Run Mode' dropdown set to 'ILM download mode, program will be download into ilm/ram and run directly in ilm/ram'.
- 5 以 Nuclei n300 为例**: Points to the 'Nuclei RISC-V Core' dropdown set to 'N307FD Core(ARCH=rv32imafdc, ABI=ilp32d)'.

Other visible fields include:

- Toolchain**: RISC-V GCC/Newlib (riscv64-unknown-elf-gcc)
- ARCH Extensions(ARCH_EXT=)**: _zba_zbb_zbc_zbs_xldsp
- Nuclei Cache Extensions**: ICache, DCache, CCM (all unchecked)
- Nuclei SMP Count**: 0
- Boot HartID**: 0
- Heap Size**: 4K
- Stack Size Per CPU**: 4K
- Enable Semihosting**: unchecked
- Standard C Library(STDCLIB=)**: newlib_nano: newlib nano without printf/scanf float
- Select NMSIS Library**: No NMSIS Library used
- Application Compile Flags**: -O2

At the bottom are buttons for '< Back', 'Next >', 'Finish', and 'Cancel'.

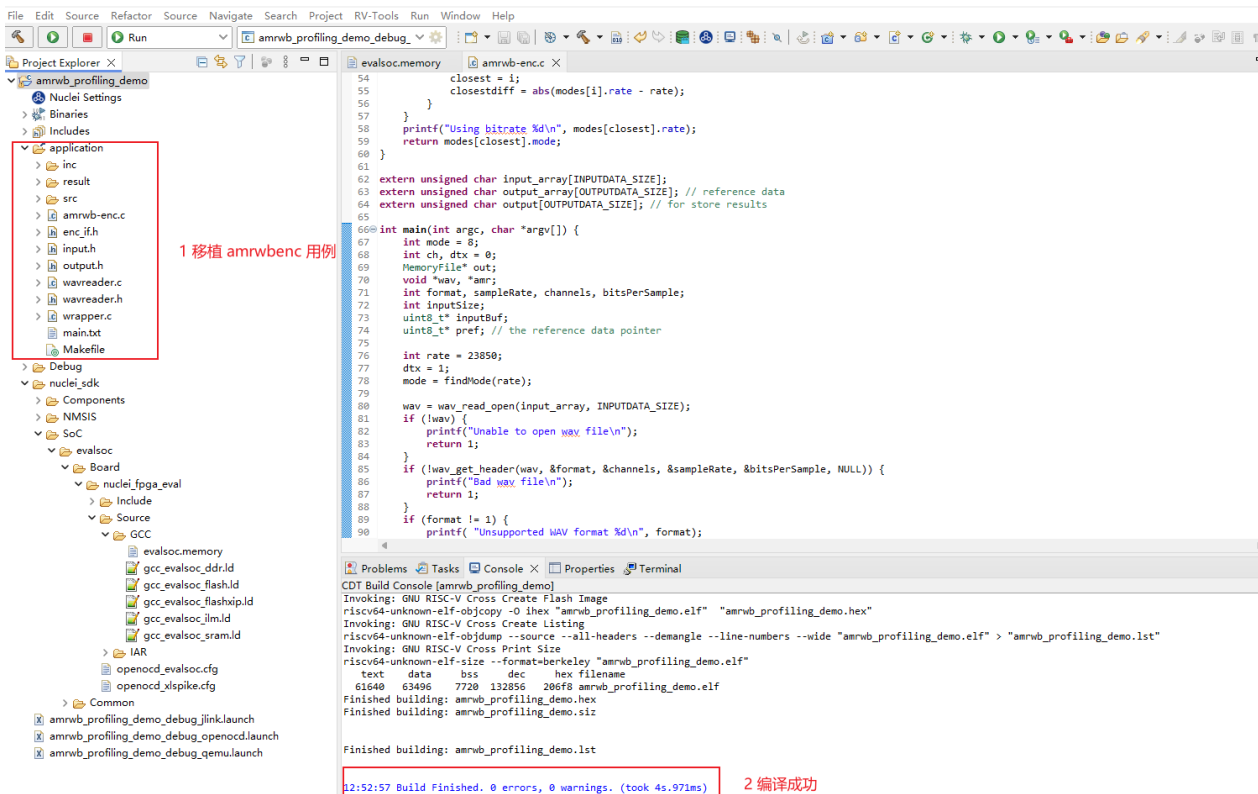
Step 2: Port the amrwbenc bare-metal use case based on the Profiling demo project

Delete the original use case in the application folder of the Profiling demo project and replace it with the amrwbenc use case, forming the following directory structure, and make sure it compiles successfully.

The project used in this example is provided here; feel free to download and use it:

[Download link for the project before optimization](#)

After downloading the zip package, you can import it directly into Nuclei Studio and run it (import steps: File->Import->Existing Projects into Workspace->Next->Select archive file->select the zip archive->next)



Note: When using Nuclei Studio in a Linux environment to import an old use case package, an error may occur (evalsoc.memory not found). This is caused by Windows path separators mixed into the Linux environment. This issue was fixed on 2026-02-09. The use case link in this document has been updated; you can re-download the new use case package, or directly fix the incorrect path. For details, refer to [Possible Issues with the Profiling and Code coverage Features](#), Issue 4

Step 3: Add gprof data collection code at the end of the use case, add the -pg compiler option, and recompile the code

Add the gprof data collection code at the end of the main function:

```

int main(int argc, char *argv[]) {
    /*
     * Code omitted
     */

    /*
     * Add gprof data collection code at the end of the main
    function
     */
    // TODO this is used for collect gprof and gcov data
    // See Components/profiling/README.md about how to set the IDE
    project properties
    extern long gprof_collect(unsigned long interface);
    gprof_collect(2);
}

```

```
    return 0;
}
```

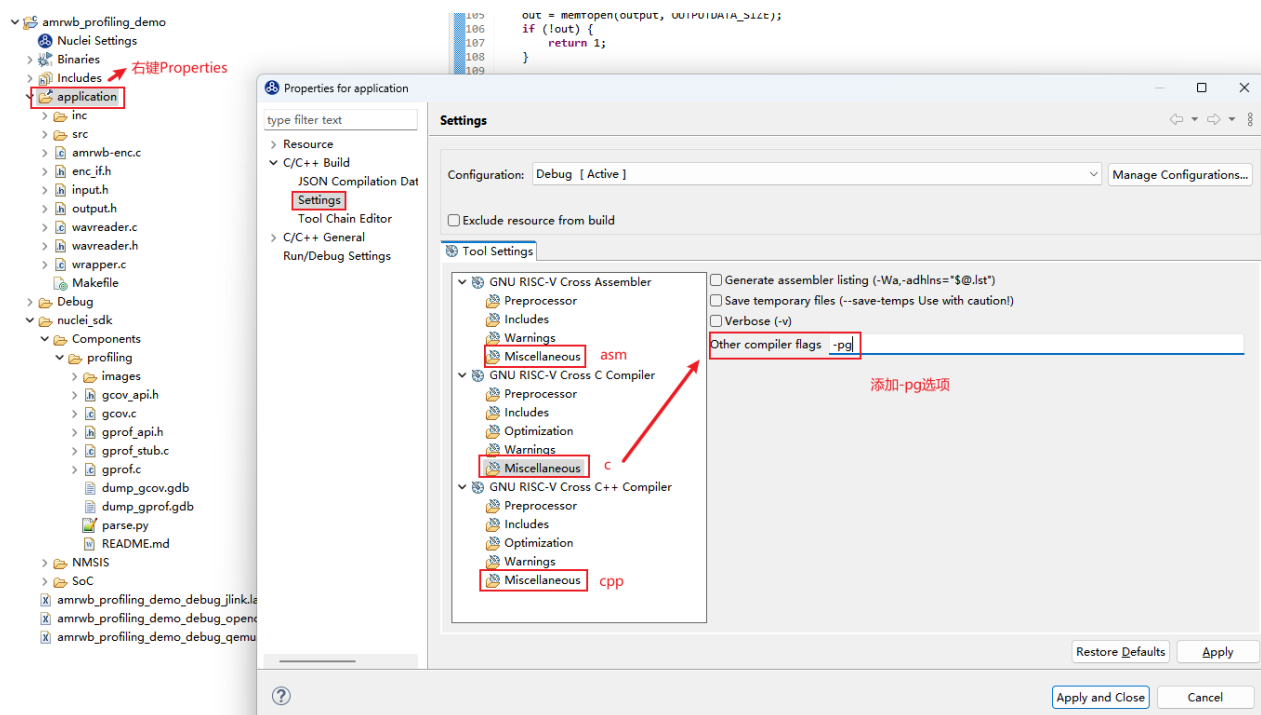
There are three ways to collect gprof data, distinguished by different input parameters:

- gprof_collect(0): Collects gprof or gcov data in a buffer; while debugging the program, a GDB script can be used to dump the gcov or gprof binary file
- gprof_collect(1): Uses semihost to write the gprof or gcov data directly into a file
- gprof_collect(2): Prints the gcov or gprof data directly to the Console or Serial Terminal; the data can then be parsed and saved to the PC using the Parse and Generate HexDump feature in the IDE

For details, refer to [Profiling README](#). Here, printing the gprof data to the serial port (Console or Serial Terminal) is used as an example.

Add the -pg compiler option to the code that needs profiling, then recompile the code:

Note: Select application and add the -pg compiler option to the key code. This use case contains only C code, so the -pg compiler option only needs to be added to the C code



Step 4: Run the program

There are several ways to run the program:

- qemu emulator (no hardware required; a quick run-through of the flow, but the test results are not accurate)
- On-board testing (data collected based on a timer)

- Based on xl_cpumodel (Nuclei Near Cycle Model), see: [Demonstrating Nuclei Model NICE/VNICE Instruction Acceleration Through Profiling](#)

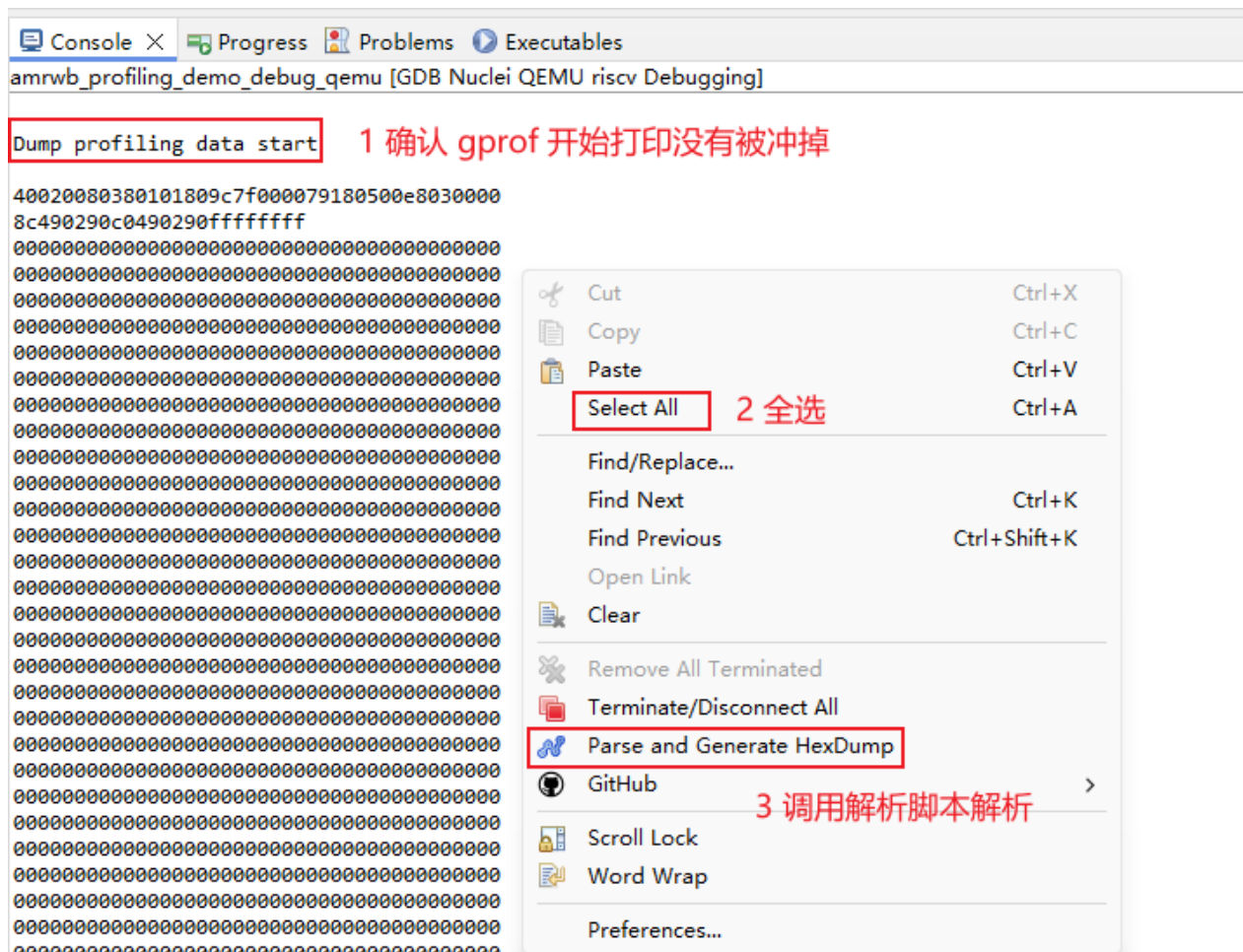
This article only covers two methods: qemu simulation and on-board testing. The data collected by qemu is printed to the Console, while the actual on-board run outputs to the Serial Terminal of Nuclei Studio.

Step 5: Parse the gprof data

Start parsing the gprof data. Note: Some issues may be encountered in this step; for solutions, refer to [Possible Issues with the Profiling and Code coverage Features](#)

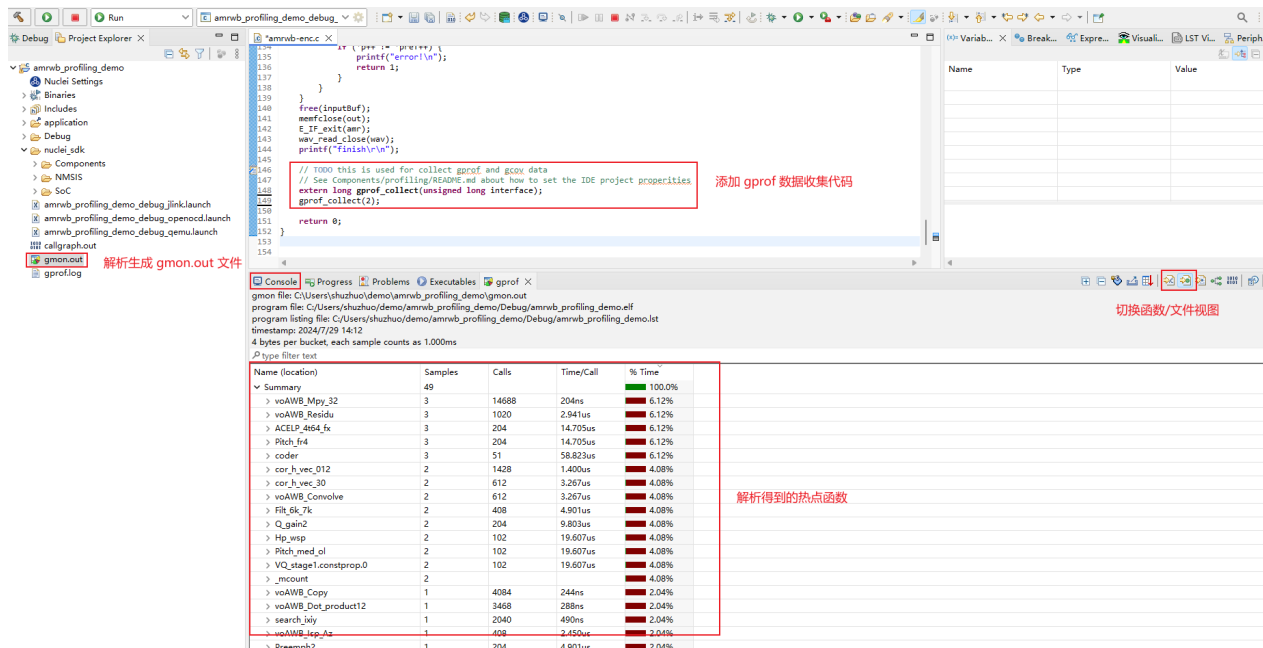
- Testing on qemu, with logs printed to the Console

Note: qemu is only used for demonstration purposes. If accurate hotspot functions are desired, on-board testing is required.



After parsing is complete, a gmon.out file is generated in the current project directory. Double-

click to open it:

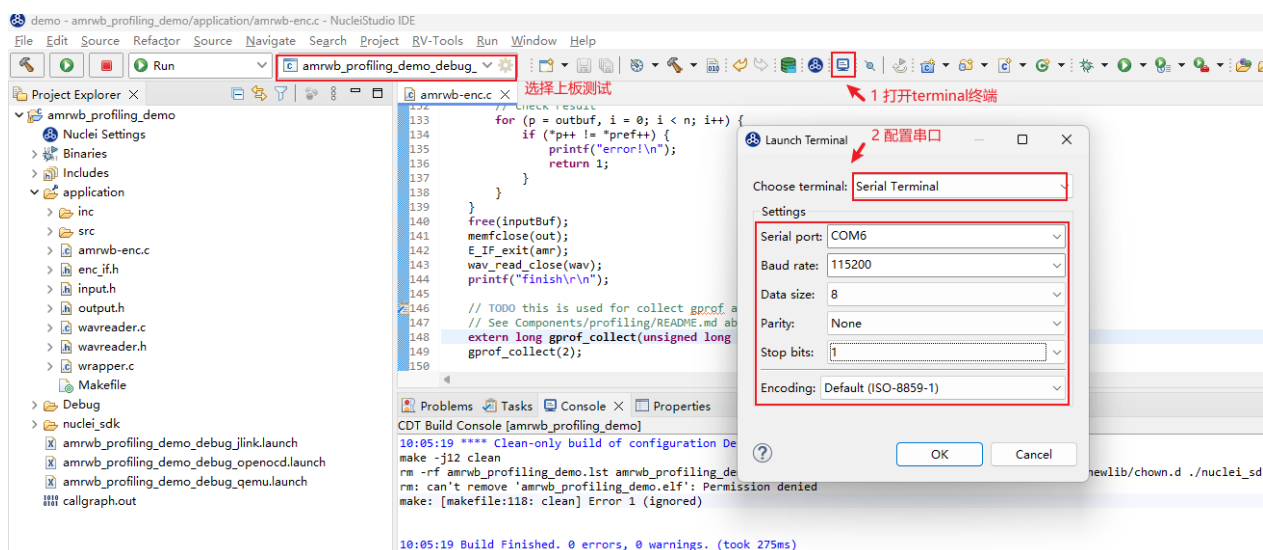


• On-board testing

The steps for on-board testing are similar to those for qemu; the only difference is that the gprof data is output to the Serial Terminal.

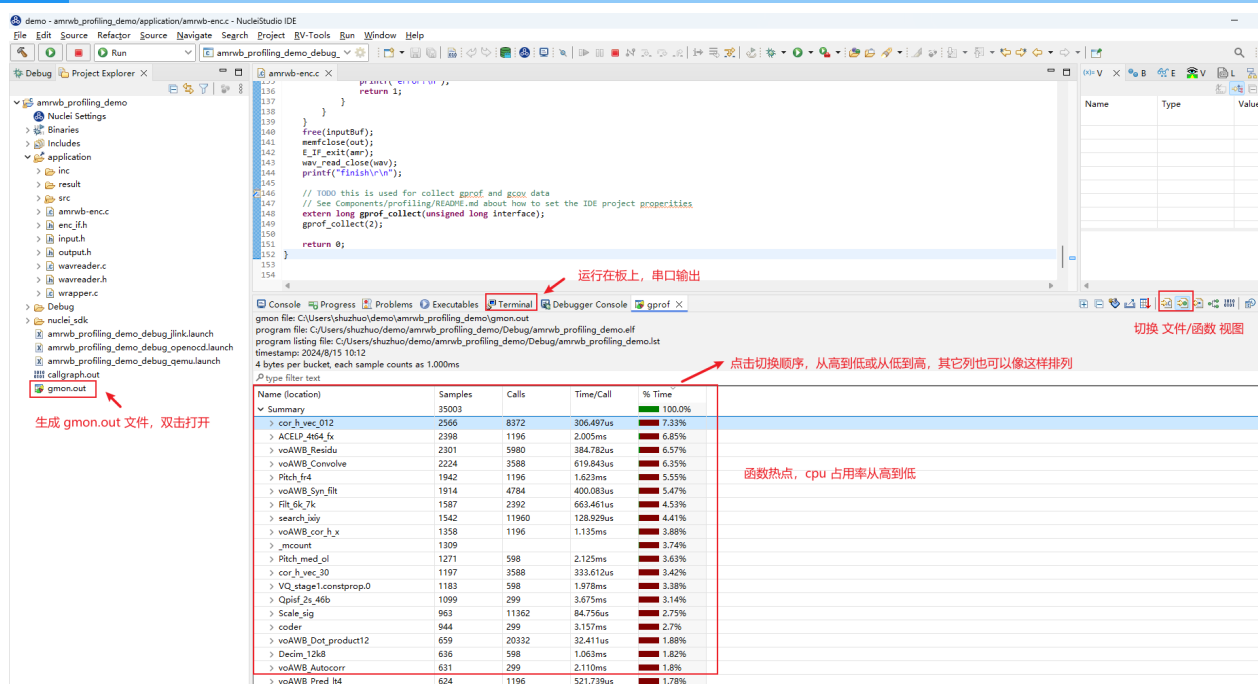
Configure the Serial Terminal:

Note: If the serial terminal tool is already open, make sure to clear the serial output before each gprof run (right-click -> Clear Terminal) to avoid affecting data parsing.



Similarly, select all the logs, right-click and choose the Parse and Generate HexDump feature, and a gmon.out file will be generated in the project folder. After refreshing the project, you can double-click to open this gmon.out file.

The figure below shows the gprof data obtained from an actual run on the board:



The TOP5 hotspot functions thus obtained are (actual on-board test):

```
cor_h_vec_012
ACELP_4t64_fx
voAWB_Residu
voAWB_Convolve
voAWB_Syn_filt
```

Once the hotspot functions are identified, optimization can begin from them; optimizing the TOP functions often achieves twice the result with half the effort.

Step 6: Optimize the hotspot functions

There are several methods for optimizing hotspot functions:

- Adjust compiler parameters: use optimization levels such as O2/O3/Ofast for the entire project or for individual operators, and enable optimization options such as `-finline-functions` and `-funroll-all-loops`
- Optimize at the algorithm level: implement the hotspot functions with better algorithms
- Optimize using RISC-V extension instructions (RVP/RVV extensions, etc.)

Here, the RVP extension is used as an example. The hotspot functions are optimized with the RVP extension in descending order of hotness. Make sure the hardware in use supports the RVP extension.

Example:

The TOP1 hotspot function is `cor_h_vec_012`. Analyze the function and try to optimize it using the RVP extension:

In the code below, the sections delimited by `#if defined __riscv_xldspn3x` represent code optimized with Nuclei N3 P extension instructions. `__RV_DSMALDA` is a Nuclei N3 P extension instruction that performs 4 int16 multiplications in one pass, accumulates the results, and stores the result in an int64 variable.

For the Intrinsic APIs of these instructions, refer to [Nuclei P Extension Instruction Intrinsic API](#)

For the detailed RVP instruction manual, please contact Nuclei System Technology.

The optimized project is shown below. Compared with the project before optimization, only the `cor_h_vec_012` operator was optimized:

[Download link for the optimized project](#)

The code snippet optimized with Nuclei N3 P extension instructions is as follows:

```
void cor_h_vec_012(
    Word16
    h[], /* (i) scaled impulse
    response */
    Word16 vec[], /* (i) scaled vector
    (/8) to correlate with h[] */
    Word16 track, /* (i) track to
    use */
    Word16 sign[], /* (i) sign
    vector */
    Word16 rrixix[]
    [NB_POS], /* (i) correlation of h[x] with h[x] */
    Word16 cor_1[], /* (o) result of
    correlation (NB_POS elements) */
    Word16 cor_2[] /* (o) result of
    correlation (NB_POS elements) */
)
{
    Word32 i, j, pos, corr;
    Word16 *p0, *p1, *p2, *p3, *cor_x, *cor_y;
    Word32 L_sum1, L_sum2;
    cor_x = cor_1;
    cor_y = cor_2;
    p0 = rrixix[track];
    p3 = rrixix[track+1];
    pos = track;

    for (i = 0; i < NB_POS; i+=2)
    {
        p1 = h;
        p2 = &vec[pos];
```

```

#if defined __riscv_xxldspn3x
    Word32 tmp1, tmp2;
    int64_t sum64_1, sum64_2;
    int64_t p64_1, p64_2;
    sum64_1 = 0;
    sum64_2 = 0;
    for (j=62-pos ;(j - 4) >= 0; j -= 4)
    {
        p64_1 = *__SIMD64(p1)++;
        tmp1 = __RV_PKBB16(*(p2 + 1), *p2);
        tmp2 = __RV_PKBB16(*(p2 + 3), *(p2 + 2));
        p64_2 = __RV_DPACK32(tmp2, tmp1);
        sum64_1 = __RV_DSMALDA(sum64_1, p64_1, p64_2);

        tmp1 = __RV_PKBB16(*(p2 + 2), *(p2 + 1));
        tmp2 = __RV_PKBB16(*(p2 + 4), *(p2 + 3));
        p64_2 = __RV_DPACK32(tmp2, tmp1);
        sum64_2 = __RV_DSMALDA(sum64_2, p64_1, p64_2);
        p2 += 4;
    }
    L_sum1 = (Word32)sum64_1;
    L_sum2 = (Word32)sum64_2;
    for ( ;j >= 0; j--)
    {
        L_sum1 += *p1 * *p2++;
        L_sum2 += *p1++ * *p2;
    }
#endif

    L_sum1 += *p1 * *p2;
    L_sum1 = (L_sum1 << 2);
    L_sum2 = (L_sum2 << 2);

    corr = (L_sum1 + 0x8000) >> 16;
    cor_x[i] = vo_mult(corr, sign[pos]) + (*p0++);
    corr = (L_sum2 + 0x8000) >> 16;
    cor_y[i] = vo_mult(corr, sign[pos + 1]) + (*p3++);
    pos += STEP;

    p1 = h;
    p2 = &vec[pos];
#if defined __riscv_xxldspn3x
    sum64_1 = 0;
    sum64_2 = 0;
    for (j=62-pos ;(j - 4) >= 0; j -= 4)
    {
        p64_1 = *__SIMD64(p1)++;
        tmp1 = __RV_PKBB16(*(p2 + 1), *p2);

```

```

    tmp2 = __RV_PKBB16(*(p2 + 3), *(p2 + 2));
    p64_2 = __RV_DPACK32(tmp2, tmp1);
    sum64_1 = __RV_DSMALDA(sum64_1, p64_1, p64_2);

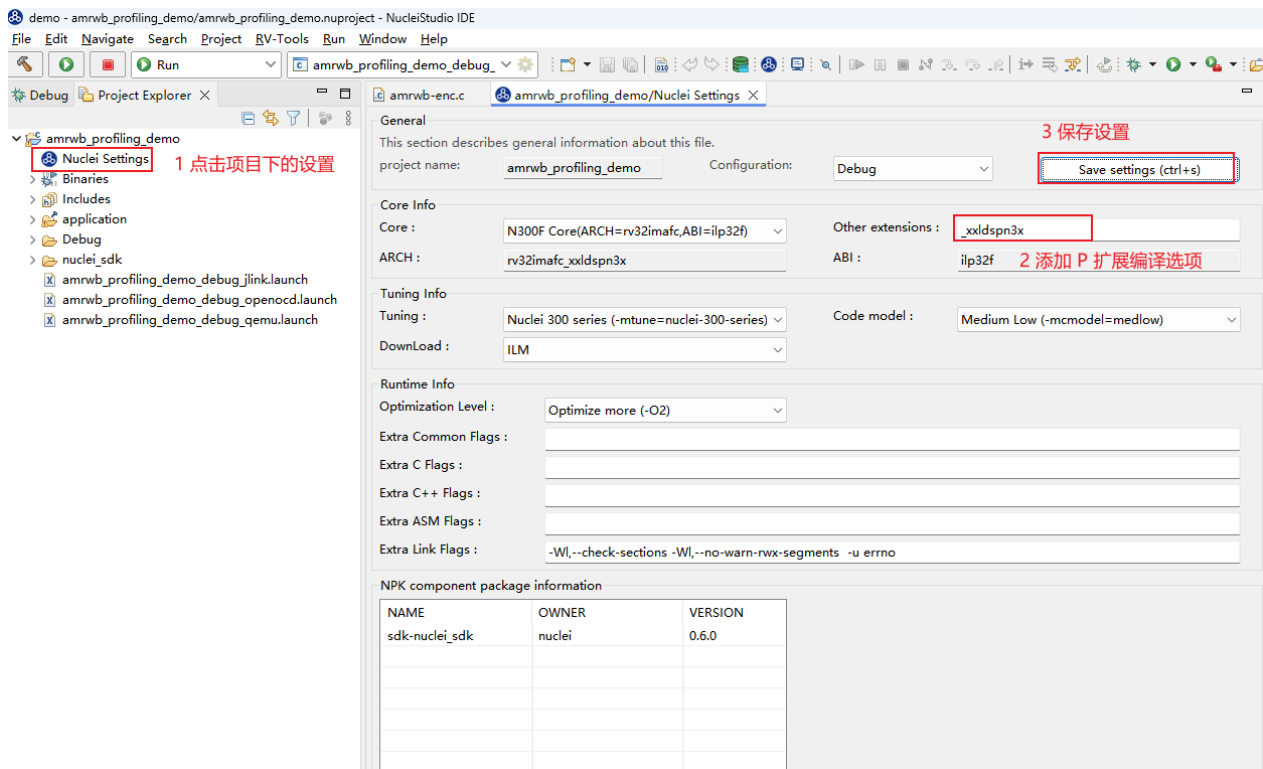
    tmp1 = __RV_PKBB16(*(p2 + 2), *(p2 + 1));
    tmp2 = __RV_PKBB16(*(p2 + 4), *(p2 + 3));
    p64_2 = __RV_DPACK32(tmp2, tmp1);
    sum64_2 = __RV_DSMALDA(sum64_2, p64_1, p64_2);
    p2 += 4;
}
L_sum1 = (Word32)sum64_1;
L_sum2 = (Word32)sum64_2;
for ( ;j >= 0; j--)
{
    L_sum1 += *p1 * *p2++;
    L_sum2 += *p1++ * *p2;
}
#endif

L_sum1 += *p1 * *p2;
L_sum1 = (L_sum1 << 2);
L_sum2 = (L_sum2 << 2);

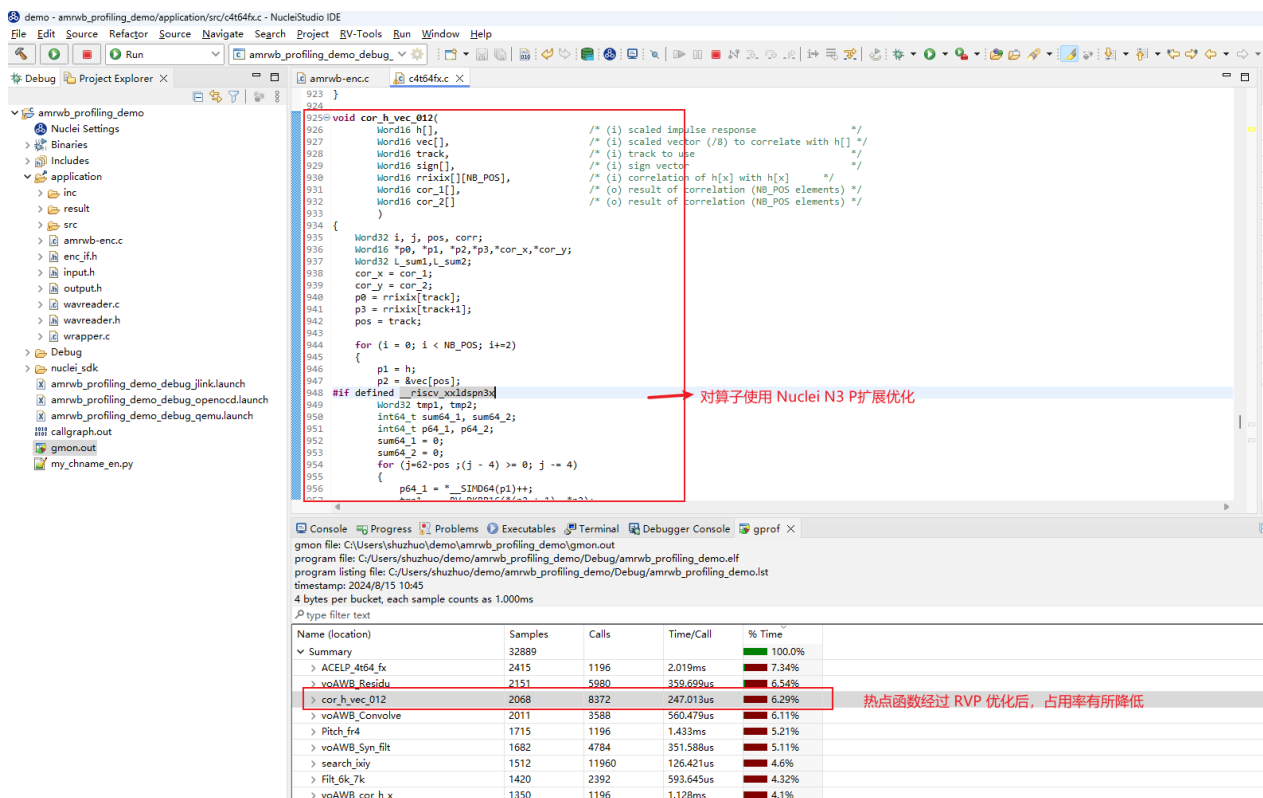
corr = (L_sum1 + 0x8000) >> 16;
cor_x[i+1] = vo_mult(corr, sign[pos]) + (*p0++);
corr = (L_sum2 + 0x8000) >> 16;
cor_y[i+1] = vo_mult(corr, sign[pos + 1]) + (*p3++);
pos += STEP;
}
return;
}

```

After optimizing this operator with the P extension, make sure to compile with the dsp extension option enabled, as shown in the figure below:



Clean the project and recompile, then run profiling again. You can see the optimization effect: the occupancy of the `cor_h_vec_012` function has decreased, and the function execution time has also been reduced.

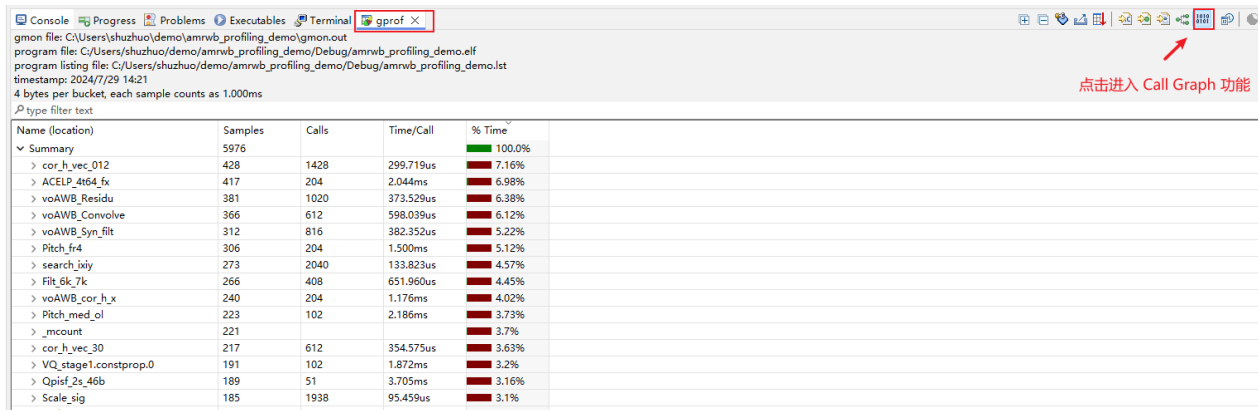


Note: The above is only a simple example. Users can analyze and optimize the hotspot functions one by one. Due to sampling and other factors during execution, the distribution of TOP

functions may fluctuate, which is normal. The final precise analysis requires counting the total number of cycles and then calculating the improvement ratio.

3 Call Graph Feature

The Call Graph in Nuclei Studio mainly obtains the function call relationships in the program by analyzing Profiling data.

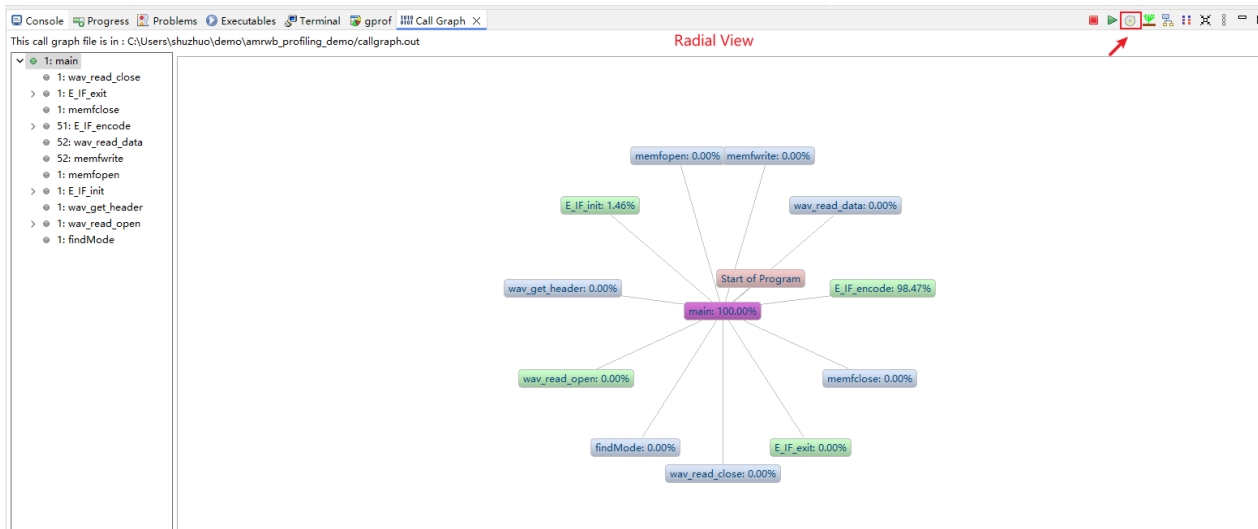


Name (location)	Samples	Calls	Time/Call	% Time
Summary	5976			100.0%
> cor_h_vec_012	428	1428	299.719us	7.16%
> ACELP_4t64_fx	417	204	2.044ms	6.98%
> voAWB_Residu	381	1020	373.529us	6.38%
> voAWB_Convolve	366	612	598.039us	6.12%
> voAWB_Syn_filt	312	816	382.352us	5.22%
> Pitch_fr4	306	204	1.500ms	5.12%
> search_iviv	273	2040	133.823us	4.57%
> Flt_6k_7k	266	408	651.960us	4.45%
> voAWB_cor_h_x	240	204	1.176ms	4.02%
> Pitch_med_ol	223	102	2.186ms	3.73%
> _mcount	221			3.7%
> cor_h_vec_30	217	612	354.575us	3.63%
> VQ_stage1.constprop.0	191	102	1.872ms	3.2%
> Qpist_2s_46b	189	51	3.705ms	3.16%
> Scale_sig	185	1938	95.459us	3.1%

The Call Graph feature includes the following views:

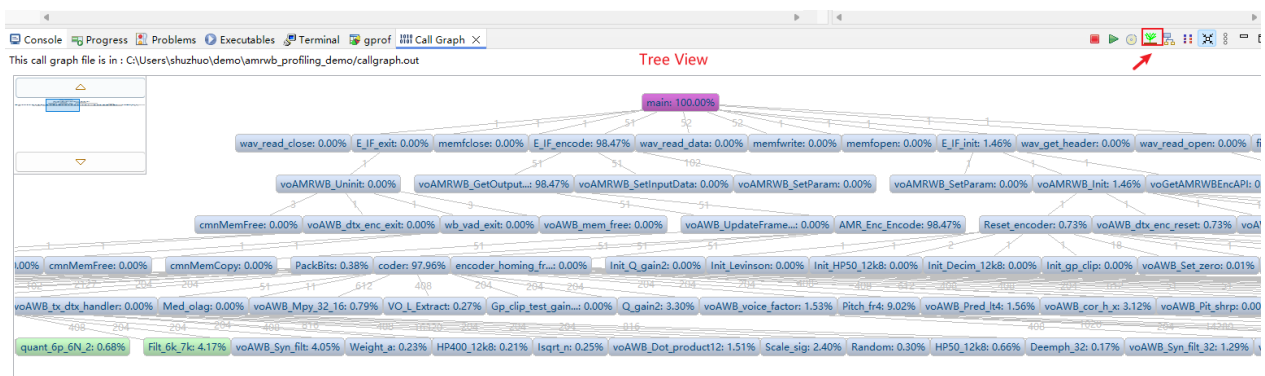
- Radial View

This view displays the call relationships of the program.



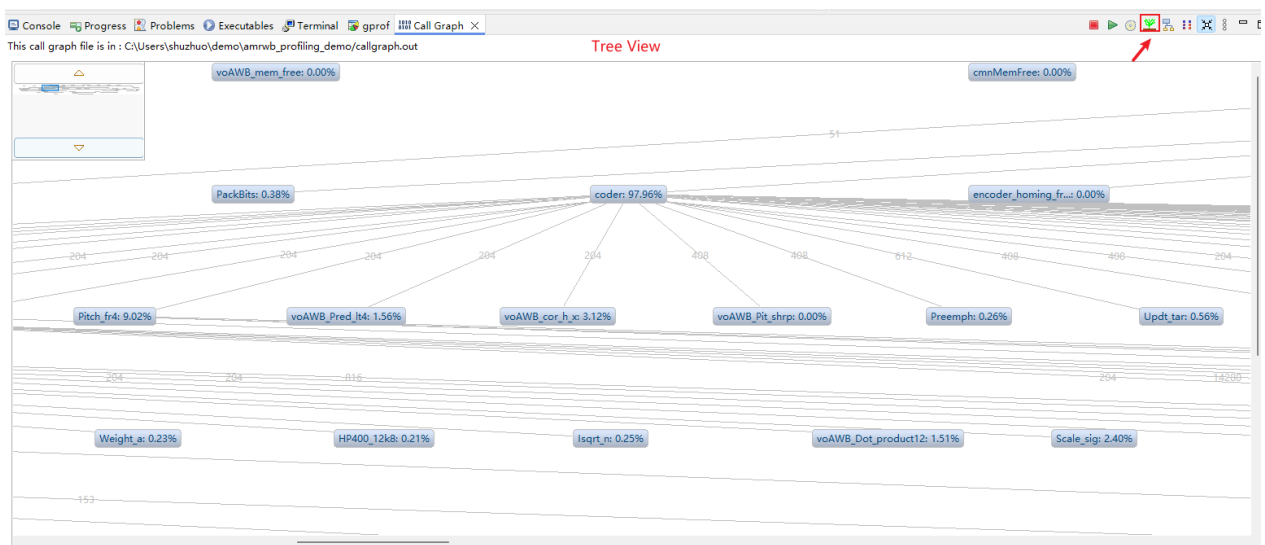
- Tree View

It displays the call relationships, time consumption ratios, call counts, and other information of the program selected in the Radial View. Selecting a function allows you to view its parent nodes, child nodes, and other information.



- Level View

Similar to the Tree View, it displays the call relationships and call counts of the program.



- Aggregate View

It presents the program's time consumption relationships very intuitively in the form of a block diagram.



4 Code coverage Feature

The Code coverage feature in Nuclei Studio is based on the gcov tool provided by the gcc compiler. During compilation, the specified source files must be compiled with the specific compiler option `-coverage`. After a successful build, an ELF file is obtained, which is then run on the actual development board to collect the required coverage files (gcda/gcno files), and finally the results are displayed graphically in the IDE.

The usage is similar to the Profiling feature; only the differences are explained here:

Step 1: Create a new Profiling demo project

Step 2: Port the amrwbench bare-metal use case based on the Profiling demo project

Step 3: Add gcov data collection code, add the `-coverage` compiler option, and recompile the code

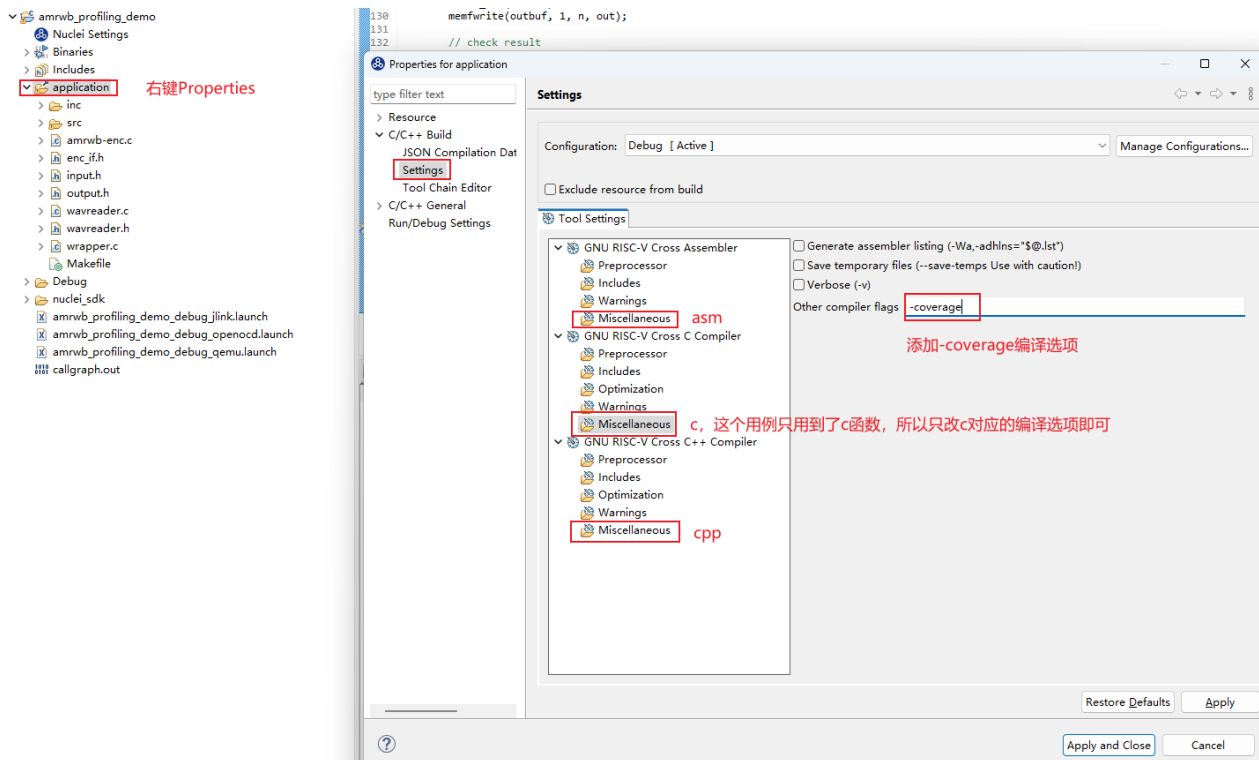
Add the gcov data collection code at the end of the main function:

```
int main(int argc, char *argv[]) {
    /*
     * Code omitted
     */

    /*
     * Add gcov data collection code at the end of the main
    function
     */
    // TODO this is used for collect gprof and gcov data
    // See Components/profiling/README.md about how to set the IDE
    project properties
    extern long gcov_collect(unsigned long interface);
    gcov_collect(2);

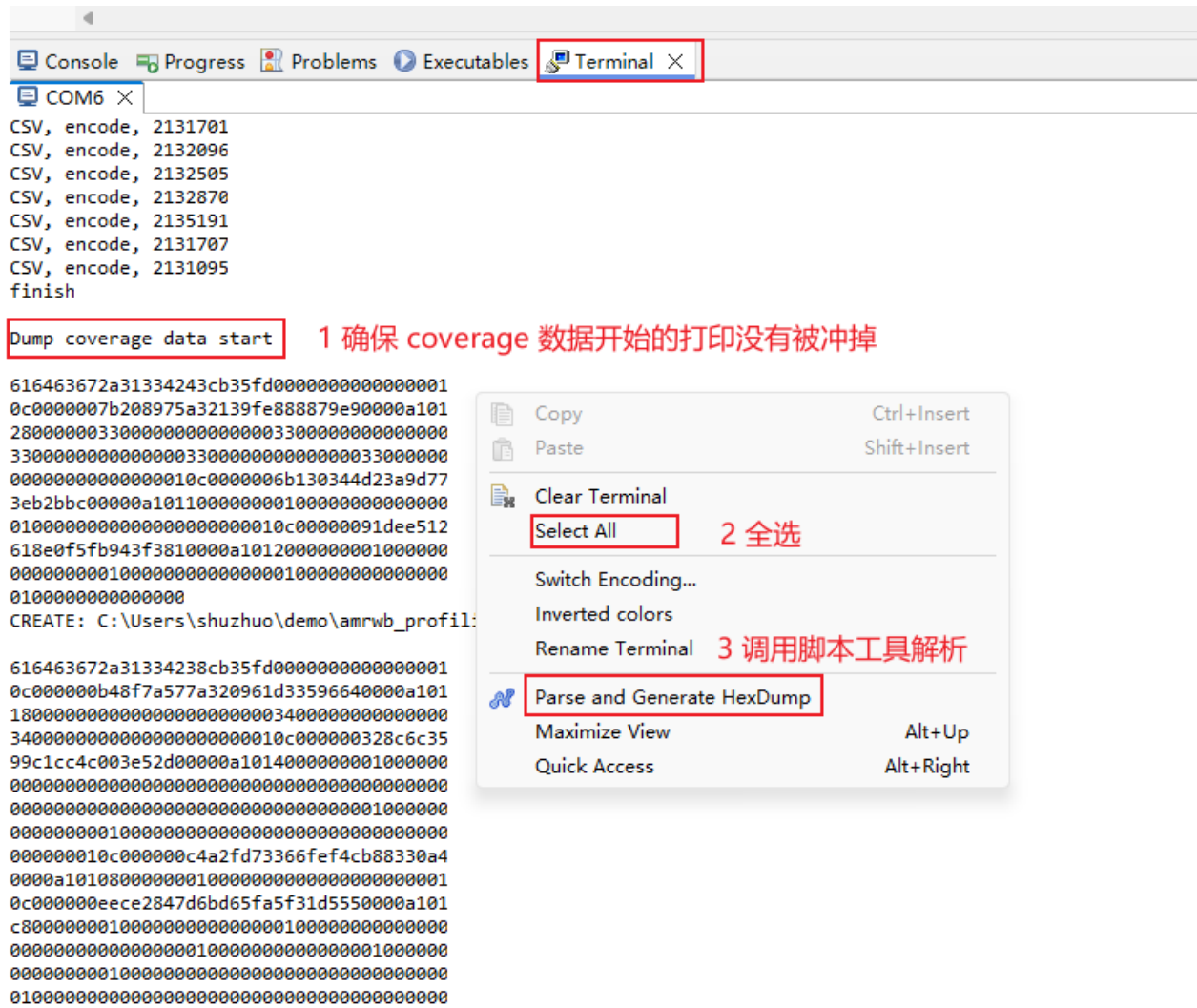
    return 0;
}
```

Add the `-coverage` compiler option and recompile the code:



Step 4: Run the program

You can run it in the qemu simulator or on the actual board (coverage statistics do not involve performance analysis, so either qemu or on-board testing works).



After parsing, gcda and gcno files are generated in the Debug->application folder; double-click to open them

The screenshot displays the Nuclei Studio IDE with the `amrwb-enc.c` file open. The code is annotated with green and red highlights to indicate execution status. A red box highlights a section of code, and a green box highlights another section. A red box around the `amrwb-enc.gcov` file in the Project Explorer is labeled "双击打开" (Double-click to open). The bottom panel shows the "gcov" view, displaying a table of coverage data for various files and functions.

绿色表示执行到了, 红色表示没有执行到

双击打开

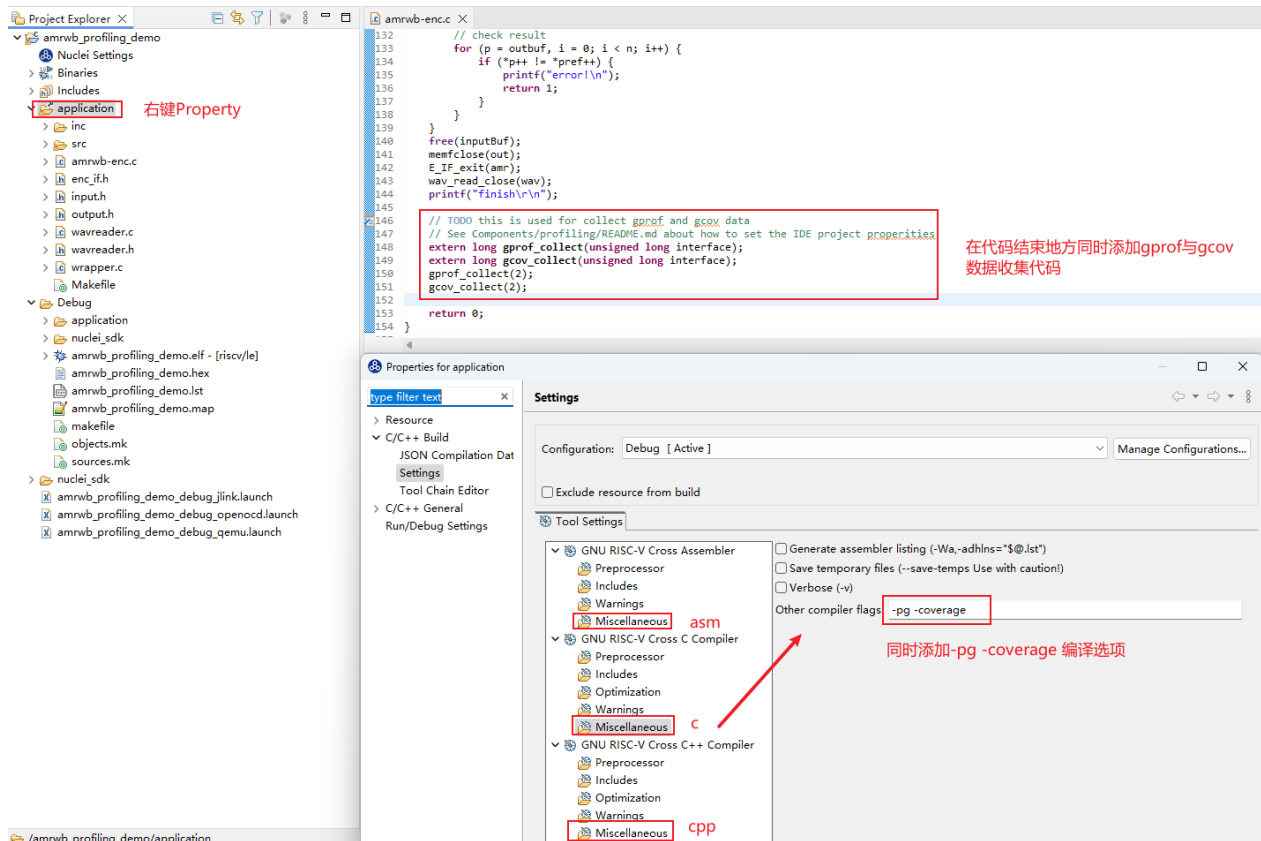
覆盖率数据

Name	Total Lines	Instrumented...	Executed Lines	Coverage %
Summary	13,285	4,516	3,630	80.38%
amrwb-enc	152	77	63	81.82%
autocorr.c	159	58	58	100.0%
az_jsp.c	265	96	94	97.92%
basic_op.h	1,174	145	122	84.14%
bits.c	206	77	43	55.84%
c2t64f.c	294	119	0	0.0%
c4t64f.c	1,067	527	407	77.23%
cmnMemo	70	20	13	65.0%
convolve.c	193	59	59	100.0%
cor_h_x.c	124	56	56	100.0%
core_featu	724	9	8	88.89%
decim54.c	144	56	56	100.0%
deemph.c	124	43	31	72.09%
dtcc	605	215	34	15.81%
g_pitch.c	72	16	16	100.0%
gpclip.c	107	31	26	83.87%
homing.c	45	7	7	100.0%
hp400.c	103	38	38	100.0%
hp50.c	104	38	38	100.0%
hp6k.c	91	29	29	100.0%
ho_wso.c	146	64	64	100.0%

5 Additional Notes

1. The Profiling and Code coverage features can be enabled at the same time. Simply add code that collects both Profiling data and Code coverage data, and add the `-pg -coverage` compiler options when compiling.

```
// TODO this is used for collect gprof and gcov data
// See Components/profiling/README.md about how to set the IDE
project properties
extern long gprof_collect(unsigned long interface);
extern long gcov_collect(unsigned long interface);
gprof_collect(2);
gcov_collect(2);
```



1. Issues that may be encountered when using Profiling:
2. Insufficient on-chip memory, with error messages in the printed logs; gprof/gcov data requires a certain amount of space
3. Incomplete data collected from the Console or Terminal leads to incorrect parsing; make sure the data has not been overwritten, and adjust the output size limit of the Console or Terminal
4. After manually deleting the gmon.out file and parsing again, a "No files have been generated" error dialog pops up

For detailed solutions to the above issues, refer to [Possible Issues with the Profiling and Code coverage Features](#)

Demonstrate NICE/VNICE Instruction Acceleration of the Nuclei Model through Profiling¶

Nuclei Model is supported on both Windows and Linux. All tests in this document were performed with the Windows version of Nuclei Studio (≥ 2025.10).

Background¶

Nuclei Model Profiling¶

In [An Example to Demonstrate the Use of Profiling and Code Coverage in Nuclei Studio](#), two run modes — qemu and on-board testing — were used to show how to import a specific program into the IDE for Profiling. Part of this document introduces how to perform Profiling on the Nuclei Model.

Advantages of Nuclei Model Profiling:

- No development board or other hardware is required
- The model has a built-in gprof feature, so Profiling files can be generated without a Profiling library or the `gcc -pg` option
- It uses instruction-level sampling, enabling instruction-level Profiling analysis

The relevant chapters of [NucleiStudio_User_Guide.pdf](#) already describe in detail how to configure simulation performance analysis for the Nuclei Model. This document demonstrates its practical application with an example.

NICE/VNICE Custom Instruction Acceleration¶

NICE/VNICE allows users to extend custom instructions based on their own applications, extending Nuclei's standard processor cores into domain-specific processors. For the detailed encoding rules of NICE, refer to the NICE Introduction section in [Nuclei_RISC-V_ISA_Spec.pdf](#). NICE is suitable for custom instructions that do not use RISC-V Vector, while VNICE is suitable for custom instructions that do use RISC-V Vector.

[demo_nice/demo_vnice](#) introduce how Nuclei's NICE/VNICE demo applications are compiled and run. This document will adapt a more common AES encryption/decryption example, focusing on how to replace hotspot functions with NICE/VNICE instructions and how to implement NICE/VNICE instructions in the model, and then use Nuclei Studio's Profiling feature to analyze program performance before and after the replacement.

Solution

Environment Preparation

Nuclei Studio: [NucleiStudio 2025.10 Windows](#)

Model Profiling

There are two ways to create a project:

- Method 1: Use the `demo_nice` or `demo_vnice` template in Nuclei Studio to port and adapt your own NICE/VNICE program
- Method 2: Import your own project into Nuclei Studio, then add code such as the NICE inline assembly header file and NICE CSR enabling

This document uses the first method to create the project. Since this demo uses VNICE instructions, a `demo_vnice` project is created, and then the AES encryption/decryption program is ported into it.

step1: Create a new `demo_vnice` project

File->New->New Nuclei RISC-V C/C++ Project, select Nuclei FPGA Evaluation Board->sdk-nuclei_sdk @0.6.0

Note: Nuclei SDK version 0.6.0 or later is required

Create Nuclei RISC-V C/C++ project using npk sdk-nuclei_sdk @0.6.0

Create project for SoC:Nuclei FPGA Evaluation SoC,Board:Nuclei FPGA Evaluation Board

Please select the relevant configuration item

Project name: 1 工程名

Project Filter by: Filters: 2 使用关键字筛选Nuclei NICE

Project Example: 3 选择demo_vnice

Toolchain:

Download/Run Mode:

Select NMSIS Library:

Nuclei RISC-V Core:

ARCH Extensions(ARCH_EXT=):

Nuclei Cache Extensions: ☐ ICache ☐ DCache ☐ CCM

Nuclei SMP Count: - +

Boot HartID: - +

Heap Size:

Stack Size Per CPU:

Enable Semihosting: ☐

Standard C Library(STDCLIB=):

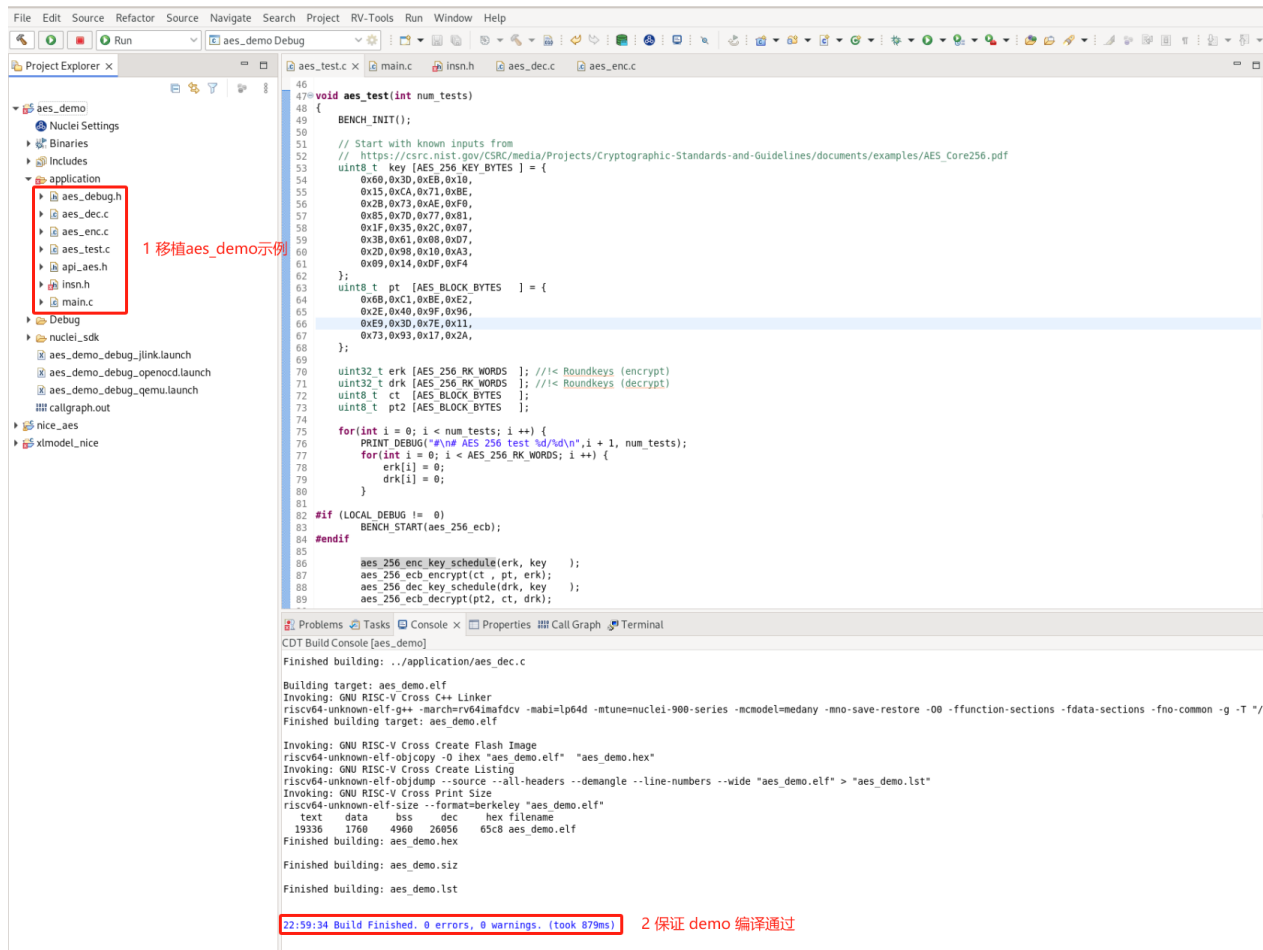
Application Compile Flags:

step2: Port the aes_demo bare-metal test case based on the demo_vnice project

When porting aes_demo, keep the insn.h inline assembly header file framework from demo_vnice so that custom NICE/VNICE instructions can be added later, and keep the CSR enabling code in main.c that runs before NICE/VNICE instructions are executed:

```
__RV_CSR_SET(CSR_MSTATUS, MSTATUS_XS);
```

The remaining original application test cases in the `demo_vnice` project can be deleted and replaced with the `aes_demo` test case, resulting in the following directory structure. Make sure it compiles successfully.



You can download our pre-ported AES encryption/decryption demo: [Download link for the AES project before optimization](#)

After downloading the zip package, you can import it directly into Nuclei Studio and run it (import steps: File->Import->Existing Projects into Workspace->Next->Select archive file->select the zip archive->next).

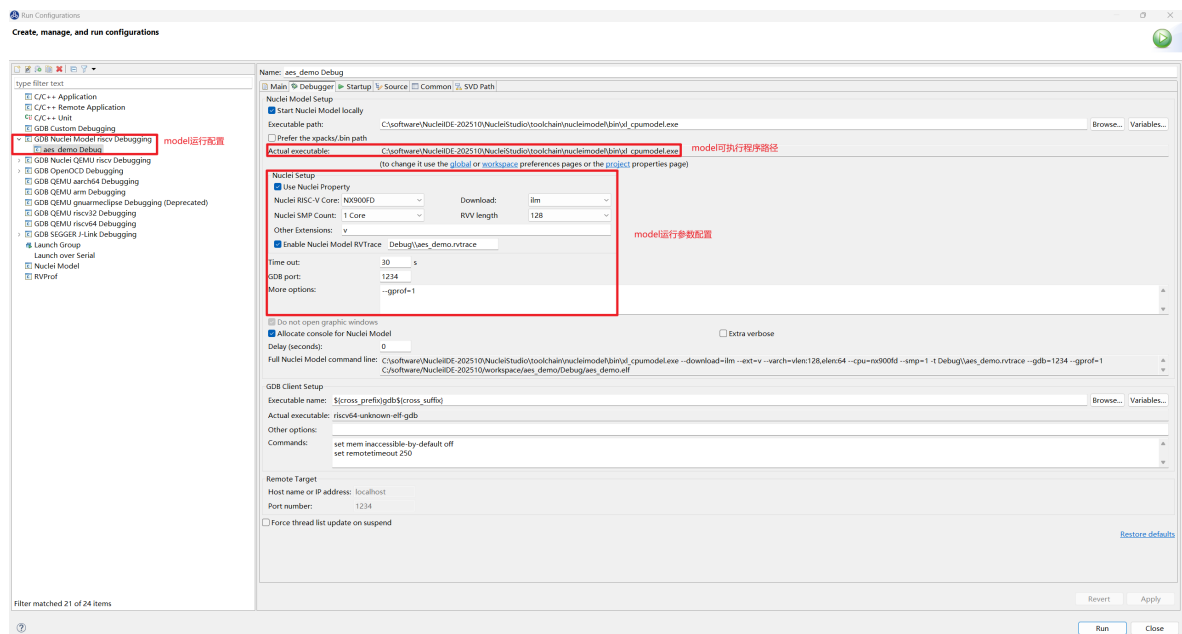
step3: Simulate the program with the model

First, enable `LOCAL_DEBUG` in `aes_debug.h` to prepare for measuring the overall cycle count of the AES algorithm.

To simulate the program with the Model, configure the GDB Nuclei Model riscv Debugging configuration in Nuclei Studio as follows:

1. Open Run Configurations from the Run option in the Nuclei Studio main menu bar

2. Select the GDB Nuclei Model riscv Debugging configuration, right-click and select New Configuration; a Model configuration page named after the project is automatically generated, and the launch bar is updated accordingly
3. In the Main tab on the right, click Search Project . . . and select the compiled elf file
4. In the Debugger tab on the right, select Browse and locate the default path of the Nuclei Model executable: NucleiStudio/toolchain/nucleimodel/bin/xl_cpumodel.exe
5. In the Debugger tab on the right, complete the model run configuration under Nuclei Setup: Nuclei RISC-V Core and Other Extensions must be consistent with the Core and Other extensions settings in Nuclei Settings. When Other Extensions is empty, this parameter is not passed. Enable Nuclei Model RVTrace means rvtrace is generated at runtime. Add -gprof=1 to More options to enable the Profiling feature. Then click Apply and Run, and the model starts running the program.



Nuclei Studio (< 2025.10) can only configure the model using Nuclei Model in Run Configurations; for Nuclei Studio (>= 2025.10), it is recommended to switch to GDB Nuclei Model riscv Debugging for configuration

When Total elapsed real time appears in the Console, the model has finished the simulation, showing that the AES algorithm consumes 161108 cycles in total.

step4: Parse the gprof data

After the model simulation completes, double-click the generated gprof*.gmon file to open it, switch to the function view, and click % Time to sort functions by CPU usage from high to low.

Note: Time/Call shows the cycle count of each function's own body in the text section, not the cycle count of the entire function; cycles consumed by its child functions are not included.

生成的 gmon 文件双击打开

切换到函数视图

函数被调用次数 函数 Time/Call 函数 cycle 数

点击切换到高到低排列

CPU 占用率

Name (location)	Samples	Calls	Time/Call	% Time
Summary	184413			100.0%
aes_mix_columns_dec	106727	13	8209 cycle	57.67%
aes_mix_columns_enc	14844	13	1141 cycle	8.05%
aes_key_schedule	9079	2	4539 cycle	4.92%
aes_ecb_decrypt	8210	1	8210 cycle	4.45%
aes_ecb_encrypt	8019	1	8019 cycle	4.39%
uart_write	4952	144	34 cycle	2.69%
putchar	3486	128	27 cycle	1.89%
aes_subbytes_shiftrows_dec	2968	14	212 cycle	1.61%
aes_subbytes_shiftrows	2954	14	211 cycle	1.6%
measure_cpu_freq	2258	2	1129 cycle	1.22%
aes_sub_word	2080	26	80 cycle	1.13%
_write	2066	5	413 cycle	1.12%
_vfprintf_r	1721	106	16 cycle	0.93%
_vfprintf_r	1555	19	81 cycle	0.84%
_vfprintf_r	1470	4	367 cycle	0.8%
aes_test	1389	1	1389 cycle	0.75%
rand	1204	48	25 cycle	0.65%
_jpmem_init	1122	1	1122 cycle	0.61%
memcpy	970	18	53 cycle	0.53%
_init_common	866			0.47%
_printf_i	816	10	81 cycle	0.44%

The TOP5 hotspot functions with the highest CPU usage are:

```

aes_mix_columns_dec
aes_mix_columns_enc
aes_key_schedule
aes_ecb_decrypt
aes_ecb_encrypt

```

Note: At this point, back up the current aes_demo project and rename the copy to aes_demo_nice. This way, both projects can be opened in Nuclei Studio at the same time, making it easy to compare the Profiling results of the project optimized with NICE/VNICE instructions against the original aes_demo project.

step5: Replace hotspot functions with NICE/VNICE instructions

In the backed-up aes_demo_nice project, the user needs to study the algorithmic characteristics of the hotspot functions and replace them with NICE/VNICE instructions to improve overall program performance.

Add `#include "insn.h"` to the two C files `aes_dec.c` and `aes_enc.c`, which contain the TOP5 hotspot functions of AES encryption/decryption, so that NICE/VNICE instruction replacements can be added.

The TOP1 hotspot function is `aes_mix_columns_dec`, which implements the inverse MixColumns of AES decryption. It takes a state matrix as input, performs the computation, and outputs the computed state matrix to the same address. It implements Load data, the inverse mix computation, and Store data. The code is as follows:

```
static void aes_mix_columns_dec(
    uint8_t    pt[16]          //!< Current block state
){
    // Col 0
    for(int i = 0; i < 4; i ++){
        uint8_t b0,b1,b2,b3;
        uint8_t s0,s1,s2,s3;

        s0 = pt[4*i+0];
        s1 = pt[4*i+1];
        s2 = pt[4*i+2];
        s3 = pt[4*i+3];

        b0 = XTE(s0) ^ XTB(s1) ^ XTD(s2) ^ XT9(s3);
        b1 = XT9(s0) ^ XTE(s1) ^ XTB(s2) ^ XTD(s3);
        b2 = XTD(s0) ^ XT9(s1) ^ XTE(s2) ^ XTB(s3);
        b3 = XTB(s0) ^ XTD(s1) ^ XT9(s2) ^ XTE(s3);

        pt[4*i+0] = b0;
        pt[4*i+1] = b1;
        pt[4*i+2] = b2;
        pt[4*i+3] = b3;
    }
}
```

Since the input and output addresses are the same, consider replacing it with a single NICE instruction. The opcode, funct3, and funct7 of the instruction can all be customized within the encoding fields. This instruction sets opcode to Custom-0, funct3 to 0, and funct7 to 0x10. Only the rs1 register is used to describe the input parameter address; rd and rs2 are not needed. The instruction is written into `insn.h` with the following inline assembly:

```
__STATIC_FORCEINLINE void custom_aes_mix_columns_dec(uint8_t* addr)
{
    int zero = 0;
    asm volatile(".insn r 0xb, 0, 0x10, x0, %1, x0" : "=r"(zero) :
```



```
"r"(addr));
}
```

The user can define a `USE_NICE` macro in `insn.h` to select whether to use NICE, and rewrite `aes_mix_columns_dec` in `aes_dec.c` as follows:

```
static void aes_mix_columns_dec(
    uint8_t    pt[16]        //!< Current block state
){

#ifdef USE_NICE
    custom_aes_mix_columns_dec(pt);
#else
    // Col 0
    for(int i = 0; i < 4; i++) {
        uint8_t b0,b1,b2,b3;
        uint8_t s0,s1,s2,s3;

        s0 = pt[4*i+0];
        s1 = pt[4*i+1];
        s2 = pt[4*i+2];
        s3 = pt[4*i+3];

        b0 = XTE(s0) ^ XTB(s1) ^ XTD(s2) ^ XT9(s3);
        b1 = XT9(s0) ^ XTE(s1) ^ XTB(s2) ^ XTD(s3);
        b2 = XTD(s0) ^ XT9(s1) ^ XTE(s2) ^ XTB(s3);
        b3 = XTB(s0) ^ XTD(s1) ^ XT9(s2) ^ XTE(s3);

        pt[4*i+0] = b0;
        pt[4*i+1] = b1;
        pt[4*i+2] = b2;
        pt[4*i+3] = b3;
    }
#endif
}
```

The TOP2 hotspot function is `aes_mix_columns_enc`, which, similar to TOP1, implements the MixColumns of AES encryption. It likewise takes a state matrix as input, performs the computation, and outputs the computed state matrix to the same address:

```
static void aes_mix_columns_enc(
    uint8_t    ct [16]        //!< Current block state
){
    for(int i = 0; i < 4; i++) {
        uint8_t b0,b1,b2,b3;
```

```

uint8_t s0,s1,s2,s3;

s0 = ct[4*i+0];
s1 = ct[4*i+1];
s2 = ct[4*i+2];
s3 = ct[4*i+3];

b0 = XT2(s0) ^ XT3(s1) ^ (s2) ^ (s3);
b1 = (s0) ^ XT2(s1) ^ XT3(s2) ^ (s3);
b2 = (s0) ^ (s1) ^ XT2(s2) ^ XT3(s3);
b3 = XT3(s0) ^ (s1) ^ (s2) ^ XT2(s3);

ct[4*i+0] = b0;
ct[4*i+1] = b1;
ct[4*i+2] = b2;
ct[4*i+3] = b3;
}
}

```

Considering that the instruction implementation may not be achievable with a single instruction, two VNICE instructions can be used to replace this algorithm: the first one loads 16 bytes of data into a Vector register, and the second one performs the computation and the store.

The opcode, funct3, and funct7 of the instructions can still be customized within the encoding fields. The first instruction uses rd to describe the Vector register and rs1 to describe the input parameter address; the second instruction uses rs1 to describe the input parameter address and rs2 to describe the input Vector register. The inline assembly for the two VNICE instructions is written into `insn.h` and defined as follows:

```

__STATIC_FORCEINLINE vint8m1_t __custom_vnice_load_v_i8m1
(uint8_t* addr)
{
    vint8m1_t rdata ;
    asm volatile(".insn r 0xb,4,0,%0,%1,x0"
                 : "=vr"(rdata)
                 : "r"(addr)
                 );
    return rdata;
}

__STATIC_FORCEINLINE void __custom_vnice_aes_mix_columns_enc_i8m1
(uint8_t *addr, vint8m1_t data)
{
    int zero = 0;
    asm volatile(".insn r 0xb,4,1,x0,%1,%2"
                 : "=r"(zero)

```

```

        : "r"(addr)
        , "vr"(data)
        );
}

```

The user rewrites `aes_mix_columns_enc` in `aes_enc.c` by defining a Vector register and using the VNICE instruction inline assembly defined above, as follows:

```

static void aes_mix_columns_enc(
    uint8_t      ct [16]          //!< Current block state
){
#ifdef USE_NICE
    uint32_t blkCnt = 16;
    size_t l;
    vint8m1_t vin;
    for (; (l = __riscv_vsetvl_e8m1(blkCnt)) > 0; blkCnt -= l) {
        vin = __custom_vnice_load_v_i8m1(ct);
        __custom_vnice_aes_mix_columns_enc_i8m1(ct, vin);
    }
#else
    for(int i = 0; i < 4; i++) {
        uint8_t b0,b1,b2,b3;
        uint8_t s0,s1,s2,s3;

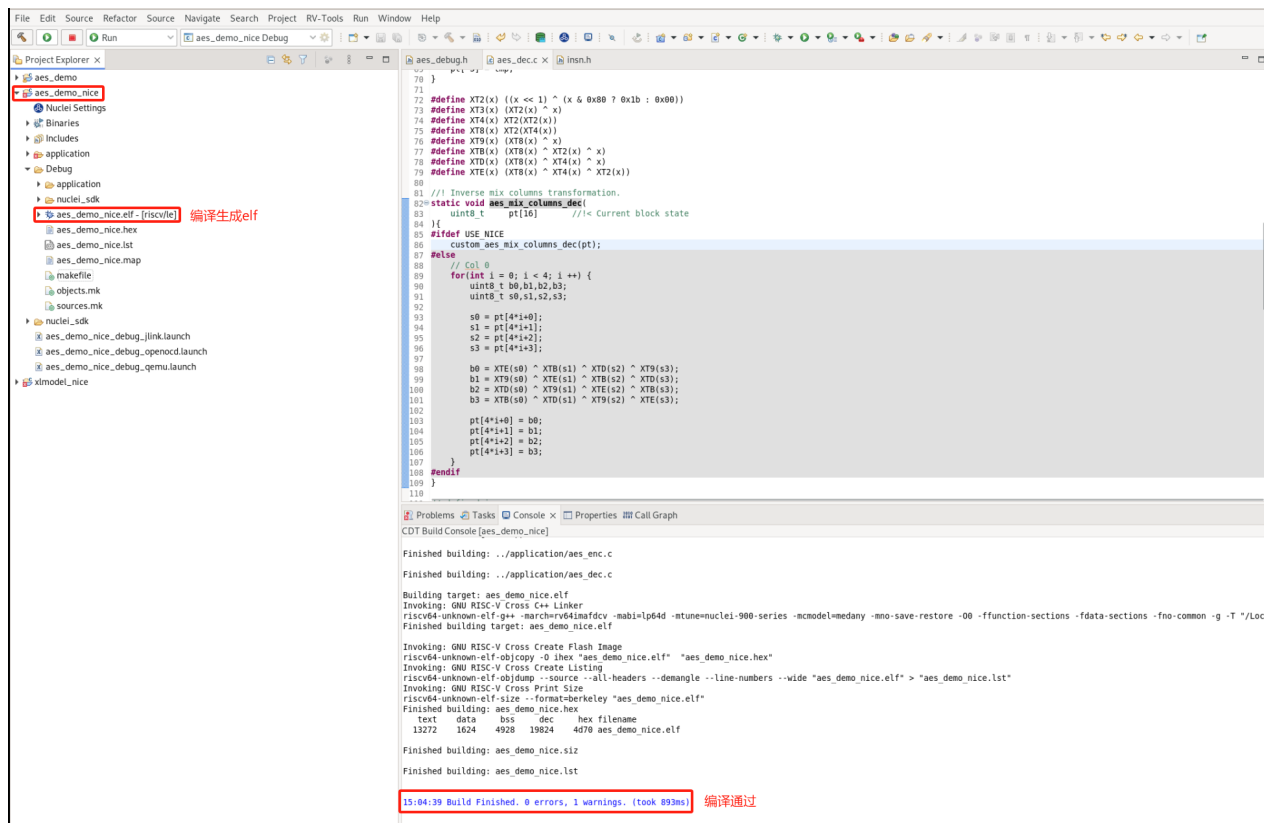
        s0 = ct[4*i+0];
        s1 = ct[4*i+1];
        s2 = ct[4*i+2];
        s3 = ct[4*i+3];

        b0 = XT2(s0) ^ XT3(s1) ^      (s2) ^      (s3);
        b1 =      (s0) ^ XT2(s1) ^ XT3(s2) ^      (s3);
        b2 =      (s0) ^      (s1) ^ XT2(s2) ^ XT3(s3);
        b3 = XT3(s0) ^      (s1) ^      (s2) ^ XT2(s3);

        ct[4*i+0] = b0;
        ct[4*i+1] = b1;
        ct[4*i+2] = b2;
        ct[4*i+3] = b3;
    }
#endif
}

```

The modified program code compiles successfully: (aes_demo_nice project)



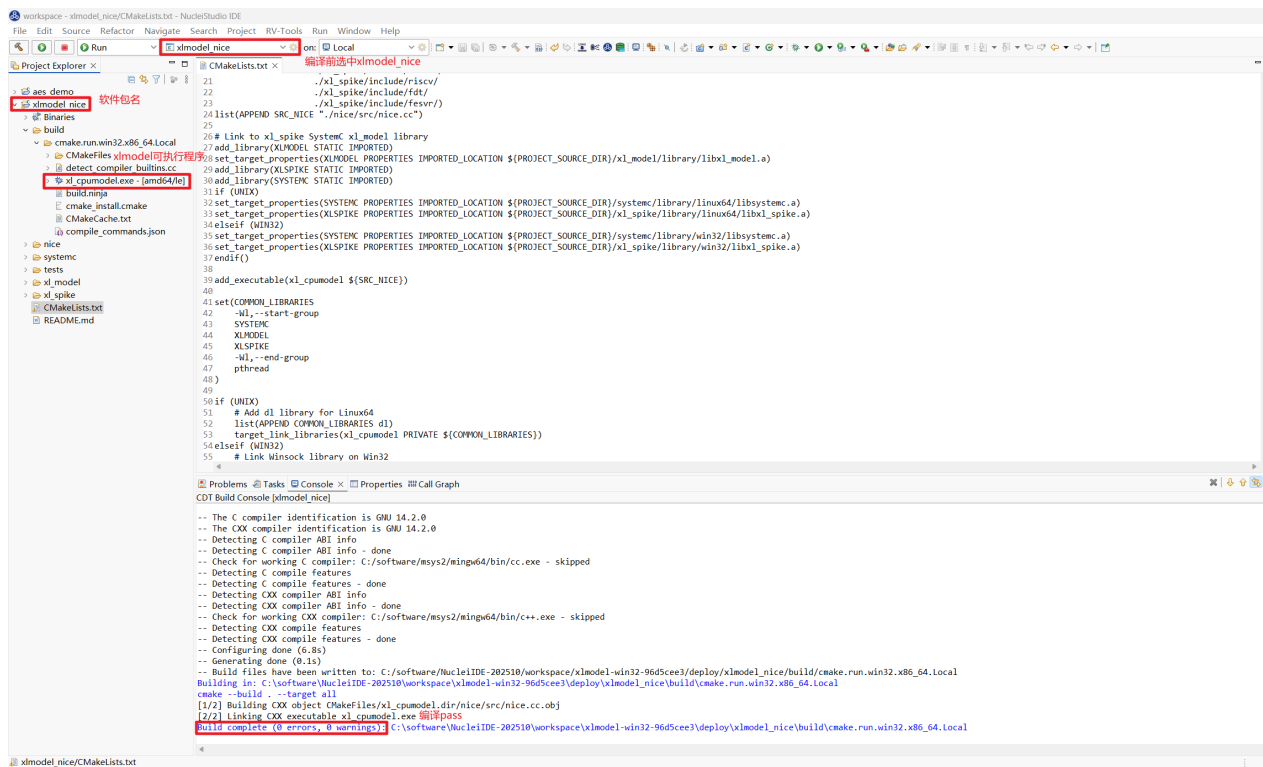
step6: Implement NICE/VNICE instructions in the Nuclei Model

First, download the original Nuclei Model software package that supports user-configured custom NICE/VNICE instructions: [Download the original model package](#). Extract the package as `xlmodel_nice`, then import it into Nuclei Studio.

Import steps: File->Import->Projects from Folder or Archive->Next->Directory->select `xlmodel_nice`->Finish

For how to use the Nuclei Model and the directory structure of the `xlmodel_nice` package, refer to the [Nuclei Model Introduction](#). `xlmodel_nice` is built with CMake and can be compiled without any modification. Before compiling, select `xlmodel_nice` in the Nuclei Studio launch bar, then click Build, and make sure the package itself compiles successfully:

For Nuclei Studio (< 2025.10), the generated elf file is located at `build/default/xl_cpumodel`



Open the `nice.cc` file. The user needs to implement all custom NICE/VNICE instructions in the `do_nice` function of this file. Currently, `do_nice` contains the NICE/VNICE instructions defined by Nuclei for `demo_nice` or `demo_vnice`; the user can refer to the comments in it to complete their own custom instructions.

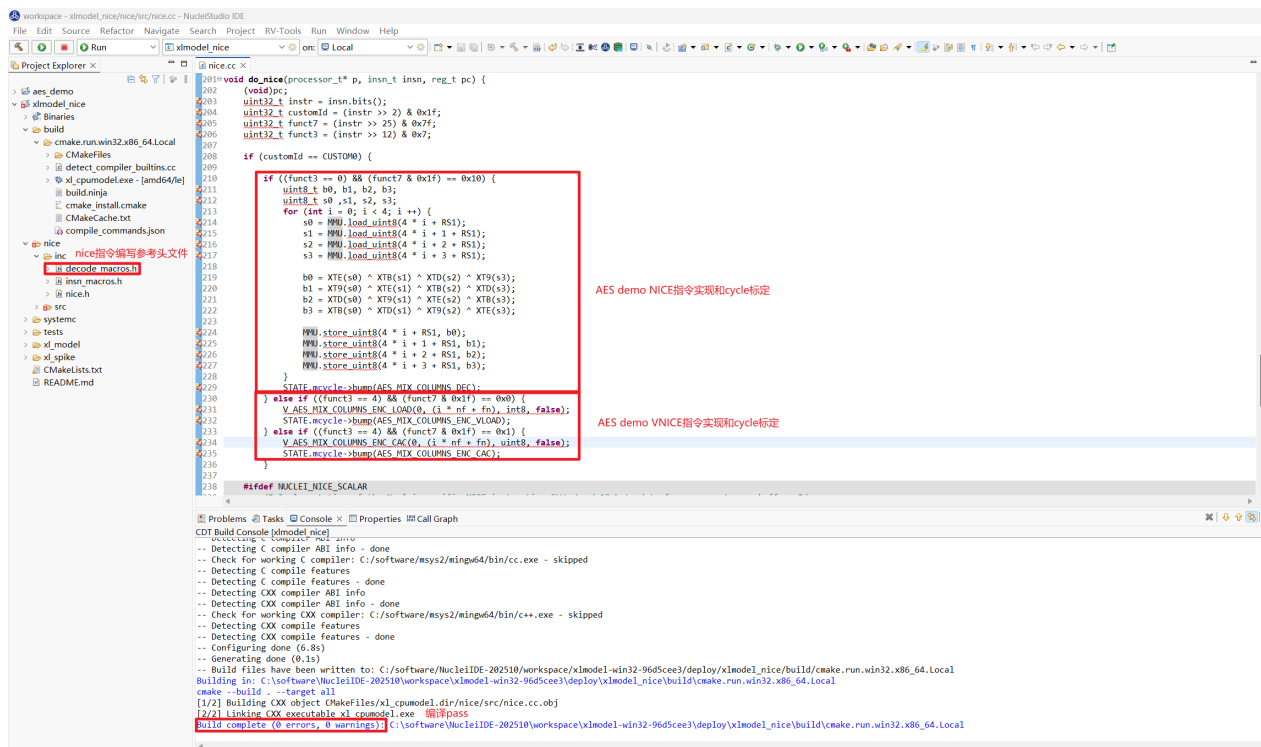
Note: When writing custom NICE/VNICE instructions, the user needs to disable the `NUCLEI_NICE_SCALAR/NUCLEI_NICE_VECTOR` macros corresponding to Nuclei's `demo_nice/demo_vnice`, so that they do not conflict with the encoding of the user's custom instructions.



In the NICE instruction implementation, the MMU macro represents memory access: use `MMU.load_uint<n>` to load memory and `MMU.store_uint<n>` to store memory. The `RD`, `RS1`, `RS2`, and `RS3` macros represent the values in their corresponding scalar registers, and the `FRS1`, `FRS2`, and `FRS3` macros represent the values in their corresponding floating-point registers. For the usage of these macros, refer to `nice/inc/decode_macros.h`.

In the VNICE instruction implementation, memory is still accessed with the MMU macro, but Vector register data is stored in the P.VU.elc class. The user can refer to `xlspike/include/riscv/v_ext_macros.h` to write the related code.

After implementing the instructions, directly specify the cycle count *n* required by the custom instruction: `STATE.mcycle->bump(n);`. Here, based on the theoretical values of implementing this algorithm in hardware with NICE/VNICE, `custom_aes_mix_columns_dec` is specified as 7 cycles, `__custom_vnice_load_v_i8m1` as 1 cycle, and `custom_vnice_aes_mix_columns_enc_i8m1` as 2 cycles.



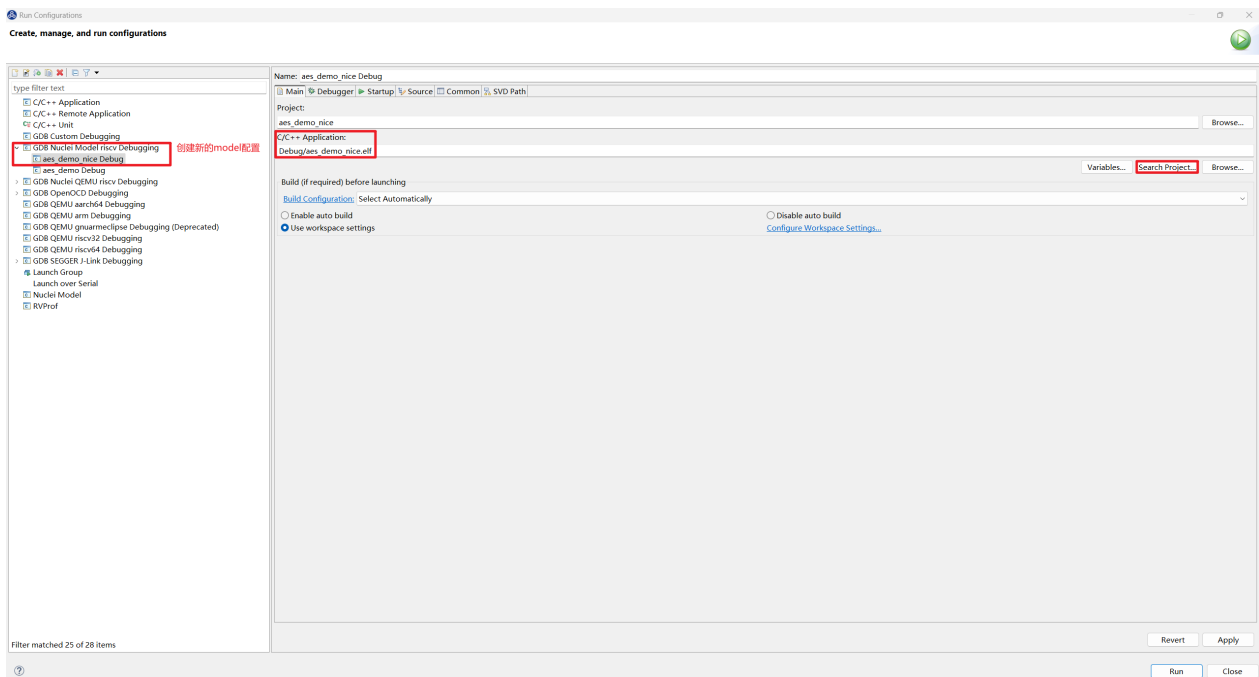
The above describes how the user adds custom NICE/VNICE instructions from the original Nuclei Model package. Next, the newly compiled model executable needs to be imported into Nuclei Studio. To avoid confusion with the name of Nuclei Studio's original model, the model can be imported into a created path such as `NucleiStudio/toolchain/nucleimodel/bin_aes/`. We provide two ways to obtain the model executable:

1. The Nuclei model software package that implements the AES demo NICE/VNICE instructions: [Model package with AES NICE instructions added](#). After compiling it, import the `xl_cpumodel` executable into the above path.
2. A pre-compiled model executable: `xl_cpumodel`. Import it directly into the above path.

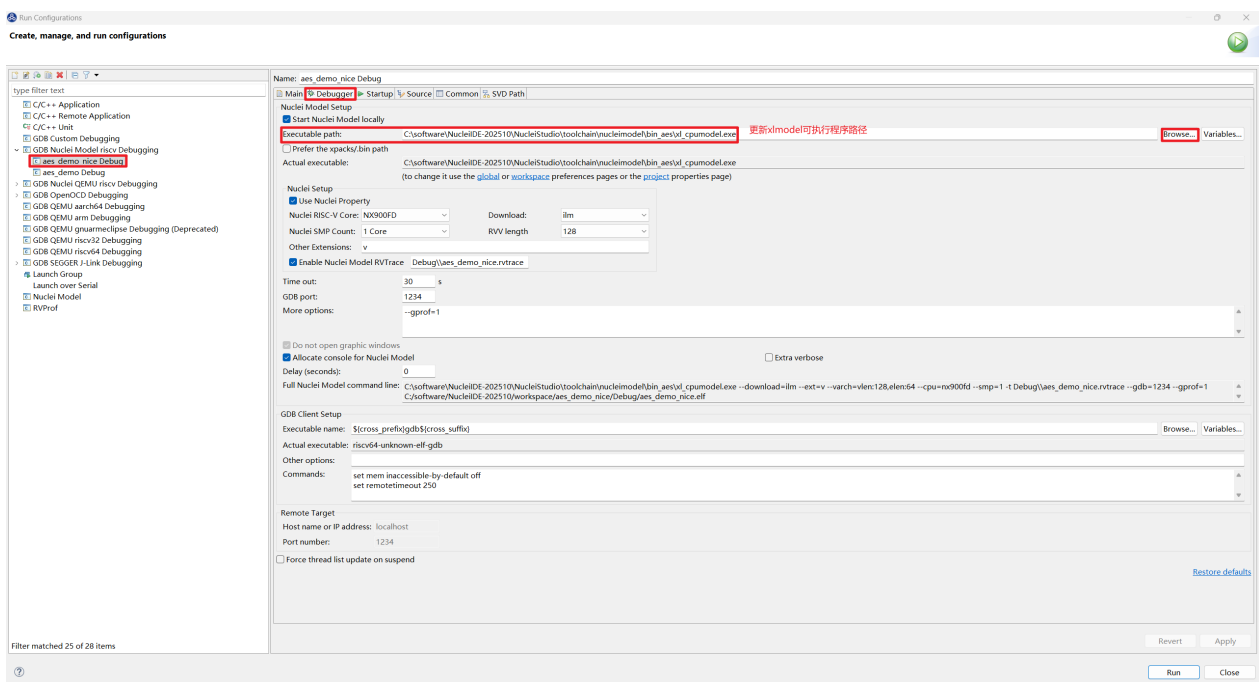
step7: Re-analyze the hotspot functions

Note: You must complete the model import described in step6 — the model implementing the NICE/VNICE instructions — into Nuclei Studio before you can use the model to Run the `aes_demo_nice` project.

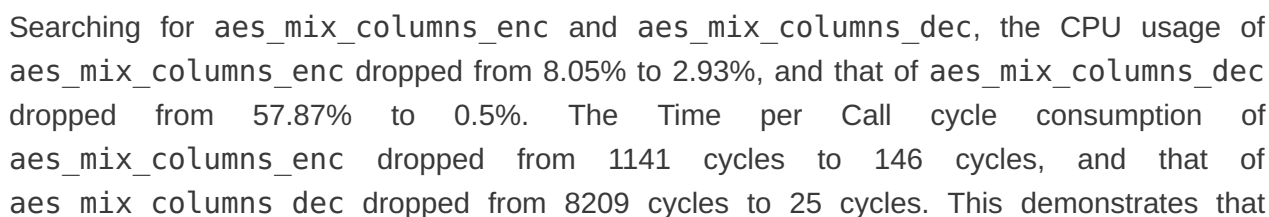
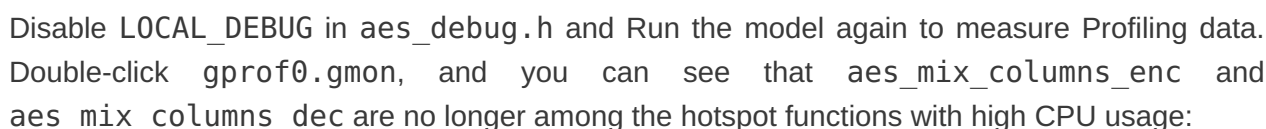
First, open Run Configurations from the Run option in the Nuclei Studio main menu bar. A new GDB Nuclei Model riscv Debugging run configuration `aes_demo_nice` Debug needs to be added for the model configuration. In the Main tab, select `aes_demo_nice.elf`:



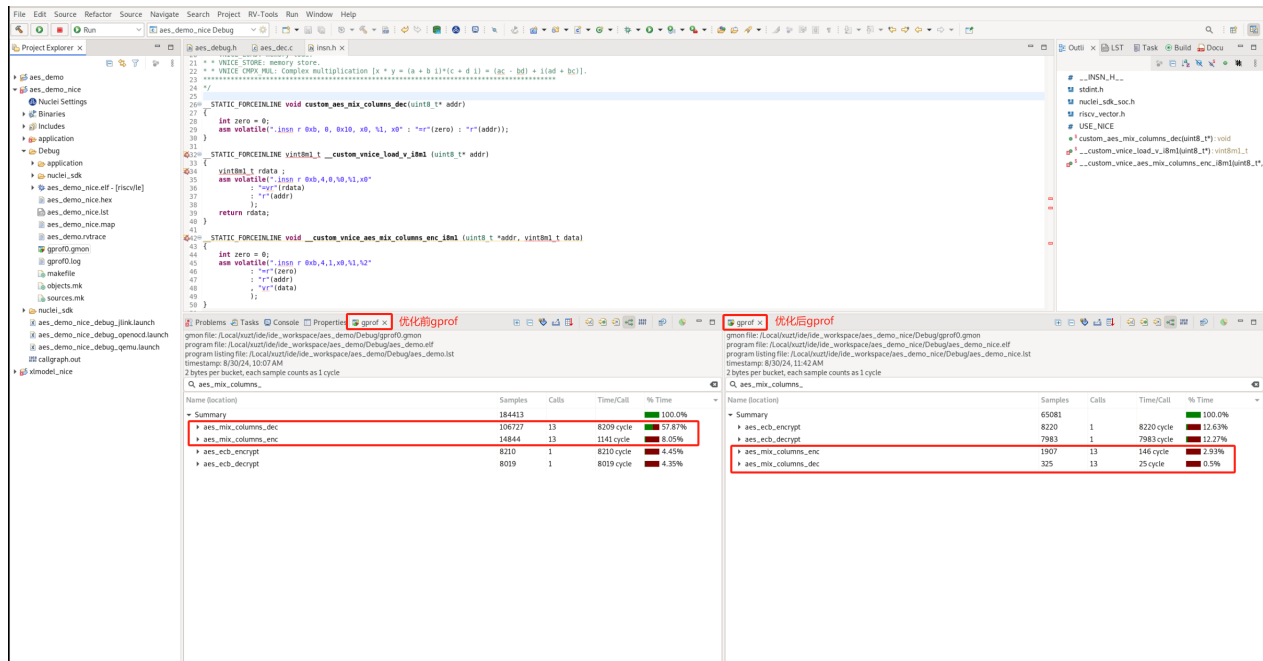
Then, in the Debugger tab, change the model Executable path to the executable path of the model newly modified in step6: `.../NucleiStudio/toolchain/nucleimodel/bin_aes/xl_cpumodel`:



Before running, enable `LOCAL_DEBUG` in `aes_debug.h` to measure the overall cycle count of the optimized AES algorithm. Select `aes_demo_nice Debug` in the Nuclei Studio launch bar and Run the model. The result shows that the total cycle count of the optimized AES algorithm dropped from 161108 before optimization to 42066 cycles.



replacing hotspot functions with NICE/VNICE instructions can greatly improve program algorithm performance.



The data statistics are as follows: (enc: aes_mix_columns_enc, dec: aes_mix_columns_dec)

Function Before Optimization NICE/VNICE Optimization

CPU Usage % (enc)	8.05	2.93
CPU Usage % (dec)	57.87	0.5
Time per Call Cycles (enc)	1,141	146
Time per Call Cycles (dec)	8,209	25

AES Program Total Before Optimization NICE/VNICE Optimization

Cycles	161,108	42,066
--------	---------	--------

AES encryption/decryption NICE/VNICE demo: [Download link for the AES project after optimization](#)

Rapid Verification of NICE/VNICE Instruction Acceleration with Nuclei Model and NICE Wizard

Nuclei Model is available for both Windows and Linux. All tests in this document were performed on the Windows version of Nuclei Studio (≥ 2025.10).

Background

xlmodel_nice

Nuclei Model continuously updates the `xlmodel_nice` software package, which allows users to customize their own NICE/VNICE implementations. Users implement the specific instruction behavior in `xlmodel_nice/nice/src/nice.cc` and build a new Nuclei Model that applications can be configured to call.

Nuclei NICE Wizard

Nuclei NICE Wizard is a NICE/VNICE instruction generation tool provided in Nuclei Studio. After configuring custom instructions, the user can automatically generate two files:

1. `insn.h`: the instruction inline assembly header file. The user needs to add the instruction inline assembly from this file into the application header file.
2. `nice.cc`: the instruction implementation file. The user needs to add the instruction decode framework from this file into `xlmodel_nice/nice/src/nice.cc`.

test code

In batch matrix operations common in AI and deep learning, there are scenarios where small matrix blocks need to be processed repeatedly. In this test, an algorithm function that performs multiplication and accumulation on multiple scalar 4x4 matrices is used as the `golden_case`. Then NICE Wizard is used to generate NICE/VNICE acceleration instructions, which are added to the test application and the `xlmodel_nice` software package project respectively and recompiled. Finally, Nuclei Model is run to observe the instruction count and cycle count of the optimized algorithm function, so as to evaluate the NICE/VNICE acceleration effect.

Solution

Environment Preparation

The NICE Wizard related features integrated in Nuclei Studio IDE must be used together with the Nuclei CPU Model - NICE Support (xlmodel_nice) software package.

Nuclei Studio:

- [NucleiStudio 202510 Windows](#)
- [NucleiStudio 202510 Linux](#)

xlmodel_nice:

- [Original xlmodel_nice package Windows](#)
- [Original xlmodel_nice package Linux](#)

Running the Original Program on Nuclei Model

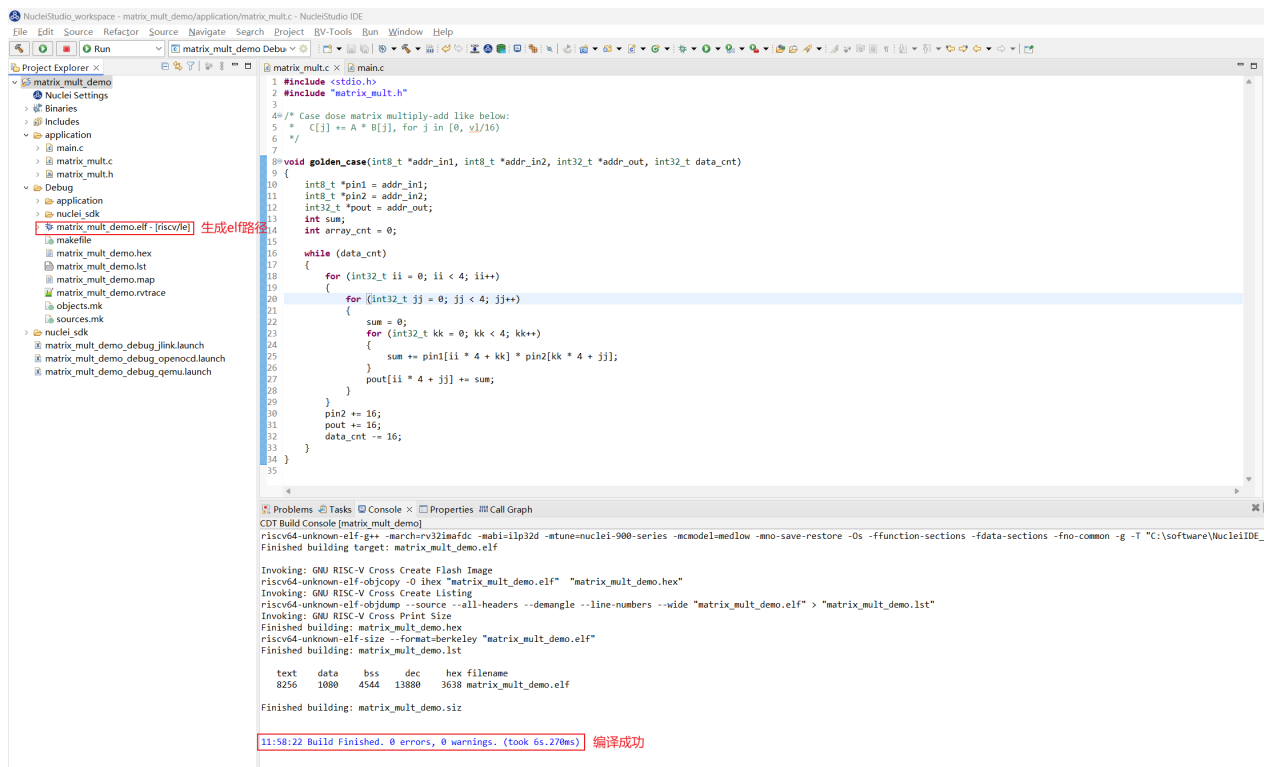
Step 1: Import the original Nuclei SDK project

[Download link for the project before optimization](#)

After downloading the zip package, you can import it directly into Nuclei Studio and run it (import steps: File->Import->Existing Projects into Workspace->Select archive file->select the zip archive->Finish).

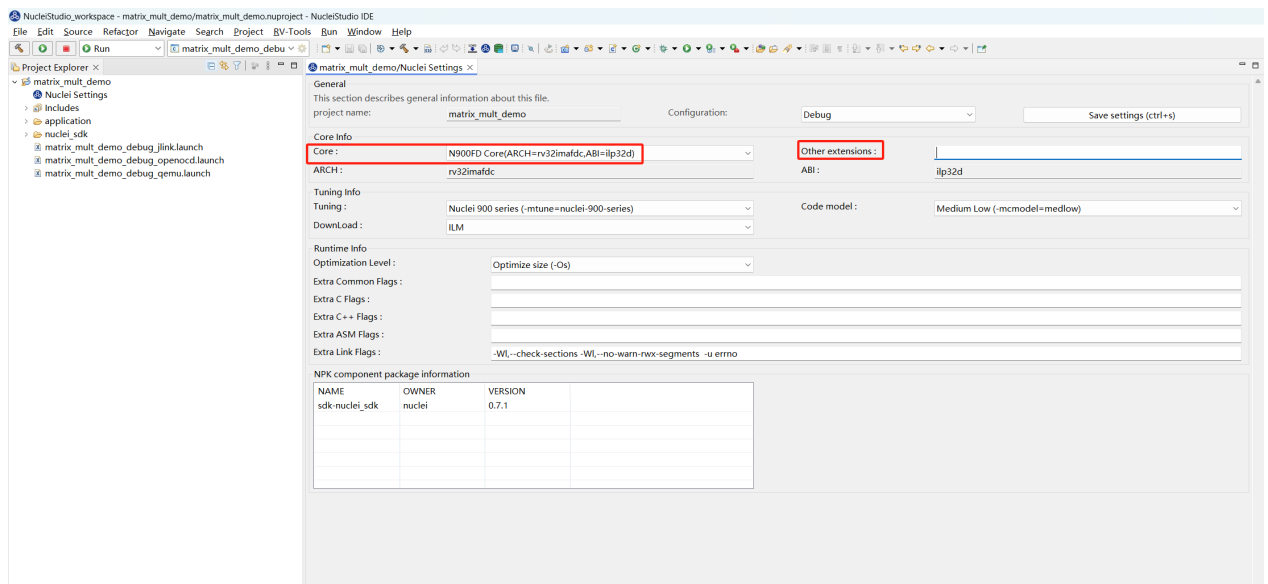
Step 2: Build the original Nuclei SDK project

Build the original project, and make sure the build succeeds and the generated elf file can be found under Debug:



Step 3: Run the original Nuclei SDK project

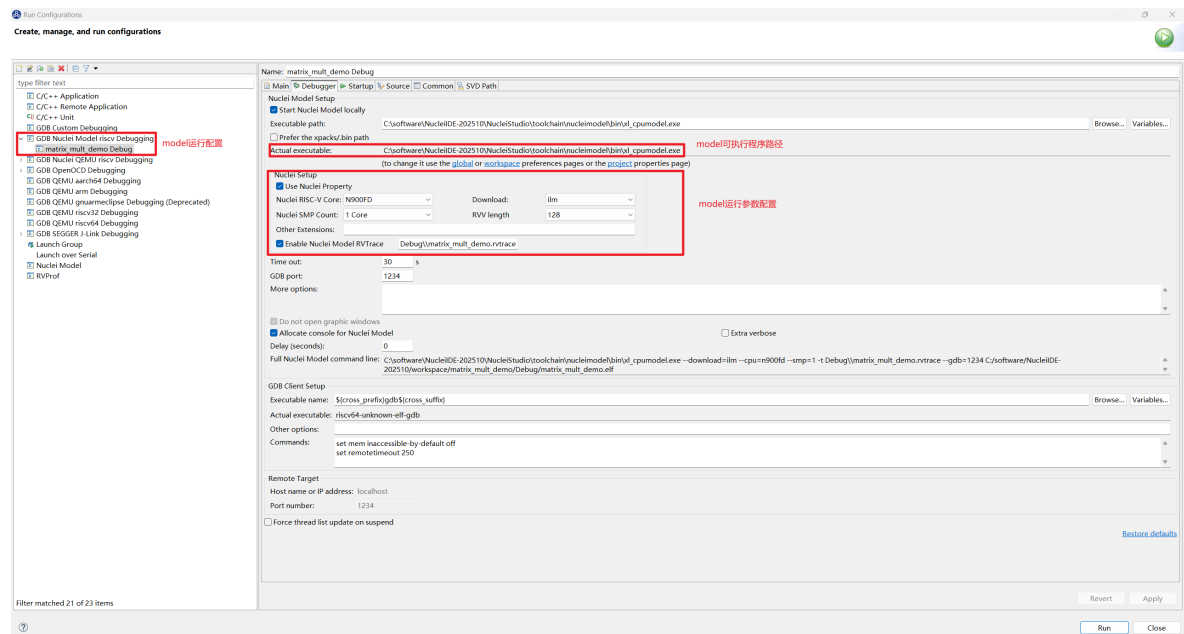
Before running the program on Nuclei Model, you first need to confirm the Core configuration and the Other extensions configuration in the project's Nuclei Settings, as these configurations need to be passed to the Model. The Core currently used is n900fd, and Other extensions is not configured.



Model simulation requires configuring the GDB Nuclei Model riscv Debugging configuration in Nuclei Studio. The steps are as follows:

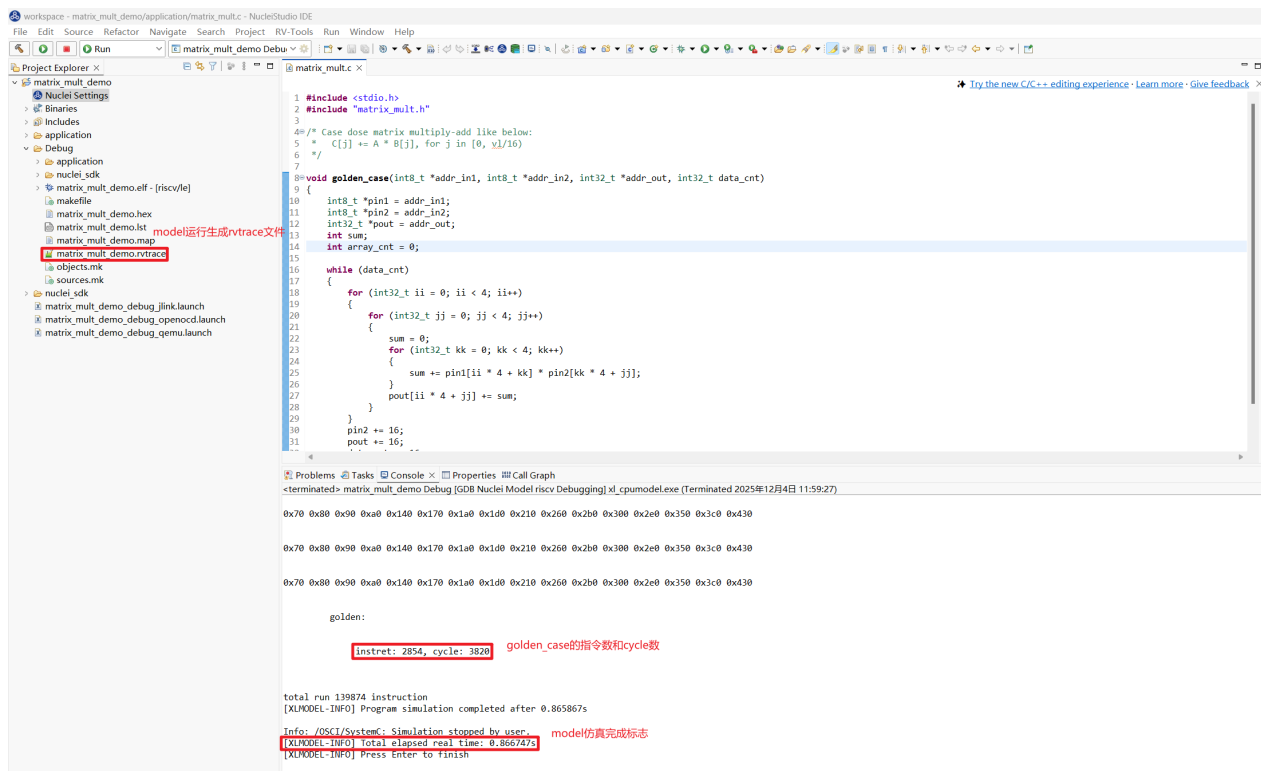
1. Open Run Configurations from the Run option in the Nuclei Studio main menu bar.

2. Select the GDB Nuclei Model riscv Debugging configuration, right-click and choose New Configuration. A Model configuration page named after the project will be automatically generated, and the launch bar will be updated accordingly.
3. In the Main tab on the right, click Search Project . . . and select the built elf file.
4. In the Debugger tab on the right, select Browse and locate the default path of the Nuclei Model executable: NucleiStudio/toolchain/nucleimodel/bin/xl_cpumodel.exe.
5. In the Nuclei Setup section of the Debugger tab on the right, complete the model run configuration. The selected Nuclei RISC-V Core and Other Extensions must be consistent with the Core and Other extensions configured in Nuclei Settings. When Other Extensions is empty, this parameter is not passed. Enable Nuclei Model RVTrace means that rvtrace is generated at runtime. Then click Apply and Run, and the model starts running the program.



Nuclei Studio (< 2025.10) can only use Nuclei Model in Run Configurations to configure the model. For Nuclei Studio (>= 2025.10), it is recommended to switch to GDB Nuclei Model riscv Debugging.

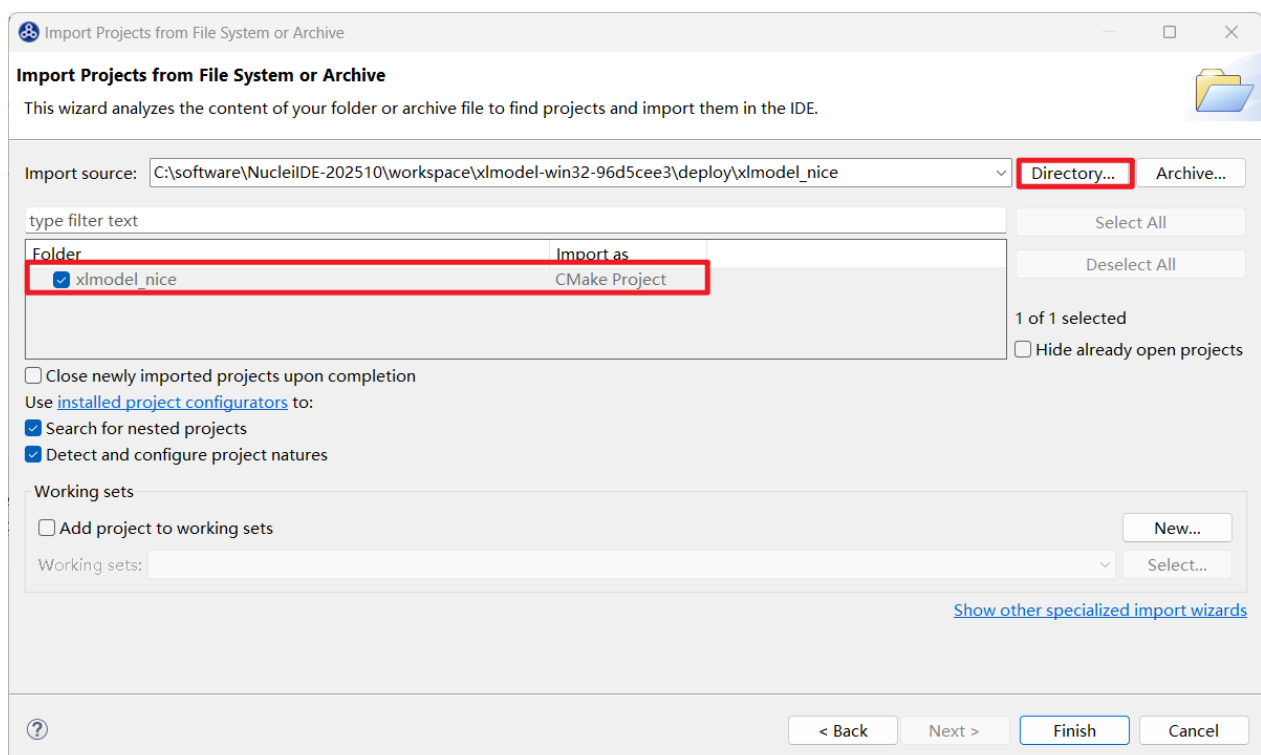
When Total elapsed real time appears in the Console, it means the model has finished the simulation. The program extracts the instruction count and cycle count of the scalar matrix multiplication algorithm function `golden_case` as follows:



NICE Instruction Replacement

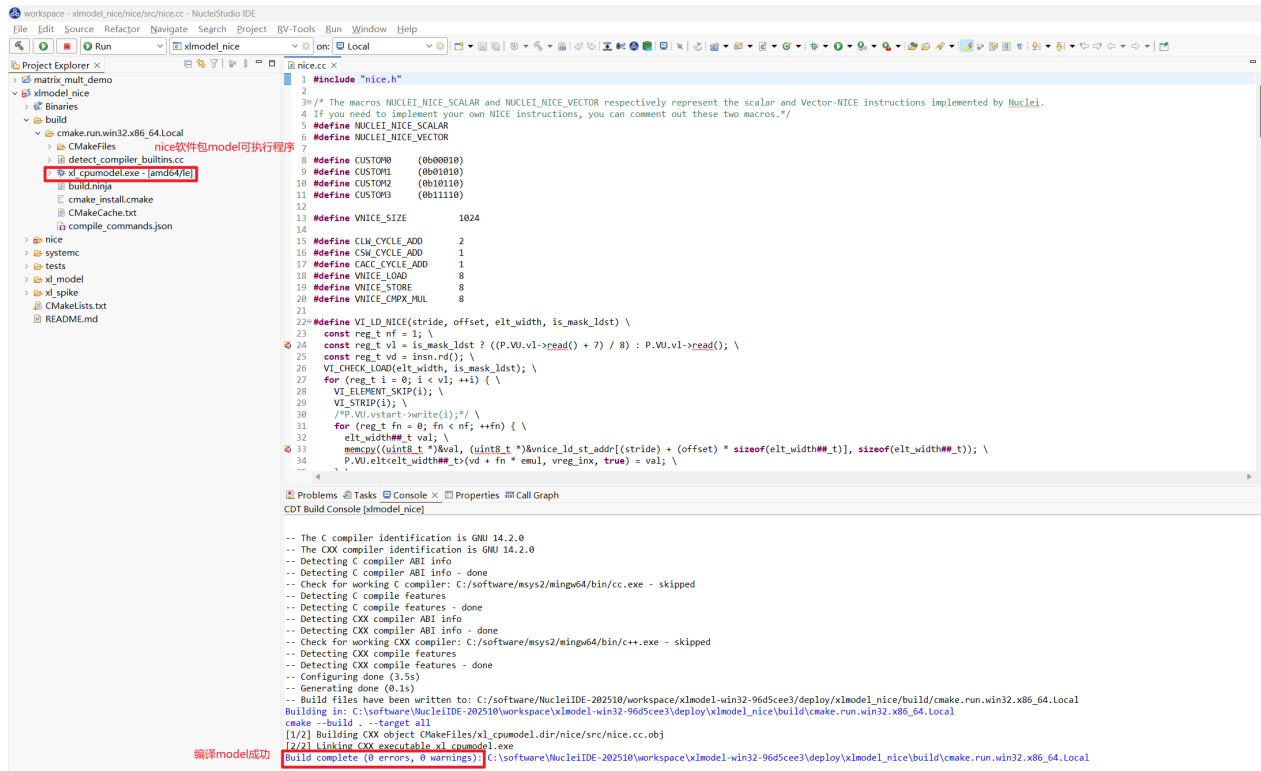
Step 1: Build the xlmodel_nice package

After downloading and unzipping the xlmodel_nice zip package, you can import it directly into Nuclei Studio and run it (import steps: File->Import->Projects from Folder or Archive->Next->Directory->select the xlmodel_nice folder->Finish).



Before building `xlmodel_nice`, you need to configure the `xlmodel` build environment first ([xlmodel_nice build environment configuration](#)). Then build to make sure the original package can be successfully compiled to generate the model executable:

For Nuclei Studio (< 2025.10), the generated elf file is located at `build/default/xl_cpumodel`.



Step 2: Use NICE Wizard to generate NICE instruction replacement

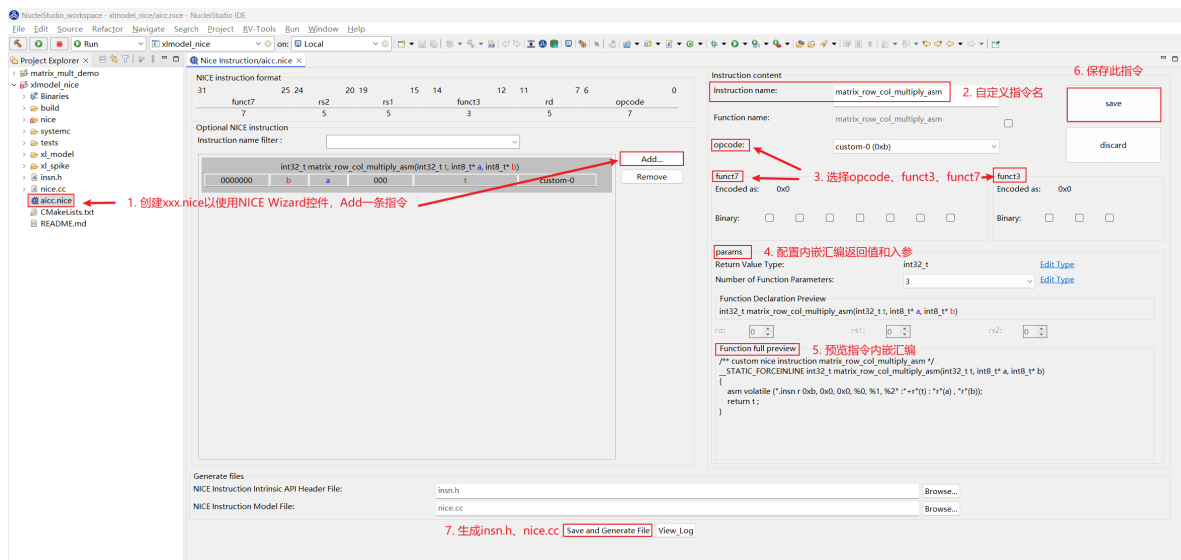
Hotspot functions of the application can be located with Nuclei Model Profiling first. For details, refer to [Demonstrate NICE/VNICE Instruction Acceleration of Nuclei Model Through Profiling](#), which will not be repeated here.

The hotspot function in this use case is known to be matrix multiply-accumulate. The computation of one row of matrix A * one column of matrix B is as follows:

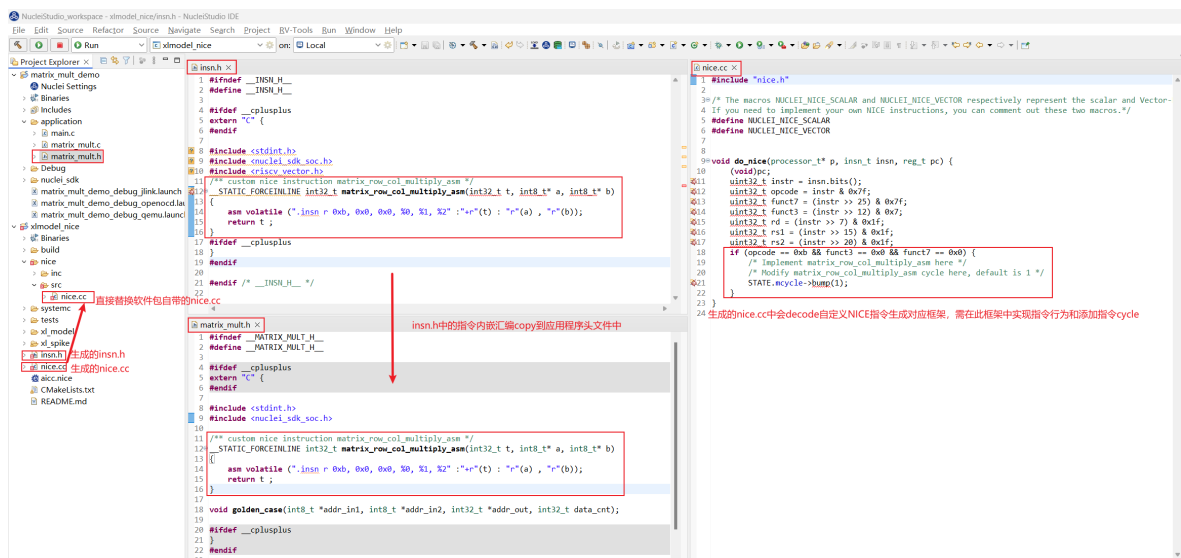
```
for (int32_t kk = 0; kk < 4; kk++)
{
    sum += pin1[ii * 4 + kk] * pin2[kk * 4 + jj];
}
```

This algorithm can be completely replaced by a single NICE instruction, with the sum value, the pin1 address, and the pin2 address as inputs, and the sum as the output.

Next, use NICE Wizard to generate the envisioned NICE instruction. The user can create a file named `aicc.nice` in the root directory of the `xlmodel_nice` project in Nuclei Studio. Once



6. Copy the NICE instruction inline assembly from the generated `insn.h` into the application header file, and replace `xlmodel_nice/src/nice.cc` directly with the generated `nice.cc`.



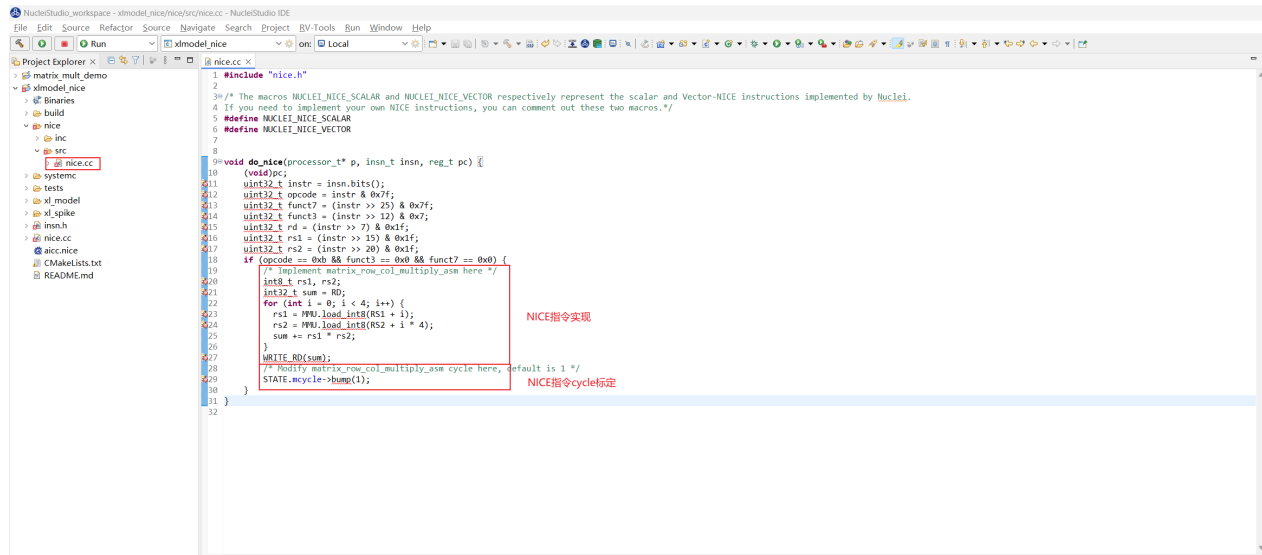
Alternatively, `insn.h` can be generated directly into the application project path for reference, which saves the manual copying of file contents each time.

Step 3: Implement the NICE instruction in `xlmodel_nice`

Open the `xlmodel_nice/nice/src/nice.cc` file and use the macros defined in `spike` to implement the NICE instruction: the `MMU` macro represents memory access; use `MMU.load_xxx<n>` for load memory and `MMU.store_xxx<n>` for store memory; the `RD`, `RS1`, `RS2`, and `RS3` macros represent the values in their corresponding scalar registers; use `WRITE_RD` to write the destination register. The usage of these macros can be found in `nice/inc/decode_macros.h`.

After implementing the instruction, directly specify the number of additional cycles n required by the custom instruction: `STATE.mcycle->bump(n);`. Here, this NICE instruction is specified to require 1 additional cycle. Since an instruction takes 1 cycle by default, this NICE instruction consumes 2 cycles in total.

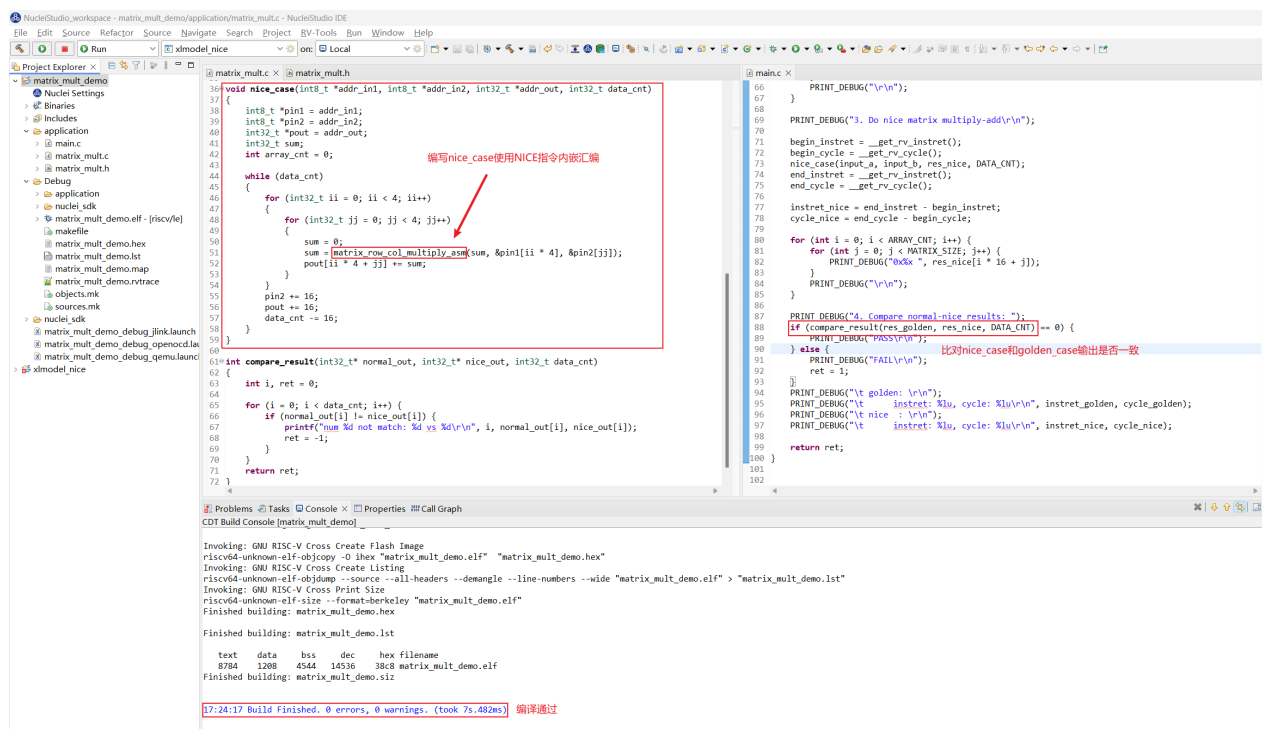
The implemented NICE instruction and its cycle specification are as follows:



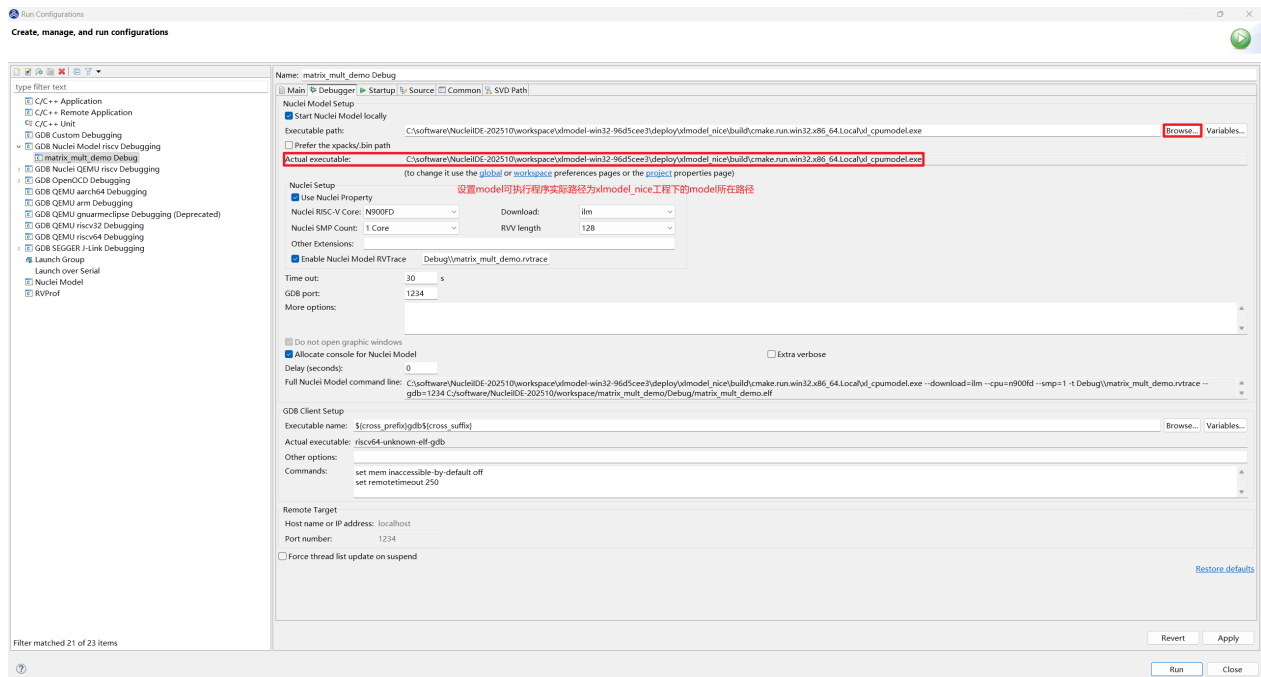
Rebuild `xlmodel_nice` and make sure the build passes.

Step 4: Rerun the program on Nuclei Model

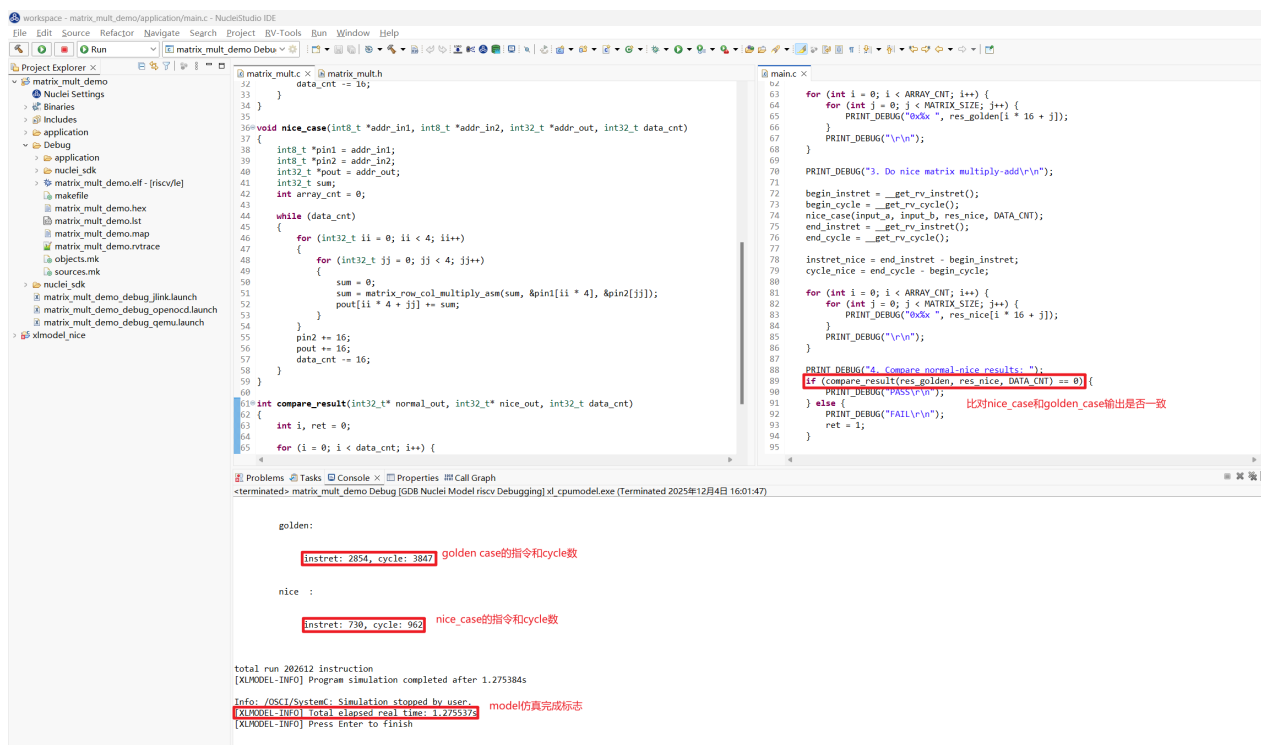
First, write an algorithm function `nice_case` with NICE instruction inline assembly for comparison with `golden_case`, add a comparison of the function output results, and then rebuild the application project:



Since the model has been rebuilt with `xlmodel_nice` into a new executable, you need to reconfigure the model executable path in the Nuclei Studio Nuclei Model configuration to `xlmodel_nice/build/default/xl_cpumodel.exe`. The rest of the configuration remains unchanged:



After Apply, Run the application again. You can find that the output results of `nice_case` and `golden_case` are identical, while the instruction count and cycle count of `nice_case` have dropped significantly. The envisioned NICE instruction is implemented correctly and has optimized the original scalar algorithm.



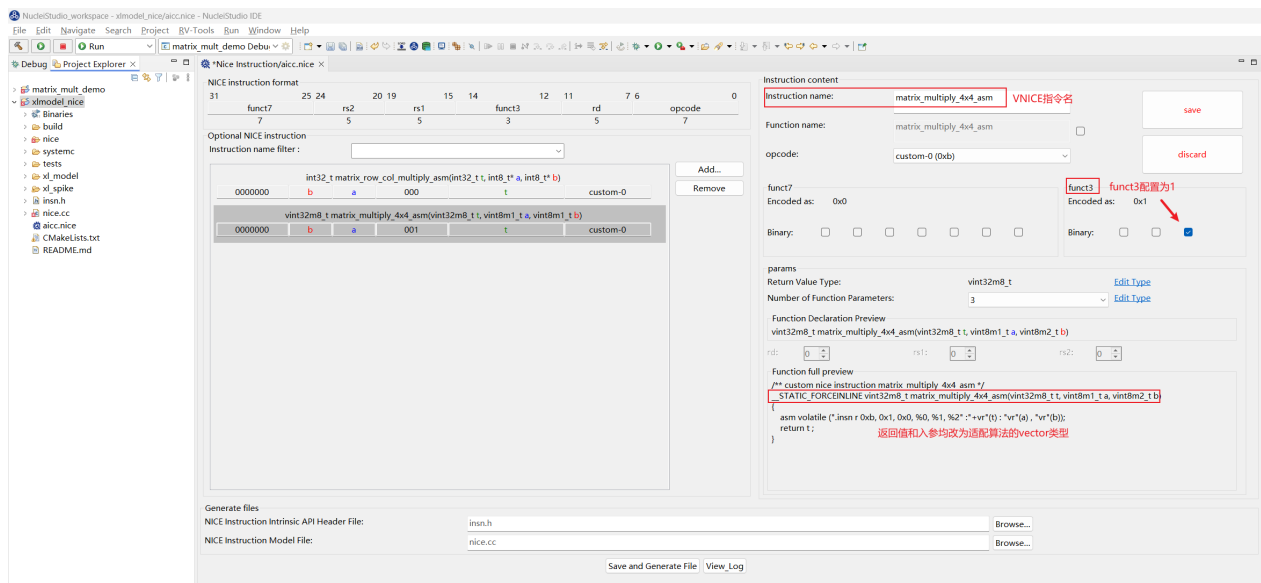
VNICE Instruction Replacement

Step 1: Use NICE Wizard to generate VNICE instruction replacement

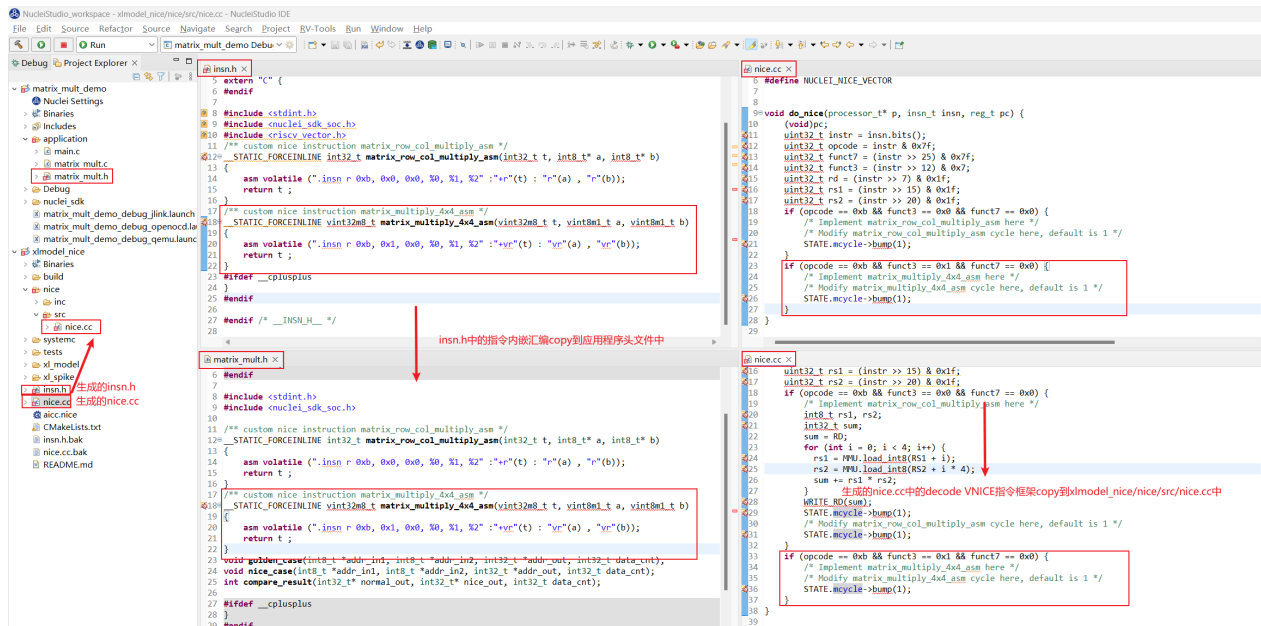
When computing with a NICE instruction, only one element of the output matrix is obtained each time, which is not efficient enough. If one instruction operation could process multiple matrix elements in parallel, the efficiency should be further improved. It is natural to think of using Vector instructions to process matrix data with higher parallelism.

The idea is to condense the complete $4 * 4$ matrix multiply-accumulate operation into a single Vector instruction. A VNICE instruction can be used to implement this behavior, with three $4 * 4$ input matrices as inputs and a $4 * 4$ output matrix as the return value.

Double-click `aicc.nice` to use NICE Wizard again to configure the envisioned instruction. The steps to generate the instruction are similar to those for generating the NICE instruction above. The differences are: configure the Instruction name field as `matrix_multiply_4x4_asm` to indicate the multiplication of two $4*4$ matrices; configure `funct3` as 1 to avoid the same encoding as the previous NICE instruction; and to match the vector data type inputs and outputs corresponding to the scalar `golden_case`, set the return value to `vin32m8_t`, the number of input parameters to 3, namely `vin32m8_t`, `vint8m1_t`, and `vint8m2_t`. The configuration interface after clicking save is as follows:



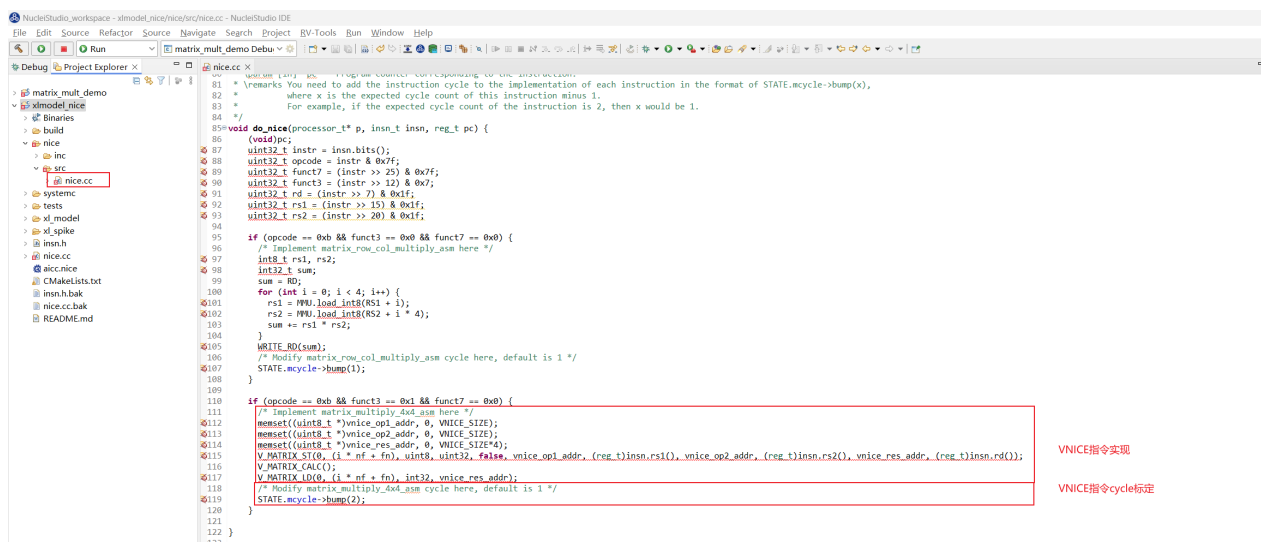
Click **Save** and **Generate File** at the bottom to overwrite the previously generated `insn.h` and `nice.cc`. At this point, `insn.h.bak` and `nice.cc.bak` will also appear in the same path. These two files are backups of the previously saved `insn.h` and `nice.cc` and will not be used. Again, copy the NICE instruction inline assembly from the generated `insn.h` into the application header file, and copy the new instruction decode framework from the generated `nice.cc` into `xlmodel_nice/nice/src/nice.cc`:



Step 2: Implement the VNICE instruction in xlmodel_nice

Implement the VNICE instruction in `xlmodel_nice/nice/src/nice.cc`: `V_MATRIX_ST` stores the vector registers input to the instruction into a custom buffer, `V_MATRIX_LD` loads the instruction output results into the RD register, and `V_MATRIX_CALC` implements the two-matrix multiply-accumulate operation. The VNICE instruction implementation can refer to the vector instruction implementation in spike: `xlmodel_nice/xl_spike/include/riscv/v_ext_macros.h`.

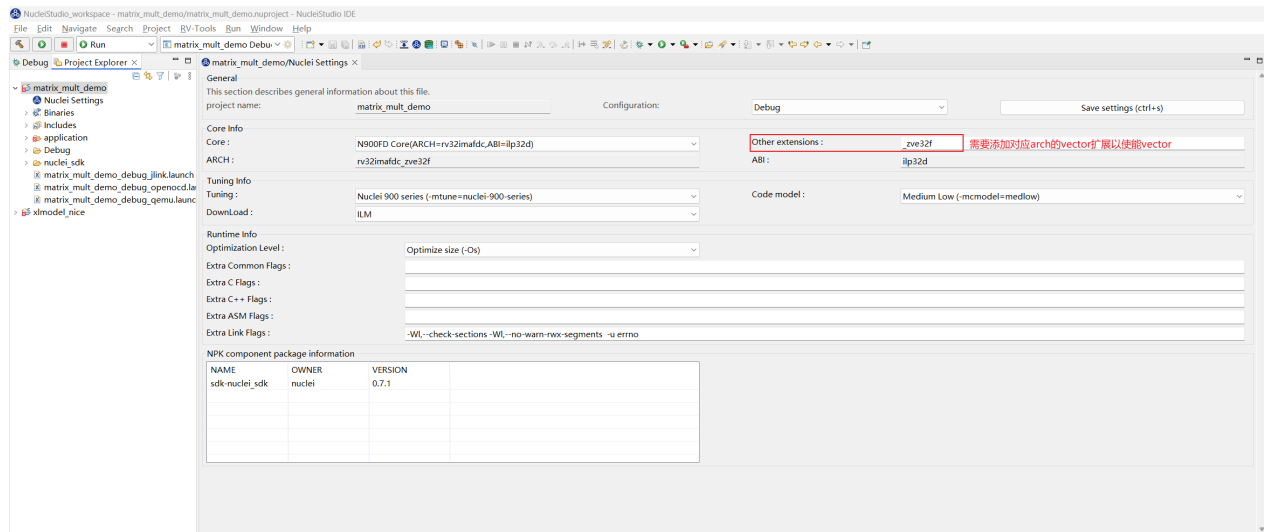
Specify that this VNICE instruction requires 2 cycles, i.e., it actually consumes 3 cycles. The implemented VNICE instruction and its cycle specification are as follows:



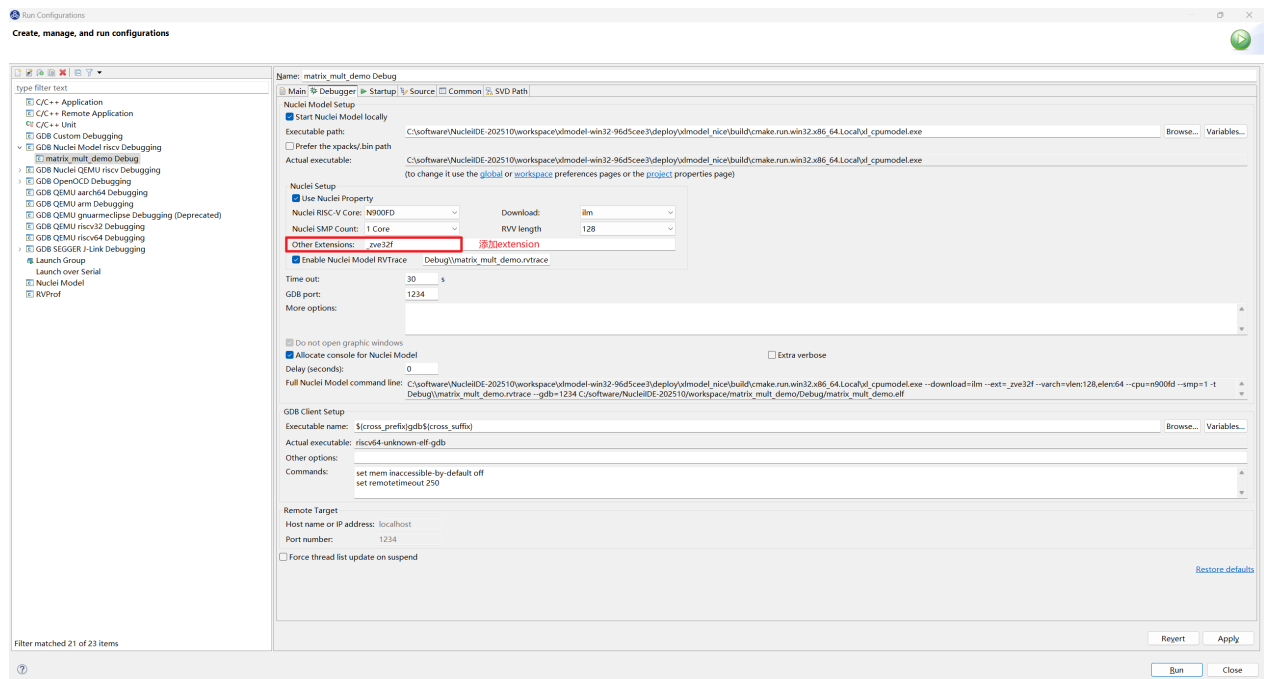
Rebuild `xlmodel_nice` again and make sure the build passes.

Step 3: Rerun the program on Nuclei Model

Since the inputs and outputs of the VNICE instruction are all vector registers, you need to configure the application's Nuclei Settings to enable the vector extension of the corresponding ARCH. Here, the `_zve32f` extension is added for `rv32imafdc`:

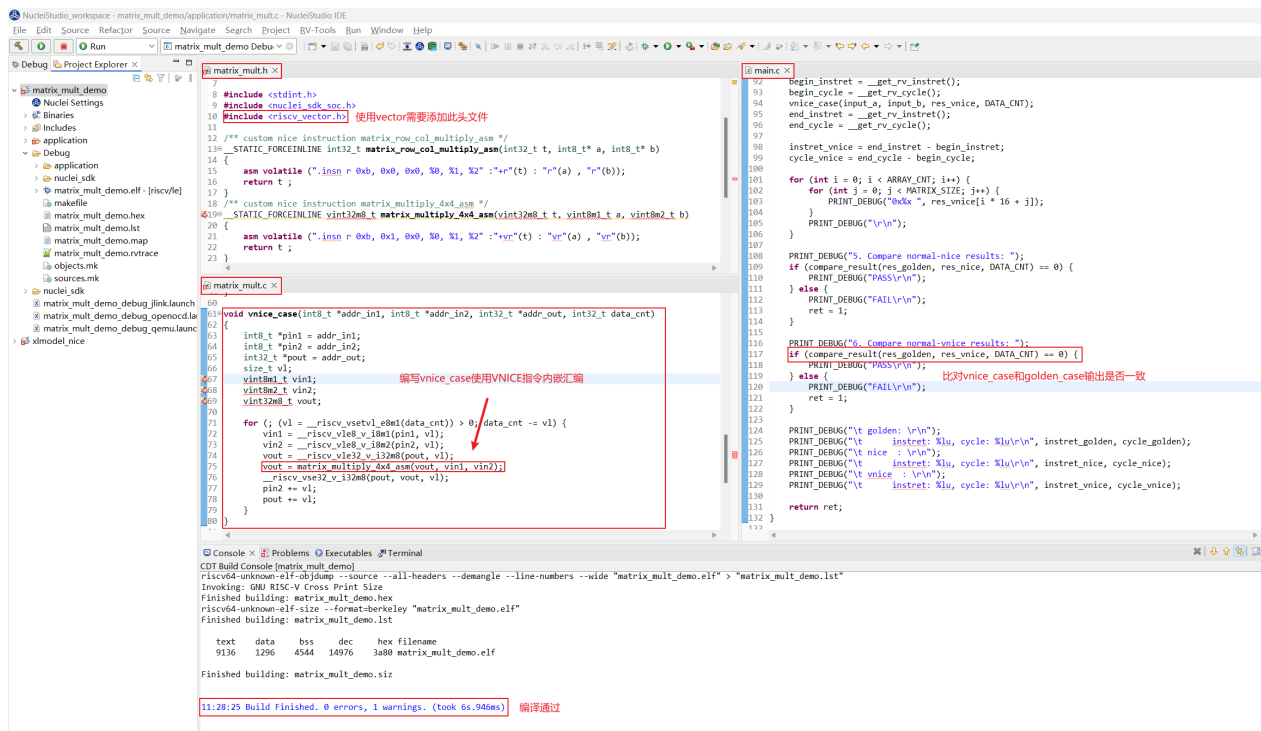


The corresponding Nuclei Model configuration also needs to add `--ext=_zve32f` to enable the model's vector functionality, then click Apply:

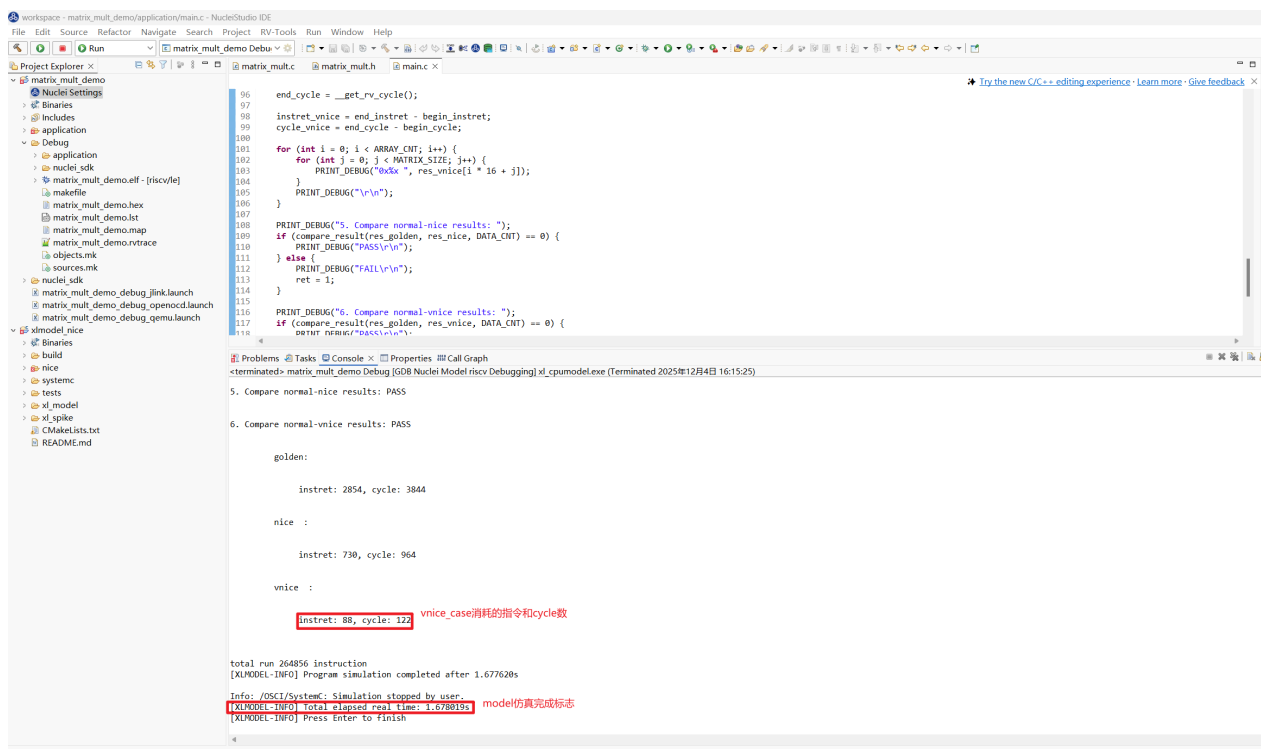


You need to write an algorithm function `vnice_case` with VNICE instruction inline assembly. The inputs and outputs required by the VNICE inline assembly need to be constructed with the corresponding vector intrinsic API. Then add a result comparison with `golden_case` and rebuild the application project.

Note: You need to add `#include <riscv_vector.h>` to the application header file to enable the vector intrinsic API.



Run the application again. You can find that the output results of `vnice_case` and `golden_case` are identical, and its instruction count and cycle count have dropped further significantly compared to `nice_case`. The envisioned VNICE instruction is implemented correctly and has accelerated the matrix multiply-accumulate algorithm by taking advantage of the high parallelism of vector.



Summary

The table below shows the instret/cycle statistics after implementing the NICE/VNICE instructions to optimize the algorithm. Compared with `golden_case`, the performance of `nice_case` after optimization is improved by about 4 times, and the performance of `vnice_case` after optimization is improved by more than 30 times.

instret/cycle	golden_case	nice_case	vnice_case	golden / nice	golden / vnice	nice / vnice
instret	2854	730	88	3.91	32.43	8.30
cycle	3844	964	122	3.99	31.51	7.90

By studying the optimization strategy of an existing algorithm, the user can quickly generate the corresponding NICE/VNICE instructions with NICE Wizard, then import the `xlmodel_nice` package into Nuclei Studio to implement the instructions, and write an application instruction-optimization case. This way, the algorithm optimization effect can be quickly verified with Nuclei Model. The entire testing process can be completed using only Nuclei Studio.

[Download link for the optimized project](#)

[Optimized `xlmodel\_nice` package](#)

Flash Programming Use Case¶

To meet users' need to download compiled binary files directly to a hardware development board, Nuclei Studio provides the Flash Programming feature. This feature allows users to quickly and conveniently download compiled binary files directly to the hardware development board, greatly improving development and debugging efficiency. Users can complete the binary file download with a single click, simplifying the workflow.

Solution¶

Environment Preparation¶

Nuclei Studio:

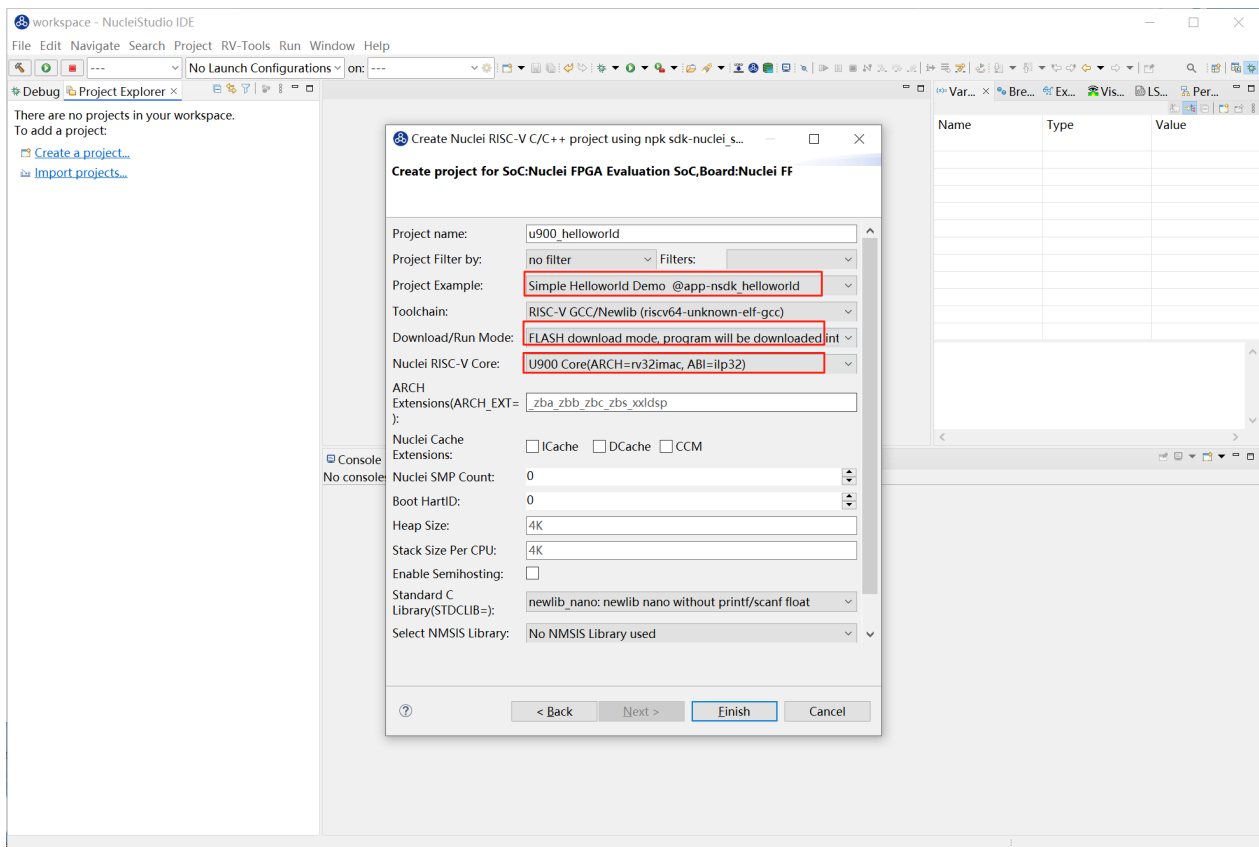
Version $\geq$ 202412 is required. Version 202502 is provided below.

- [NucleiStudio 202502 Windows](#)
- [NucleiStudio 202502 Linux](#)

Flash Programming Demo¶

step1: Create a project and program the bit file

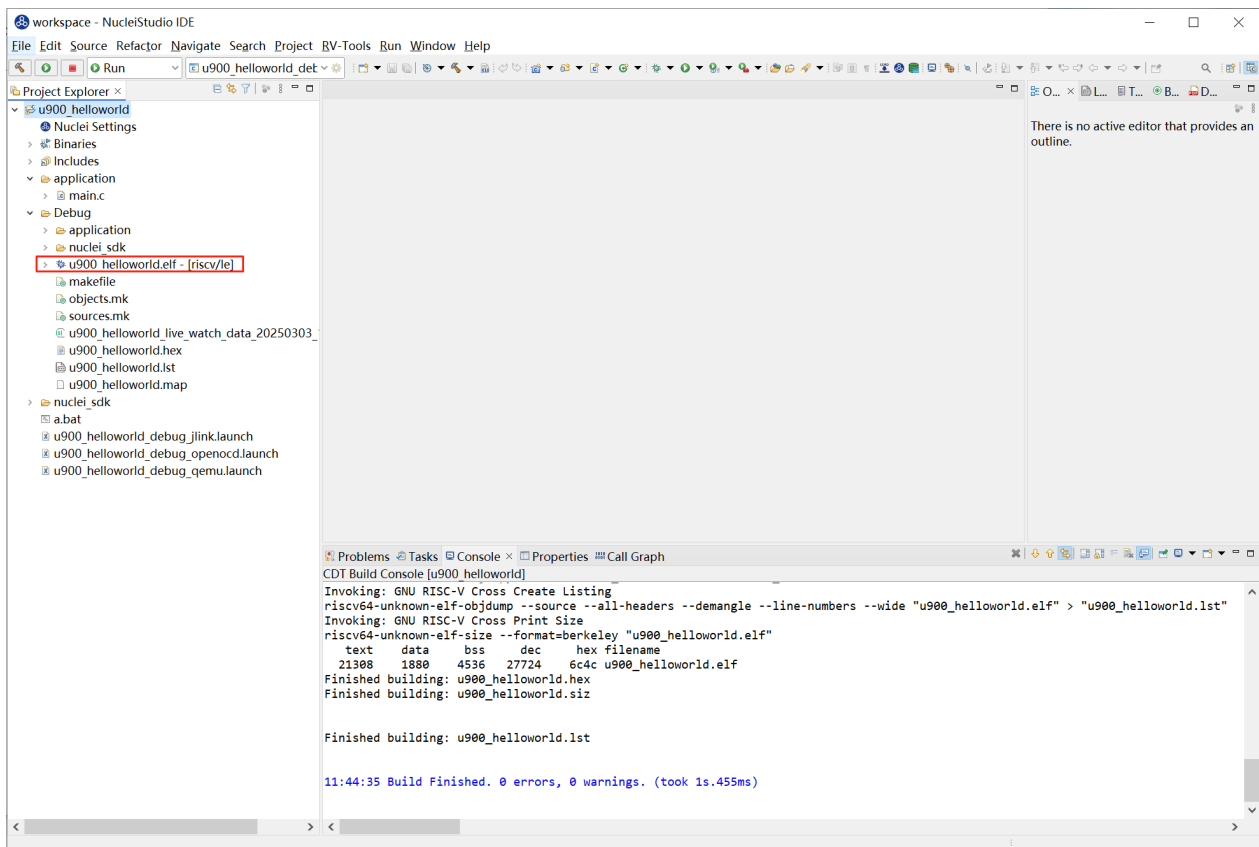
Use version 0.7.1 of sdk-nuclei_sdk to create a u900 helloworld project. Select Simple Helloworld Demo, FLASH download mode, and U900 Core in sequence, then click Finish.



Program the corresponding bit file to the development board. Here we use trace-u900_best_config_ku060_16M_e85631d489_e82e2771f_202409232110_v3.12.0.bit

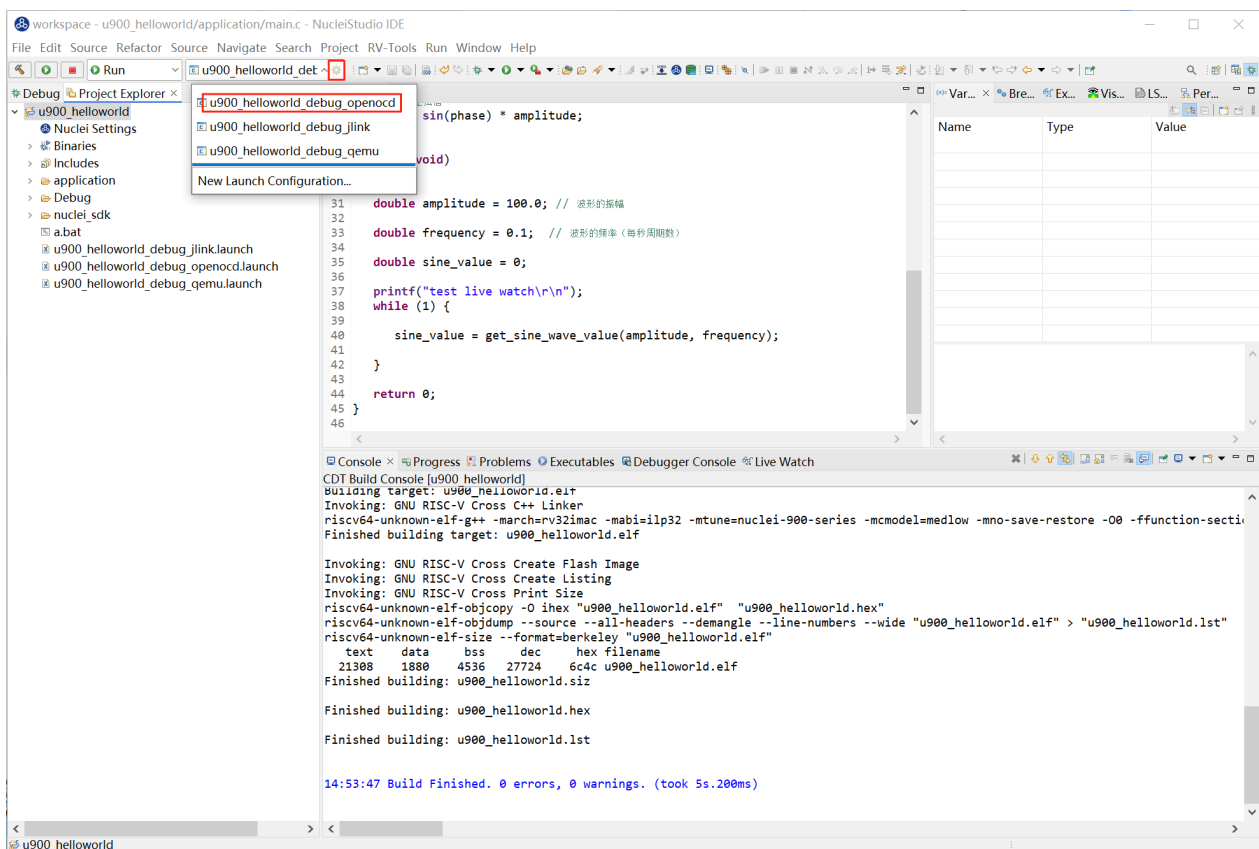
step2: Configure and build the original Nuclei SDK project

Build the original project, making sure the build succeeds and the generated elf file can be found under Debug:



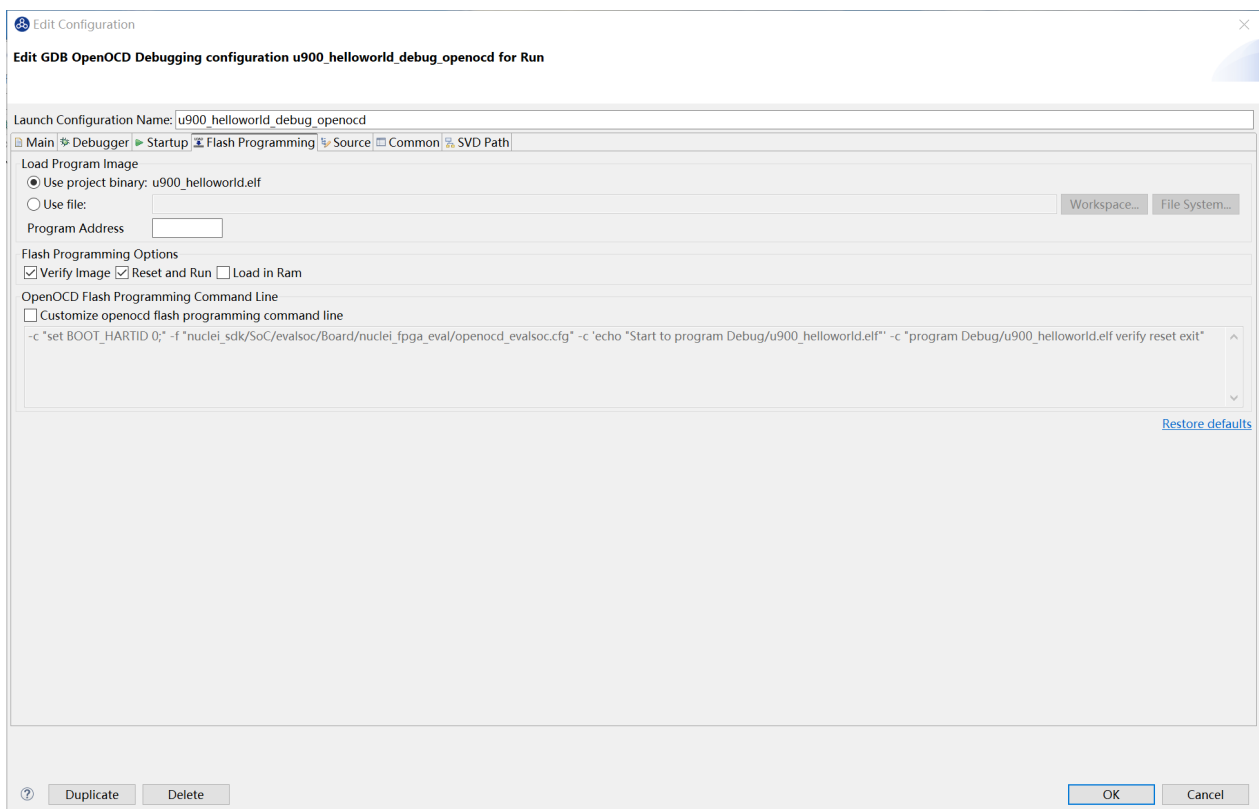
step3: Configure the Flash Programming tab

In Launch Configuration, select the corresponding debug option (openocd) and click edit to open the configuration page.



Select the Flash Programming tab to enter the configuration page.

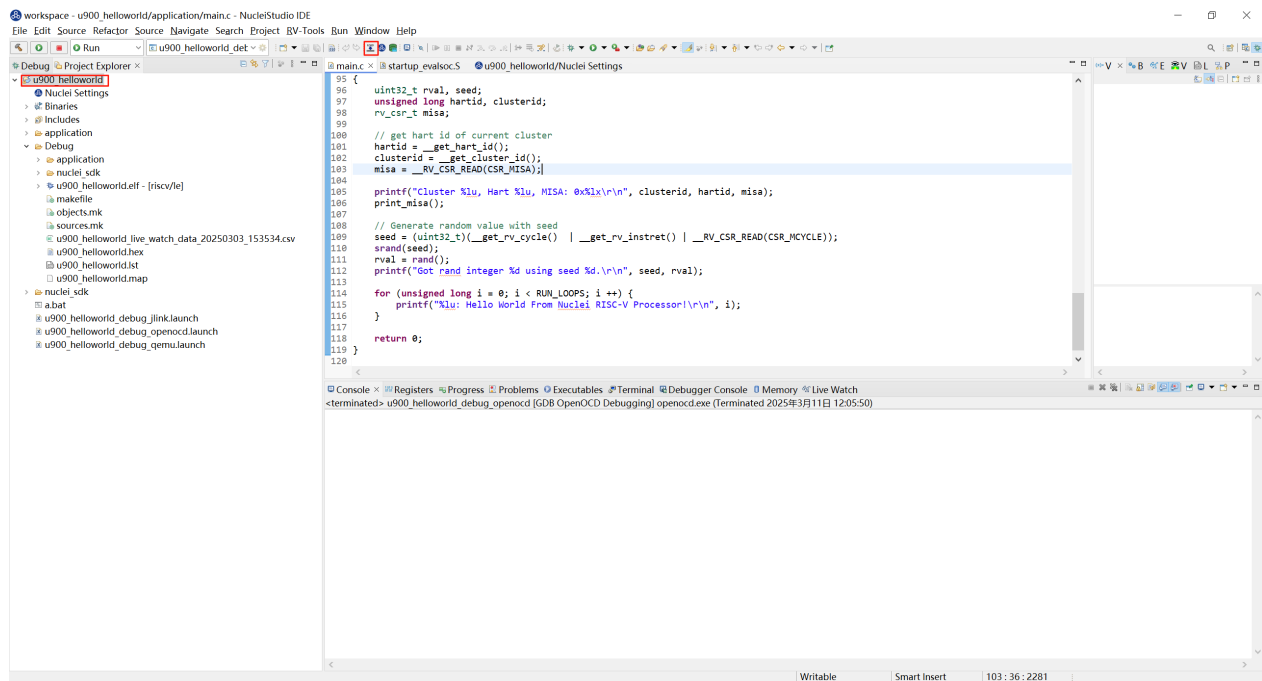
Since this is FLASH download mode, the default options verify image and reset and run are sufficient here.



For details of each configuration item, refer to the [Nuclei Development Tool Guide](#)

step4: Download

Select the project and click Flash Programming to download the binary file to the hardware development board.

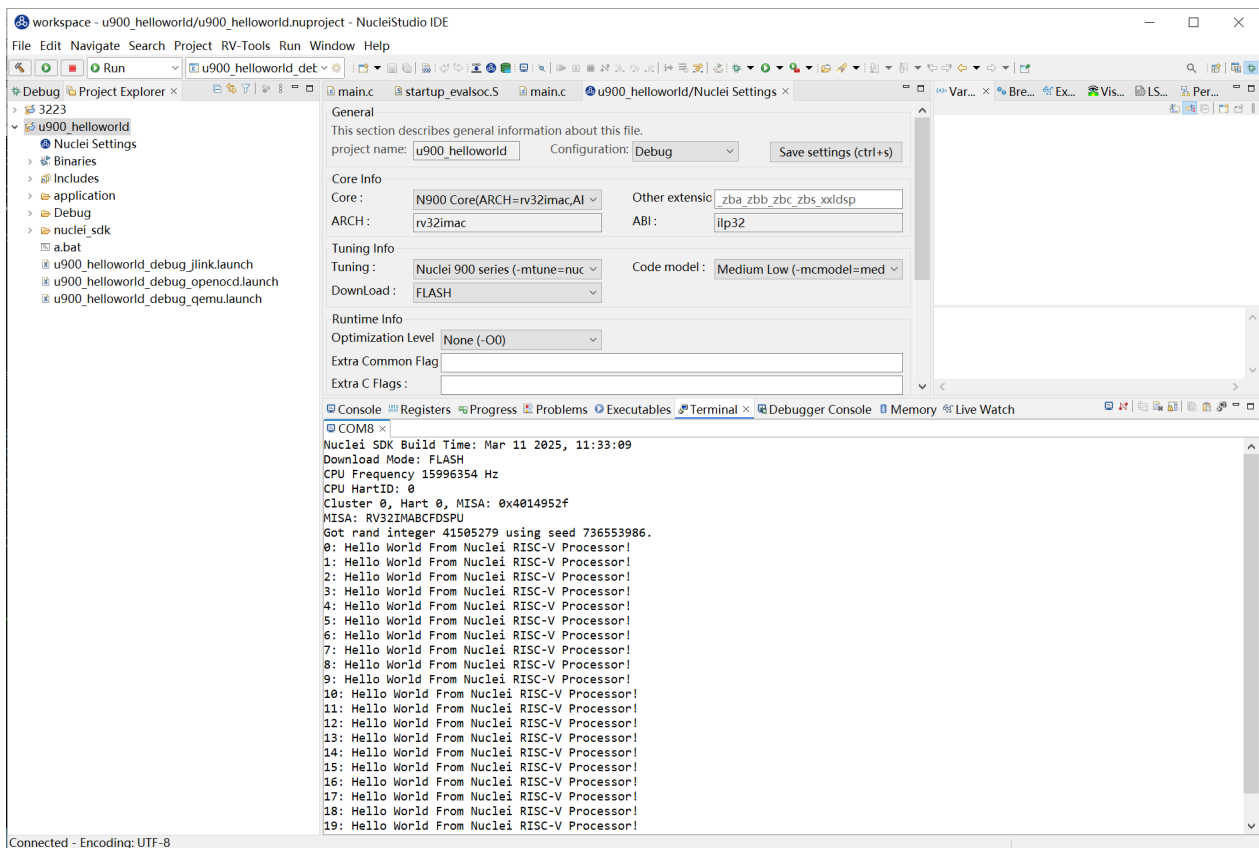
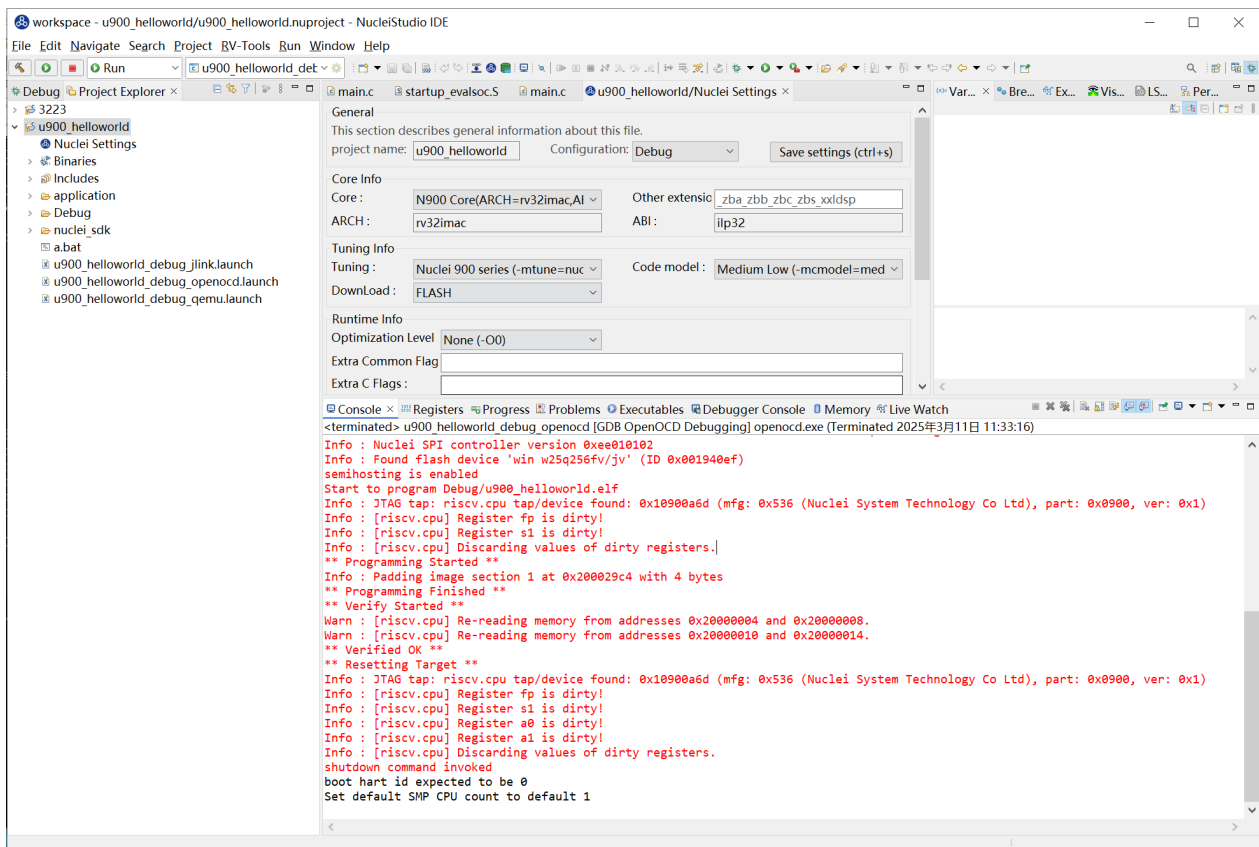


After the download succeeds, users can view the download result in the Console to confirm that the binary file has been successfully programmed into the hardware.

```

** Programming Started **
Info : Padding image section 1 at 0x200029c4 with 4 bytes
** Programming Finished **
** Verify Started **
Warn : [riscv.cpu] Re-reading memory from addresses 0x20000004 and 0x20000008.
Warn : [riscv.cpu] Re-reading memory from addresses 0x20000010 and 0x20000014.
** Verified OK **
** Resetting Target **
Info : JTAG tap: riscv.cpu tap/device found: 0x10900a6d (mfg: 0x536 (Nuclei System Technology Co Ltd), part: 0x0900, ver: 0x1)
Info : [riscv.cpu] Register fp is dirty!
Info : [riscv.cpu] Register s1 is dirty!
Info : [riscv.cpu] Register a0 is dirty!
Info : [riscv.cpu] Register a1 is dirty!
Info : [riscv.cpu] Discarding values of dirty registers.
shutdown command invoked

```

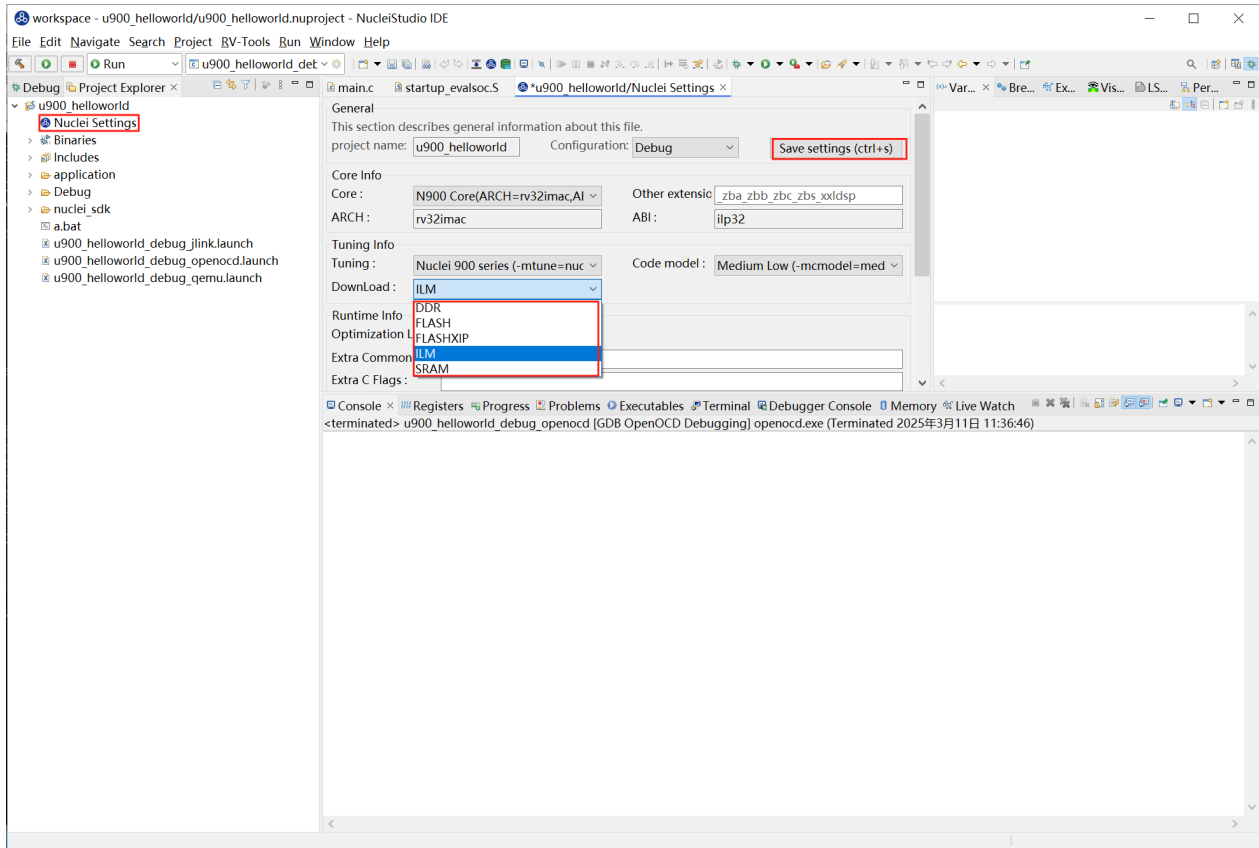


step5: Differences when downloading to memory

Nuclei Studio supports multiple download modes: DDR, FLASH, FLASHXIP, ILM, and SRAM.

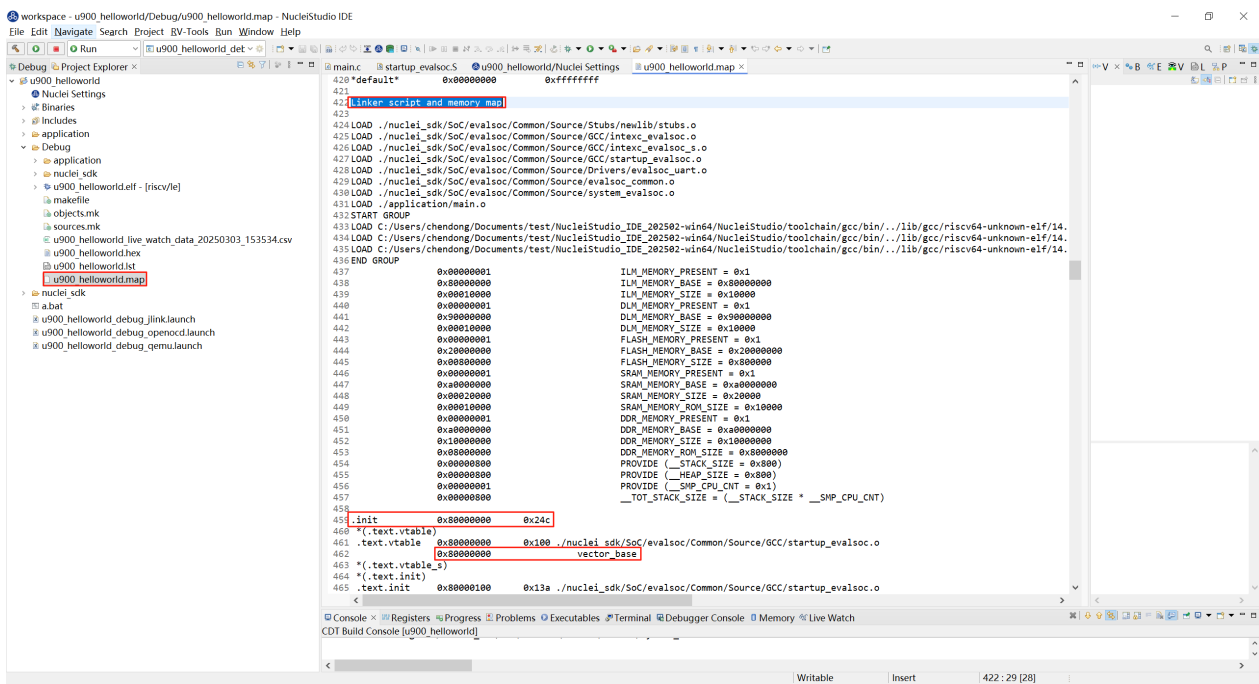
The FLASH and FLASHXIP modes can be used by following the steps above, while DDR, ILM, and SRAM download to memory, which differs from Flash. The following uses ILM as an example.

Click Nuclei Settings to open the page, select ILM in Download, and save.



Rebuild the project: clean project -> build project

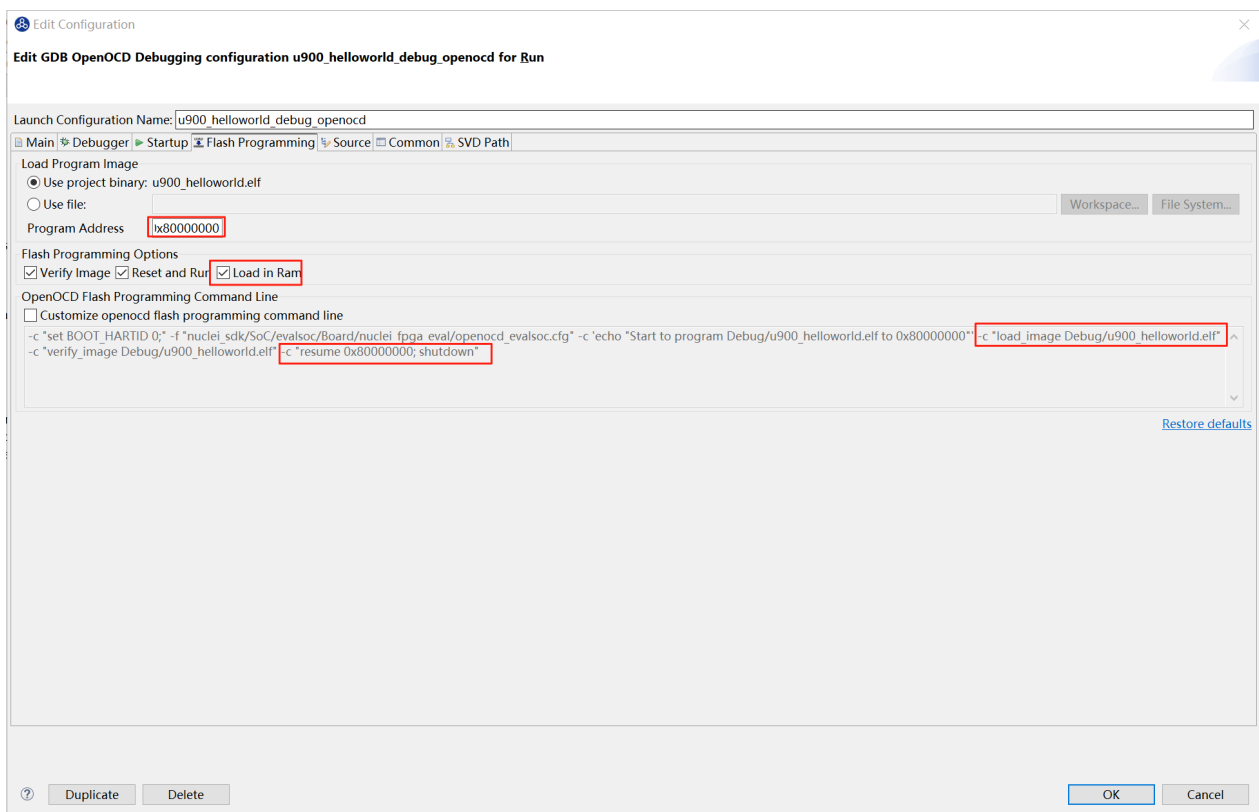
Then open the corresponding .map file, here u900_helloworld.map, and find the initial load address in it, such as 0x80000000 in the figure below.



Open the Flash Programming tab. Since the download is to memory, check Load in Ram here. At this point, the load_image command is added to the command line below.

Then enter the address obtained above, 0x80000000, in Program Address, and the command line will include the resume 0x80000000 parameter.

Click OK.

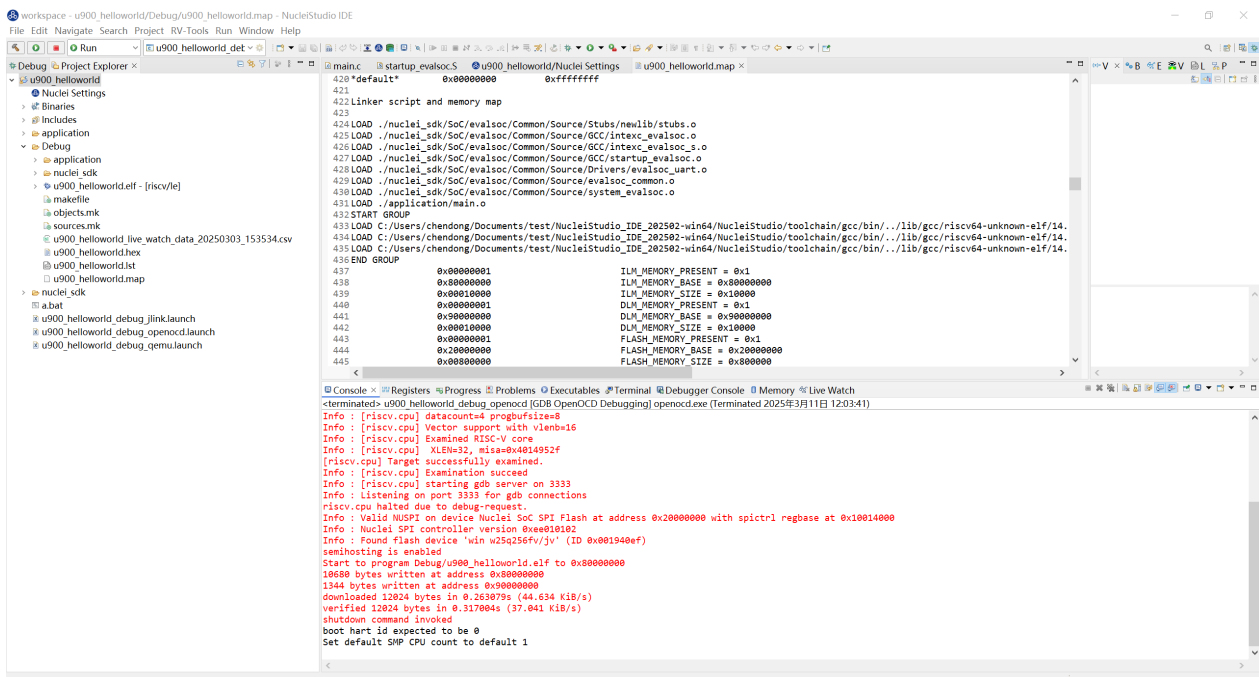


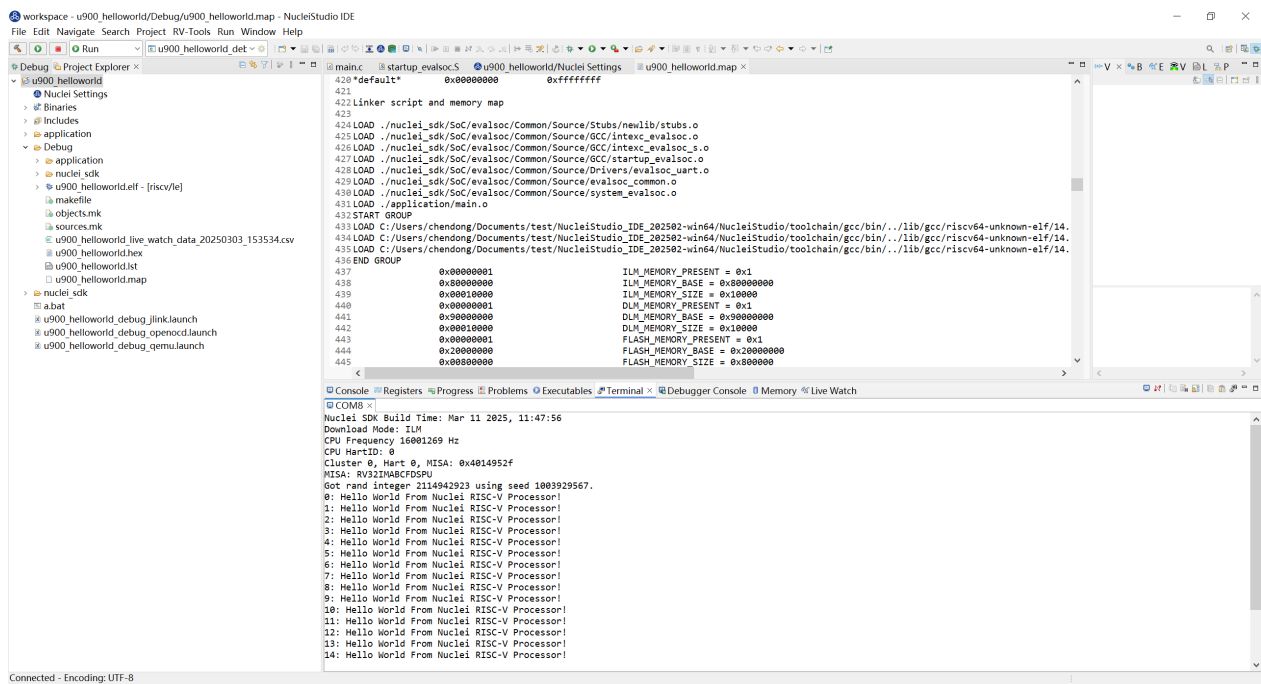
Select the project and click Flash Programming to download. The result is as follows.

```

Info : Valid NUSPI on device Nuclei SoC SPI Flash at address
0x20000000 with spictrl regbase at 0x10014000
Info : Nuclei SPI controller version 0xee010102
Info : Found flash device 'win w25q256fv/jv' (ID 0x001940ef)
semihosting is enabled
Start to program Debug/u900_helloworld.elf to 0x80000000
10680 bytes written at address 0x80000000
1344 bytes written at address 0x90000000
downloaded 12024 bytes in 0.263079s (44.634 KiB/s)
verified 12024 bytes in 0.317004s (37.041 KiB/s)
shutdown command invoked

```





step6: Possible issues

1. Error: checksum mismatch , attempting binary compare

This error occurs because the flash download and ram download modes were confused. You need to modify the Download mode in nuclei settings.

Summary

The Flash Programming feature provides users with a fast and convenient way to download compiled binary files to a hardware development board. With simple configuration, users can easily adapt to different hardware environments and ensure that binary files are programmed correctly.

Using the Live Watch Feature¶

Live Watch is a powerful real-time monitoring tool designed for developers to help you debug and optimize code more efficiently. With Live Watch, you can instantly observe how variables change while the program is running, without interrupting execution or manually adding log statements. The Live Watch feature is implemented in Nuclei Studio 2025.02. It supports automatic refreshing of variable values, ensuring that you always see the latest data changes. Its intuitive graphical interface makes it easy to manage the variables you want to monitor.

Background¶

The Live Watch feature depends on Nuclei OpenOCD version ≥ 2025.02 , and it only supports Nuclei CPUs configured with the RISC-V SBA feature. With Live Watch, developers can monitor variable changes in real time during debugging, helping to quickly locate issues and optimize code performance.

Solution¶

Environment Preparation¶

Nuclei Studio:

- [NucleiStudio 202502 Windows](#)
- [NucleiStudio 202502 Linux](#)

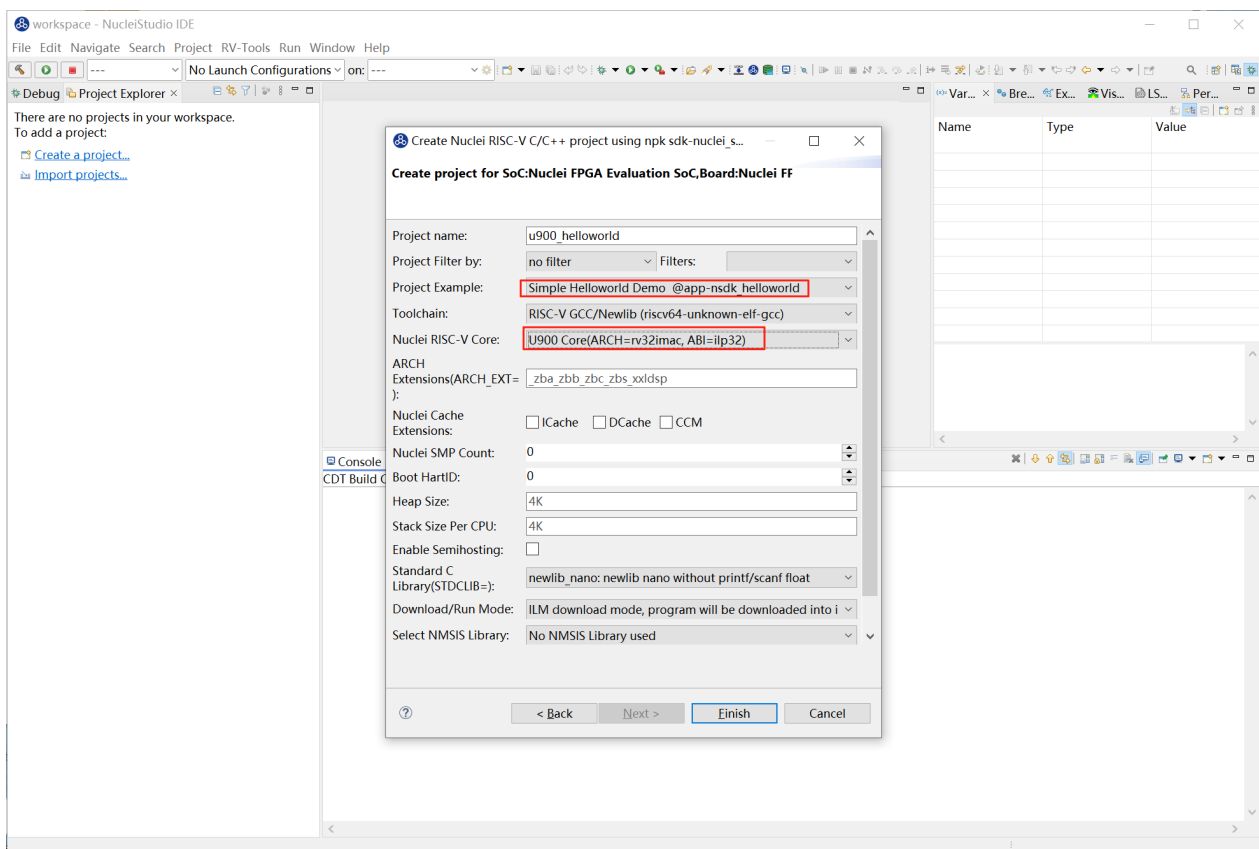
Nuclei OpenOCD:

- Make sure the installed OpenOCD version is ≥ 2025.02 and that it supports the RISC-V SBA feature.

Live Watch Usage Demo¶

Step 1: Create a project and flash the bitstream

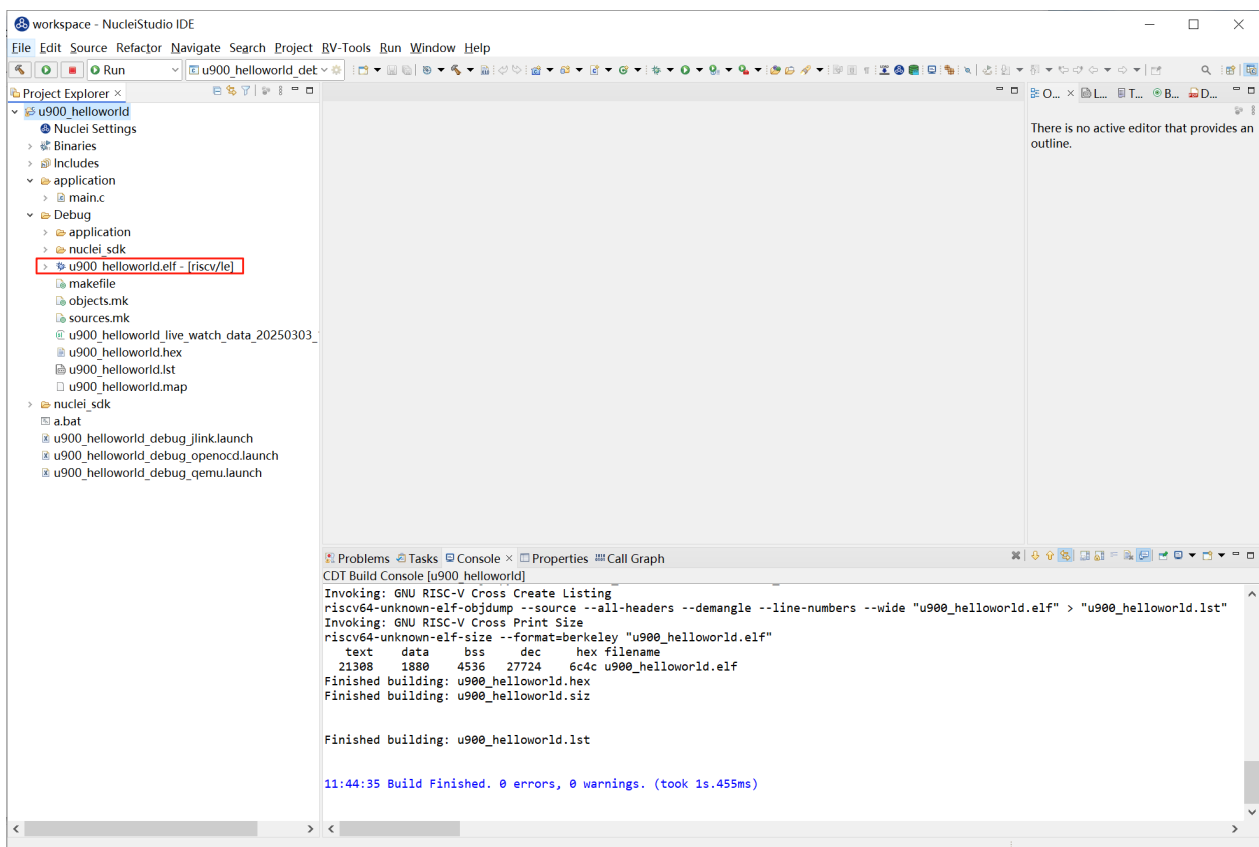
Use version 0.7.1 of `sdk-nuclei_sdk` to create a u900 helloworld project. Select Simple Helloworld Demo and U900 Core in turn, then click Finish.



Flash the corresponding bitstream to the development board. Here we use trace-u900_best_config_ku060_16M_e85631d489_e82e2771f_202409232110_v3.12.0.bit

Step 2: Build the original Nuclei SDK project

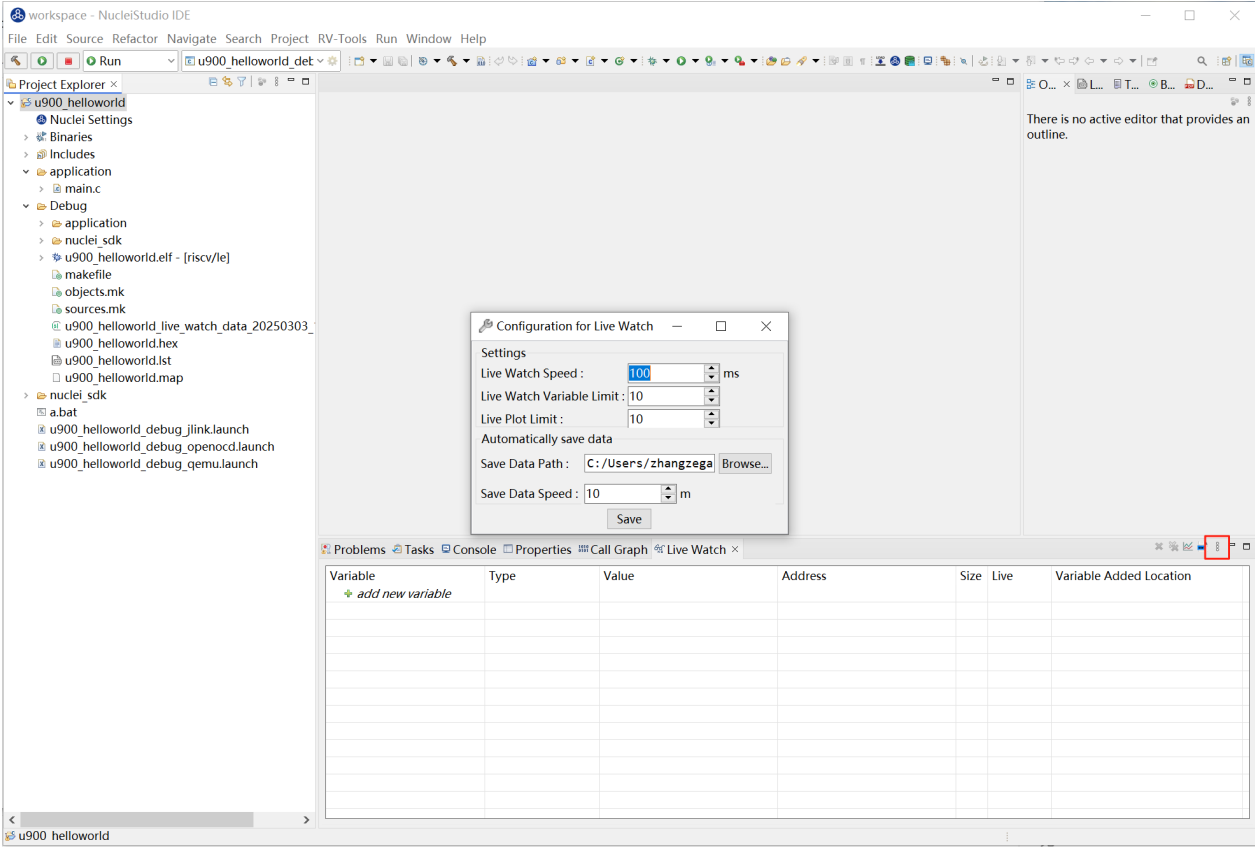
Build the original project and make sure the build succeeds and that the generated elf file can be found under Debug:



Step 3: Open the Live Watch view

Open the Live Watch view, find Live Watch Settings, and configure the relevant parameters as needed. Here we simply use the default values.

For configuration details, refer to the figure below or the [Nuclei Development Tool Guide](#)



Step 4: Run the original Nuclei SDK project

Debug and run the program, then add the variable `seed` you want to observe in the Live Watch view.

If you want to view the variable's change curve through Live Plot, select that record, right-click it, and choose **Toggle Live Plot** from the pop-up menu. The Live Plot tool will then pop up.

workspace - u900_helloworld/application/main.c - NucleiStudio IDE

File Edit Source Refactor Source Navigate Search Project RV-Tools Run Window Help

Debug × Project Explorer × main.c × [main()] at main.c:101 0x800011c0 × startup_evalsoc.S ×

u900_helloworld_debug_openocd [GDB OpenOCD]

u900_helloworld.elf

Thread #1 (Suspended : Breakpoint)

main() at main.c:101 0x800011c0

openocd.exe

riscv64-unknown-elf-gdb

```

95 {
96     uint32_t rval, seed;
97     unsigned long hartid, clusterid;
98     rv_csr_t misa;
99
100     // get hart id of current cluster
101     hartid = __get_hart_id();
102     clusterid = __get_cluster_id();
103     misa = __RV_CSR_READ(CSR_MISA);
104
105     printf("Cluster %lu, Hart %lu, MISA: 0x%x\n", clusterid, hartid, misa);
106     print_misa();
107
108     // Generate random value with seed
109     seed = (uint32_t)(__get_rv_cycle() | __get_rv_instret() | __RV_CSR_READ(CSR_MISA));
110     srand(seed);
111     rval = rand();
112     printf("Got rand integer %d using seed %d.\n", seed, rval);
113
114     for (unsigned long i = 0; i < RUN_LOOPS; i++) {
115         printf("%lu: Hello World From Nuclei RISC-V Processor!\n", i);
116     }
117
118     return 0;
119 }
120

```

Variable Type Value Address Size Live Variable Address

Variable	Type	Value	Address	Size	Live	Variable Address
seed	uint32_t	319487		4	☒	main() at C:\...

add new variable

Remove

Remove All

Toggle Live Watch

Toggle Live Plot

Save as CSV

Number Format

Value

Time(H:M:S.MS)

seed

When the project runs at full speed, you can see the variable's value changing at the configured Live Watch Speed. The curve plotted by Live Plot is shown below.

workspace - u900_helloworld/application/main.c - NucleiStudio IDE

File Edit Source Refactor Source Navigate Search Project RV-Tools Run Window Help

Debug × Project Explorer × main.c × [main()] at main.c:101 0x800011c0 × startup_evalsoc.S ×

u900_helloworld_debug_openocd [GDB OpenOCD]

u900_helloworld.elf

Thread #1 (Running : User Request)

openocd.exe

riscv64-unknown-elf-gdb

```

95 {
96     uint32_t rval, seed;
97     unsigned long hartid, clusterid;
98     rv_csr_t misa;
99
100     // get hart id of current cluster
101     hartid = __get_hart_id();
102     clusterid = __get_cluster_id();
103     misa = __RV_CSR_READ(CSR_MISA);
104
105     printf("Cluster %lu, Hart %lu, MISA: 0x%x\n", clusterid, hartid, misa);
106     print_misa();
107
108     // Generate random value with seed
109     seed = (uint32_t)(__get_rv_cycle() | __get_rv_instret() | __RV_CSR_READ(CSR_MISA));
110     srand(seed);
111     rval = rand();
112     printf("Got rand integer %d using seed %d.\n", seed, rval);
113
114     for (unsigned long i = 0; i < RUN_LOOPS; i++) {
115         printf("%lu: Hello World From Nuclei RISC-V Processor!\n", i);
116     }
117
118     return 0;
119 }
120

```

Variable Type Value Address Size Live Variable Address

Variable	Type	Value	Address	Size	Live	Variable Address
seed	uint32_t	319487	0x9000ffb4	4	☒	main() at C:\...

add new variable

Live Plot

Value

Time(H:M:S.MS)

seed

As more and more data nodes accumulate over time, the data nodes will be hidden. You can right-click Suspend in Live Plot to pause, then scroll the mouse wheel to zoom in on the curve. Once zoomed in to a certain level, the nodes will be displayed, and you can hover the mouse over a node to view its data details. Click Continue to let Live Plot resume plotting the curve.

The screenshot displays the NucleiStudio IDE interface. The top panel shows the source code for `main.c` at address `0x800011c0`. The code includes a function `main()` that initializes variables `seed`, `hartid`, and `clusterid`, and prints a message. The bottom panel shows the **Live Watch** and **Live Plot** windows.

Live Watch Table:

Variable	Type	Value	Address	Size	Live	Variable At
seed	uint32_t	317439	0x9000ffb4	4	☒	main() at C
+ add new variable						

Live Plot:

The Live Plot window shows a graph titled "Live Plot". The Y-axis is labeled "Value" and ranges from `3.17439E5` to `3.17439E5`. The X-axis is labeled "Time(H:M:S.MS)" and ranges from `00:00:50` to `00:01:45`. A red line represents the variable `seed`. A tooltip shows "Suspend" and "Continue" buttons.

Select the seed row and right-click it to save the variable's results as a CSV file for review and use.

The screenshot shows the NucleiStudio IDE interface. The main editor displays the source code of `main.c`. The bottom panel is divided into two sections: a table for live-watched variables and a corresponding plot.

Variable	Type	Value	Address	Size	Live	Variable Address
seed	uint32_t	317439	0x317439	4	☒	main() at C:\...

The 'Live Plot' graph shows the value of 'seed' over time. The y-axis is labeled 'Value' and ranges from 3.17439E5 to 3.17439E5. The x-axis is labeled 'Time(H:M:S.MS)' and ranges from 00:03:35 to 00:04:30. A single red line represents the value of 'seed', which remains constant at 3.17439E5.

Live Watch also automatically saves the queried data results to the Save Data Path. You can find the corresponding CSV data files at the address specified by Save Data Path.

The screenshot shows the NucleiStudio IDE interface with the 'Configuration for Live Watch' dialog box open. The dialog contains the following settings:

- Live Watch Speed: 100 ms
- Live Watch Variable Limit: 10
- Live Plot Limit: 10
- Save Data Path: C:/Users/zhangzega (highlighted with a red box)
- Save Data Speed: 10 m

The 'Live Plot' graph shows the value of 'seed' over time. The y-axis is labeled 'Value' and ranges from 3.17439E5 to 3.17439E5. The x-axis is labeled 'Time(H:M:S.MS)' and ranges from 00:04:50 to 00:05:45. A single red line represents the value of 'seed', which remains constant at 3.17439E5.

电脑 > Windows (C:) > 用户 > workspace > u900_helloworld > Debug

名称	修改日期	类型
application	2025/3/10 11:44	文件夹
nuclei_sdk	2025/3/10 11:44	文件夹
makefile	2025/3/10 11:44	文件
objects.mk	2025/3/5 12:06	Makefile 源文件
sources.mk	2025/3/5 12:04	Makefile 源文件
u900_helloworld.elf	2025/3/10 11:44	ELF 文件
u900_helloworld.hex	2025/3/10 11:44	HEX 文件
u900_helloworld.lst	2025/3/10 11:44	LST 文件
u900_helloworld.map	2025/3/10 11:44	MAP 文件
u900_helloworld_live_watch_data_20250303_153534.csv	2025/3/3 15:39	XLS 工作表

Summary

The Live Watch feature provides developers with a powerful real-time monitoring tool that greatly improves debugging efficiency and code optimization capabilities. By using Live Watch effectively, developers can handle complex debugging tasks more easily and improve development productivity.

Tutorial for Adding Custom Assembly Instructions in LLVM¶

All examples below use 32-bit instructions

Recognizing the Custom Extension Name¶

The following uses the xnice extension as an example

File: `llvm/lib/Target/RISCV/RISCVFeatures.td`

Content to add:

```
def FeatureVendorXnice
  : RISCVExtension<"xnice", 1, 0,
    "'Xnice' (Xnice extension)">;
def HasVendorXnice
  : Predicate<"Subtarget->hasVendorXnice()">,
  AssemblerPredicate<(all_of FeatureVendorXnice),
    "'Xnice' (Xnice extension)">;
```

Note: In `RISCVExtension`, the first `xnice` is the actual extension name recognized by the LLVM compiler, the following `1, 0` is the version number of the extension, and the second `Xnice` is only used for the extension's functional description

Recognizing Custom Assembly Instructions¶

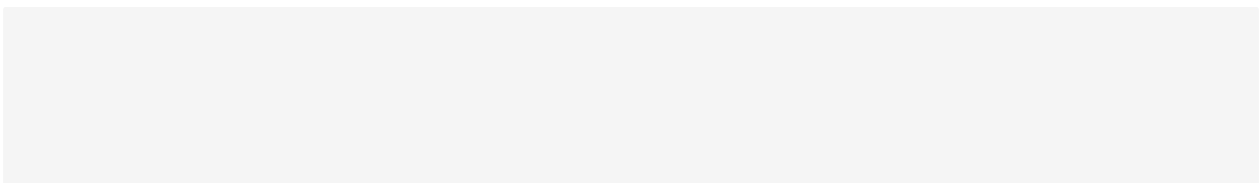
The following uses the addition of a standard R-type nice instruction as an example

1. Add the corresponding decoder

File: `llvm/lib/Target/RISCV/Disassembler/RISCVDisassembler.cpp`

Function: `DecodeStatus RISCVDisassembler::getInstruction32()`

Content:



```
TRY_TO_DECODE_FEATURE(RISCV::FeatureVendorXnice,
DecoderTableXnice32,
    "Xnice extension");
```

1. Create the encoding/decoding file

Create an encoding/decoding file named `RISCVInstrInfoXnice.td` under `llvm/lib/Target/RISCV/`, and include this file in `llvm/lib/Target/RISCV/RISCVInstrInfo.td`

```
include "RISCVInstrInfoXnice.td"
```

1. Instruction encoding

Assume the assembly format of the nice instruction is `nice rd, rs1, rs2`, and it uses the encoding space of the custom3 region reserved by RISC-V. The encoding steps are as follows:

- Create a class named `XniceInstr` to describe the unified format of all instructions in the XNICE extension. Since it is an R-type instruction, it can be directly inherited from the pre-defined `RVInstR` class in LLVM; otherwise, you need to inherit from another matching class or write a new instruction class from the base class. The declarations of all instruction classes in LLVM are located in `llvm/lib/Target/RISCV/RISCVInstrFormats.td`
- Use `Predicates` to specify the extension the nice instruction belongs to (`[HasVendorXnice]`) and the decoder it uses (`DecoderNamespace = "Xnice"`)
- Use `def` to add an instruction definition. You only need `XniceInstr` and to fill in the parameters missing from the class declaration to complete the encoding of an instruction. For example, a custom R-type instruction only requires declaring `funct7`, `funct3`, and the assembly instruction name again

Complete example:

```
let hasSideEffects = 0, mayLoad = 0, mayStore = 0 in {
  class XniceInstr<bits<7> funct7, bits<3> funct3, string opcodestr>
    : RVInstR<funct7, funct3, OPC_CUSTOM_3, (outs GPR:$rd),
      (ins GPR:$rs1, GPR:$rs2), opcodestr, "$rd, $rs1,
$rs2">;
}

let Predicates = [HasVendorXnice], DecoderNamespace = "Xnice" in {
  def NICE : XniceInstr<0b1111010, 0b001, "nice">;
}
```

`OPC_CUSTOM_3` is a macro already reserved in `llvm/lib/Target/RISCV/RISCVInstrFormats.td`. If you use a different encoding space, you can directly look it up and change it

The uppercase `NICE` after `def` is generally used for intrinsics or auto-vectorization calls. When only doing assembly, you can simply provide the uppercase format of the assembly instruction name for differentiation

In addition, the above example does not restrict the usage scenarios of the instruction under RV32/64, so it can be recognized under both RV32 and RV64. If you need to restrict it to RV32 only, you need to additionally specify the restriction together with the extension in `Predicates`, for example `[HasVendorXxlczbitop, IsRV32]`

Usage Instructions¶

Usage is the same as with GCC. You only need to pass `xnice` to the LLVM compiler via the `-march` option, for example `-march=rv32imafdc_xnice`

References¶

LLVM TableGen syntax reference <https://llvm.org/docs/TableGen/ProgRef.html>

PLCT's example of adding custom RISC-V instructions in LLVM <https://www.bilibili.com/video/BV1JR4y1J7he>

Extension recognition and assembly implementation of Nuclei's custom VPU instructions <https://github.com/riscv-mcu/llvm-project/commit/f5d025b9800f3cd662e93c11eb7c7b0f65ca4472>

How to Use the DebugMap Registers Provided by Nuclei to Analyze Error Scenarios¶

First, Confirm That the Hardware Supports the DebugMap Feature¶

There is a signal called `dm_map_enable` at the top level of the core. Tying this signal to 1 enables the DebugMap feature.

For details, refer to the *Debug Control Interface* section of the *Nuclei_CPU_Debug_Function_Specification.pdf* document

What Are DebugMap Registers¶

The DebugMap feature allows you to inspect the internal state of the Core via OpenOCD when the Core is hung. Several internal states are mapped into DM registers. Currently, only the following three states are mapped:

- 00: Commit PC(i0 for dual issue)
- 16: ICache miss address(ICache is supported)
- 32: DCache address waiting for retire(DCache is supported)

For details, refer to the *CFR0 (Custom Feature Register0)* section of the *Nuclei_CPU_Debug_Function_Specification.pdf* document

DebugMap Output Information in OpenOCD¶

When using OpenOCD to connect to an FPGA/chip, you will often see output similar to the following:

```
Info : coreid=0, nuclei debug map reg 00: 0xa00003ac, 16: 0xa0003240, 32: 0x10003014
```

- coreid indicates which core's debug-map information is currently being output

Possible Error Scenarios¶

Error scenario 1:

```
Info : Using libusb driver
Info : clock speed 1000 kHz
Info : JTAG tap: riscv.cpu tap/device found: 0x10900a6d (mfg:
0x536 (Nuclei System Technology Co Ltd), part: 0x0900, ver: 0x1)
Info : [riscv.cpu] datacount=4 progbufsize=8
Info : coreid=0, nuclei debug map reg 00: 0xa0000496, 16:
0xa0003140, 32: 0x10002ff8
Error: [riscv.cpu] Unable to halt hart 0. dmcontrol=0x00000001,
dmstatus=0x00400ca2
Error: [riscv.cpu] Fatal: Hart 0 failed to halt during examine()
Warn : target riscv.cpu examination failed
Info : [riscv.cpu] datacount=4 progbufsize=8
Error: Hart 0 doesn't exist.
Error: Fatal: Failed to read s0 from hart 0.
Info : [riscv.cpu] datacount=4 progbufsize=8
Error: Hart 0 doesn't exist.
Error: Fatal: Failed to read s0 from hart 0.
Info : starting gdb server for riscv.cpu on 22800
Info : Listening on port 22800 for gdb connections
Error: Target not examined yet
```

Error scenario 2:

```
Info : libusb_open() failed with LIBUSB_ERROR_NOT_FOUND
Info : no device found, trying D2xx driver
Info : D2xx device count: 2
Info : Connecting to "(null)" using D2xx mode...
Info : clock speed 1000 kHz
Info : JTAG tap: riscv0.cpu tap/device found: 0x10300a6d (mfg:
0x536 (Nuclei System Technology Co Ltd), part: 0x0300, ver: 0x1)
Info : [riscv0.cpu] datacount=4 progbufsize=2
Info : coreid=0, nuclei debug map reg 00: 0xa0000496, 16:
0xa0003140, 32: 0x10002ff8
Info : Examined RISC-V core; found 1 harts
Info : hart 0: XLEN=32, misa=0x40001127
[riscv0.cpu] Target successfully examined.
Info : starting gdb server for riscv0.cpu on 3333
Info : Listening on port 3333 for gdb connections
Started by GNU MCU Eclipse
Info : Listening on port 6666 for tcl connections
Info : Listening on port 4444 for telnet connections
```



```

Info : accepting 'gdb' connection on tcp/3333
Warn : Prefer GDB command "target extended-remote :3333" instead
of "target remote :3333"
Error: Timed out after 2s waiting for busy to go low
(abstracets=0x2001004). Increase the timeout with riscv
set_command_timeout_sec.
Error: Abstract command ended in error 'busy'
(abstracets=0x2001104)

```

How to Correctly Use DebugMap to Analyze Error Scenarios¶

- After the Core-hang symptom occurs, you need to use OpenOCD to connect to the FPGA/ chip again without power-cycling or resetting. Only the DebugMap output at this point can be used to analyze the error scenario.
- "00": The currently committed PC — indicates the most recently committed PC. From this information, you can roughly infer which PC location the CPU has reached.
- "16": If ICache is configured, this records the most recent address issued by the ICache (ILM addresses are not recorded for now). In theory, the ICache has 2 outstanding transactions; what is recorded is the address of the earliest issued transaction that has not yet returned a response.
- "32": If DCache is configured, this records the most recent address issued by the DCache (DLM and Mem can also be recorded; addresses issued to PPI/FIO are not recorded for now). In theory, the DCache has many outstanding transactions; what is recorded is the address of the earliest issued transaction that has not yet returned a response.

Reading Other DebugMap Registers via OpenOCD¶

OpenOCD provides a set of commands, *nuclei expose_cpu_core* and *nuclei examine_cpu_core*, which can be used to read other DebugMap registers. For details, see https://doc.nucleisys.com/nuclei_tools/openocd/intro.html#debug-map-feature

Command implementation and usage in OpenOCD: [source code](#)

- Note that the *nuclei expose_cpu_core* command must be used before the init command
- *nuclei examine_cpu_core* is used after the init command, and can also be used after a gdb/telnet connection is established. Note that sending commands from gdb to OpenOCD requires the monitor keyword: *monitor nuclei examine_cpu_core*

Tutorial for Adding Custom Assembly Instructions in binutils

The following examples all use 32-bit instructions.

Recognizing the Custom Extension Name

The following uses the xnice extension as an example.

File: bfd\elfxx-riscv.c

riscv_supported_vendor_x_ext[] function:

```
static struct riscv_supported_ext riscv_supported_vendor_x_ext[] ={
    {"xnice",  ISA_SPEC_CLASS_DRAFT, 1, 0, 0},
}
```

Tips: This function is responsible for adding the extension name and version number, where the first two values 1, 0 are the extension's version number.

riscv_multi_subset_supports function:

```
/* Each instuction is belonged to an instruction class
   INSN_CLASS_*.
   Call riscv_subset_supports to make sure if the instuction is
   valid. */

bool
riscv_multi_subset_supports (riscv_parse_subset_t *rps,
                             enum riscv_insn_class insn_class)
{
    switch (insn_class)
    {
        case INSN_CLASS_XNICE:
            return riscv_subset_supports (rps, "xnice");
    }
}
```

Tips: The content to be added goes inside the switch statement, establishing the association between INSN_CLASS_XNICE, which corresponds to the instructions of the added xnice extension, and the xnice extension itself.

`riscv_implicit_subsets[]` function: (optional)

```
/* Please added in order since this table is only run once time.
 */
static struct riscv_implicit_subset riscv_implicit_subsets[] = {
    {"xnice", "+zve32x", check_implicit_always},
}
```

Tips: This function controls whether the custom `xnice` extension depends on other extensions. If there are no dependencies, nothing needs to be added. Assuming it depends on the `zve32x` extension, you need to add the dependency in the format shown above within this function. If it depends on multiple extensions, continue adding them after the `zve32x` extension.

File `include\opcode\riscv.h`

```
enum riscv_insn_class
{
    INSN_CLASS_XNICE,
}
```

Tips: This file is mainly responsible for declaring `INSN_CLASS_XNICE` in the `riscv_insn_class` enumeration.

Recognizing Custom Assembly Instructions

The following uses adding a standard R-type nice instruction as an example.

1. Add the instruction encoding

Assume the assembly format of this nice instruction is `nice rd, rs1, rs2`, and it uses the encoding space of the RISC-V reserved custom3 region. Its encoding is:

Inst. format	Func7	rs2	rs1	Func3	rd	opcode
<code>xnice rd,rs1,rs2</code>	1011101	rs2	rs1	001	rd	1111011

- Generate the opcode macros required by the compiler (it is recommended to use the `riscv-opcodes` repository at <https://github.com/riscv/riscv-opcodes/tree/master>)

```
git clone https://github.com/riscv/riscv-opcodes.git
```

```
cd riscv-opcodes/extensions/unratified/
```

```
vim rv_xnice //在该文件夹下创建xnice扩展指令文件(文件名规则是rv_name), 并
```

根据指令模板添加一条指令

```
nice rd rs1 rs2 31..25=0x5D 14..12=1 6..2=0x1E 1..0=3 //此为需要添加的指令
```

```
cd ../../
```

```
make EXTENSIONS='unratified/rv_xnice'
```

After the above steps, the opcode macros are obtained in the `riscv-opcodes/encoding.out.h` file, as shown below:

```
#define MATCH_NICE 0xba00107b
#define MASK_NICE 0xfe00707f
```

Note: You can also generate the macros manually based on the encoding. The rule is: for the encoding of `MATCH_NICE`, all undefined bit positions are 0 and the remaining positions stay unchanged. For the encoding of `MASK_NICE`, all undefined bit positions are 0 and all remaining positions are 1.

- In the `include\opcode\riscv-opc.h` file, add the macros generated above.
- Add the association between the extension and the extension's instruction encodings

File: `opcodes\riscv-opc.c`

`riscv_opcodes[]` function:

```
const struct riscv_opcode riscv_opcodes[] =
{
/* name, xlen, isa, operands, match, mask, match_func, pinfo. */
{"xnice", 0, INSN_CLASS_XNICE, "d,s,t", MATCH_XNICE, MASK_XNICE,
match_opcode, 0 },
}
```

Tips: The first 0 means this instruction has no xlen requirement. d,s,t correspond to rd,rs1,rs2 respectively; the corresponding mapping relationships can be found in the `validate_riscv_insn` function in the `gas/config/tc-riscv.c` file.

Usage Instructions

When using it, pass `xnice` to the compiler through the `-march` option, for example `-march=rv32imafdc_xnice`.

References:¶

Modifying binutils to add assembly instructions on RISC-V:

<https://blog.cyyself.name/add-compile-instr-for-riscv/>

Extension recognition and assembly implementation of Nuclei custom VPU instructions:

<https://github.com/riscv-mcu/riscv-binutils-gdb/commit/c8806f4bd8c1a1673ec61ad3badfc3d490fa52f7>

Guide to Using the Nuclei Cross-Trigger Feature in OpenOCD

Feature Overview

To meet the need for synchronized halt and resume in AMP multicore debugging, the Nuclei RISC-V CPU implements the cross-trigger feature. OpenOCD has integrated the following two synchronization control features:

1. Synchronized Halt Group (halt_group) - When any core in the group halts, all other members automatically halt in sync
2. Synchronized Resume Group (resume_group) - When any core in the group resumes execution, all other members automatically resume in sync

Basic command format:

```
# add target to halt_group
nuclei cti halt_group on $_TARGETNAME0 $_TARGETNAME1

# remove target from halt_group
nuclei cti halt_group off $_TARGETNAME0 $_TARGETNAME1

# add target to resume_group
nuclei cti resume_group on $_TARGETNAME0 $_TARGETNAME1

# remove target from resume_group
nuclei cti resume_group off $_TARGETNAME0 $_TARGETNAME1
```

Configuration File Examples

1. Synchronized Halt Group Configuration

```
adapter_khz      1000

interface ftdi
ftdi_vid_pid 0x0403 0x6010
ftdi_oscan1_mode off

transport select jtag
```

```

ftdi_layout_init 0x0008 0x001b
ftdi_layout_signal nSRST -oe 0x0020 -data 0x0020
ftdi_layout_signal TCK -data 0x0001
ftdi_layout_signal TDI -data 0x0002
ftdi_layout_signal TDO -input 0x0004
ftdi_layout_signal TMS -data 0x0008
ftdi_layout_signal JTAG_SEL -data 0x0100 -oe 0x0100

set _CHIPNAME0 riscv0
jtag newtap $_CHIPNAME0 cpu -irlen 5 -expected-id 0x10900a6d
set _TARGETNAME0 $_CHIPNAME0.cpu
target create $_TARGETNAME0 riscv -chain-position $_TARGETNAME0 -
coreid 0

set _CHIPNAME1 riscv1
jtag newtap $_CHIPNAME1 cpu -irlen 5 -expected-id 0x10900a6d
set _TARGETNAME1 $_CHIPNAME1.cpu
target create $_TARGETNAME1 riscv -chain-position $_TARGETNAME1 -
coreid 0

init
#reset

if {[ info exists pulse_srst]} {
    ftdi_set_signal nSRST 0
    ftdi_set_signal nSRST z
}

# add targets to the halt group
nuclei cti halt_group on $_TARGETNAME0 $_TARGETNAME1

foreach t [target names] {
    targets $t
    halt
}

```

2. Synchronized Resume Group Configuration¶

```

adapter_khz      1000

interface ftdi
ftdi_vid_pid 0x0403 0x6010
ftdi_oscan1_mode off

transport select jtag

```

```

ftdi_layout_init 0x0008 0x001b
ftdi_layout_signal nSRST -oe 0x0020 -data 0x0020
ftdi_layout_signal TCK -data 0x0001
ftdi_layout_signal TDI -data 0x0002
ftdi_layout_signal TD0 -input 0x0004
ftdi_layout_signal TMS -data 0x0008
ftdi_layout_signal JTAG_SEL -data 0x0100 -oe 0x0100

set _CHIPNAME0 riscv0
jtag newtap $_CHIPNAME0 cpu -irlen 5 -expected-id 0x10900a6d
set _TARGETNAME0 $_CHIPNAME0.cpu
target create $_TARGETNAME0 riscv -chain-position $_TARGETNAME0 -
coreid 0

set _CHIPNAME1 riscv1
jtag newtap $_CHIPNAME1 cpu -irlen 5 -expected-id 0x10900a6d
set _TARGETNAME1 $_CHIPNAME1.cpu
target create $_TARGETNAME1 riscv -chain-position $_TARGETNAME1 -
coreid 0

init
#reset

if {[ info exists pulse_srst]} {
    ftdi_set_signal nSRST 0
    ftdi_set_signal nSRST z
}

# add target to resume_group
nuclei cti resume_group on $_TARGETNAME0 $_TARGETNAME1

foreach t [target names] {
    targets $t
    halt
}

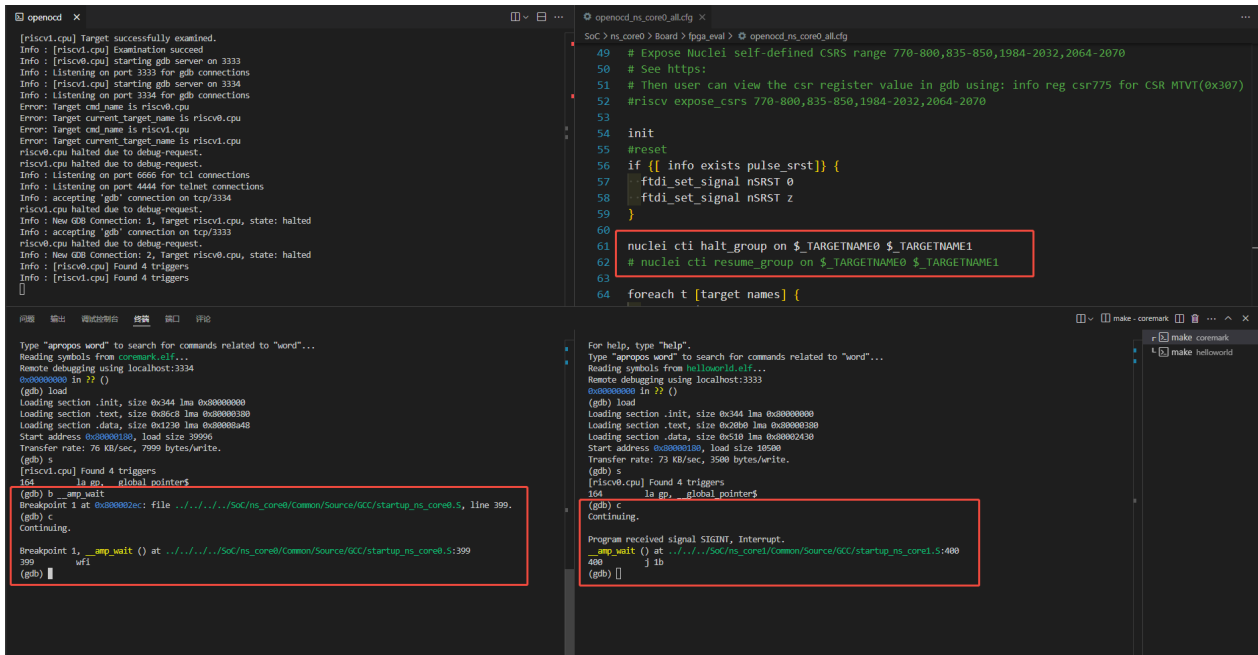
```

Command-Line Verification Steps¶

1. Synchronized Halt Group Verification¶

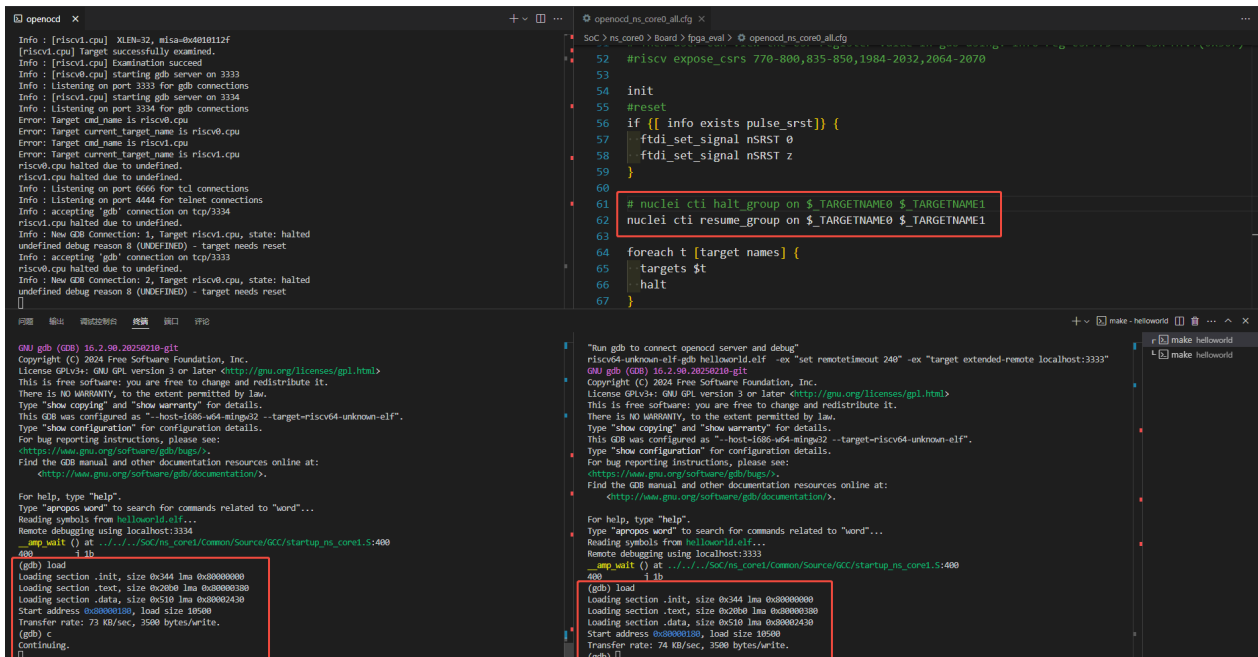
1. Targets have been added to the halt_group in the configuration file
2. Load different firmware onto the two cores
3. Set a breakpoint only in the __amp_wait() function of core0
4. Execution flow: resume core1 first, then resume core0

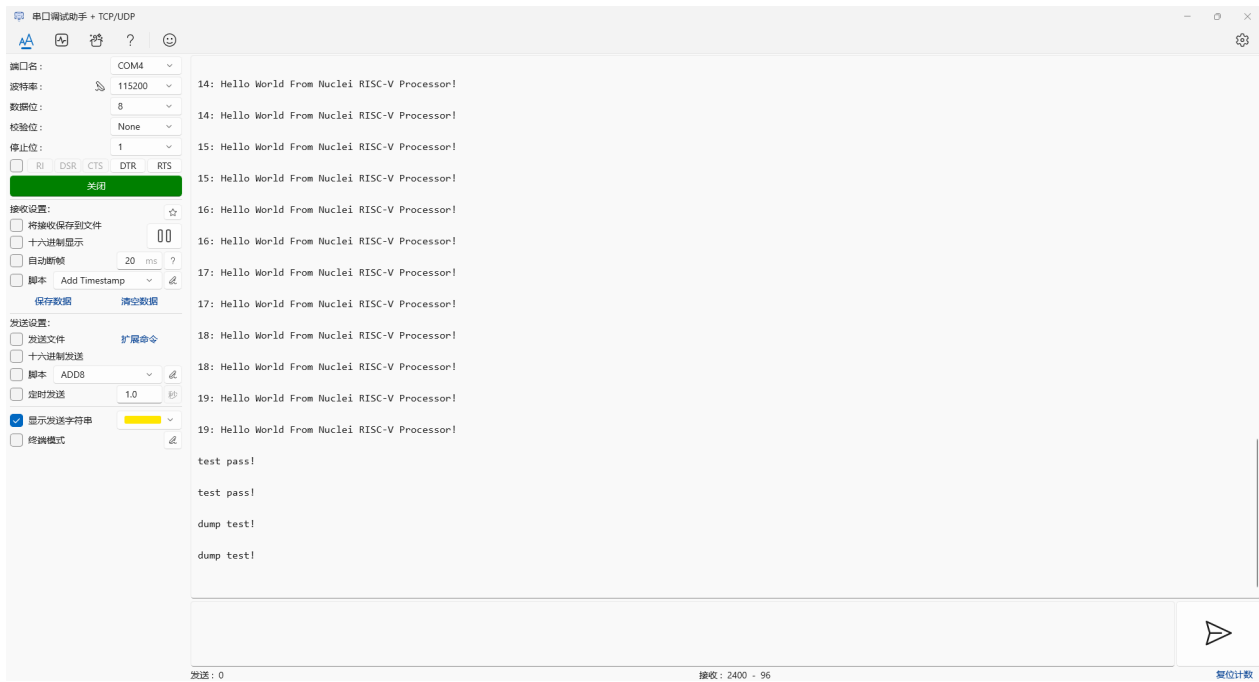
5. Expected result: when core0 hits the breakpoint and halts, core1 halts in sync



2. Synchronized Resume Group Verification

1. Targets have been added to the `resume_group` in the configuration file
2. Load the same helloworld firmware onto both cores
3. Send the `continue/resume` command to core0 only:
4. Expected result: the serial port output shows both cores running simultaneously

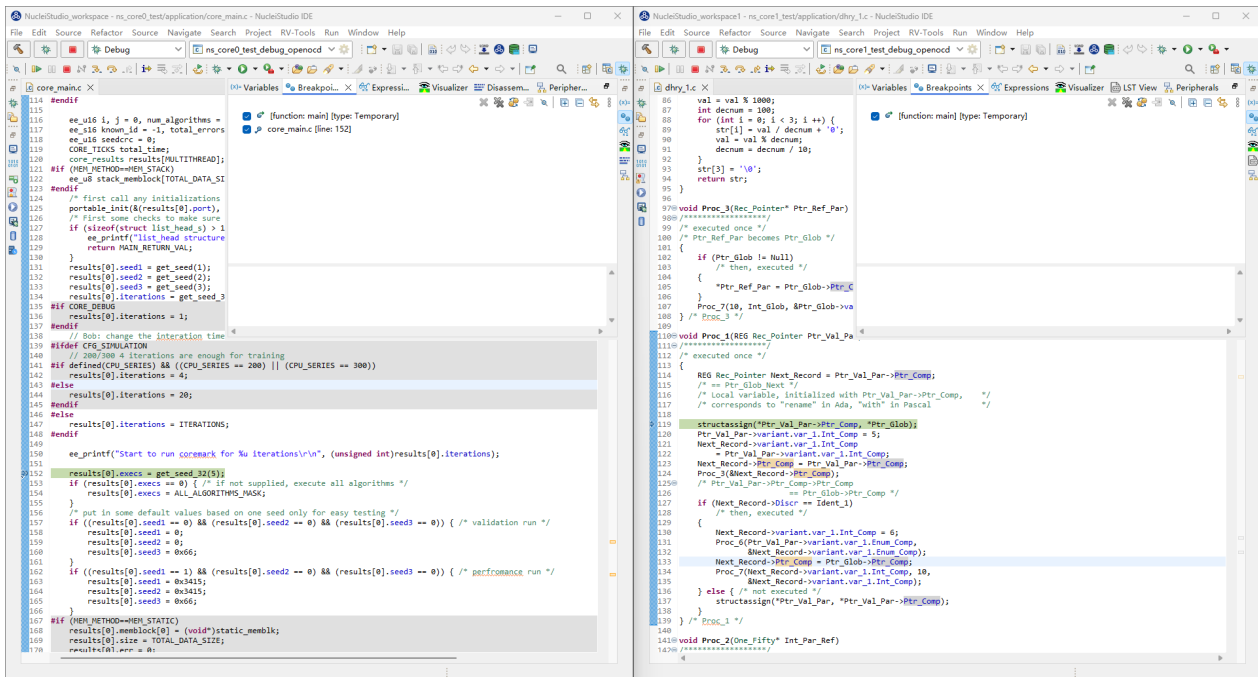




IDE Verification Steps

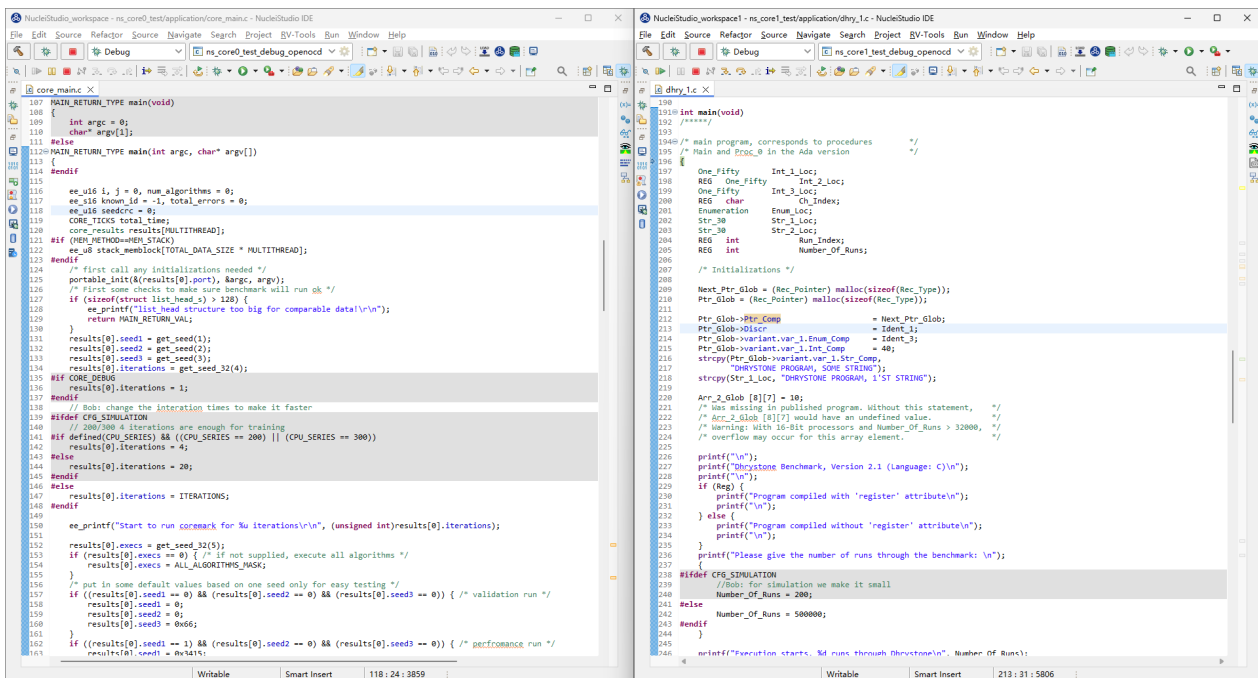
1. Synchronized Halt Group Verification

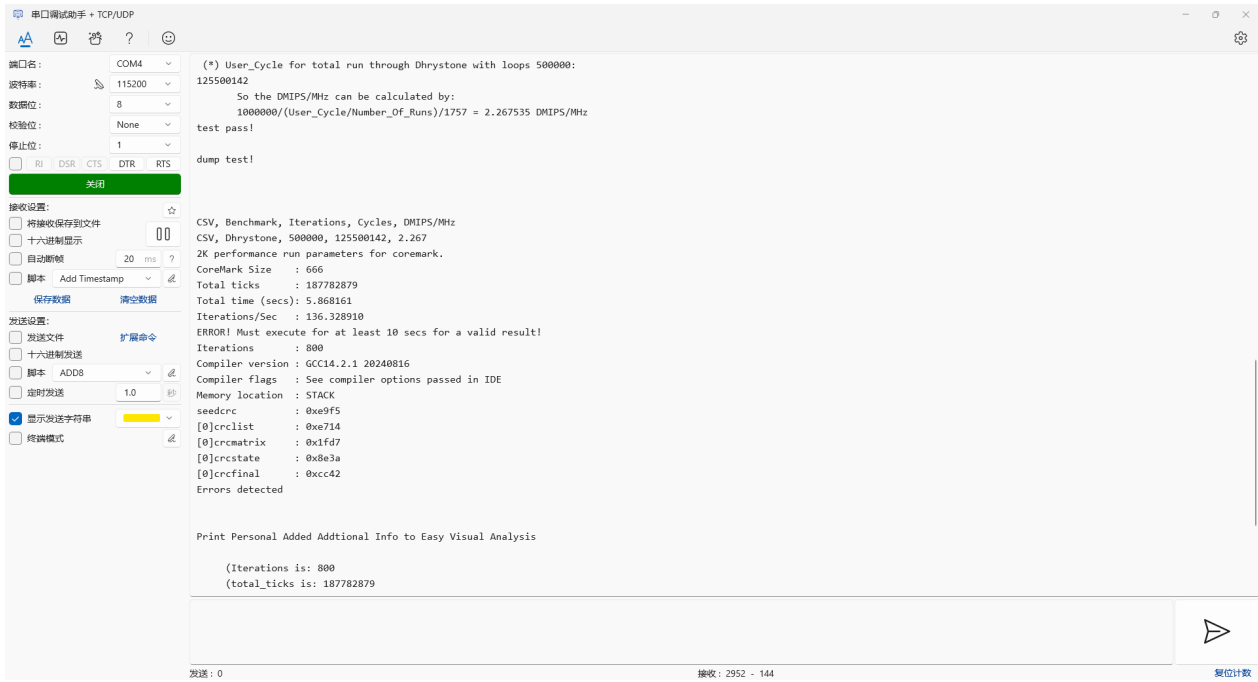
1. halt_group has been configured in the configuration file
2. Load different firmware onto the two cores
3. Set a breakpoint at line 152 of core_main.c in core0
4. Operation sequence:
5. Start core1 first
6. Then start core0
7. Expected result: when core0 hits the breakpoint, core1 halts in sync



2. Synchronized Resume Group Verification

1. resume_group has been configured in the configuration file
2. Load different firmware onto the two cores
3. Start core0 only
4. Expected result: the serial port output shows both cores running simultaneously





User Guide for FreeRTOS Debugging Support in OpenOCD

By updating your Nuclei Studio IDE to version 202502 and downloading sdk-nuclei_sdk 0.7.1, together with the modifications described below, you can debug FreeRTOS using OpenOCD.

Environment Preparation

Nuclei Studio:

- [NucleiStudio 202502 Windows](#)
- [NucleiStudio 202502 Linux](#)

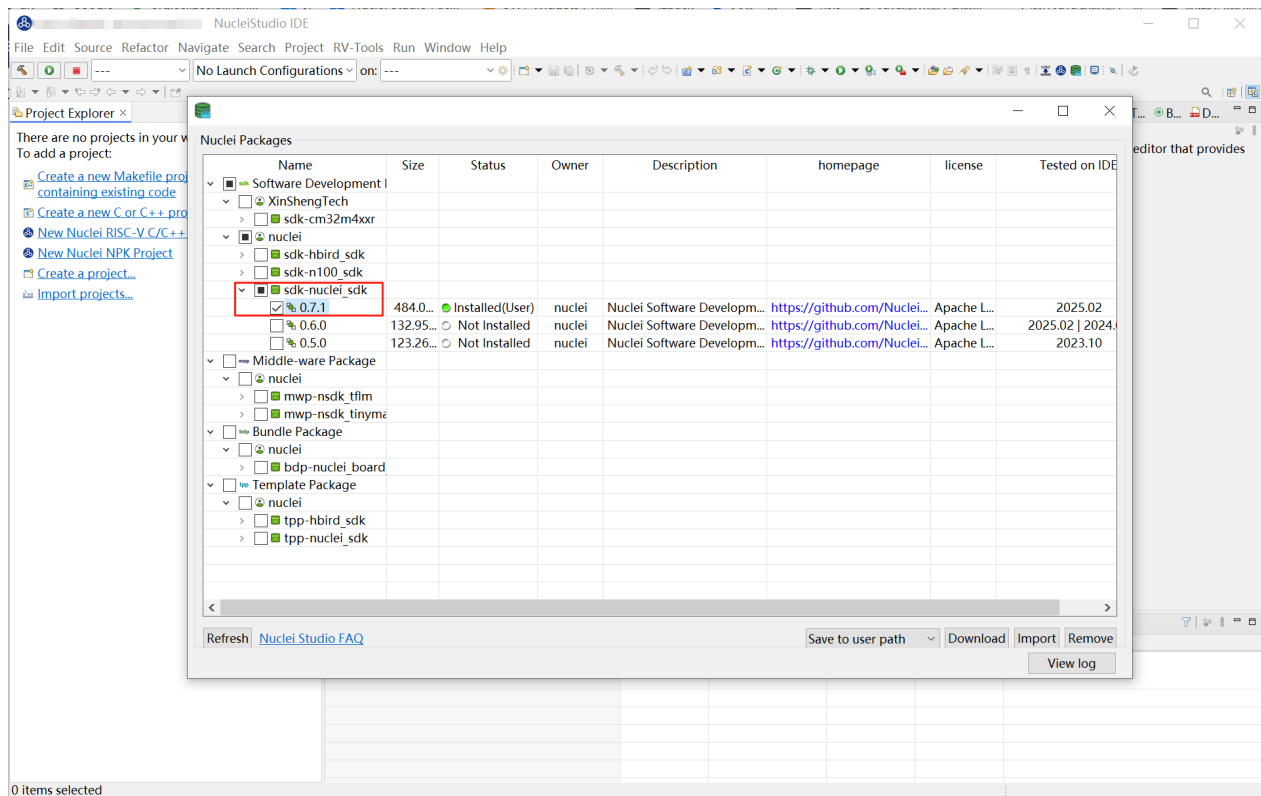
Nuclei OpenOCD:

- The OpenOCD bundled with NucleiStudio 202502 is sufficient.

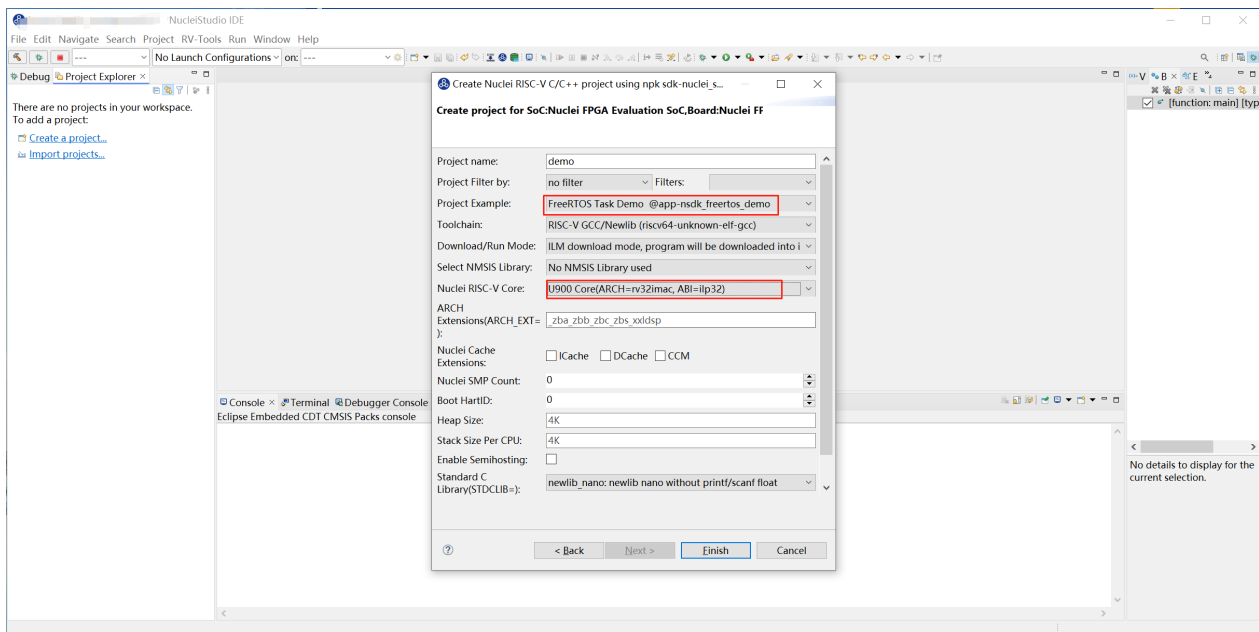
Usage Steps

Step 1: Create the Original Project

Download sdk-nuclei_sdk version 0.7.1 in NucleiStudio IDE.



Create a 900 project, as shown below.

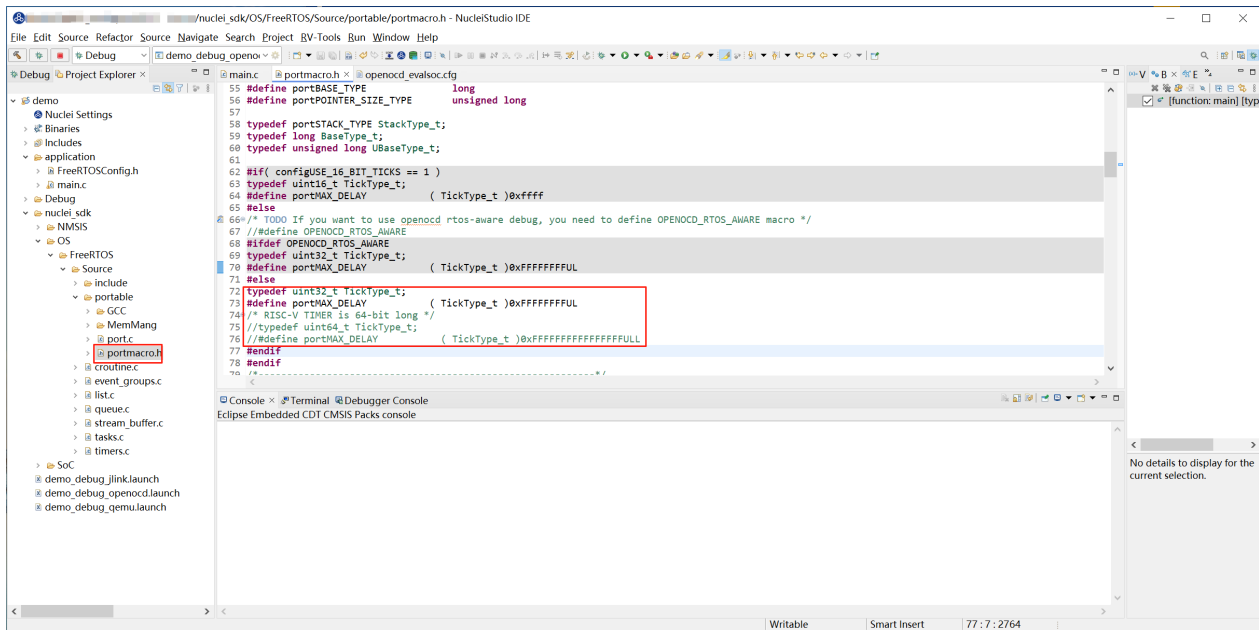


Program the corresponding bitstream to the development board. In this guide, we use u900_best_config_ku060_50M_c1dd7f44af_915aefa97_202504141013_v4.1.0.bit.

Step 2: Modify the portmacro.h Content

After the project is created, locate nuclei_sdk\OS\FreeRTOS\Source\portable\portmacro.h and modify the file content as follows:

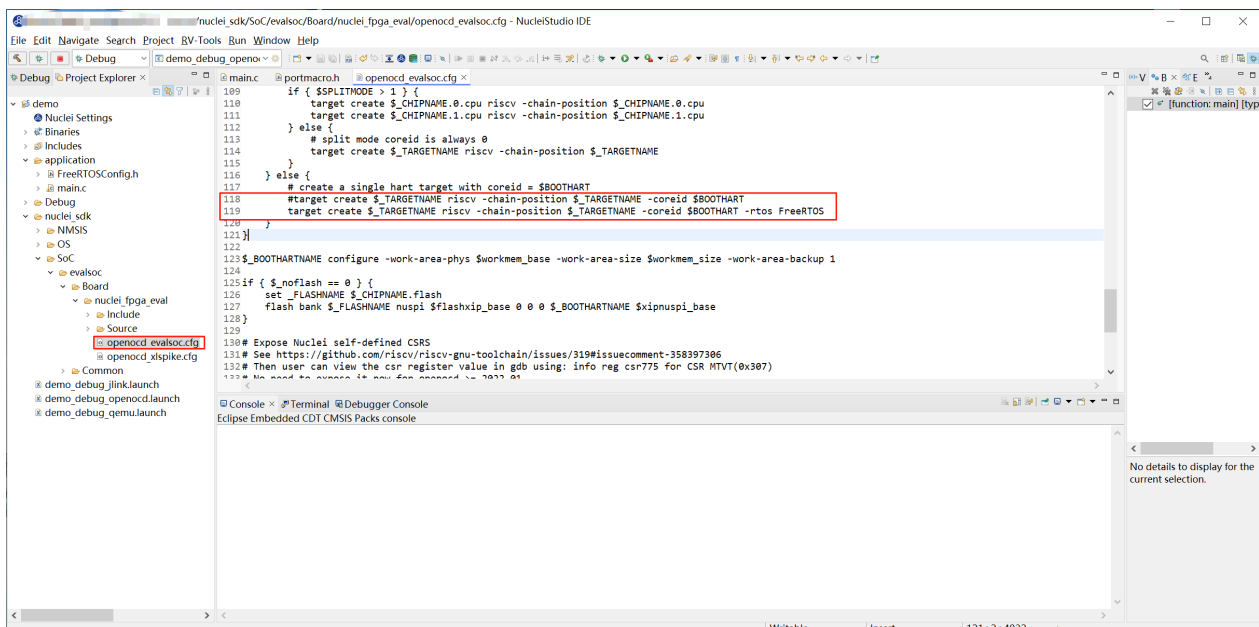
```
typedef uint32_t TickType_t;
#define portMAX_DELAY ( TickType_t )0xFFFFFFFFUL
/* RISC-V TIMER is 64-bit long */
//typedef uint64_t TickType_t;
//#define portMAX_DELAY
( TickType_t )0xFFFFFFFFFFFFFFFFULL
```



Step 3: Modify the openocd_evalsoc.cfg Content

Locate `nuclei_sdk/SoC/evalsoc/Board/nuclei_fpga_eval/openocd_evalsoc.cfg` and modify line 118:

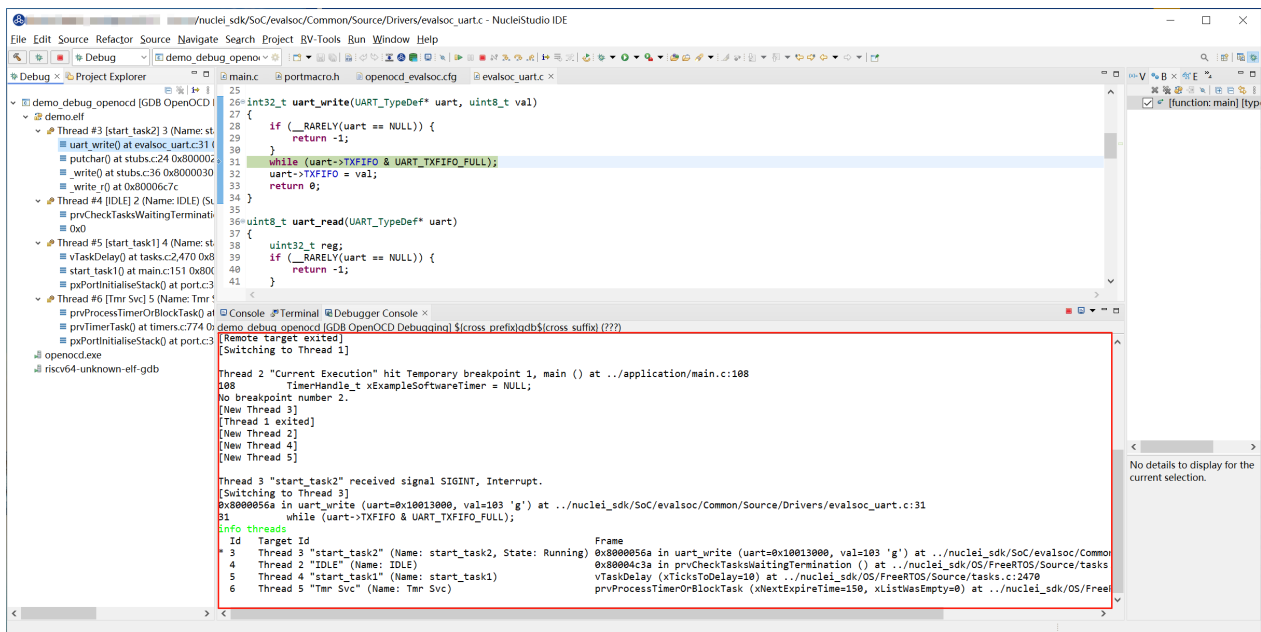
```
target create $_TARGETNAME riscv -chain-position $_TARGETNAME -
coreid $BOOTHART -rtos FreeRTOS
```



Step 4: Debug the Project with OpenOCD

Run the program in Debug mode and open the Debugger Console view.

In the Debugger Console view, type `info threads` and press `Enter`.



Usage Notes

Currently only FreeRTOS is supported; Zephyr, ThreadX, and UCOSII are not supported.

How to Debug with Multiple Hummingbird Debuggers Simultaneously¶

Problem Description¶

Nuclei System Technology's Hummingbird Debugger uses the [FTDI-FT2232H](#) as its USB interface bridge chip. When multiple Hummingbird Debuggers are connected at the same time, how can you distinguish between different debuggers? How do you configure OpenOCD to recognize a specific Hummingbird Debugger?

Solution¶

The FT2232H provides a configurable Serial Number that can be used to distinguish between different debuggers.

Download FT_PROG¶

FT_PROG is a tool for programming the EEPROM inside the FT2232H. It can be used to view and modify the FT2232H's Serial Number.

FT_PROG download page: <https://ftdichip.com/utilities/>

You can find the download link on this page, as shown in the figure below:

FT_PROG 3.12.61.670 - EEPROM Programming Utility

FT_PROG is a free EEPROM programming utility for use with FTDI devices. It is used for modifying EEPROM content.

PLEASE NOTE – The use of some of these utilities by an end user may result in a device being rendered useless.

FT_PROG is available for download by clicking [here](#).

The full FT_PROG User Guide can be downloaded [here](#).

Please Note: FT_PROG requires the Microsoft .NET Framework 4.0 installed on your system to run the application.
[id=17851&WT.mc_id=MSCOM_EN_US_DLC_DETAILS_121LSUS007996](#)

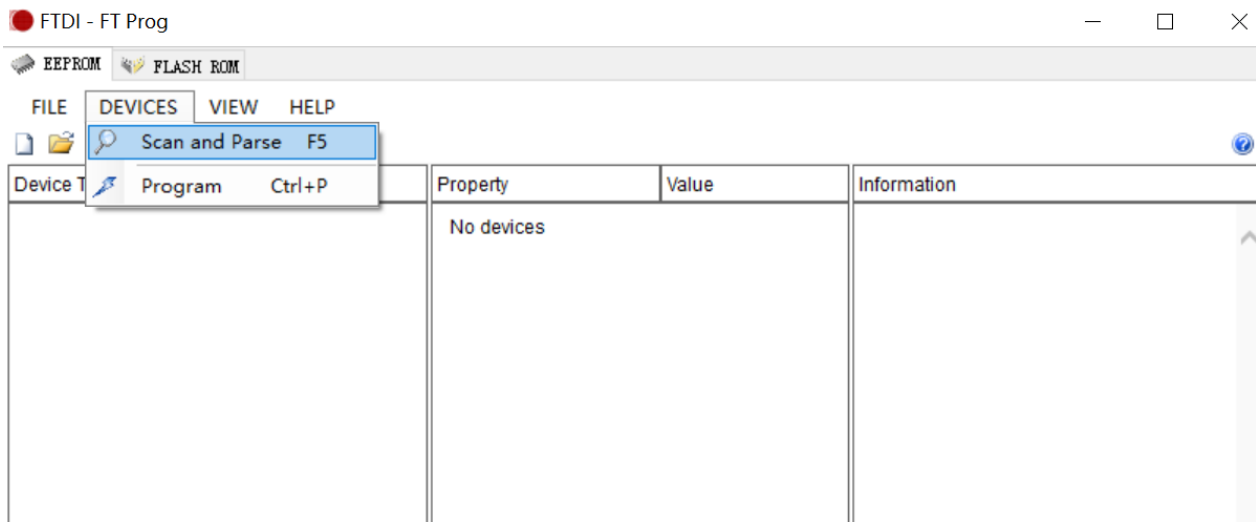
If your system does not have .NET 4.0 installed please download the file from the above link. To install, double click

After downloading and installing, an FT_Prog tool icon will be created on the desktop.

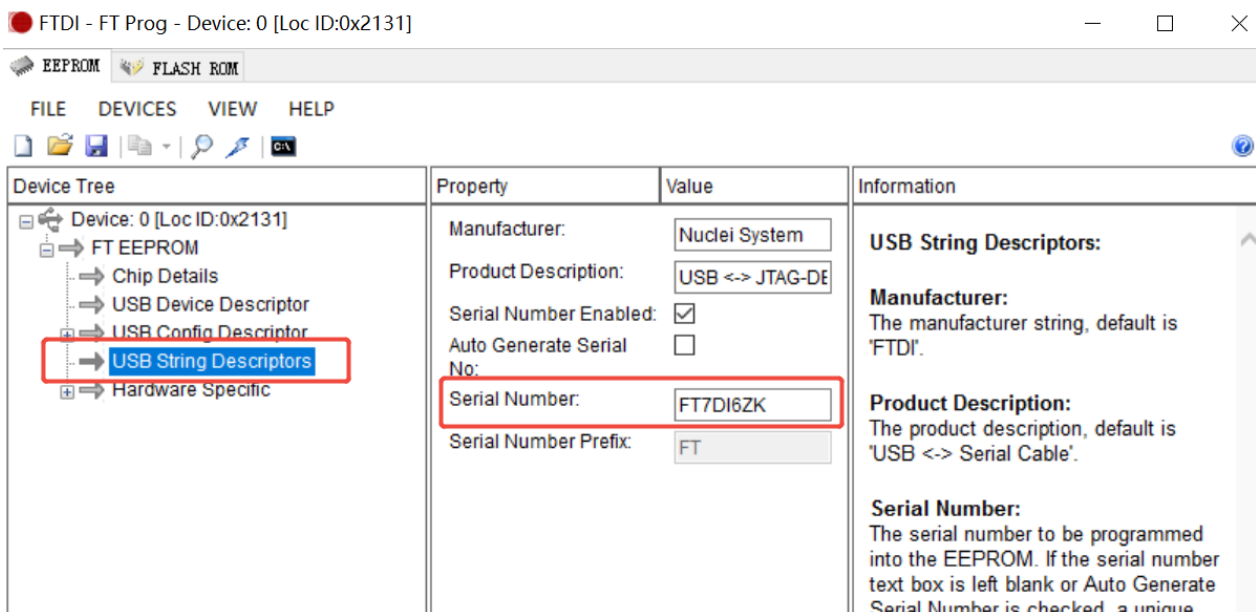
View the Serial Number

Using the FT_PROG tool, you can view the FT2232H's Serial Number.

1. Connect the Hummingbird Debugger If you cannot distinguish between multiple Hummingbird Debuggers, it is recommended to connect only one Hummingbird Debugger first.
2. Open the FT_PROG tool Click the FT_PROG icon to open the tool.
3. Scan for devices Click Scan and Parse in the DEVICES menu to scan for connected Hummingbird Debuggers.



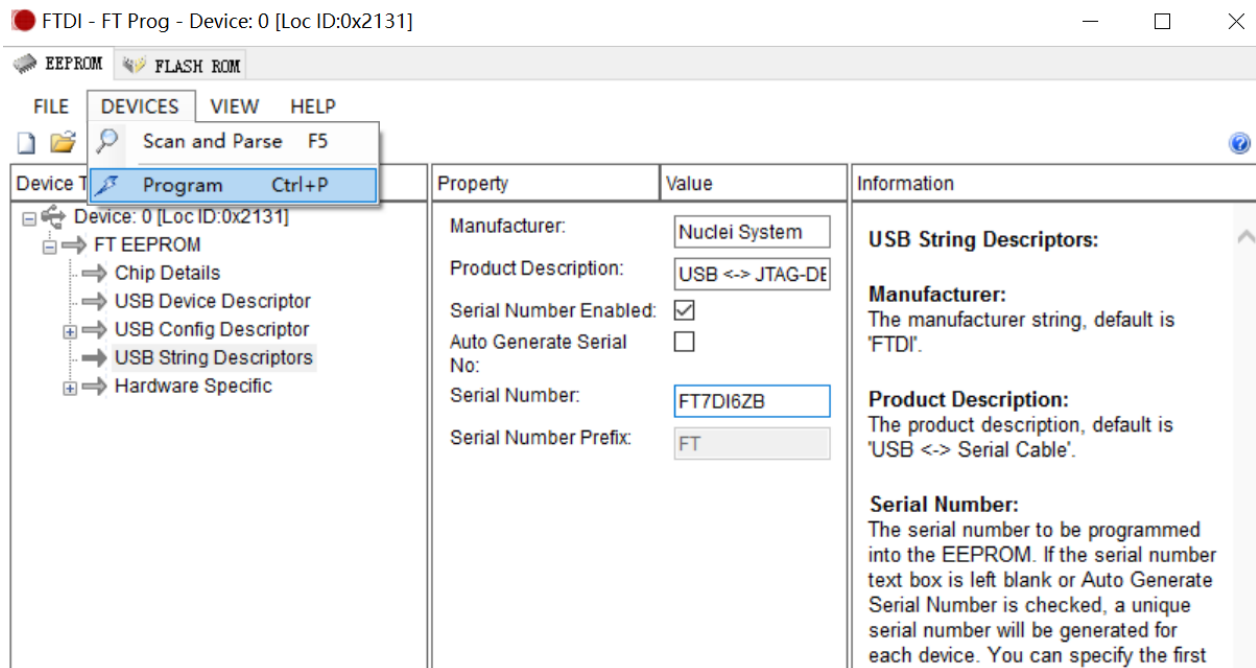
1. View the Serial Number You can view the Hummingbird Debugger's Serial Number under Serial Number in the USB String Descriptors section.



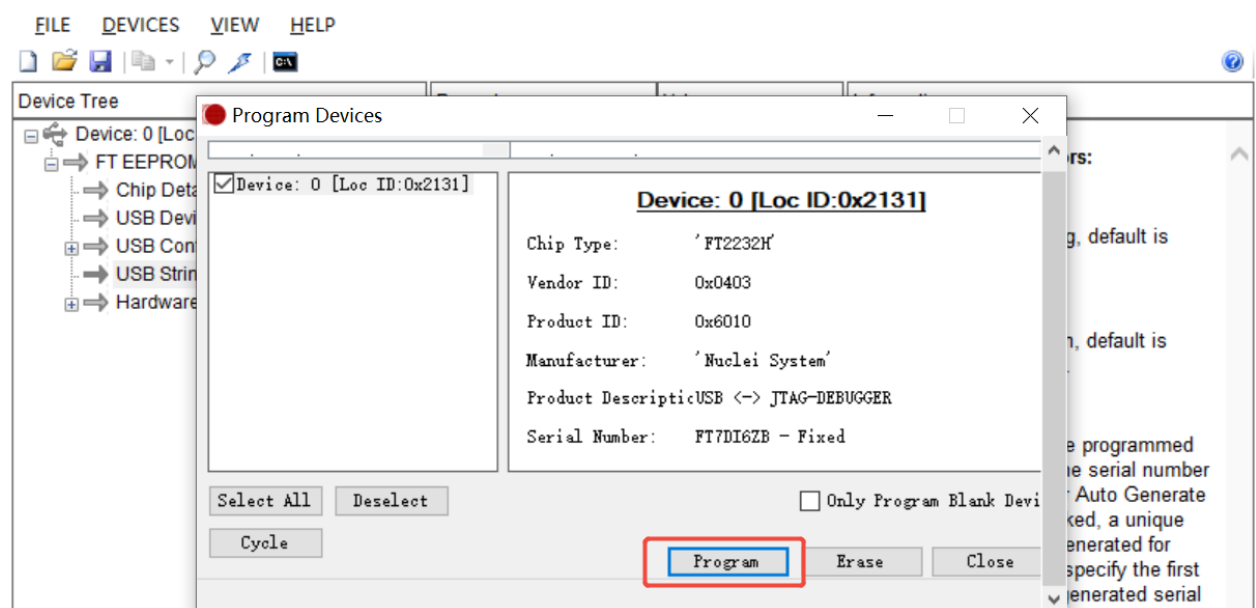
Modify the Serial Number

You can modify the Hummingbird Debugger's Serial Number on the same page where you view it.

For example, in the figure below, I changed the original Serial Number FT7DI6ZK to FT7DI6ZB



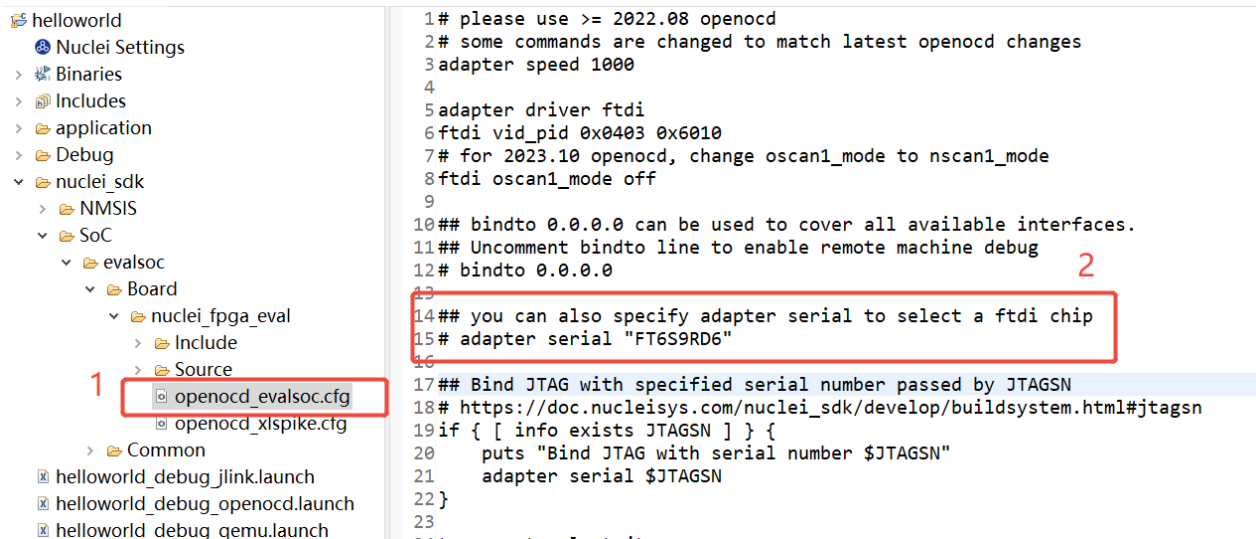
Then, using the Program option in the DEVICES menu, you can write the modified Serial Number into the FT2232H's EEPROM.



Note: Multiple Hummingbird Debuggers must be assigned different Serial Numbers to distinguish between them.

Update the OpenOCD Configuration

When using the Nuclei FPGA Evaluation Board, open the project's OpenOCD configuration file in Nuclei Studio, and you will see the following content:



Linux

Modify the `openocd_evalsoc.cfg` file according to the instructions in the red box in the figure:

```
# Note: remove the comment symbol # before adapter serial
adapter serial "<Serial Number>"
```

Replace `<Serial Number>` with the actual Serial Number.

Once modified, the project can be debugged using the Hummingbird Debugger with the specified Serial Number.

Windows

Note: On Windows, you need to append A to the actual Serial Number for the setting to be valid.

For example, if the actual Serial Number is FT7DI6ZB, the OpenOCD configuration file needs the following setting:

```
adapter serial "FT7DI6ZBA"
```

References

- [Nuclei Studio FAQs — How to select correct FTDI debugger?](#)

- [FTDI Utilities](#)
- [User Guide for FTDI FT_PROG Utility](#)

Quick Porting from evalsoc to customsoc Based on Nuclei SDK

Overview

Nuclei Eval SoC (evalsoc for short) is a SoC provided by Nuclei System Technology for evaluating Nuclei CPUs. It features On-Chip SRAMs, UART, SPI, and more.

[Nuclei SDK](#) and [Nuclei N100 SDK](#) provide an evalsoc-based software development platform. After evaluating Nuclei CPUs with evalsoc, customers often want to quickly adapt the corresponding SDK to their own SoC (referred to as customsoc in this document).

- Nuclei SDK mainly supports Nuclei 200/300/600/900/1000 series RISC-V CPUs, for rapid software evaluation and development on EvalSoC based on these CPU series
- Nuclei N100 SDK mainly supports Nuclei 100 series RISC-V CPUs, for rapid software evaluation and development on EvalSoC based on these CPU series

Solution

Depending on the CPU series to be ported and adapted, pull the latest corresponding SDK repository, or directly use the SDK included in the CPU delivery package.

Environment Preparation

Adaptation Modifications

If the accompanying files were generated with the `nuclei_gen` tool, simply replace the files with the same names directly — this is simple and less error-prone. If modifying manually, pay attention to the files and modification points mentioned below.

Do not rename any directories or files yet. Modify the following files step by step.

1 Modify the CPU feature description macro file

The `SoC/evalsoc/Common/Include/cpufeature.h` file defines the `#define` macros related to the features and parameters supported by customsoc. The `nuclei_gen` tool in the CPU delivery package automatically generates this file; simply replace it.

2 Modify the CPU feature ISA configuration

The SoC/evalsoc/cpufeature.mk file defines the CORE (whether single/double-precision floating point is supported) and ARCH_EXT (whether the b and v extensions are supported, etc.) of customsoc. The nuclei_gen tool in the CPU delivery package automatically generates this file; simply replace it.

3 Modify the memory map of link addresses

SoC/evalsoc/Board/nuclei_fpga_eval/Source/GCC/evalsoc.memory describes the BASE addresses and SIZEs of ILM/DLM/FLASH/SRAM/DDR, as well as the size of the code sections. The nuclei_gen tool in the CPU delivery package automatically generates this file; simply replace it.

4 Modify the OpenOCD configuration file

OpenOCD establishes a gdb server port with the CPU via JTAG, which is used by GDB for debugging and loading.

SoC/evalsoc/Board/nuclei_fpga_eval/openocd_evalsoc.cfg is the OpenOCD configuration description file. The nuclei_gen tool in the CPU delivery package automatically generates this file; simply replace it. Key parameters are as follows:

```
# TODO: variables should be replaced by nuclei_gen
set workmem_base      0x80000000
set workmem_size      0x10000
set flashxip_base     0x20000000
set xipnuspi_base     0x10014000
```

5 Modify the Systimer frequency

In SoC/evalsoc/Common/Include/evalsoc.h, modify SOC_TIMER_FREQ to the actual Systimer frequency of customsoc (consult your SoC hardware designers).

```
// The unit is Hz; for example, for 32768 Hz, enter 32768 here
#define SOC_TIMER_FREQ          customsoc_systimer_freq
```

6 Modify the CPU core frequency

In SoC/evalsoc/Common/Source/system_evalsoc.c, SystemCoreClock = get_cpu_freq() automatically calculates the CPU core frequency (depending on Systimer). It can be modified directly to the customsoc core frequency.

```
// The unit is Hz; for example, for 50 MHz, enter 50000000 here
SystemCoreClock = customsoc_cpu_freq;
```

7 Modify the UART driver

The UART IP of evalsoc is an evaluation version.

Do not modify the `uart_xxx` API names in `evalsoc_uart.c` and `evalsoc_uart.h`, because some stub functions under `SoC/evalsoc/Common/Source/Stubs` use the UART APIs.

The UART driver is located in `SoC/evalsoc/Common/Source/Drivers/evalsoc_uart.c` and `SoC/evalsoc/Common/Include/evalsoc_uart.h`. If a different UART IP is used, adapt it according to the actual UART register definitions.

8 Modify the UART baud rate

`SoC/evalsoc/Common/Source/system_evalsoc.c`: `uart_init(SOC_DEBUG_UART, 115200);` the baud rate is generally 115200.

9 Modify `_premain_init`

Some initialization that needs to run before the main function can be placed in this function.

If there are other related configurations such as IOMUX and PLL, they can be implemented in the `_premain_init` function in `SoC/evalsoc/Common/Source/system_evalsoc.c`; if not, this step can be skipped.

10 Remove the code used internally by Nuclei

`SoC/evalsoc/Common/Source/system_evalsoc.c`: the `SIMULATION_EXIT` macro definition is a marker used for Nuclei internal simulation, and can be defined as empty.

```
#define SIMULATION_EXIT(ret)    {}
```

11 Check peripheral addresses

It is recommended not to modify these when configuring the CPU; keep them consistent with evalsoc.

The SOC_DEBUG_UART used by the UART is defined as UART0.

- The base addresses of peripherals are determined by EVALSOC_PERIPHS_BASE, which is defined in SoC/evalsoc/Common/Include/cpufeature.h (generated by the nuclei_gen tool; just copy and overwrite it). Generally no further modification is needed.
- The offset addresses of peripherals are defined in SoC/evalsoc/Common/Include/evalsoc.h. Search for Peripheral memory map; generally no modification is needed.

```
#define UART0_BASE          (EVALSOC_PERIPH_BASE +
0x13000)                  /*!< (UART0) Base Address */
#define QSPI0_BASE          (EVALSOC_PERIPH_BASE +
0x14000)                  /*!< (QSPI0) Base Address */
#define UART0               ((UART_TypeDef *) UART0_BASE)
```

Test Run

Once the above modifications are complete, you can test whether the SoC works properly.

Since this is modified on the basis of evalsoc, the related names have not yet been changed to customsoc.

Therefore, still use SOC=evalsoc BOARD=nuclei_fpga_eval.

```
# Test helloworld application
## cd to helloworld application directory
cd application/baremetal/helloworld
## clean and build helloworld application for ncstar_eval board
make SOC=evalsoc BOARD=nuclei_fpga_eval clean all
## connect your board to PC and install jtag driver, open UART
terminal
## set baudrate to 115200bps and then upload the built application
## to the fpga board using openocd, and you can check the
## run message in UART terminal
make SOC=evalsoc BOARD=nuclei_fpga_eval upload
```

If it runs normally and prints Hello World From Nuclei RISC-V Processor, then there are basically no problems. If you need to run more cases, please refer to the following application example documentation to confirm whether they run successfully.

- Nuclei SDK: https://doc.nucleisys.com/nuclei_sdk/design/app.html
- Nuclei N100 SDK: https://doc.nucleisys.com/nuclei_n100_sdk/design/app.html

Adjust Names

There are quite a few places to rename, so they are not listed here. Ultimately, just make sure compilation passes.

After the test passes, you can change the file names and directory names involving evalsoc to customsoc, and replace the macro names/file names starting with eval/EVAL with custom.

```
# After the modifications, test again
make SOC=customsoc BOARD=nuclei_fpga_custom upload
```

At this point, the SDK has shed the eval logo and become an SDK for custom.

Streamline the Code

Because Nuclei SDK/N100 SDK supports evaluation and internal testing of multiple Nuclei CPU series, it has to cover a great many scenarios, so there is some redundant code. It is recommended to streamline and remove code only after reading the SDK documentation and becoming familiar with the code framework.

IAR Project

- The IAR project has a dedicated linker script, located at SoC/evalsoc/Board/nuclei_fpga_eval/Source/IAR/*.icf. The IAR linker script is currently not generated by the nuclei_gen tool, so you need to manually check and adjust the base addresses and sizes of ROM_region32/ILM_region32/RAM_region32. Here, "from" represents the base address, and "size" indicates the size of that region.

```
define region ROM_region32 = mem:[from 0x20000000 size 0x800000];
define region ILM_region32 = mem:[from 0x80000000 size 0x10000];
define region RAM_region32 = mem:[from 0x90000000 size 0x10000];
```

- The IAR project is located at ideprojects/iar and is also prebuilt for evalsoc, so it can run directly before the names are adjusted. If the names have been adjusted, the paths and file names have changed, and the project needs to be recreated. It is recommended to open the ewp file in a text editor and search for the keyword "eval" to replace it.

```
diff --git a/ideprojects/iar/baremetal/coremark.ewp b/ideprojects/iar/baremetal/coremark.ewp
index 3eed66a8..17443eae 100644
--- a/ideprojects/iar/baremetal/coremark.ewp
+++ b/ideprojects/iar/baremetal/coremark.ewp
```

```

@@ -434,8 +434,8 @@
        <option>
            <name>CCIncludePath2</name>
            <state>$PROJ_DIR$\\..\\..\\..\\NMSIS\\Core\\Include</
state>
-           <state>$PROJ_DIR$\\..\\..\\..\\
\\SoC\\evalsoc\\Board\\nuclei_fpga_eval\\Include</state>
-           <state>$PROJ_DIR$\\..\\..\\..\\
\\SoC\\evalsoc\\Common\\Include</state>
+           <state>$PROJ_DIR$\\..\\..\\..\\
\\SoC\\customsoc\\Board\\nuclei_fpga_custom\\Include</state>
+           <state>$PROJ_DIR$\\..\\..\\..\\
\\SoC\\customsoc\\Common\\Include</state>
            <state>$PROJ_DIR$\\..\\..\\..\\
\\application\\baremetal\\benchmark\\coremark</state>
        </option>

```

IDE Project Support

If you want Nuclei Studio IDE to support the custom SoC, you need to modify the names involving "eval" in the following files. For the syntax format of npk.yml, see [2.4. Nuclei Studio NPK Introduction](#).

```

evalsoc/Common/npk.yml
evalsoc/Board/nuclei_fpga_eval/npk.yml

```

References

- [Nuclei 200/300/600/900/1000 Eval SoC](#)
- [Port your SoC into Nuclei SDK](#)
- [Nuclei 100 Eval SoC](#)
- [Port your SoC into Nuclei N100 SDK](#)

Using the OVERLAY Command in Linker Scripts

Problem Description

The SRAM inside a CPU Core is fast, small in capacity, and large in area. In embedded systems, this on-Core RAM is very likely unable to hold all functions. To solve this problem, one approach is to use the OVERLAY command in the linker script. The OVERLAY command provided by GNU `ld` allows multiple sections to be "overlaid" on the same memory region. Several sections can share the same runtime VMA (Virtual Memory Address); at runtime, the loading and unloading of these overlay sections must be managed manually.

So how do you use the OVERLAY command in Nuclei Studio IDE? This article provides a sample program demonstrating how to use the OVERLAY command.

Solution

Sample Program

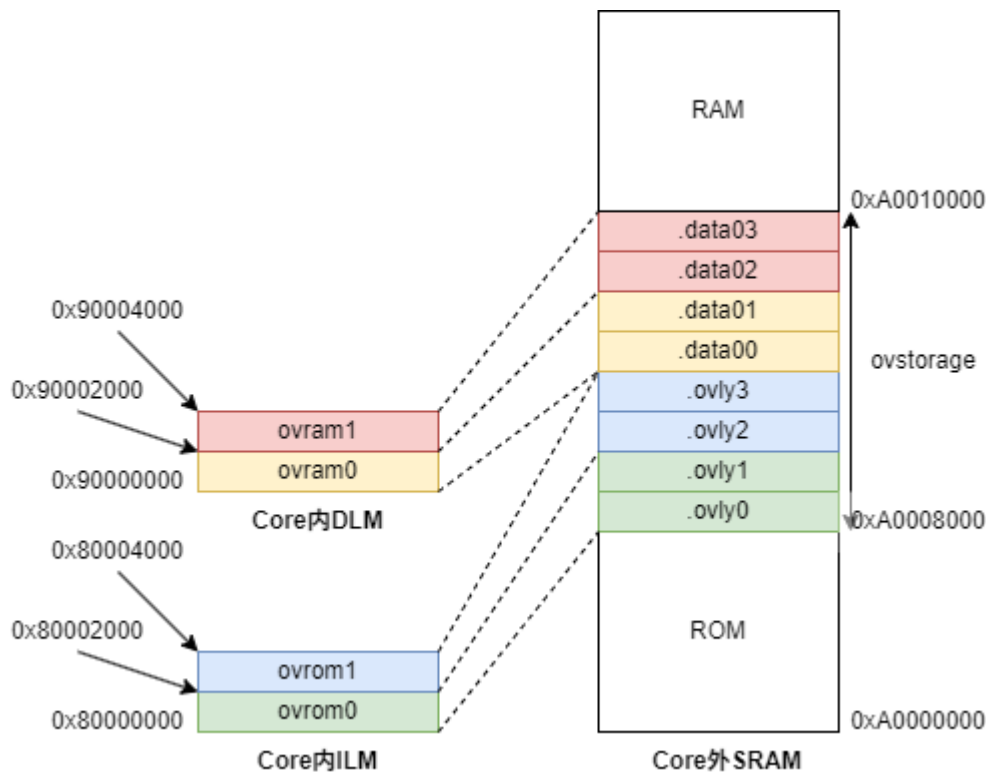
[demo_overlay](#) is a sample project created with [Nuclei Studio IDE 2025.02](#). It supports both Linux and Windows platforms and demonstrates how to use the OVERLAY command in a linker script.

The code in the sample project is essentially the same as the code provided in [Overlay Sample Program](#). The main differences are that the test results are printed in the main function, and that the linker script has been modified according to the address mapping of evalsoc.

```
├─ bar.c
├─ baz.c
├─ foo.c
├─ grbx.c
├─ overlays.c
├─ ovlymgr.c
└─ ovlymgr.h
```

The original code can be obtained from [gdb/testsuite/gdb.base](#).

Overlay Layout



On our evalsoc, if a program cannot fit entirely into the Core's ILM/DLM, the approach shown in the figure above can be adopted: some sections are dynamically loaded into the ILM/DLM in the form of overlays and executed there. The LMA (Load Memory Address) of all eight sections `.ovlyx` and `.data0x` is located in the SRAM outside the Core, but their VMAs are inside the Core, and some of them overlap.

Writing the Linker Script

In the `MEMORY` command, first define the memory regions you need — for example, the four Core-internal regions `ovrom0`, `ovrom1`, `ovram0`, `ovram1`, and the Core-external region `ovstorage`. The size of each memory region can be adjusted according to the actual size of the code and data.

```
MEMORY
{
    rom (rxa!w) : ORIGIN =
SRAM_MEMORY_BASE,                LENGTH =
SRAM_MEMORY_ROM_SIZE
    ovstorage (rwa) : ORIGIN = SRAM_MEMORY_BASE +
SRAM_MEMORY_ROM_SIZE, LENGTH = SRAM_OVLY_STORAGE_SIZE
    ram (wxa!r) : ORIGIN = SRAM_MEMORY_BASE + SRAM_MEMORY_ROM_SIZE +
SRAM_OVLY_STORAGE_SIZE,    LENGTH = SRAM_MEMORY_SIZE -
SRAM_MEMORY_ROM_SIZE - SRAM_OVLY_STORAGE_SIZE
```

```

    ovrom0 (rwx) : ORIGIN = ILM_MEMORY_BASE,    LENGTH =
    ILM_OVLY_SIZE0
    ovrom1 (rwx) : ORIGIN = ILM_MEMORY_BASE + ILM_OVLY_SIZE0,
    LENGTH = ILM_OVLY_SIZE1
    ovram0 (rwx) : ORIGIN = DLM_MEMORY_BASE,    LENGTH =
    DLM_OVLY_SIZE0
    ovram1 (rwx) : ORIGIN = DLM_MEMORY_BASE + DLM_OVLY_SIZE0,
    LENGTH = DLM_OVLY_SIZE1
}

```

The OVERLAY command must be placed inside the SECTIONS command. For example, the code below places the two sections .ovly0 and .ovly1 into the same VMA address range represented by ovrom0, while their LMAs are contiguously located in the address range represented by ovstorage.

```

OVERLAY :
{
    .ovly0 { *foo.o(.text .text.*) }
    .ovly1 { *bar.o(.text .text.*) }
} >ovrom0 AT>ovstorage

```

Referring to [Automatic Overlay Debugging](#), the following code in the linker script exposes the VMAs, LMAs, and count of the overlay sections to C code through the _ovly_table and _novlys variables; the dynamic loading and unloading of sections is then further implemented in ovlymgr.c.

```

/* _ovly_table used for gdb debug overlay sections */
_ovly_table = .;
_ovly0_entry = .;
LONG(ABSOLUTE(ADDR(.ovly0)));
LONG(SIZEOF(.ovly0));
LONG(LOADADDR(.ovly0));
LONG(0);
...
_novlys = .;
LONG((_novlys - _ovly_table) / 16);

```

Test Results

Examining the map file generated after compilation, you can see that the corresponding code sections and data sections are all allocated with the expected VMAs and LMAs. For example, .ovly0 and .ovly1 share the same VMA 0x80000000, while their LMAs are 0xa0008000 and 0xa0008028 respectively.

```

.ovly0          0x80000000      0x28 load address 0xa0008000
*foo.o(.text .text.*)
.text.foo       0x80000000      0x28 ./application/foo.o
               0x80000000      foo
               [!provide]      PROVIDE
(__load_start_ovly0 = LOADADDR (.ovly0))
               [!provide]      PROVIDE
(__load_stop_ovly0 = (LOADADDR (.ovly0) + SIZEOF (.ovly0)))

.ovly1          0x80000000      0x28 load address 0xa0008028
*bar.o(.text .text.*)
.text.bar       0x80000000      0x28 ./application/bar.o
               0x80000000      bar
               [!provide]      PROVIDE
(__load_start_ovly1 = LOADADDR (.ovly1))
               [!provide]      PROVIDE
(__load_stop_ovly1 = (LOADADDR (.ovly1) + SIZEOF (.ovly1)))

```

In the main function, `OverlayLoad` is used to switch between different functions, and finally the return values of each function are accumulated to verify the accumulated result.

```

/* load .text and .data for `foo` */
OverlayLoad (0);
OverlayLoad (4);
a = foo (1);
/* load .text and .data for `bar` */
OverlayLoad (1);
OverlayLoad (5);
b = bar (1);
/* load .text and .data for `baz` */
OverlayLoad (2);
OverlayLoad (6);
c = baz (1);
/* load .text and .data for `grbx` */
OverlayLoad (3);
OverlayLoad (7);
d = grbx (1);

e = a + b + c + d;
if (e != ('f' + 'o' + 'o'
        + 'b' + 'a' + 'r'
        + 'b' + 'a' + 'z'
        + 'g' + 'r' + 'b' + 'x')) {
    printf ("Overlay Test FAIL\r\n");
} else {

```

```
printf ("Overlay Test PASS\r\n");  
}
```

Testing through QEMU simulation or on an FPGA development board yields the following result, where Overlay Test PASS indicates that the result meets expectations.

```
Nuclei SDK Build Time: Sep 25 2025, 17:02:19  
Download Mode: SRAM  
CPU Frequency 16003235 Hz  
CPU HartID: 0  
Overlay Test PASS
```

Notes¶

1. When a data section overlay is about to replace a section, its data must be saved — that is, an "unload" operation is required to save the data into external SRAM; a code section, on the other hand, is read-only and therefore does not need to be unloaded.
2. In the sample project, neither the ILM nor the DLM goes through the cache, so cache coherency does not need to be considered. However, if the VMA of an overlay section is located in a cacheable region, cache coherency generally must be taken into account, unless there is hardware-supported snooping between the ICache and DCache. For more details, refer to the implementation of `ovlymgr.c` in the sample project.

References¶

- [How Overlays Work](#)
- [Overlay Commands](#)
- [Automatic Overlay Debugging](#)
- [Overlay Sample Program](#)